



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра информационных технологий в атомной энергетике (ИТвАЭ)

КУРСОВАЯ РАБОТА

по дисциплине «Автоматизированные системы управления элементами
промышленных и административных объектов»

Тема: «Автоматизированное управление для системы инженерии для ядерной энергии»

Студент Павел Никитов
Бергесов

Группа ИКБО-50-23

(подпись студента)

Работа представлена к защите _____ (дата)

Руководитель Щербаков К.В.

Работа допущена к защите _____ (дата)

/ Щербаков К.В.
(подпись и ФИО
руководителя)

Оценка по итогам защиты отлично

/ Щербаков Кирилл Владимирович, старший преподаватель
_____/_____

(подписи, дата, ф.и.о., должность, звание, уч. степень двух преподавателей, принявших защиту)

РТУ МИРЭА 202 6 г.



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра информационных технологий в атомной энергетике (ИТВАЭ)

Утверждаю

Заведующий кафедрой ИТВАЭ

Боридько С.И.

(подпись)

28 августа 2025 г.

ЗАДАНИЕ

на выполнение курсовой работы

по дисциплине «Автоматизированные системы управления элементами
промышленных и административных объектов»

Студент Павлов Никита Сергеевич

Группа ИКБО-50-23

Тема «Автоматизированная система управления для обеспечения инвентарем для лыжного спорта».

Цель курсовой работы:

Выполнить проектирование элементов автоматизированной системы управления (АСУ) в соответствии с темой курсовой работы.

Задачи курсовой работы:

С помощью технологии OSINT осуществить сбор информации об организации по направлению деятельности в соответствии с темой курсовой работы.

Разработать функциональные и нефункциональные требования к АСУ в соответствии с темой курсовой работы с использованием реверс-инжиниринга.

Выполнить моделирование АСУ в соответствии с темой курсовой работы с использованием средств визуализации стандарта UML с разбиением АСУ на модули (обязательное представление диаграмм вариантов использования для каждого модуля, диаграммы последовательности для основного процесс деятельности организации и диаграммы классов).

Выполнить выбор и описание программно-аппаратной платформы, сформировать перечень программного обеспечения для обеспечения всех функций АСУ, указанных в требованиях и отображенных на UML диаграммах.

Определить функцию АСУ, которая в наименьшей степени обеспечивается выбранным программным обеспечением. Выбрать IDE, библиотеки для создания программного модуля для

проектируемой АСУ, реализующего данную функцию. Представить исходный код разработанного модуля.

Построить ER-диаграмму на основе классовой диаграммы UML, выбрать систему управления базами (СУБД) данных и реализовать схему данных в выбранной СУБД.

Выбрать серверное и клиентское оборудование для обеспечения выполнения функций, отображенных на UML диаграммах, построить схему сети с использованием средств виртуализации. Выбрать среду виртуализации, создать виртуальный стенд, реализующий виртуальную инфраструктуру АСУ.

Создать программный модуль с Web-интерфейсом для работы с базой данных АСУ.

Предложить решение по обеспечению безопасности для АСУ, выбрав средства обеспечения безопасности обработки и передачи информации в соответствии со схемой сети.

Требования к курсовой работе:

По результатам выполнения курсовой работы формируется отчет. Выполнение каждой задачи курсовой работы подтверждается разделом отчета с обязательными выводами о проделанной работе. Все результаты выполнения задач курсовой работы, представленные в отчете, должны быть получены лично автором, все заимствования оформляются со ссылкой на авторитетный источник, указанный в списке использованной литературы в конце отчета.

Срок представления к защите курсовой работы:

до « » _____ 202__ г.

Задание на курсовую работу выдал


Подпись руководителя

Щербаков К.В.
(ФИО руководителя)
« » _____ 202__ г.

Задание на курсовую работу получил


Подпись обучающегося

Павлов Н.С.
(ФИО обучающегося)
« » _____ 202__ г.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	8
1. Структура и описание проектируемой АСУ	9
1.1. Описание участка организации, требующего автоматизации	9
1.1.1. Бизнес-процессы автоматизируемого участка	9
1.1.2. Пользователи автоматизированной системы	9
1.2. Тип автоматизированной системы управления	10
1.3. Описание проектируемой АСУП	11
1.3.1. Техническое оснащение	11
1.3.2. Программная часть	11
1.4. Жизненный цикл и нормативно-правовое обеспечение проектирования АСУП	12
1.4.1. Нормативно-правовая база и стандарты	12
1.4.2. Жизненный цикл автоматизированной системы	13
2. Функциональные и нефункциональные требования к АСУ	15
2.1. Функциональные требования	15
2.2. Нефункциональные требования	17
3. UML моделирование проектируемой АСУ	20
3.1. Определение модулей, пользователей и функций АСУ	20
3.1.1. Пользователи системы	20
3.1.2. Модуль 1: «Управление закупками, планирование и бюджетный контроль»	20
3.1.3. Модуль 2: «Складской учет, логистика и управление имуществом»	25
3.1.4. Модуль 3: «Администрирование и отчетность»	29
3.2. Построение диаграмм вариантов использования	33
3.2.1. Модуль 1: «Управление закупкам, планирование и бюджетный контроль»	33
3.2.2. Модуль 2: «Складской учет, логистика и управление имуществом»	34
3.2.3. Модуль 3: «Администрирование и отчетность»	35
3.3. Построение диаграмм последовательности	36
3.3.1. Процесс 1: Формирование и согласование заявки на инвентарь	36
3.3.2. Процесс 2: Приемка и маркировка поступившего инвентаря на складе	37
3.3.3. Процесс 3: Выдача инвентаря спортсмену и контроль возврата	38
3.4. Построение диаграммы классов	40
3.4.1. Описание диаграммы	40

3.4.1.1.	Пакет «Пользователи».....	40
3.4.1.1.1.	Пользователь (абстрактный)	40
3.4.1.1.2.	Спортсмен (наследует Пользователь).....	40
3.4.1.1.3.	Тренер (наследует Пользователь).....	41
3.4.1.1.4.	Кладовщик (наследует Пользователь).....	41
3.4.1.1.5.	МенеджерПоЗакупкам (наследует Пользователь)	41
3.4.1.1.6.	Руководитель (наследует Пользователь).....	41
3.4.1.1.7.	Бухгалтер (наследует Пользователь)	42
3.4.1.1.8.	Администратор (наследует Пользователь).....	42
3.4.1.2.	Пакет «Закупки и планирование»	42
3.4.1.2.1.	ЗаявкаНаИнвентарь	42
3.4.1.2.2.	ПозицияЗаявки	42
3.4.1.2.3.	ЗаказПоставщику	43
3.4.1.2.4.	Поставщик	43
3.4.1.3.	Пакет «Склад и учет инвентаря»	43
3.4.1.3.1.	Номенклатура.....	43
3.4.1.3.2.	ЕдиницаИнвентаря	44
3.4.1.3.3.	ВыдачаИнвентаря	44
3.4.1.3.4.	ПриходнаяНакладная.....	44
3.4.1.4.	Связи между классами (Relationships).....	45
3.4.2.	Схематичное представление.....	46
4.	Описание и доработка программно-аппаратной платформы для проектируемой АСУ	47
4.1.	Построение таблицы аппаратного обеспечения АСУ по выбранной тематике и обеспечивающих подсистем	47
4.2.	Построение таблицы программного обеспечения для выполнения всех функций, описанных в требованиях и отображенных на UML диаграммах	49
4.3.	Анализ функций АСУ и определение функции для доработки	51
4.4.	Анализ существующих языков программирования и IDE	52
4.5.	Обоснование выбора языка, IDE и библиотек	53
4.6.	Генерация программного кода	55
5.	ER-диаграмма для проектируемой АСУ	56
5.1.	Выделение и описание свойств и методов выбранных классов.....	56
5.1.1.	Класс «Пользователь» (абстрактный)	56
5.1.2.	Класс «Спортсмен» (наследует Пользователь)	56
5.1.3.	Класс «Тренер» (наследует Пользователь).....	56
5.1.4.	Класс «Кладовщик» (наследует Пользователь).....	56
5.1.5.	Класс «МенеджерПоЗакупкам» (наследует Пользователь).....	57
5.1.6.	Класс «Руководитель» (наследует Пользователь).....	57
5.1.7.	Класс «Бухгалтер» (наследует Пользователь).....	57

5.1.8.	Класс «Администратор» (наследует Пользователь)	57
5.1.9.	Класс «ЗаявкаНаИнвентарь»	57
5.1.10.	Класс «ПозицияЗаявки»	57
5.1.11.	Класс «ЗаказПоставщику».....	58
5.1.12.	Класс «Поставщик»	58
5.1.13.	Класс «Номенклатура»	58
5.1.14.	Класс «ЕдиницаИнвентаря»	59
5.1.15.	Класс «ВыдачаИнвентаря».....	59
5.1.16.	Класс «ПриходнаяНакладная»	59
5.2.	Построение ER-диаграммы на основе классовой диаграммы	60
5.2.1.	Описание таблиц и полей	60
5.2.1.1.	Таблицы группы «Пользователи и роли»	60
5.2.1.2.	Таблицы группы «Справочники»	62
5.2.1.3.	Таблицы группы «Закупки и планирование».....	63
5.2.1.4.	Таблицы группы «Складской учет»	65
5.2.1.5.	Таблицы группы «Вспомогательные»	66
5.2.2.	Связи между таблицами.....	67
5.2.3.	Соответствие свойств классов и полей ER-диаграммы	68
5.2.4.	ER-диаграмма	70
5.3.	Выбор реляционной СУБД.....	71
5.3.1.	Рассмотренные СУБД	71
5.3.2.	Критерии сравнения.....	71
5.3.3.	Преимущества и недостатки.....	72
5.3.3.1.	PostgreSQL	72
5.3.3.2.	MySQL.....	72
5.3.3.3.	Microsoft SQL Server	73
5.3.4.	Обоснование выбора СУБД.....	73
5.4.	Создание базы данных в выбранной СУБД и заполнение данными..	74
5.4.1.	Создание структуры БД	74
5.4.2.	Заполнение базы тестовыми данными	75
5.4.3.	Тестовый запрос на выборку	79
6.	Схема сети для проектируемой АСУ	80
6.1.	Расчет характеристик оборудования	80
6.1.1.	Серверное оборудование	80
6.1.1.1.	Сервер баз данных и приложений (основной)	80
6.1.1.1.1.	Исходные данные.....	80
6.1.1.1.2.	Расчёт оперативной памяти (RAM)	80
6.1.1.1.3.	Расчёт процессора (CPU).....	81
6.1.1.1.4.	Расчёт дисковой подсистемы	81
6.1.1.1.5.	Выбранная конфигурация	82
6.1.1.2.	Сервер аналитики и прогнозирования.....	82
6.1.1.2.1.	Исходные данные.....	82
6.1.1.2.2.	Расчёт оперативной памяти (RAM)	82

6.1.1.2.3.	Расчёт процессора (CPU).....	83
6.1.1.2.4.	Расчёт дисковой подсистемы	83
6.1.1.2.5.	Выбранная конфигурация	83
6.1.1.3.	Сервер резервного копирования	84
6.1.1.3.1.	Исходные данные.....	84
6.1.1.3.2.	Расчёт дискового пространства.....	84
6.1.1.3.3.	Расчёт RAM.....	84
6.1.1.3.4.	Расчёт процессора (CPU).....	85
6.1.1.3.5.	Выбранная конфигурация	85
6.1.2.	Клиентское оборудование	85
6.1.2.1.	Автоматизированные рабочие места (АРМ)	85
6.1.2.1.1.	Исходные данные.....	85
6.1.2.1.2.	Расчёт оперативной памяти (RAM)	86
6.1.2.1.3.	Расчёт процессора (CPU).....	86
6.1.2.1.4.	Расчёт дисковой подсистемы	86
6.1.2.1.5.	Выбранная конфигурация	86
6.1.2.2.	Мобильные устройства (планшеты)	87
6.1.2.2.1.	Исходные данные.....	87
6.1.2.2.2.	Расчёт оперативной памяти (RAM)	87
6.1.2.2.3.	Расчёт процессора (CPU).....	87
6.1.2.2.4.	Расчёт встроенной памяти.....	87
6.1.2.2.5.	Дополнительные требования.....	88
6.1.2.2.6.	Выбранная конфигурация	88
6.1.2.3.	Терминалы сбора данных (ТСД)	88
6.1.2.3.1.	Исходные данные.....	88
6.1.2.3.2.	Расчёт оперативной памяти (RAM)	88
6.1.2.3.3.	Расчёт процессора (CPU).....	89
6.1.2.3.4.	Расчёт встроенной памяти.....	89
6.1.2.3.5.	Требования к сканеру	89
6.1.2.3.6.	Выбранная конфигурация	89
6.1.2.4.	Принтеры этикеток.....	90
6.1.2.4.1.	Исходные данные.....	90
6.1.2.4.2.	Расчёт характеристик.....	90
6.1.2.4.3.	Выбранная конфигурация	90
6.1.3.	Сетевое оборудование.....	91
6.1.3.1.	Коммутаторы	91
6.1.3.1.1.	Исходные данные.....	91
6.1.3.1.2.	Расчёт пропускной способности	91
6.1.3.1.3.	Выбранная конфигурация	91
6.1.3.2.	Маршрутизатор / Межсетевой экран (NGFW).....	92
6.1.3.2.1.	Исходные данные.....	92
6.1.3.2.2.	Расчёт пропускной способности	92
6.1.3.2.3.	Расчёт RAM и CPU	92
6.1.3.2.4.	Выбранная конфигурация	92
6.1.3.3.	VPN-шлюз.....	93

6.2.	Схема компьютерной сети АСУ МТО	93
6.2.1.	Состав оборудования:	93
6.2.2.	Логическая структура сети:	94
6.2.3.	Графическое представление	95
6.3.	Описание и сравнение сред виртуализации	96
6.3.1.	Рассмотренные среды виртуализации.....	96
6.3.2.	Критерии сравнения.....	96
6.3.3.	Преимущества и недостатки.....	97
6.3.4.	Обоснование выбора среды виртуализации	98
6.4.	Проектирование виртуального стенда.....	99
6.4.1.	Общая архитектура стенда	99
6.4.2.	Состав виртуальных машин.....	99
6.4.3.	Сетевая топология.....	100
6.4.4.	Распределение программных компонентов АСУ по ВМ.....	101
6.5.	Реализация виртуального стенда	102
7.	Web-интерфейс для работы с базой данных АСУ	108
7.1.	Информационный веб-портал	108
7.1.1.	Выбор инструмента разработки	108
7.1.2.	Структура web-портала.....	108
7.1.3.	Описание разделов портала	109
7.1.3.1.	Главная страница (Неро).....	109
7.1.3.2.	О системе	109
7.1.3.2.1.	Карточки пользователей.....	110
7.1.3.2.2.	Основные функции системы	110
7.1.3.2.3.	Преимущества автоматизации	110
7.1.3.3.	Результаты работы АСУ	110
7.1.3.3.1.	Табы для переключения между блоками результатов....	110
7.1.3.3.2.	Слайдер отзывов	111
7.1.3.4.	Контакты	111
7.1.4.	Исходный код	111
7.1.5.	Результат разработанного портала.....	112
7.2.	Веб-портал для работы с БД.....	116
7.2.1.	Обоснование выбора технологий	116
7.2.2.	Архитектура модуля и структура проекта	117
7.2.3.	Описание функциональности	118
7.2.3.1.	Дашборд.....	118
7.2.3.2.	Спортсмены	119
7.2.3.3.	Выдачи	119
7.2.3.4.	Склад	120
7.2.3.5.	Статистика	120
7.2.4.	Исходный код	121
7.2.5.	Результаты работы модуля	121

8. Средства защиты информации для проектируемой АСУ	125
8.1. Угрозы безопасности	125
8.1.1. Угрозы безопасности хранения и обработки информации на рабочих станциях.....	125
8.1.2. Угрозы безопасности передачи информации по компьютерной сети	126
8.2. Защита информации, хранимой и обрабатываемой на рабочих станциях	128
8.2.1. Анализ защищаемой информации и платформы рабочих станций	128
8.2.2. Обзор и сравнение программных средств защиты	129
8.2.2.1. Антивирусная защита	129
8.2.2.2. Защита от несанкционированного доступа (СЗИ от НСД) .	129
8.2.2.3. Управление мобильными устройствами (MDM)	130
8.2.2.4. Предотвращение утечек данных (DLP)	130
8.2.3. Выбранный комплекс программных средств защиты.....	131
8.3. Обеспечение безопасности передачи информации по компьютерной сети	132
8.3.1. Анализ защищаемых каналов передачи данных	132
8.3.2. Обзор и сравнение средств защиты сетевой передачи.....	133
8.3.2.1. Межсетевые экраны нового поколения (NGFW).....	133
8.3.2.2. Системы обнаружения и предотвращения вторжений (IDS/IPS)	134
8.3.2.3. VPN-шлюзы	134
8.3.2.4. Средства защиты Wi-Fi сетей	135
8.3.2.5. Средства шифрования каналов и мониторинга трафика	135
8.3.3. Выбранный комплекс средств защиты сети	136
Заключение	139
Список источников	142
Приложения	144
Приложение 1. Программный код модуля для доработки	145
Приложение 2. SQL скрипты создания и заполнения БД	153
Приложение 3. Исходный код информационного веб-портала	163
Приложение 4. Исходный код портала для работы с БД	183

ВВЕДЕНИЕ

В современных условиях эффективность спортивных организаций, школ и клубов во многом зависит от своевременного обеспечения качественным инвентарем. Лыжный спорт предъявляет высокие требования к снаряжению: лыжи, ботинки, крепления, смазки и палки должны не только соответствовать стандартам безопасности, но и поступать в нужном количестве к началу сезона.

Традиционные методы управления закупками (работа в электронных таблицах, хаотичная работа с поставщиками) приводят к ряду проблем: дефицит или избыток товаров на складе, срыв сроков поставки, нецелевое использование бюджета и сложность планирования. Особенно остро эти вопросы стоят в условиях импортозамещения и работы с множеством поставщиков.

Таким образом, актуальной задачей является внедрение автоматизированной системы управления предприятием (АСУП), ориентированной на учет, закупку и распределение лыжного снаряжения.

Целью данной курсовой работы является выполнение проектирования элементов автоматизированной системы управления для обеспечения инвентарем лыжного спорта.

1. Структура и описание проектируемой АСУ

1.1. Описание участка организации, требующего автоматизации

Объектом автоматизации является отдел снабжения и логистики спортивной организации (например, лыжной федерации, спортивной школы или проката), занимающийся обеспечением спортсменов и воспитанников необходимым инвентарем.

1.1.1. Бизнес-процессы автоматизируемого участка

Ключевым бизнес-процессом является управление закупками. Он включает сбор заявок от тренеров, формирование сводной ведомости потребностей, поиск поставщиков и создание заказов. Система автоматически контролирует соответствие заявок выделенному бюджету.

Вторым важным процессом является складской учет и логистика. При поступлении снаряжения система фиксирует его на складе, присваивает уникальные номера (или штрих-коды) и отслеживает дальнейшее движение: передача спортсмену, передача на тестирование, списание изношенного инвентаря.

1.1.2. Пользователи автоматизированной системы

Эксплуатация системы предполагает участие нескольких категорий пользователей:

- **Менеджер по закупкам.** Формирует заказы поставщикам, отслеживает статусы поставок, контролирует дебиторскую задолженность и исполнение контрактов.
- **Кладовщик / Администратор склада.** Отвечает за приемку товара, размещение на складе и отгрузку (выдачу) снаряжения спортсменам, используя терминалы сбора данных.

- **Тренер.** Выступает в роли инициатора заявок. Он видит, какой инвентарь есть в наличии у школы, и подает заявки на обеспечение своей группы (или конкретного спортсмена) к предстоящим сборам или соревнованиям.
- **Спортсмен (или его представитель).** Через личный кабинет может видеть историю полученного снаряжения, сроки возврата и наличие задолженностей.
- **Руководитель / Бухгалтер.** Контролирует бюджет закупок, анализирует отчеты по расходованию средств и движению материальных ценностей.
- **Администратор.** Осуществляет обслуживание и настройку системы, поддержание ее в работоспособном состоянии, восстановление после сбоев, управление пользователями.

1.2. Тип автоматизированной системы управления

Для решения поставленных задач предлагается разработка автоматизированной системы управления предприятием (АСУП) класса ERP (Enterprise Resource Planning) или специализированной товароучетной системы с модулями закупок и складского учета.

Основные характеристики системы:

- Учетная система (ведение номенклатуры, контрагентов, документооборот);
- Система поддержки принятия решений (аналитика закупок);
- Система с элементами электронного документооборота.

Основными функциями АСУП являются:

- ведение справочников (спортсмены, снаряжение, поставщики);
- регистрация заявок на снаряжение;
- учет движения товаров (приход, расход, внутреннее перемещение, возврат);

- автоматическое формирование отчетов для бухгалтерии;
- контроль исполнения закупок и поставок.

Система должна обеспечивать работу в режиме 24/7 с разграничением прав доступа для каждой роли.

1.3. Описание проектируемой АСУП

Проектируемая система включает следующие компоненты:

1.3.1. Техническое оснащение

- Серверное оборудование (для хранения базы данных и приложения).
- Автоматизированные рабочие места (АРМ) специалистов (менеджер, кладовщик, тренер) с доступом к системе через веб-интерфейс или толстый клиент.
- Оборудование для маркировки: принтеры этикеток, сканеры штрих-кодов (для быстрой идентификации единиц снаряжения при выдаче и возврате).

1.3.2. Программная часть

Программное обеспечение системы состоит из:

- модуля управления закупками;
- модуля складского учета;
- модуля администрирования и отчетности.

Режимы работы:

- Плановый режим: автоматическое формирование заказов поставщикам при достижении минимального остатка (нормы запаса);
- Ручной режим: создание разовых закупок под конкретное мероприятие;

- Режим администрирования: настройка прав пользователей, ведение справочников.

Алгоритмы управления включают:

- автоматический расчет дефицита снаряжения перед началом сезона;
- блокировка выдачи инвентаря спортсмену, имеющему задолженность по возврату;
- контроль сроков эксплуатации (автоматическое списание инвентаря, бывшего в использовании более нормативного срока).

1.4. Жизненный цикл и нормативно-правовое обеспечение проектирования АСУП

1.4.1. Нормативно-правовая база и стандарты

При разработке системы учитываются положения следующих нормативных актов и стандартов:

1. ГОСТ 34.601-90 «Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания». Данный стандарт определяет этапность работ и состав документации.
2. ГОСТ Р ИСО/МЭК 12207 (в части процессной модели жизненного цикла программных средств).
3. Федеральный закон № 329-ФЗ «О физической культуре и спорте в Российской Федерации» (в части требований к материально-технической базе спортивных школ и федераций).
4. Локальные акты организации (Приказы о проведении инвентаризации, нормы списания спортивного инвентаря).

1.4.2. Жизненный цикл автоматизированной системы

Проектирование АСУП для отдела снабжения лыжным инвентарем включает следующие стадии согласно ГОСТ 34.601-90:

1. Формирование требований к АСУ (Предпроектное обследование):
 - Анализ текущих бизнес-процессов (ручной сбор заявок, работа с таблицами Excel).
 - Выявление проблемных мест (дефицит размеров, потеря инвентаря, задержки поставок).
 - Определение целей автоматизации (прозрачность остатков, ускорение закупок).
2. Разработка концепции АСУ (Эскизный проект):
 - Выбор архитектуры системы (клиент-серверная или веб-ориентированная).
 - Разработка прототипа интерфейса АРМ кладовщика и менеджера.
 - Обоснование выбора типа системы: модульная товароучетная система с элементами планирования ресурсов (ERP).
3. Техническое проектирование:
 - Проектирование логической структуры базы данных (сущности: «Спортсмен», «Снаряжение», «Поставщик», «Заявка»).
 - Разработка алгоритмов управления запасами (расчет точки заказа, автопланирование списания).
 - Создание схемы информационных потоков между тренером, складом и бухгалтерией.
4. Разработка и тестирование (Ввод в действие):
 - Написание программного кода модулей «Складской учет» и «Управление закупками».
 - Настройка печатных форм (Акт приема-передачи снаряжения спортсмену, Заявка на закупку).

- Интеграция с оборудованием штрих-кодирования (сканеры и принтеры этикеток).
 - Предварительные испытания на тестовых данных (моделирование зимнего сезона закупок).
5. Сопровождение АСУ (Эксплуатация и модернизация):
- Обучение конечных пользователей (менеджера, тренеров и кладовщика) работе с интерфейсом.
 - Опытная эксплуатация системы в течение одного лыжного сезона (ноябрь – март).
 - Анализ эффективности (сравнение процента неликвидных остатков на складе до и после внедрения).
 - Актуализация справочников номенклатуры в связи с появлением новых моделей лыж и смазок.

2. Функциональные и нефункциональные требования к АСУ

2.1. Функциональные требования

Таблица 2.1 – Функциональные требования

№	Требование	Обоснование	Источник информации / аналог
1	Ведение единого справочника номенклатуры. Система должна позволять создавать и редактировать карточки товаров (лыжи, ботинки, крепления и т.д.) с указанием характеристик: размер, ростовка, тип, производитель, нормативный срок службы.	Необходимо для однозначной идентификации инвентаря и автоматизации учета.	1С:ERP Управление холдингом, SAP ERP
2	Ведение справочника контрагентов. Система должна хранить полную информацию о поставщиках (название, ИНН, контакты, договоры) и спортсменах/тренерах.	Обеспечивает прозрачность взаиморасчетов и истории взаимодействия.	1С:Управление торговлей, CRM системы
3	Регистрация заявок на снаряжение. Тренер должен иметь возможность создавать электронную заявку на инвентарь для своей группы с указанием необходимого количества и желаемых сроков поставки.	Автоматизация процесса сбора потребностей, исключение бумажных заявок.	Системы класса SRM
4	Автоматический расчет потребности в закупках. Система должна анализировать поступившие заявки от тренеров, сравнивать с текущими остатками на складе и формировать план закупок.	Исключение дефицита или избытка товаров, оптимизация бюджета.	Системы класса Forecast Management
5	Бюджетный контроль заявок и заказов. Система должна проверять соответствие суммы заявки/заказа остаткам лимитов бюджета, выделенного на закупку, и блокировать превышение.	Предотвращение нецелевого использования средств, усиление финансовой дисциплины.	1С:ERP
6	Формирование заказов поставщикам. На основе утвержденных планов закупок менеджер должен иметь возможность создавать заказы поставщикам, печатать их и отправлять контрагентам.	Централизованное управление закупками, контроль обязательств.	1С:Управление холдингом
7	Контроль статусов поставок. Система должна позволять отслеживать статус заказа (отправлен, в пути, доставлен) и уведомлять кладовщика о поступлении ожидаемого товара.	Повышение прозрачности логистики, своевременная приемка товаров.	Логистические модули ERP

8	Приемка товаров на склад. Кладовщик должен иметь возможность создавать документ "Приходная накладная" на основе заказа поставщику, с возможностью выборочной приемки по количеству и качеству.	Автоматизация складского учета, фиксация фактического поступления.	1С:Управление торговлей
9	Маркировка единиц инвентаря. Система должна генерировать и печатать штрих-коды для каждой поступающей единицы инвентаря (или комплекта) для последующей идентификации.	Обеспечение прослеживаемости движения каждой единицы снаряжения.	EAN-13, Code-128, системы "Штрих-М"
10	Учет движения товаров. Система должна фиксировать все операции: поступление, перемещение между складами, выдачу спортсмену, возврат от спортсмена, списание.	Полный контроль местонахождения и состояния каждой единицы инвентаря.	1С:Бухгалтерия, складские модули
11	Выдача инвентаря спортсменам. Кладовщик, используя сканер штрих-кода, должен быстро оформить выдачу инвентаря конкретному спортсмену, зафиксировав дату выдачи и планируемую дату возврата.	Ускорение процесса выдачи, исключение ошибок, привязка инвентаря к ответственному лицу.	Библиотечные системы, системы проката
12	Контроль возврата инвентаря. Система должна автоматически отслеживать просрочки возврата и уведомлять об этом тренера, спортсмена и кладовщика.	Снижение потерь инвентаря, повышение дисциплины.	Системы управления прокатом
13	Блокировка выдачи должникам. Система не должна позволять выдавать новый инвентарь спортсмену, за которым числится невозвращенное или просроченное снаряжение.	Обеспечение сохранности имущества.	Библиотечные системы
14	Автоматическое списание инвентаря. По истечении нормативного срока службы или при фиксации негодности, система должна позволять создавать акт на списание и перемещать инвентарь в соответствующую категорию.	Поддержание актуальности данных о пригодном к использованию имуществе.	ЕАМ системы (управление основными фондами)
15	Инвентаризация склада. Система должна поддерживать проведение инвентаризации: формирование инвентаризационной описи, ввод фактических остатков, формирование акта расхождений.	Контроль сохранности имущества, выявление хищений или ошибок учета.	1С:Бухгалтерия
16	Формирование отчетов для руководителя. Система должна предоставлять руководителю отчеты: оборотно-сальдовая ведомость по складу, отчет по закупкам (план/факт), анализ дефицита/избытка, статистика использования инвентаря.	Обеспечение руководителя достоверной информацией для принятия управленческих решений.	ВІ-системы, встроенные отчеты ERP

17	Формирование отчетов для бухгалтерии. Система должна формировать отчеты по движению материальных ценностей для передачи в бухгалтерскую программу (например, 1С:Бухгалтерия) или выгрузки в формате MS Excel.	Снижение трудозатрат бухгалтера на ручной ввод данных, исключение ошибок.	Любая товароучетная система
18	Ведение истории единицы инвентаря. Система должна хранить полную историю "жизни" каждой единицы: дата поступления, кому выдавалась, когда возвращена, когда списана.	Обеспечение прозрачности, анализ интенсивности использования инвентаря.	Системы управления активами, прослеживаемость в ERP
19	Наличие личного кабинета спортсмена/тренера. Спортсмен (тренер) через веб-интерфейс должен видеть, какой инвентарь за ним закреплен, сроки возврата, подавать заявки на новый инвентарь.	Повышение вовлеченности пользователей, снижение нагрузки на менеджера.	Порталы самообслуживания в CRM и ERP
20	Управление взаимоотношениями с поставщиками. Система должна хранить историю закупок у каждого поставщика (цены, сроки, качество), что позволит анализировать их надежность.	Оптимизация выбора поставщиков, повышение эффективности закупок.	Системы класса SRM

2.2. Нефункциональные требования

Таблица 2.2 – Нефункциональные требования

№	Требование	Обоснование	Источник информации / аналог
1	Многопользовательский доступ. Система должна обеспечивать одновременную работу не менее 50 пользователей (менеджеры, кладовщики, тренеры) без потери производительности.	Организация имеет несколько отделов и удаленных пользователей (тренеры на сборах).	Технические требования к серверным решениям (напр., 1С)
2	Разграничение прав доступа (ролевая модель). Система должна строго разграничивать доступ к функциям и данным в соответствии с ролями пользователей (менеджер, кладовщик, тренер, руководитель, бухгалтер).	Обеспечение информационной безопасности, защита от случайных или намеренных ошибок, конфиденциальность данных.	Стандарты безопасности (ISO 27001), реализовано в большинстве ERP/CRM
3	Круглосуточная доступность (24/7). Система должна быть доступна пользователям в любое время, особенно в период активной подготовки к соревнованиям и в сезон выдачи инвентаря.	Обеспечение непрерывности бизнес-процессов, возможность работы в любое удобное время.	Требования к веб-сервисам и корпоративным системам (SLA 99.9%)

4	Время отклика интерфейса. Большинство операций (открытие формы, сохранение документа) должны выполняться не более чем за 2 секунды при средней нагрузке.	Обеспечение комфортной работы пользователей, повышение производительности труда.	Требования к юзабилити ПО, стандарты производительности
5	Интуитивно понятный интерфейс (Usability). Интерфейс должен быть простым и понятным для пользователей с разным уровнем компьютерной грамотности (тренеры, спортсмены).	Снижение затрат на обучение персонала, минимизация ошибок при вводе данных.	Принципы проектирования UI/UX (материалы NN/g, ГОСТ Р ИСО 9241)
6	Интеграция с бухгалтерскими системами. Должна быть обеспечена возможность двухстороннего обмена данными с внешними бухгалтерскими программами (например, 1С:Бухгалтерия, SAP).	Исключение двойного ввода данных, обеспечение корректности бухгалтерского и налогового учета.	Требования к корпоративным информационным системам
7	Резервное копирование. Система должна выполнять автоматическое резервное копирование базы данных по расписанию (ежедневно) с возможностью быстрого восстановления.	Защита от потери данных в случае сбоев оборудования или ошибок ПО.	Стандарты администрирования БД (напр., для PostgreSQL, MS SQL)
8	Устойчивость к сбоям и отказоустойчивость. Критические компоненты системы (сервер БД) должны иметь резервирование для минимизации простоев.	Обеспечение непрерывности работы в "горячий" сезон.	Требования к архитектуре ПО (standby-серверы, кластеризация)
9	Масштабируемость. Возможность наращивания производительности системы (путем добавления ресурсов сервера) при увеличении количества пользователей или объема данных без кардинальной переработки кода.	Обеспечение развития организации и роста потребностей в автоматизации.	Требования к современным веб-приложениям и ERP
10	Защита данных от несанкционированного доступа. Вся конфиденциальная информация (персональные данные спортсменов, коммерческая информация о поставщиках) должна храниться и передаваться в зашифрованном виде.	Соответствие законодательству (152-ФЗ "О персональных данных"), защита коммерческой тайны.	Стандарты безопасности (PCI DSS, ГОСТ 28147-89)
11	Аудит и логирование действий пользователей. Система должна фиксировать все значимые действия пользователей (создание/удаление документов, изменение справочников) с указанием даты, времени и имени пользователя.	Возможность расследования инцидентов, определение ответственного за ошибки.	Требования к финансовым и государственным ИС
12	Кроссплатформенность веб-клиента. Веб-интерфейс системы должен корректно работать во всех современных	Обеспечение доступа для пользователей с любым программным	Стандарты веб-разработки (W3C)

	браузерах (Chrome, Firefox, Safari, Edge) на разных устройствах (ПК, планшеты).	обеспечением, включая мобильные устройства.	
13	Возможность работы с торговым оборудованием. Система должна поддерживать интеграцию со сканерами штрих-кода и принтерами этикеток популярных производителей.	Необходимо для быстрой и безошибочной работы кладовщика на складе.	Драйверы и SDK производителей оборудования
14	Адаптивность интерфейса для мобильных устройств. Веб-интерфейс для тренеров и спортсменов должен быть удобен для использования на экранах смартфонов.	Тренеры и спортсмены часто находятся вне офиса и используют мобильные устройства.	Принципы Mobile First, responsive design
15	Надежность и стабильность работы. Система должна работать без сбоев (некритических ошибок, зависаний) в течение длительного времени (uptime не менее 99.5% в рабочее время).	Обеспечение доверия пользователей к системе и исключение простоев в работе.	Требования к промышленному ПО
16	Модульность архитектуры. Система должна состоять из слабосвязанных модулей (закупки, склад, отчеты), что позволит модернизировать или заменять их по отдельности.	Упрощение разработки, тестирования и дальнейшего сопровождения системы.	Принципы проектирования ПО (микросервисы, SOA)
17	Сопровождение и поддержка. Должна быть предусмотрена возможность обновления системы (исправления ошибок, добавление нового функционала) без длительного простоя.	Адаптация к меняющимся потребностям бизнеса и законодательства.	Методологии DevOps, CI/CD
18	Производительность при формировании отчетов. Сложные аналитические отчеты за длительный период (например, за сезон) должны формироваться не более 1-2 минут.	Руководитель не должен тратить много времени на ожидание данных для принятия решения.	Требования к BI-системам
19	Импортонезависимость. Желательно использование СУБД и программного обеспечения, входящих в Единый реестр российских программ для ЭВМ и БД.	Соответствие политике импортозамещения, снижение рисков, связанных с санкциями.	Реестр отечественного ПО, Постановление Правительства РФ
20	Локализация. Весь интерфейс и отчетность должны быть на русском языке, с поддержкой российских форматов дат, чисел и документооборота.	Соответствие российским стандартам делопроизводства, удобство для пользователей.	ГОСТ Р 6.30-2003 (требования к оформлению документов)

3. UML моделирование проектируемой АСУ

3.1. Определение модулей, пользователей и функций АСУ

3.1.1. Пользователи системы

В системе выделено 7 основных ролей пользователей:

1. Спортсмен
2. Тренер
3. Кладовщик / Администратор склада
4. Менеджер по закупкам
5. Руководитель
6. Бухгалтер
7. Администратор

3.1.2. Модуль 1: «Управление закупками, планирование и бюджетный контроль»

Назначение модуля

Автоматизация процессов планирования потребностей в инвентаре, бюджетирования, формирования и контроля заказов поставщикам, а также аналитика эффективности закупок.

Возможности модуля

- Сбор и агрегация заявок от тренеров с возможностью прикрепления обосновывающих документов.
- Автоматический расчет потребности в закупке на основе заявок, остатков и сезонных коэффициентов.
- Многокомпонентный бюджетный контроль (проверка по статьям расходов, источникам финансирования).
- Формирование и согласование заказов поставщикам.

- Ведение справочника поставщиков и истории взаимоотношений с ними (SRM).
- Мониторинг исполнения контрактов и сроков поставки.
- Прогнозирование потребностей на основе исторических данных.

Таблица 3.1 – Интерфейсные функции модуля 1

№	Интерфейсная функция	Спортсмен	Тренер	Менеджер по закупкам	Руководитель	Бухгалтер
1.1	Просмотр лимитов бюджета на период			+	+	+
1.2	Создание заявки на инвентарь	+	+			
1.3	Редактирование своей заявки	+	+			
1.4	Просмотр статуса своих заявок	+	+	+		
1.5	Утверждение (согласование) заявок			+	+	
1.6	Отклонение заявки с комментарием			+	+	
1.7	Формирование плана закупок на период			+	+	
1.8	Корректировка плана закупок			+	+	
1.9	Создание заказа поставщику			+		
1.10	Редактирование заказа поставщику			+		
1.11	Просмотр реестра заказов поставщикам			+	+	+
1.12	Изменение статуса заказа (отправлен, в пути)			+		
1.13	Анализ надежности поставщиков			+	+	
1.14	Сравнение цен поставщиков			+	+	
1.15	Просмотр истории закупок у поставщиков			+	+	+

Таблица 3.2 – Соответствие интерфейсных и внутрисистемных функций модуля 1

Интерфейсная функция	Внутрисистемная функция
1.1 Просмотр лимитов бюджета на период	1.IS1 Запрос к БД для получения данных о бюджетных лимитах за указанный период
	1.IS2 Расчет доступного остатка бюджета с учетом утвержденных заявок и заказов
1.2 Создание заявки на инвентарь	1.IS3 Сохранение заявки в БД со статусом "Черновик"
	1.IS4 Проверка корректности введенных данных (размеры, количество, обязательные поля)
	1.IS5 Автоматическое определение доступного бюджета для данной заявки
	1.IS6 Проверка прав доступа на редактирование (только свой черновик)
1.3 Редактирование своей заявки	1.IS7 Обновление данных заявки с сохранением истории изменений
	1.IS8 Пересчет стоимости заявки при изменении количества или номенклатуры
	1.IS9 Запрос к БД для получения списка заявок пользователя с текущими статусами

1.5 Утверждение (согласование) заявок	1.IS10 Отображение текущего этапа согласования и ответственного лица
	1.IS11 Проверка полномочий пользователя на утверждение заявок
	1.IS12 Проверка соответствия суммы заявки остаткам бюджетных лимитов
	1.IS13 Изменение статуса заявки на "Утверждена"
	1.IS14 Блокировка средств бюджета под утвержденную заявку
	1.IS15 Отправка уведомления автору заявки о решении
1.6 Отклонение заявки с комментарием	1.IS16 Изменение статуса заявки на "Отклонена"
	1.IS17 Сохранение комментария причины отклонения в БД
	1.IS18 Отправка уведомления автору заявки с указанием причины
1.7 Формирование плана закупок на период	1.IS19 Запуск алгоритма расчета потребности (анализ утвержденных заявок)
	1.IS20 Анализ текущих остатков на складе по каждой позиции
	1.IS21 Учет минимальных страховых запасов при расчете потребности
	1.IS22 Генерация проекта плана закупок с группировкой по номенклатуре
	1.IS23 Группировка потребности по предпочтительным поставщикам
1.8 Корректировка плана закупок	1.IS24 Сохранение изменений в проекте плана закупок
	1.IS25 Пересчет итоговой суммы плана закупок
	1.IS26 Логирование факта корректировки с указанием автора
1.9 Создание заказа поставщику	1.IS27 Проверка наличия действующего договора с поставщиком
	1.IS28 Сохранение заказа в БД с присвоением уникального номера
	1.IS29 Резервирование средств бюджета под созданный заказ
	1.IS30 Изменение статуса включенных в заказ позиций плана на "Заказано"
1.10 Редактирование заказа поставщику	1.IS31 Проверка возможности редактирования (статус "Черновик" или "Отправлен")
	1.IS32 Обновление данных заказа в БД
	1.IS33 Пересчет общей суммы заказа и корректировка резервирования бюджета
1.11 Просмотр реестра заказов поставщикам	1.IS34 Запрос к БД для получения списка заказов с фильтрацией по периоду/поставщику
	1.IS35 Расчет общей суммы заказов за период
1.12 Изменение статуса заказа	1.IS36 Обновление статуса заказа в БД (сформирован, отправлен, в пути, доставлен)
	1.IS37 Логирование изменения статуса с указанием даты и времени
	1.IS38 Отправка уведомления кладовщику при статусе "Доставлен"
1.13 Анализ надежности поставщиков	1.IS39 Сбор статистики по поставщику: количество выполненных заказов
	1.IS40 Расчет среднего срока поставки по каждому поставщику
	1.IS41 Анализ количества рекламаций и возвратов по поставщику

	1.IS42 Расчет рейтинга поставщика на основе истории поставок
1.14 Сравнение цен поставщиков	1.IS43 Выборка цен на аналогичные товары от разных поставщиков
	1.IS44 Анализ динамики цен за период
	1.IS45 Формирование сравнительной таблицы с учетом доставки и условий
1.15 Просмотр истории закупок у поставщиков	1.IS46 Запрос к БД всех заказов указанного поставщика за период
	1.IS47 Формирование статистики по номенклатуре и суммам

3.1.3. Модуль 2: «Складской учет, логистика и управление имуществом»

Назначение модуля

Автоматизация операций по приемке, хранению, маркировке, выдаче, возврату, списанию и техническому обслуживанию лыжного инвентаря. Полная прослеживаемость жизненного цикла каждой единицы.

Возможности модуля

- Приемка товаров по заказам с контролем качества.
- Генерация и печать штрих-кодов / QR-кодов для маркировки единиц инвентаря.
- Учет движения товаров (приход, расход, перемещение между складами/локациями).
- Выдача инвентаря спортсменам и контроль возврата с напоминаниями.
- Учет технического обслуживания (ремонт, подготовка смазки).
- Проведение инвентаризации и анализ расхождений.

Таблица 3.3 – Интерфейсные функции модуля 2

№	Интерфейсная функция	Спортсмен	Тренер	Кладовщик	Менеджер по закупкам	Руководитель	Бухгалтер
2.1	Просмотр текущих остатков на складе		+	+	+	+	+
2.2	Просмотр остатков по размерам/ростовкам		+	+	+	+	+
2.3	Приемка товара по накладной			+			
2.4	Частичная приемка товара (расхождения)			+			
2.5	Печать штрих-кодов для инвентаря			+			
2.6	Выдача инвентаря спортсмену			+			
2.7	Возврат инвентаря от спортсмена			+			
2.8	Просмотр истории выдач/возвратов	+	+	+		+	+
2.9	Просмотр текущей задолженности	+	+	+			
2.10	Создание акта на списание инвентаря			+			
2.11	Утверждение акта на списание					+	
2.12	Проведение инвентаризации (ввод факт. остатков)			+			+
2.13	Формирование инвентаризационной описи			+			+
2.14	Просмотр карточки единицы инвентаря			+	+		
2.15	Перемещение товаров между складами			+			
2.16	Резервирование инвентаря под мероприятие		+	+			
2.17	Оформление ТО инвентаря			+			
2.18	Контроль качества (фиксация брака)		+	+			

Таблица 3.4 – Соответствие интерфейсных и внутрисистемных функций модуля 2

Интерфейсная функция	Внутрисистемная функция
2.1 Просмотр текущих остатков на складе	2.IS1 Запрос к БД для получения актуальных количественных остатков по всем позициям
	2.IS2 Расчет доступного остатка (с учетом резервов)
2.2 Просмотр остатков по размерам/ростовкам	2.IS3 Запрос к БД для получения остатков с детализацией по характеристикам товара
	2.IS4 Группировка результатов по размерной сетке

2.3 Приемка товара по накладной	2.IS5 Поиск заказа в БД по номеру накладной или заказа
	2.IS6 Сверка поступающих позиций с заказом
	2.IS7 Создание документа "Приходная накладная"
	2.IS8 Увеличение количественных остатков на складе
	2.IS9 Изменение статуса заказа на "Исполнен"
2.4 Частичная приемка товара (расхождения)	2.IS10 Фиксация расхождений по количеству в документе приемки
	2.IS11 Создание документа "Акт расхождений"
	2.IS12 Запуск процесса претензионной работы с поставщиком
	2.IS13 Оприходование только фактически принятого количества
2.5 Печать штрих-кодов для инвентаря	2.IS14 Генерация уникальных идентификаторов (штрих-кодов) для каждой единицы товара
	2.IS15 Проверка уникальности сгенерированных кодов
	2.IS16 Формирование макета для печати с читаемым штрих-кодом и текстом
2.6 Выдача инвентаря спортсмену	2.IS17 Проверка спортсмена на наличие задолженностей
	2.IS18 Поиск единиц инвентаря по штрих-коду
	2.IS19 Создание документа "Выдача"
	2.IS20 Привязка единиц инвентаря к спортсмену в БД
	2.IS21 Уменьшение свободного остатка на складе
	2.IS22 Фиксация даты планируемого возврата
2.7 Возврат инвентаря от спортсмена	2.IS23 Поиск выдачи по штрих-коду возвращаемого инвентаря
	2.IS24 Создание документа "Возврат"
	2.IS25 Проверка состояния возвращаемого инвентаря (фиксация дефектов)
	2.IS26 Отвязка инвентаря от спортсмена
	2.IS27 Увеличение свободного остатка на складе
2.8 Просмотр истории выдач/возвратов	2.IS28 Запрос к БД всех операций по конкретному спортсмену или единице инвентаря
	2.IS29 Формирование хронологической ленты событий
2.9 Просмотр текущей задолженности	2.IS30 Запрос к БД списка спортсменов с просроченным инвентарем
	2.IS31 Расчет количества дней просрочки по каждой позиции
2.10 Создание акта на списание инвентаря	2.IS32 Поиск единиц инвентаря по критериям (истек срок, непригодность)
	2.IS33 Формирование документа "Акт на списание"
	2.IS34 Изменение статуса единиц инвентаря на "На списании"
2.11 Утверждение акта на списание	2.IS35 Проверка полномочий на утверждение списания

	2.IS36 Изменение статуса единиц инвентаря в БД на "Списано"
	2.IS37 Уменьшение стоимости активов в учетной системе
2.12 Проведение инвентаризации	2.IS38 Формирование инвентаризационной описи с текущими остатками
	2.IS39 Ввод фактических остатков по результатам пересчета
	2.IS40 Сравнение фактических остатков с учетными
	2.IS41 Формирование списка расхождений (излишки/недостачи)
	2.IS42 Выгрузка данных для печати инвентаризационной ведомости
2.13 Формирование инвентаризационной описи	2.IS43 Запрос к БД полной информации о конкретной единице по штрих-коду
2.14 Просмотр карточки единицы инвентаря	2.IS44 Отображение полной истории движения единицы
2.15 Перемещение товаров между складами	2.IS45 Создание документа "Перемещение"
	2.IS46 Уменьшение остатка на складе-отправителе
	2.IS47 Увеличение остатка на складе-получателе
2.16 Резервирование инвентаря под мероприя- тие	2.IS48 Поиск свободного инвентаря под заданные параметры
	2.IS49 Изменение статуса единиц инвентаря на "Зарезервировано"
	2.IS50 Привязка резерва к мероприятию в календаре
2.17 Оформление ремонта инвентаря	2.IS51 Изменение статуса единицы инвентаря на "В ремонте"
	2.IS52 Создание заявки на ремонт с описанием неисправности
	2.IS53 Фиксация даты начала и планируемой даты окончания ремонта
2.18 Контроль качества (фиксация брака)	2.IS54 Создание документа "Акт о браке"
	2.IS55 Изменение статуса единицы инвентаря на "Брак"
	2.IS56 Инициирование процесса возврата поставщику или списания

3.1.4. Модуль 3: «Администрирование и отчетность»

Назначение модуля

Управление доступом к системе, настройка справочников и предоставление аналитической информации руководству и бухгалтерии.

Возможности модуля

- Управление пользователями и ролями (разграничение прав доступа).
- Ведение единых справочников (номенклатура, контрагенты).
- Формирование управленческих отчетов (обороты-сальдовые ведомости, анализ закупок, статистика использования).
- Формирование отчетов для бухгалтерии (движение ТМЦ) и их выгрузка.
- Просмотр и анализ логов действий пользователей (аудит)

Таблица 3.5 – Интерфейсные функции модуля 3

№	Интерфейсная функция	Трени- нер	Кладов- щик	Менеджер по за- купкам	Руководи- тель	Бухгал- тер	Администра- тор
3.1	Управление пользователями (создание, блокировка)						+
3.2	Назначение ролей и прав доступа						+
3.3	Ведение справочника номенклатуры			+			+
3.4	Ведение справочника контрагентов			+			+
3.5	Ведение справочника складов/мест хранения		+				+
3.6	Формирование отчета по закупкам (план/факт)			+	+	+	
3.7	Формирование ОСВ по складу				+	+	
3.8	Формирование отчета по движению ТМЦ				+	+	
3.9	Формирование отчета по дебиторам/должникам	+	+		+		
3.10	Формирование отчета для бухгалтерии (выгрузка)					+	
3.11	Просмотр логов действий пользователей						+
3.12	Настройка параметров системы (общих)						+
3.13	Настройка бизнес-процессов согласования						+
3.14	Просмотр статистики использования инвентаря	+	+	+	+		
3.15	Формирование отчета по износу инвентаря		+		+	+	
3.16	Выгрузка отчетов в Excel/PDF				+	+	
3.17	Настройка резервного копирования						+

Таблица 3.6 – Соответствие интерфейсных и внутрисистемных функций модуля 3

Интерфейсная функция	Внутрисистемная функция
3.1 Управление пользователями (создание, блокировка)	3.IS1 Создание новой учетной записи пользователя в БД
	3.IS2 Блокировка/разблокировка учетной записи
	3.IS3 Редактирование данных пользователя
	3.IS4 Удаление пользователя (или пометка на удаление)
	3.IS5 Логирование всех действий с учетными записями
3.2 Назначение ролей и прав доступа	3.IS6 Установка соответствия пользователь-роль в таблице прав доступа
	3.IS7 Настройка прав доступа к конкретным функциям
	3.IS8 Проверка непротиворечивости назначенных прав
3.3 Ведение справочника номенклатуры	3.IS9 Добавление новой позиции в справочник номенклатуры
	3.IS10 Редактирование характеристик существующих позиций
	3.IS11 Удаление позиций (или пометка на удаление)
	3.IS12 Импорт номенклатуры из Excel
3.4 Ведение справочника контрагентов	3.IS13 Добавление нового контрагента (поставщика)
	3.IS14 Редактирование данных контрагента
	3.IS15 Ведение истории договоров с контрагентами
3.5 Ведение справочника складов/мест хранения	3.IS16 Добавление/редактирование информации о складах
	3.IS17 Настройка структуры мест хранения (стеллажи, ячейки)
3.6 Формирование отчета по закупкам (план/факт)	3.IS18 Выборка данных из таблиц "Заказы поставщикам" за период
	3.IS19 Сравнение плановых показателей с фактическими закупками
	3.IS20 Агрегация данных и вычисление итоговых показателей
3.7 Формирование ОСВ по складу	3.IS21 Выборка всех складских операций за период
	3.IS22 Расчет начального остатка, прихода, расхода, конечного остатка
	3.IS23 Группировка по номенклатурным группам
3.8 Формирование отчета по движению ТМЦ	3.IS24 Детальная выборка всех операций по каждой номенклатуре
	3.IS25 Формирование ведомости движения с указанием дат и документов
3.9 Формирование отчета по дебиторам/должникам	3.IS26 Выборка всех спортсменов с просроченным инвентарем
	3.IS27 Расчет суммы штрафных санкций (если применимо)
	3.IS28 Группировка по тренерам/группам
3.10 Формирование отчета для бухгалтерии	3.IS29 Формирование данных для передачи в бухгалтерскую систему

	3.IS30 Выгрузка данных в требуемом формате (Excel, 1C, XML)
3.11 Просмотр логов действий пользователей	3.IS31 Запись в лог-файл любого значимого действия пользователя
	3.IS32 Выборка и фильтрация записей лога для отображения
	3.IS33 Экспорт логов для анализа
3.12 Настройка параметров системы (общих)	3.IS34 Сохранение конфигурационных параметров системы в БД
	3.IS35 Применение настроек без перезагрузки системы
3.13 Настройка бизнес-процессов согласования	3.IS36 Настройка маршрутов согласования заявок
	3.IS37 Определение правил эскалации при нарушении сроков
	3.IS38 Сохранение параметров в конфигурационных таблицах
3.14 Просмотр статистики использования инвентаря	3.IS39 Анализ частоты выдачи различных видов инвентаря
	3.IS40 Расчет коэффициента загрузки инвентаря
	3.IS41 Формирование рейтинга наиболее востребованных позиций
3.15 Формирование отчета по износу инвентаря	3.IS42 Анализ сроков эксплуатации каждой единицы инвентаря
	3.IS43 Расчет процента износа и прогнозирование замены
	3.IS44 Формирование плана обновления инвентаря
3.16 Выгрузка отчетов в Excel/PDF	3.IS45 Преобразование данных отчета в требуемый формат
	3.IS46 Генерация файла для скачивания пользователем
3.17 Настройка резервного копирования	3.IS47 Создание задания в планировщике для автоматического бэкапа БД
	3.IS48 Настройка периодичности и места хранения резервных копий
	3.IS49 Запуск внепланового резервного копирования
	3.IS50 Проверка целостности созданных резервных копий

3.2.2. Модуль 2: «Складской учет, логистика и управление имуществом»

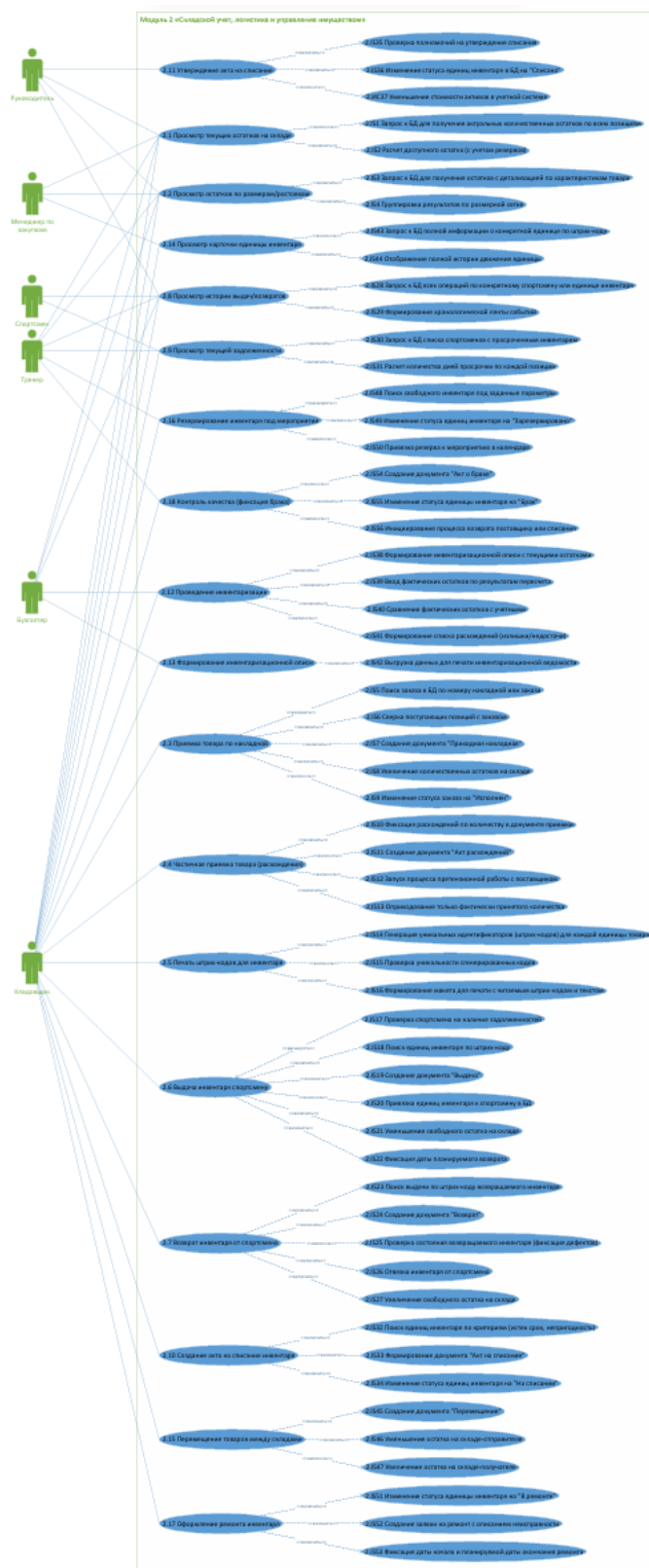


Рисунок 3.2 – Диаграмма вариантов использования модуля 2

3.2.3. Модуль 3: «Администрирование и отчетность»

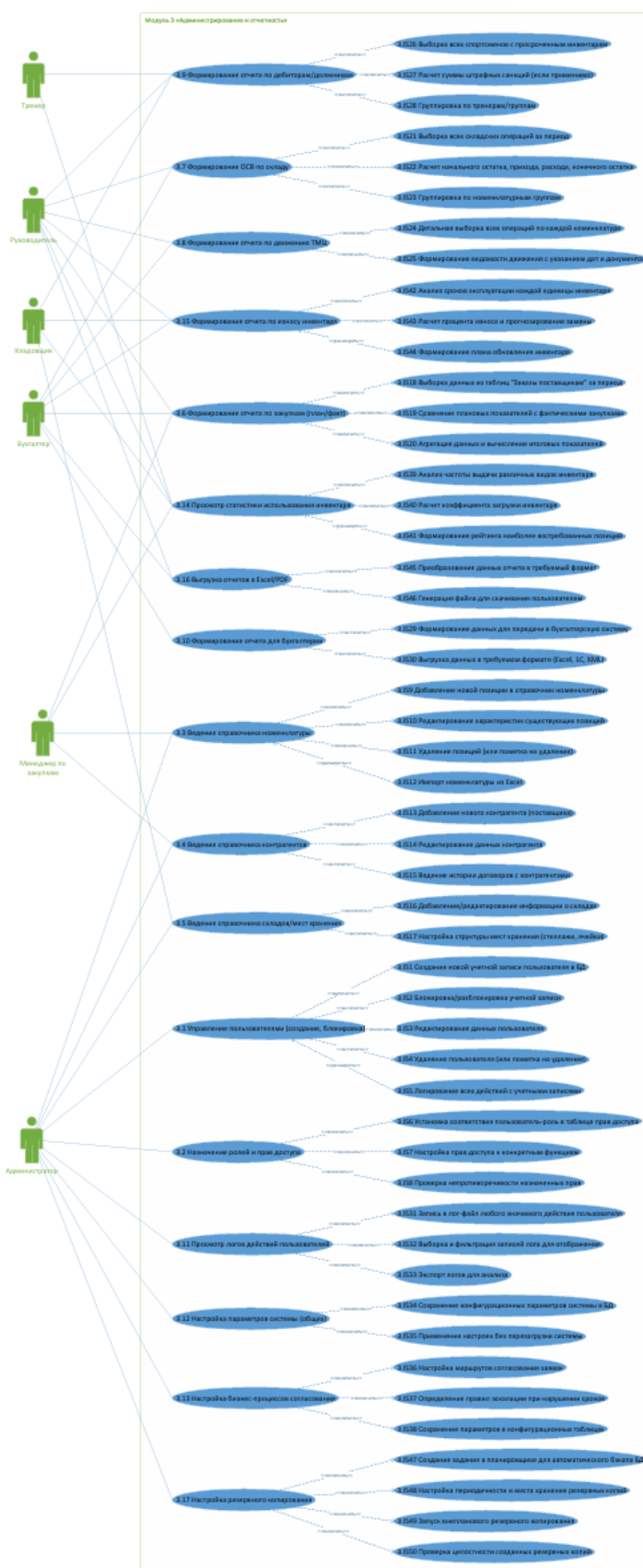


Рисунок 3.3 – Диаграмма вариантов использования модуля 3

3.3. Построение диаграмм последовательности

Для выполнения задачи выделены 3 процесса в рамках деятельности отдела снабжения и логистики спортивной организации:

- Процесс 1: Формирование и согласование заявки на инвентарь.
- Процесс 2: Приемка и маркировка поступившего инвентаря на складе.
- Процесс 3: Выдача инвентаря спортсмену и контроль возврата.

3.3.1. Процесс 1: Формирование и согласование заявки на инвентарь

Участники (Actors): Тренер, Менеджер по закупкам.

Модули АСУ: Модуль 1 (Управление закупками), Модуль 3 (Администрирование и отчетность).

Описание основного потока (success):

- Тренер заполняет форму заявки.
- Система проверяет корректность данных.
- Заявка сохраняется со статусом «На согласовании» и направляется менеджеру.
- Менеджер проверяет соответствие заявки бюджету (запрос у системы остатков бюджета и инвентаря).
- Менеджер, а затем и система подтверждают успешное согласование
- Система блокирует средства.
- Тренер получает уведомление об утверждении.

Альтернативный поток (failure):

- Если сумма заявки превышает остаток бюджета, менеджер и система отклоняют заявку
- Система уведомляет тренера с указанием причины.

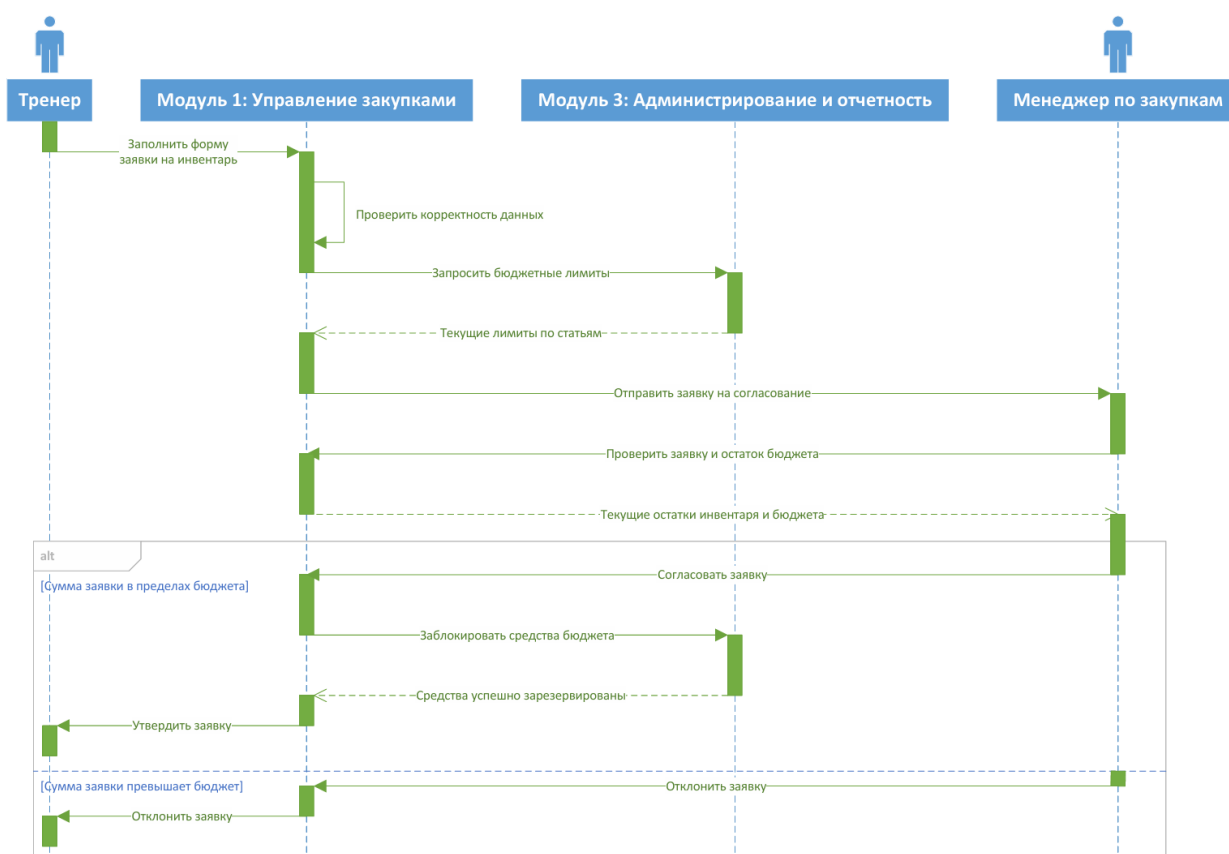


Рисунок 3.4 – Диаграмма последовательности для Процесса 1

3.3.2. Процесс 2: Приемка и маркировка поступившего инвентаря на складе

Участники (Actors): Кладовщик, Менеджер по закупкам.

Модули АСУ: Модуль 1 (Управление закупками), Модуль 2 (Складской учет).

Описание основного потока:

- Кладовщик находит заказ поставщику в системе.
- Система отображает ожидаемые позиции.
- Кладовщик подтверждает приемку (количество и качество сходятся).
- Система создает приходную накладную и увеличивает остатки.
- Система генерирует уникальные штрих-коды для каждой единицы инвентаря.

- Система уведомляет менеджера о б успешном исполнении заказа

Альтернативный поток (partial receipt):

- Если фактическое количество товара расходится с заказом, кладовщик фиксирует расхождение. Система создает акт расхождений и оприходует только фактически принятое количество.
- Система направляет уведомление менеджеру для инициализации претензионной работы с поставщиком

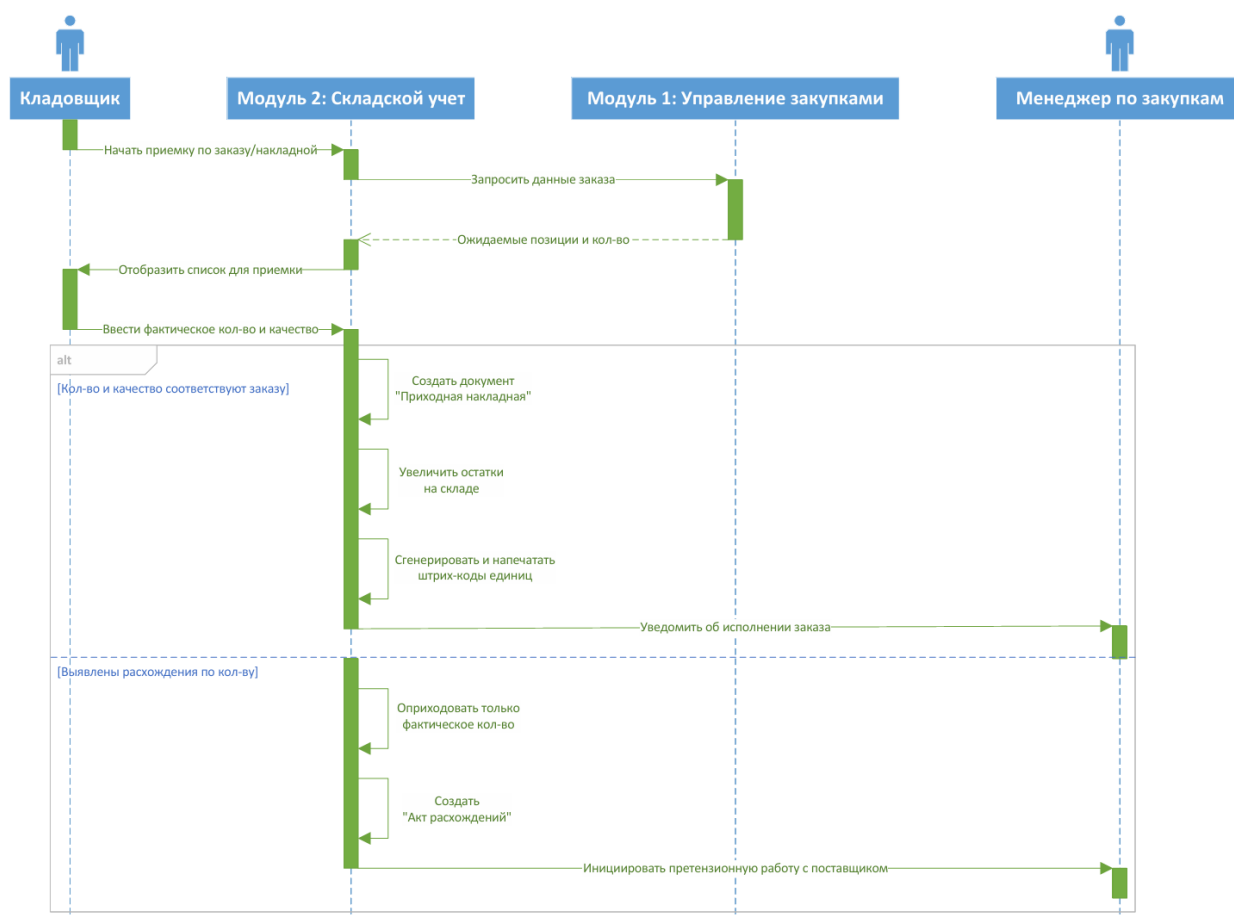


Рисунок 3.5 – Диаграмма последовательности для Процесса 2

3.3.3. Процесс 3: Выдача инвентаря спортсмену и контроль возврата

Участники (Actors): Спортсмен, Кладовщик.

Модули АСУ: Модуль 2 (Складской учет), Модуль 3 (Администрирование и отчетность).

Описание основного потока:

- Спортсмен предъявляет идентификатор (или называет ФИО).
- Кладовщик сканирует штрих-код выдаваемого инвентаря.
- Система проверяет спортсмена на наличие задолженностей.
- Если задолженностей нет, система оформляет выдачу, привязывает инвентарь к спортсмену и уменьшает остатки на складе.

Альтернативный поток (debtor block):

- Если за спортсменом числится просроченный или невозвращенный инвентарь, система блокирует выдачу и отображает предупреждение.

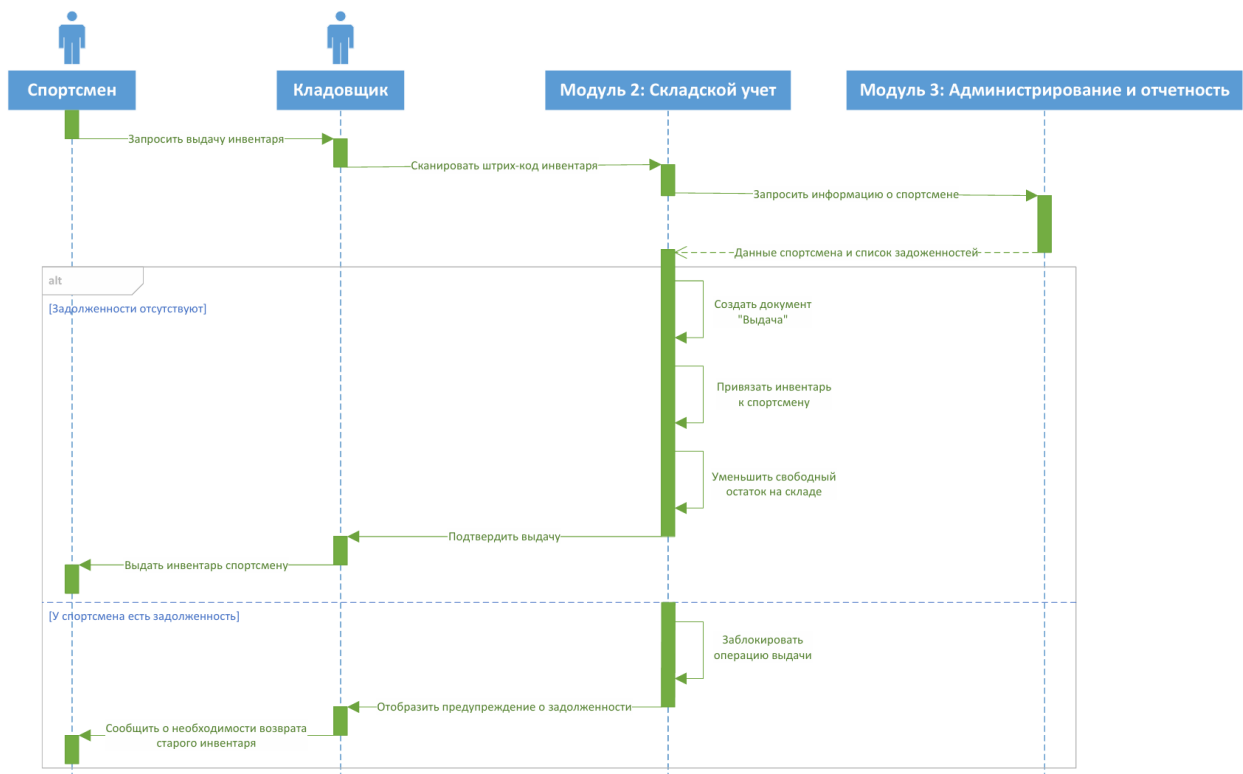


Рисунок 3.6 – Диаграмма последовательности для Процесса 3

3.4. Построение диаграммы классов

Диаграмма содержит 16 классов, сгруппированных в 3 пакета:

- Пользователи (7 классов + 1 абстрактный)
- Закупки и планирование (4 класса)
- Склад и учет инвентаря (4 класса)

3.4.1. Описание диаграммы

3.4.1.1. Пакет «Пользователи»

3.4.1.1.1. Пользователь (абстрактный)

Таблица 3.7 – Поля и методы класса "Пользователь"

Поле/метод	Тип	Описание
ФИО	string	Полное имя
Email	string	Рабочая почта
Телефон	string	Контактный номер
Логин	string	Имя для входа
ПарольHash	string	Хеш пароля
Роль	enum	Роль в системе
ДатаРегистрации	date	Дата создания учетки
Активен	bool	Статус блокировки
Авторизоваться(логин, пароль)	bool	Проверка доступа
СменитьПароль(старый, новый)	void	Обновление пароля
ВыйтиИзСистемы()	void	Завершение сессии

3.4.1.1.2. Спортсмен (наследует Пользователь)

Таблица 3.8 – Поля и методы класса "Спортсмен"

Поле/метод	Тип	Описание
СпортивныйРазряд	string	КМС, МС и т.д.
ГодРождения	int	Возрастная группа
ГруппаID	int	Связь с группой
ТренерID	int	Связь с тренером
ПросмотретьМойИнвентарь()	List<Выдача>	Актуальный инвентарь
ПодатьЗаявкуНаИнвентарь(инвентарь)	void	Создание заявки
ПросмотретьЗадолженность()	List<Выдача>	Просроченный инвентарь

3.4.1.1.3. Тренер (наследует Пользователь)

Таблица 3.9 – Поля и методы класса "Тренер"

Поле/метод	Тип	Описание
Специализация	string	Вид лыжного спорта
Категория	string	Тренерская категория
Спортсмены	List<Спортсмен>	Состав группы
СоздатьЗаявкуНаГруппу(группа)	void	Массовая заявка
УтвердитьЗаявкуСпортсмена(заявка)	void	Подтверждение
ПросмотретьДолжниковПоГруппе(группа)	List<Спортсмен>	Контроль возврата

3.4.1.1.4. Кладовщик (наследует Пользователь)

Таблица 3.10 – Поля и методы класса "Кладовщик"

Поле/метод	Тип	Описание
СкладID	int	Привязка к складу
НомерУдостоверения	string	Документ МОЛ
ПринятьТовар(накладная)	void	Оприходование
ВыдатьИнвентарь(спортсмен, штрихкод)	void	Оформление выдачи
ПринятьВозврат(штрихкод)	void	Оформление возврата
ПровестиИнвентаризацию()	void	Сверка остатков
СформироватьАктСписания(штрихкод)	void	Списание ТМЦ

3.4.1.1.5. МенеджерПоЗакупкам (наследует Пользователь)

Таблица 3.11 – Поля и методы класса "МенеджерПоЗакупкам"

Поле/метод	Тип	Описание
Отдел	string	Название подразделения
СформироватьПланЗакупок()	void	На основе заявок
СоздатьЗаказПоставщику()	void	Оформление заказа
АнализироватьРынокПоставщиков()	Report	Сравнение цен
КонтролироватьИсполнениеЗаказа(заказ)	void	Отслеживание статусов

3.4.1.1.6. Руководитель (наследует Пользователь)

Таблица 3.12 – Поля и методы класса "Руководитель"

Поле/метод	Тип	Описание
Должность	string	Директор, зам. и т.д.
УровеньДоступа	int	1-3
УтвердитьБюджетЗакупок()	void	Финансовое согласование
ПросмотретьОтчетПоЗакупкам()	Report	План/факт анализ
УтвердитьСписание()	void	Подписание актов

3.4.1.1.7. Бухгалтер (наследует Пользователь)

Таблица 3.13 – Поля и методы класса "Бухгалтер"

Поле/метод	Тип	Описание
Участок	string	Материальный стол
СформироватьОСВ()	Report	Оборотно-сальдовая ведомость
ПроверитьПриходныеДокументы()	void	Контроль первички
ВыгрузитьДанныеВ1С()	File	Экспорт для бухучета

3.4.1.1.8. Администратор (наследует Пользователь)

Таблица 3.14 – Поля и методы класса "Администратор"

Поле/метод	Тип	Описание
УровеньПрав	enum	Полный доступ
СоздатьПользователя(пользователь, роль)	void	Регистрация нового
НазначитьРоль()	void	Настройка прав
НастроитьРезервноеКопирование()	void	Администрирование БД
ПросмотретьЛогДействий()	List<Log>	Аудит системы

3.4.1.2. Пакет «Закупки и планирование»

3.4.1.2.1. ЗаявкаНаИнвентарь

Таблица 3.15 – Поля и методы класса "ЗаявкаНаИнвентарь"

Поле/метод	Тип	Описание
НомерЗаявки	string	Номер документа
ДатаСозданияЗаявки	datetime	Время создания
СтатусЗаявки	enum	Черновик/НаСогласовании/ Утверждена/Отклонена
АвторЗаявкиID	int	FK на Пользователь (Тренер/Спортсмен)
ЗаявкуУтвердилID	int	FK на Пользователь (Руководитель)
СуммаЗаявкиПлановая	decimal	Ожидаемая стоимость
КомментарийКЗаявке	string	Примечание
РассчитатьСумму()	decimal	Сумма по позициям
Утвердить(пользователь)	void	Смена статуса
Отклонить(причина)	void	Отказ с комментарием

3.4.1.2.2. ПозицияЗаявки

Таблица 3.16 – Поля и методы класса "ПозицияЗаявки"

Поле/метод	Тип	Описание
ЗаявкаID	int	FK на ЗаявкаНаИнвентарь
НоменклатураID	int	FK на Номенклатура
КоличествоПозиций	int	Запрошенное кол-во
ЦенаПозицииОжидаемая	decimal	Ориентировочная цена
ПриоритетПозиции	enum	Высокий/Средний/Низкий

ПолучитьСумму()	decimal	Кол-во × Цена
ПроверитьДоступность(складID)	bool	проверяет, есть ли товар в наличии

3.4.1.2.3. ЗаказПоставщику

Таблица 3.17 – Поля и методы класса "ЗаказПоставщику"

Поле/метод	Тип	Описание
НомерЗаказа	string	Номер заказа
ДатаСозданияЗаказа	date	Дата оформления
СтатусЗаказа	enum	Черновик/Отправлен/ВПути/Доставлен/Закрыт
ПоставщикID	int	FK на Поставщик
МенеджерID	int	FK на МенеджерПоЗакупкам
СуммаИтоговая	decimal	Общая стоимость
ОжидаемаяДатаПоставки	date	Плановый срок
СформироватьИзПлана()	void	Автозаполнение позиций
ОтправитьПоставщику()	void	Смена статуса + уведомление
ИзменитьСтатус(статус)	void	Обновление состояния

3.4.1.2.4. Поставщик

Таблица 3.18 – Поля и методы класса "Поставщик"

Поле/метод	Тип	Описание
Наименование	string	Название организации
ИНН	string	ИНН
КПП	string	КПП
ЮридическийАдрес	string	Адрес
КонтактноеЛицо	string	ФИО менеджера
ТелефонПоставщика	string	Контактный номер
EmailПоставщика	string	Для заказов
РейтингПоставщика	decimal	Расчетный показатель надежности
РассчитатьРейтинг()	decimal	На основе истории поставок
ПолучитьИсториюЗаказов()	List<ЗаказПоставщику>	Все заказы

3.4.1.3. Пакет «Склад и учет инвентаря»

3.4.1.3.1. Номенклатура

Таблица 3.19 – Поля и методы класса "Номенклатура"

Поле/метод	Тип	Описание
АртикулТовара	string	Уникальный код
НаименованиеТовара	string	Название товара
КатегорияТовара	enum	Лыжи/Ботинки/Палки/Крепления/Смазка
ЕдиницаИзмерения	string	шт/пара/комплект
НормативныйСрокСлужбы	int	В месяцах
МинОстаток	int	Страховой запас

Производитель	string	Бренд
ПолучитьТекущийОстаток(складID)	int	Количество на складе
ПолучитьДоступныйОстаток(складID)	int	С учетом резервов
ПроверитьНеобходимостьЗакупки()	bool	Остаток < МинОстаток

3.4.1.3.2. ЕдиницаИнвентаря

Таблица 3.20 – Поля и методы класса "ЕдиницаИнвентаря"

Поле/метод	Тип	Описание
Штрихкод	string	Уникальный номер единицы
НоменклатураID	int	FK на Номенклатура
СерийныйНомер	string	Заводской номер
СтатусИнвентаря	enum	НаСкладе/Выдан/ВРемонте/Списан/Брак
СкладID	int	Текущее местоположение
ДатаПоступления	date	Дата прихода
ДатаСписания	date	NULL если активна
СтоимостьЗакупки	decimal	Цена приобретения
РазмерИнвентаря	string	Ростовка/размер
ПримечаниеКИнвертарю	string	Особые отметки
ВыдатьСпортсмену(спортсменID)	void	Смена статуса на «Выдан»
ПринятьВозврат()	void	Смена статуса на «НаСкладе»
Списать(причина)	void	Смена статуса на «Списан»
ОтправитьВРемонт()	void	Смена статуса на «ВРемонте»
ПолучитьИсториюДвижения()	List<Операция>	Полная история

3.4.1.3.3. ВыдачаИнвентаря

Таблица 3.21 – Поля и методы класса "ВыдачаИнвентаря"

Поле/метод	Тип	Описание
ЕдиницаИнвентаряID	int	FK на ЕдиницаИнвентаря
СпортсменID	int	FK на Спортсмен
КладовщикID	int	FK на Кладовщик
ДатаВыдачи	datetime	Когда выдано
ПлановаяДатаВозврата	date	Срок возврата
ФактическаяДатаВозврата	date	NULL до возврата
СтатусВыдачи	enum	Активна/Завершена/Просрочена
СостояниеПриВыдаче	string	Новое/б/у
СостояниеПриВозврате	string	Норма/Повреждено
ОформитьВозврат(состояние)	void	Закрытие выдачи
ПроверитьПросрочку()	bool	ДатаВозврата < Сегодня
ПродлитьСрок(новаяДата)	void	Продление возврата

3.4.1.3.4. ПриходнаяНакладная

Таблица 3.22 – Поля и методы класса "ПриходнаяНакладная"

Поле/метод	Тип	Описание
НомерНакладной	string	Номер документа

ДатаПрихода	date	Дата приемки
ЗаказID	int	FK на ЗаказПоставщику
КладовщикID	int	FK на Кладовщик
СтатусНакладной	enum	Оприходована/СРасхождениями
СуммаФактическая	decimal	Итоговая сумма
ОприходоватьТовар()	void	Создание ЕдиницИнвентаря
СоздатьАктРасхождений(позиции)	void	При несовпадении

3.4.1.4. Связи между классами (Relationships)

Таблица 3.23 – Связи между классами

Связь	Тип	Множественность	Описание
Пользователь ← Спортсмен	Наследование	—	
Пользователь ← Тренер	Наследование	—	
Пользователь ← Кладовщик	Наследование	—	
Пользователь ← МенеджерПоЗакупкам	Наследование	—	
Пользователь ← Руководитель	Наследование	—	
Пользователь ← Бухгалтер	Наследование	—	
Пользователь ← Администратор	Наследование	—	
Тренер → Спортсмен	Ассоциация	1 → 0..*	У тренера много спортсменов
ЗаявкаНаИнвентарь → ПозицияЗаявки	Композиция	1 → 1..*	Заявка состоит из позиций
ЗаявкаНаИнвентарь → Пользователь (Автор)	Ассоциация	* → 1	Кто создал
ПозицияЗаявки → Номенклатура	Ассоциация	* → 1	Что заказывают
ЗаказПоставщику → Поставщик	Ассоциация	* → 1	Кому заказ
ЗаказПоставщику → МенеджерПоЗакупкам	Ассоциация	* → 1	Кто оформил
ПриходнаяНакладная → ЗаказПоставщику	Ассоциация	* → 0..1	Связь с заказом
ПриходнаяНакладная → Кладовщик	Ассоциация	* → 1	Кто принял
ЕдиницаИнвентаря → Номенклатура	Ассоциация	* → 1	Тип товара
ЕдиницаИнвентаря → ПриходнаяНакладная	Ассоциация	* → 1	По какому документу поступила
ВыдачаИнвентаря → ЕдиницаИнвентаря	Ассоциация	* → 1	Что выдано
ВыдачаИнвентаря → Спортсмен	Ассоциация	* → 1	Кому выдано
ВыдачаИнвентаря → Кладовщик	Ассоциация	* → 1	Кто выдал

3.4.2. Схематичное представление

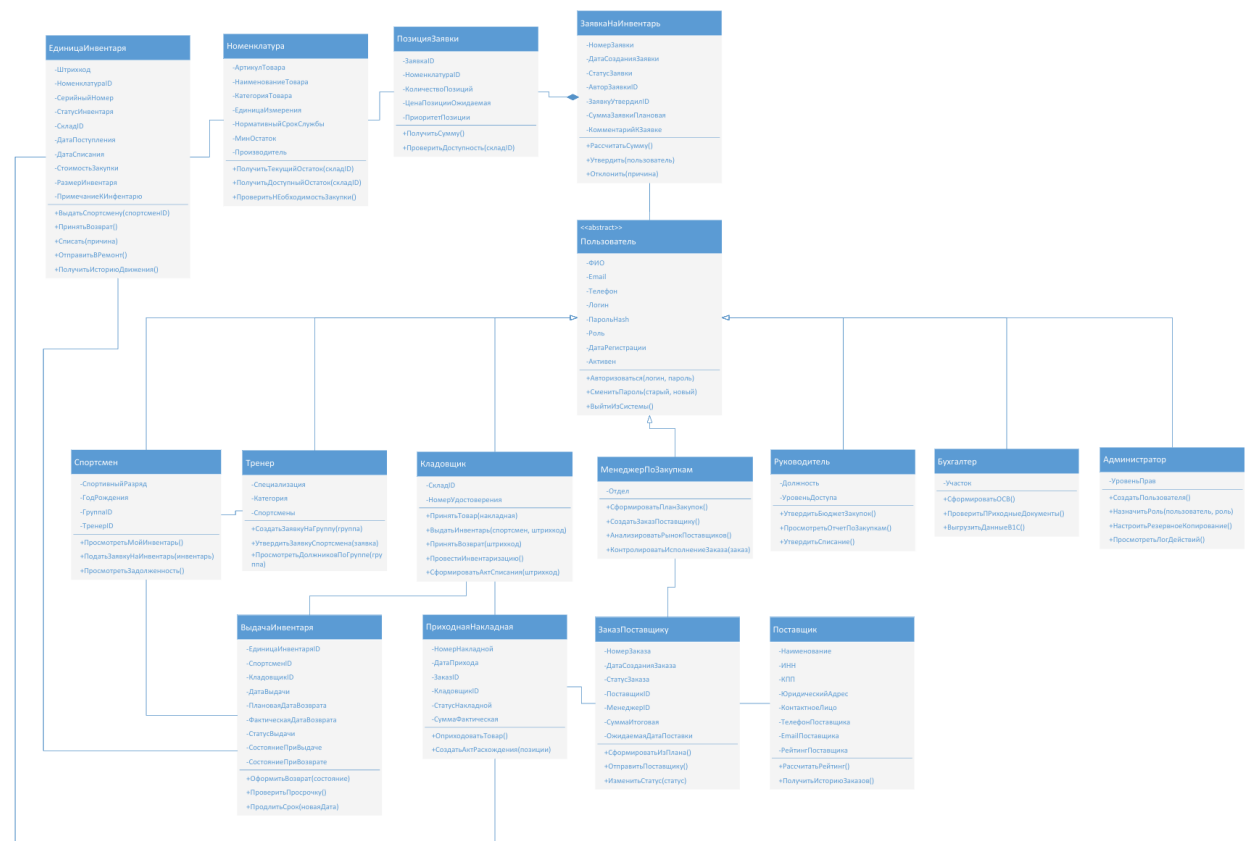


Рисунок 3.7 – Диаграмма классов

4. Описание и доработка программно-аппаратной платформы для проектируемой АСУ

4.1. Построение таблицы аппаратного обеспечения АСУ по выбранной тематике и обеспечивающих подсистем

Таблица 4.1 – Аппаратное обеспечение АСУ

№	Наименование оборудования	Тип / Модель (рекомендуемая)	Количество	Назначение	Обеспечиваемые функции / UML элементы
1	Основной модуль АСУ				
1.1	Сервер баз данных	Сервер на базе 1x Intel Xeon Silver 4309Y (8 ядер), 32 ГБ RAM, 2x SSD 480 ГБ (RAID 1)	1	Хранение данных, обработка транзакций	Все функции модулей 1, 2, 3; диаграммы последовательности (сохранение заявок, заказов, операций)
1.2	Сервер приложений	Сервер на базе 1x Intel Xeon Silver 4310 (12 ядер), 32 ГБ RAM, SSD 1 ТБ	1	Выполнение бизнес-логики, API, веб-сервер	Модули 1, 2, 3; все варианты использования
1.3	Сервер резервного копирования	Сервер с 2x HDD 4 ТБ (RAID 1)	1	Резервное копирование БД и файлов	Нефункциональное требование №7 (резервное копирование)
1.4	Сетевое оборудование	Коммутатор уровня L3 (например, Cisco Catalyst 9300)	2	Обеспечение сетевой связности	Нефункциональное требование №3 (доступность 24/7)
1.5	Межсетевой экран (Firewall)	Аппаратный firewall (например, UserGate или Ideco)	1	Защита от НСД, фильтрация трафика	Нефункциональное требование №10 (защита данных)
2	Аналитическая подсистема (BI)				
2.1	Сервер аналитики (OLAP)	Сервер на базе 1x Intel Xeon Silver 4309Y (8 ядер), 64 ГБ RAM, SSD 2 ТБ	1	Хранение аналитических данных, построение отчетов	Функции: 3.6-3.10 (отчеты по закупкам, ОСВ, движение ТМЦ); Генератор Отчетов (диаграмма классов)
2.2	Хранилище данных (Data Warehouse)	СХД на базе 2x HDD 4 ТБ (RAID 1)	1	Долгосрочное хранение исторических данных	Функция 1.15 (история закупок), 2.8 (история выдач/возвратов)

2.3	ETL-сервер	Виртуальная машина на сервере аналитики (2.1)	1	Извлечение, трансформация и загрузка данных из OLTP в OLAP	Функции аналитической подсистемы; подготовка данных для ГенераторОтчетов
3	Подсистема информационного обеспечения (Web/Доступ)				
3.1	Веб-сервер (Frontend)	Виртуальная машина на сервере приложений (1.2)	1	Хостинг веб-интерфейса, статического контента	Функция 3.12-3.17 (настройки, выгрузка); личный кабинет спортсмена/тренера
3.2	Терминалы сбора данных (ТСД)	Сканер штрих-кода (например, Zebra TC21)	2	Приемка, выдача, инвентаризация	Функции 2.3-2.7, 2.12, 2.15; Кладовщик (диаграмма классов)
3.3	Принтер этикеток	Термотрансферный принтер (Zebra ZD421)	1	Печать штрих-кодов для инвентаря	Функция 2.5 (печать штрих-кодов)
3.4	Автоматизированные рабочие места (АРМ)	Рабочие станции (ПК)	8-10	Работа пользователей (менеджеры, кладовщики, бухгалтер)	Все интерфейсные функции для всех ролей
3.5	Мобильные устройства	Планшеты/смартфоны (iOS/Android)	10+	Доступ тренеров и спортсменов к личным кабинетам	Нефункциональное требование №14 (адаптивность для мобильных устройств)
3.6	Защищенный планшет для тренера	Getac T800 / Panasonic Toughpad (морозостойкий, IP68)	2-3	Работа тренера на трассе, подача заявок, просмотр остатков	Функции 1.2-1.4, 2.16; Тренер (диаграмма классов)
3.7	Сервер видеонаблюдения склада	NVR-регистратор + IP-камеры (4-8 шт.)	1 комплект	Контроль доступа к дорогостоящему инвентарю	Нефункциональное требование №10 (защита данных)

4.2. Построение таблицы программного обеспечения для выполнения всех функций, описанных в требованиях и отображенных на UML диаграммах

Таблица 4.2 – Программное обеспечение АСУ

№	Наименование ПО	Версия / Тип	Назначение	Обеспечиваемые функции / UML элементы
1	Основной модуль АСУ			
1.1	Операционная система серверов	Astra Linux Special Edition (или Red OS)	Базовая ОС для серверного оборудования	Соответствие импортозамещению (Нефункциональное требование №19)
1.2	СУБД	PostgreSQL 17 (или Postgres Pro Enterprise)	Хранение данных, транзакционная обработка	Все функции модулей 1-3; Журнал Операций, Бюджетный Счет, все документы
1.3	Сервер приложений (Backend)	Spring Boot (Java 21) / .NET Core 8	Бизнес-логика, REST API, обработка запросов	Все внутрисистемные функции (IS1-IS56); Сервис Авторизации, Сервис Уведомлений
1.4	Веб-сервер (Frontend)	React.js / Vue.js	Пользовательский интерфейс (SPA)	Все интерфейсные функции для всех ролей; личные кабинеты
1.5	Система резервного копирования	Bacula / Veeam Backup	Автоматическое резервное копирование	Нефункциональное требование №7 (резервное копирование)
1.6	Средства мониторинга	Zabbix / Prometheus + Grafana	Мониторинг состояния серверов, оповещение	Нефункциональное требование №8 (устойчивость к сбоям)
1.7	Контейнеризация	Docker + Docker Compose	Управление микросервисами, масштабирование	Нефункциональное требование №9 (масштабируемость)
2	Аналитическая подсистема (BI)			
2.1	OLAP-сервер	ClickHouse / Apache Druid	Аналитическая обработка больших данных	Функции 3.6-3.10, 3.14-3.15 (отчеты, статистика)
2.2	ETL-инструмент	Apache NiFi / Talend Open Studio	Извлечение, трансформация, загрузка данных	Передача данных из OLTP в OLAP
2.3	BI-платформа (отчеты)	Redash / Apache Superset	Визуализация данных, дашборды	Функции: отчет по закупкам (3.6), ОСВ (3.7), движение ТМЦ (3.8), статистика использования (3.14)
2.4	Система прогнозирования	Python (pandas, scikit-learn)	Прогнозирование потребностей в закупках	Функция 1.7 (формирование плана закупок) — алгоритм расчета потребности

2.5	Аналитика поставщиков (SRM)	Модуль на базе Python	Оценка надежности, сравнение цен	Функции 1.13 (анализ надежности), 1.14 (сравнение цен)
2.6	Модуль учета износа лыжного инвентаря	Python (pandas) + алгоритмы прогнозирования	Расчет остаточного ресурса лыж, прогноз замены	Функция 3.15 (отчет по износу); ЕдиницаИнвентаря (диаграмма классов)
3	Подсистема информационного обеспечения (Web/Доступ)			
3.1	Веб-сервер (Reverse Proxy)	Nginx / Apache	Проксирование, балансировка, SSL-терминация	Нефункциональное требование №12 (кроссплатформенность), доступ через глобальную сеть
3.2	Сервер аутентификации	Keycloak / WSO2 Identity Server	Централизованная аутентификация, SSO, MFA	СервисАвторизации (диаграмма классов); Нефункциональное требование №2 (ролевая модель)
3.3	Система управления API	Kong / Apache APISIX	Управление API-шлюзами, rate limiting	Взаимодействие между модулями; Нефункциональное требование №1 (многопользовательский доступ)
3.4	Сервер электронной почты	Postfix + Dovecot	Отправка уведомлений	СервисУведомлений (диаграмма классов); функции 1.5-1.6 (уведомления о согласовании)
3.5	Мессенджер/Уведомления	Telegram Bot API / Mattermost	Push-уведомления	СервисУведомлений (диаграмма классов); Нефункциональное требование №12 (контроль возврата)
3.6	ОС для АРМ	Astra Linux	Базовая ОС для работы сотрудников с ПК	Соответствие импортозамещению (Нефункциональное требование №19)
3.7	Офисный пакет (АРМ)	LibreOffice / Р7-Офис	Просмотр и экспорт отчетов	Функция 3.16 (выгрузка отчетов в Excel/PDF)
3.8	Клиентское ПО для ТСД	Android (AOSP) + специализированное приложение	Работа сканеров штрих-кода	Функции кладовщика 2.3-2.7, 2.12, 2.15
3.9	Драйверы торгового оборудования	SDK производителя (Zebra, Honeywell)	Интеграция со сканерами и принтерами	Нефункциональное требование №13 (работа с торговым оборудованием)
3.10	Мобильное приложение тренера (PWA)	React PWA + IndexedDB	Работа без интернета на сборах, синхронизация при подключении	Функции 1.2-1.4, 2.1-2.2, 2.8-2.9; Нефункциональное требование №14

4.3. Анализ функций АСУ и определение функции для доработки

Выбранная функция для разработки: Функция 1.7 — «Формирование плана закупок на период» (из Модуля 1: Управление закупками, планирование и бюджетный контроль).

Внутрисистемные функции, реализуемые модулем:

- 1.IS19 — Запуск алгоритма расчета потребности (анализ утвержденных заявок)
- 1.IS20 — Анализ текущих остатков на складе по каждой позиции
- 1.IS21 — Учет минимальных страховых запасов при расчете потребности
- 1.IS22 — Генерация проекта плана закупок с группировкой по номенклатуре

Обоснование выбора:

1. Высокая алгоритмическая сложность — прогнозирование требует применения методов машинного обучения (временные ряды, сезонность).

2. Специфичность предметной области — лыжный спорт имеет выраженную сезонность:

- Подготовка к зимнему сезону (август-октябрь)
- Пик использования (ноябрь-март)
- Межсезонное обслуживание (апрель-июль)

3. Учет множества факторов:

- Исторические данные по расходу инвентаря
- Сезонные коэффициенты спроса
- Нормативный износ лыж (зависит от интенсивности использования)
- Планируемые соревнования и сборы
- Рост/смена состава спортсменов (изменение размеров)

4. Экономическая значимость — ошибки прогнозирования приводят к:

- Дефициту (срыв тренировочного процесса)
- Избытку (заморозка бюджета, устаревание моделей)

5. Отсутствие готовых решений — типовые ERP-системы не учитывают специфику износа лыжного инвентаря.

4.4. Анализ существующих языков программирования и IDE

Таблица 4.3 – Рассмотренные языки программирования для задачи прогнозирования

Язык	Преимущества	Недостатки	Применимость к задаче
Python	Богатейшая экосистема для ML (pandas, scikit-learn, Prophet, statsmodels); быстрая разработка; интерактивный анализ в Jupyter	Динамическая типизация; ниже производительность на больших данных; GIL ограничивает многопоточность	Высокая — идеален для прототипирования и реализации ML-моделей прогнозирования
R	Специализирован для статистического анализа; отличная визуализация (ggplot2); множество пакетов для временных рядов	Сложный синтаксис; меньшее сообщество в РФ; сложнее интеграция с Java-бэкендом	Средняя — хорош для исследовательского анализа, но сложен в production
Java	Высокая производительность; строгая типизация; отличная интеграция с существующей архитектурой (Spring Boot)	Ограниченная поддержка ML-библиотек (Weka, Smile уступают Python); громоздкий код	Средняя — хорош для production-контура, но не для разработки моделей
Julia	Высокая производительность; синтаксис, близкий к математическому	Молодая экосистема; мало библиотек; сложность развертывания	Низкая — неоправданные риски для production

Таблица 4.4 – Рассмотренные IDE для разработки ML-модуля

IDE	Поддерживаемые языки	Преимущества	Недостатки
PyCharm Professional	Python, SQL, JavaScript	Лучшая поддержка научных библиотек; встроенный Jupyter Notebook; отладка DataFrame; интеграция с БД	Платная (бесплатна для студентов)

JupyterLab	Python, R, Julia	Интерактивная разработка; визуализация «на лету»; идеален для экспериментов	Не подходит для полноценного production-кода; нет рефакторинга
VS Code	Python, R, SQL	Бесплатный; отличные расширения для Python; поддержка Jupyter внутри редактора	Требует настройки окружения
DataSpell	Python, R, SQL	Специализированная IDE для Data Science от JetBrains; встроенный Jupyter	Платная; избыточна для небольшого модуля

4.5. Обоснование выбора языка, IDE и библиотек

Выбранный язык программирования: Python 3.12

Обоснование:

- Библиотеки для временных рядов — Prophet (Facebook/Meta), statsmodels, scikit-learn — готовые реализации алгоритмов прогнозирования.
- Быстрое прототипирование — возможность проверить гипотезы в Jupyter Notebook перед интеграцией.
- Интеграция с Java-бэкендом — через REST API или экспорт модели в PMML/ONNX.
- Визуализация — matplotlib, seaborn, plotly для построения графиков прогнозов.
- Работа с данными — pandas для ETL-операций, загрузки из PostgreSQL.
- Соответствие архитектуре — Python указан как язык аналитической подсистемы (позиция 2.4-2.5).

Выбранная IDE: PyCharm Professional 2026.1

Обоснование:

- Бесплатная лицензия для студентов — доступ к Professional версии.

- Встроенная поддержка Jupyter Notebook — можно вести разработку модели в .ipynb и затем экспортировать в .py.
- Интеграция с БД — просмотр таблиц PostgreSQL, выполнение SQL-запросов без переключения.
- Отладка DataFrame — просмотр содержимого pandas DataFrame в удобном табличном виде.
- Поддержка научных библиотек — автодополнение для pandas, numpy, sklearn.

Таблица 4.5 – Выбранные библиотеки Python

Библиотека	Версия	Назначение	Обоснование
pandas	2.2.0	Обработка и анализ данных	Стандарт для работы с табличными данными, загрузка из SQL
numpy	1.26.0	Численные вычисления	Базовые математические операции, работа с массивами
statsmodels	0.14.1	Статистический анализ	Экспоненциальное сглаживание (Holt-Winters), тесты на стационарность
scikit-learn	1.4.0	Машинное обучение	Метрики качества прогноза (MAE, MAPE, RMSE)
psycopg2	2.9.9	Подключение к PostgreSQL	Загрузка исторических данных из БД АСУ
sqlalchemy	2.0.25	ORM для Python	Удобная работа с БД через pandas
loguru	0.7.2	Логирование	Удобное логирование процесса прогнозирования
joblib	1.3.2	Сериализация моделей	Сохранение обученной модели для повторного использования
python-dotenv	1.0.0	Переменные окружения	Загрузка конфигурации из файла .env

4.6. Генерация программного кода

Листинги с программным кодом приведены в Приложение 1. Программный код модуля для доработки

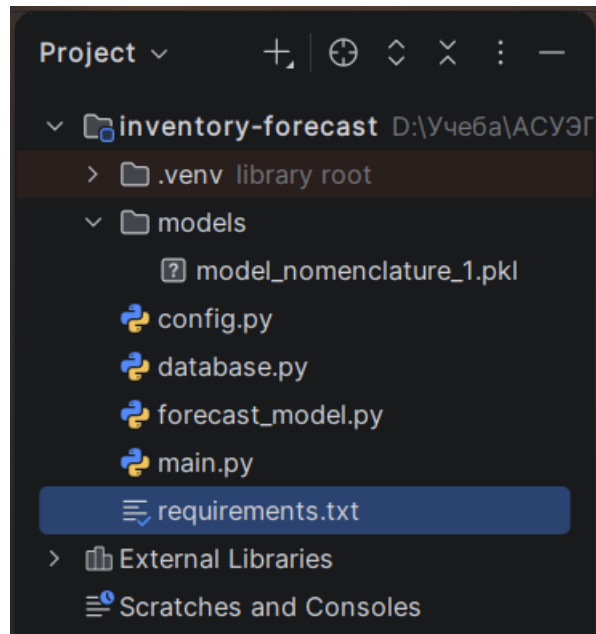


Рисунок 4.1 – Структура разрабатываемого модуля

5. ER-диаграмма для проектируемой АСУ

5.1. Выделение и описание свойств и методов выбранных классов

5.1.1. Класс «Пользователь» (абстрактный)

Таблица 5.1 - Свойства и методы класса "Пользователь"

Свойство	Тип данных	Методы, использующие свойство
ФИО	string	Авторизоваться()
Email	string	Авторизоваться(), СменитьПароль()
Телефон	string	—
Логин	string	Авторизоваться(), СменитьПароль()
ПарольHash	string	Авторизоваться(), СменитьПароль()
Роль	enum	Авторизоваться()
ДатаРегистрации	date	—
Активен	bool	Авторизоваться()

5.1.2. Класс «Спортсмен» (наследует Пользователь)

Таблица 5.2 - Свойства и методы класса "Спортсмен"

Свойство	Тип данных	Методы, использующие свойство
СпортивныйРазряд	string	ПросмотретьМойИнвентарь()
ГодРождения	int	—
ГруппаID	int	ПодатьЗаявкуНаИнвентарь()
ТренерID	int	ПросмотретьМойИнвентарь()

5.1.3. Класс «Тренер» (наследует Пользователь)

Таблица 5.3 - Свойства и методы класса "Тренер"

Свойство	Тип данных	Методы, использующие свойство
Специализация	string	СоздатьЗаявкуНаГруппу()
Категория	string	—
Спортсмены	List<Спортсмен>	СоздатьЗаявкуНаГруппу(), ПросмотретьДолжниковПоГруппе()

5.1.4. Класс «Кладовщик» (наследует Пользователь)

Таблица 5.4 - Свойства и методы класса "Кладовщик"

Свойство	Тип данных	Методы, использующие свойство
СкладID	int	ПринятьТовар(), ПровестиИнвентаризацию()
НомерУдостоверения	string	—

5.1.5. Класс «МенеджерПоЗакупкам» (наследует Пользователь)

Таблица 5.5 - Свойства и методы класса "МенеджерПоЗакупкам"

Свойство	Тип данных	Методы, использующие свойство
Отдел	string	СформироватьПланЗакупок()

5.1.6. Класс «Руководитель» (наследует Пользователь)

Таблица 5.6 - Свойства и методы класса "Руководитель"

Свойство	Тип данных	Методы, использующие свойство
Должность	string	УтвердитьБюджетЗакупок()
УровеньДоступа	int	УтвердитьБюджетЗакупок(), УтвердитьСписание()

5.1.7. Класс «Бухгалтер» (наследует Пользователь)

Таблица 5.7 - Свойства и методы класса "Бухгалтер"

Свойство	Тип данных	Методы, использующие свойство
Участок	string	СформироватьОСВ()

5.1.8. Класс «Администратор» (наследует Пользователь)

Таблица 5.8 - Свойства и методы класса "Администратор"

Свойство	Тип данных	Методы, использующие свойство
УровеньПрав	enum	СоздатьПользователя(), НазначитьРоль()

5.1.9. Класс «ЗаявкаНаИнвентарь»

Таблица 5.9 - Свойства и методы класса "ЗаявкаНаИнвентарь"

Свойство	Тип данных	Методы, использующие свойство
НомерЗаявки	string	РассчитатьСумму()
ДатаСозданияЗаявки	datetime	Утвердить(), Отклонить()
СтатусЗаявки	enum	Утвердить(), Отклонить()
АвторЗаявкиID	int	Утвердить(), Отклонить()
ЗаявкуУтвердилID	int	Утвердить()
СуммаЗаявкиПлановая	decimal	РассчитатьСумму()
КомментарийКЗаявке	string	Отклонить()

5.1.10. Класс «ПозицияЗаявки»

Таблица 5.10 - Свойства и методы класса "ПозицияЗаявки"

Свойство	Тип данных	Методы, использующие свойство
ЗаявкаID	int	ПолучитьСумму()
НоменклатураID	int	ПолучитьСумму(), ПроверитьДоступность()
КоличествоПозиций	int	ПолучитьСумму(), ПроверитьДоступность()
ЦенаПозицииОжидаемая	decimal	ПолучитьСумму()

ПриоритетПозиции	enum	—
------------------	------	---

5.1.11. Класс «ЗаказПоставщику»

Таблица 5.11 - Свойства и методы класса "ЗаказПоставщику"

Свойство	Тип данных	Методы, использующие свойство
НомерЗаказа	string	ОтправитьПоставщику()
ДатаСозданияЗаказа	date	СформироватьИзПлана()
СтатусЗаказа	enum	ИзменитьСтатус(), ОтправитьПоставщику()
ПоставщикID	int	СформироватьИзПлана(), ОтправитьПоставщику()
МенеджерID	int	СформироватьИзПлана()
СуммаИтоговая	decimal	СформироватьИзПлана()
ОжидаемаяДатаПоставки	date	ИзменитьСтатус()

5.1.12. Класс «Поставщик»

Таблица 5.12 - Свойства и методы класса "Поставщик"

Свойство	Тип данных	Методы, использующие свойство
Наименование	string	ПолучитьИсториюЗаказов()
ИНН	string	—
КПП	string	—
ЮридическийАдрес	string	—
КонтактноеЛицо	string	—
ТелефонПоставщика	string	—
EmailПоставщика	string	—
РейтингПоставщика	decimal	РассчитатьРейтинг()

5.1.13. Класс «Номенклатура»

Таблица 5.13 - Свойства и методы класса "Номенклатура"

Свойство	Тип данных	Методы, использующие свойство
АртикулТовара	string	—
НаименованиеТовара	string	ПолучитьТекущийОстаток()
КатегорияТовара	enum	ПроверитьНеобходимостьЗакупки()
ЕдиницаИзмерения	string	—
НормативныйСрокСлужбы	int	ПроверитьНеобходимостьЗакупки()
МинОстаток	int	ПолучитьДоступныйОстаток(), ПроверитьНеобходимостьЗакупки()
Производитель	string	—

5.1.14. Класс «ЕдиницаИнвентаря»

Таблица 5.14 - Свойства и методы класса " ЕдиницаИнвентаря"

Свойство	Тип данных	Методы, использующие свойство
Штрихкод	string	ВыдатьСпортсмену(), ПринятьВозврат()
НоменклатураID	int	ВыдатьСпортсмену()
СерийныйНомер	string	ПолучитьИсториюДвижения()
СтатусИнвертаря	enum	ВыдатьСпортсмену(), ПринятьВозврат(), Списать(), ОтправитьВРемонт()
СкладID	int	ПринятьВозврат()
ДатаПоступления	date	ПолучитьИсториюДвижения()
ДатаСписания	date	Списать()
СтоимостьЗакупки	decimal	Списать()
РазмерИнвентаря	string	ВыдатьСпортсмену()
ПримечаниеКИнвентарю	string	—

5.1.15. Класс «ВыдачаИнвентаря»

Таблица 5.15 - Свойства и методы класса " ВыдачаИнвентаря"

Свойство	Тип данных	Методы, использующие свойство
ЕдиницаИнвентаряID	int	ОформитьВозврат(), ПроверитьПросрочку()
СпортсменID	int	ОформитьВозврат(), ПродлитьСрок()
КладовщикID	int	ОформитьВозврат()
ДатаВыдачи	datetime	ПроверитьПросрочку(), ПродлитьСрок()
ПлановаяДатаВозврата	date	ПроверитьПросрочку(), ПродлитьСрок()
ФактическаяДатаВозврата	date	ОформитьВозврат()
СтатусВыдачи	enum	ОформитьВозврат(), ПроверитьПросрочку()
СостояниеПриВыдаче	string	—
СостояниеПриВозврате	string	ОформитьВозврат()

5.1.16. Класс «ПриходнаяНакладная»

Таблица 5.16 - Свойства и методы класса " ПриходнаяНакладная"

Свойство	Тип данных	Методы, использующие свойство
НомерНакладной	string	ОприходоватьТовар()
ДатаПрихода	date	ОприходоватьТовар()
ЗаказID	int	ОприходоватьТовар(), СоздатьАктРасхождений()
КладовщикID	int	ОприходоватьТовар()
СтатусНакладной	enum	ОприходоватьТовар(), СоздатьАктРасхождений()
СуммаФактическая	decimal	ОприходоватьТовар(), СоздатьАктРасхождений()

5.2. Построение ER-диаграммы на основе классовой диаграммы

5.2.1. Описание таблиц и полей

Диаграмма содержит 19 таблиц, сгруппированных по логическим модулям:

- Пользователи и роли (8 таблиц): user, athlete, coache, storekeeper, purchase_manager, manager, accountant, administrator
- Справочники (4 таблицы): nomenclature, supplier, warehouse, group
- Закупки (3 таблицы): purchase_request, purchase_request_item, purchase_order
- Склад и выдачи (3 таблицы): inventory_item, receipt_invoice, issuance_document
- Вспомогательные (1 таблица): operation_log

5.2.1.1. Таблицы группы «Пользователи и роли»

Таблица 5.17 – Структура таблицы «user»

Поле	Тип данных	Ограничения	Описание
user_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
full_name	VARCHAR(200)	NOT NULL	Полное имя
email	VARCHAR(100)	UNIQUE, NOT NULL	Рабочая почта
phone	VARCHAR(20)		Контактный телефон
login	VARCHAR(50)	UNIQUE, NOT NULL	Логин для входа
password_hash	VARCHAR(255)	NOT NULL	Хеш пароля
role	VARCHAR(30)	NOT NULL	Роль в системе
registration_date	DATE	DEFAULT CURRENT DATE	Дата регистрации
is_active	BOOLEAN	DEFAULT TRUE	Статус активности
user_created_at	TIMESTAMP	DEFAULT NOW()	Служебное поле
user_updated_at	TIMESTAMP	DEFAULT NOW()	Служебное поле

Таблица 5.18 – Структура таблицы «athlete»

Поле	Тип данных	Ограничения	Описание
athlete_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
user_id	INTEGER	NOT NULL, REFERENCES user(user_id) ON DELETE CASCADE	Связь с пользователем
sports_category	VARCHAR(50)		Спортивный разряд
birth_year	INTEGER		Год рождения
group_id	INTEGER	REFERENCES group(group_id)	Группа спортсмена
coach_id	INTEGER	REFERENCES coach(coach_id)	Тренер
has_active_debt	BOOLEAN	DEFAULT FALSE	Наличие задолженности

Таблица 5.19 – Структура таблицы «coach»

Поле	Тип данных	Ограничения	Описание
coach_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
user_id	INTEGER	NOT NULL, REFERENCES user(user_id) ON DELETE CASCADE	Связь с пользователем
specialization	VARCHAR(100)		Специализация
coach_category	VARCHAR(50)		Тренерская категория

Таблица 5.20 – Структура таблицы «storekeeper»

Поле	Тип данных	Ограничения	Описание
storekeeper_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
user_id	INTEGER	NOT NULL, REFERENCES user(user_id) ON DELETE CASCADE	Связь с пользователем
warehouse_id	INTEGER	REFERENCES warehouse(warehouse_id)	Привязка к складу
certificate_number	VARCHAR(50)		Номер удостоверения

Таблица 5.21 – Структура таблицы «purchase_manager»

Поле	Тип данных	Ограничения	Описание
purchase_manager_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
user_id	INTEGER	NOT NULL, REFERENCES user(user_id) ON DELETE CASCADE	Связь с пользователем
department	VARCHAR(100)		Отдел

Таблица 5.22 – Структура таблицы «manager»

Поле	Тип данных	Ограничения	Описание
manager_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
user_id	INTEGER	NOT NULL, REFERENCES user(user_id) ON DELETE CASCADE	Связь с пользователем
position	VARCHAR(100)		Должность
manager access level	INTEGER	DEFAULT 1	Уровень доступа

Таблица 5.23 – Структура таблицы «accountant»

Поле	Тип данных	Ограничения	Описание
accountant_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
user_id	INTEGER	NOT NULL, REFERENCES user(user_id) ON DELETE CASCADE	Связь с пользователем
section	VARCHAR(100)		Участок работы

Таблица 5.24 – Структура таблицы «administrator»

Поле	Тип данных	Ограничения	Описание
administrator_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
user_id	INTEGER	NOT NULL, REFERENCES user(user_id) ON DELETE CASCADE	Связь с пользователем
administrator access level	VARCHAR(30)	DEFAULT 'FULL'	Уровень прав

5.2.1.2. Таблицы группы «Справочники»

Таблица 5.25 – Структура таблицы «nomenclature»

Поле	Тип данных	Ограничения	Описание
nomenclature_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
article	VARCHAR(50)	UNIQUE, NOT NULL	Артикул
nomenclature name	VARCHAR(200)	NOT NULL	Наименование
nomenclature category	VARCHAR(50)	NOT NULL	Категория товара
unit	VARCHAR(20)	NOT NULL	Единица измерения
standard_service_life	INTEGER		Нормативный срок службы (мес.)
min stock level	INTEGER	DEFAULT 0	Минимальный остаток
manufacturer	VARCHAR(100)		Производитель

nomenclature_created_at	TIMESTAMP	DEFAULT NOW()	Служебное поле
-------------------------	-----------	---------------	----------------

Таблица 5.26 – Структура таблицы «supplier»

Поле	Тип данных	Ограничения	Описание
supplier_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
supplier name	VARCHAR(200)	NOT NULL	Наименование
inn	VARCHAR(12)	UNIQUE, NOT NULL	ИНН
kpp	VARCHAR(9)		КПП
legal address	TEXT		Юридический адрес
contact person	VARCHAR(200)		Контактное лицо
supplier phone	VARCHAR(20)		Телефон
supplier_email	VARCHAR(100)		Email
rating	DECIMAL(3,2)	DEFAULT 0	Рейтинг надежности
supplier created at	TIMESTAMP	DEFAULT NOW()	Служебное поле

Таблица 5.27 – Структура таблицы «warehouse»

Поле	Тип данных	Ограничения	Описание
warehouse id	SERIAL	PRIMARY KEY	Уникальный идентификатор
warehouse name	VARCHAR(100)	NOT NULL	Наименование склада
address	TEXT		Адрес
type	VARCHAR(50)		Тип склада

Таблица 5.28 – Структура таблицы «group»

Поле	Тип данных	Ограничения	Описание
group_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
group name	VARCHAR(100)	NOT NULL	Название группы
coach_id	INTEGER	REFERENCES coach(coach_id)	Тренер группы
group created at	TIMESTAMP	DEFAULT NOW()	Служебное поле

5.2.1.3. Таблицы группы «Закупки и планирование»

Таблица 5.29 – Структура таблицы «purchase_request»

Поле	Тип данных	Ограничения	Описание
purchase_request_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
purchase_request_number	VARCHAR(50)	UNIQUE, NOT NULL	Номер заявки
purchase request created date	TIMESTAMP	DEFAULT NOW()	Дата создания

purchase_request_status	VARCHAR(30)	NOT NULL, DEFAULT 'DRAFT'	Статус
author_id	INTEGER	NOT NULL, REFERENCES user(user_id)	Автор заявки
approved_by	INTEGER	REFERENCES user(user_id)	Кто утвердил
planned amount	DECIMAL(14,2)		Плановая сумма
purchase request comment	TEXT		Комментарий
purchase request created at	TIMESTAMP	DEFAULT NOW()	Служебное поле
purchase request updated at	TIMESTAMP	DEFAULT NOW()	Служебное поле

Таблица 5.30 – Структура таблицы «purchase_request_item»

Поле	Тип данных	Ограничения	Описание
purchase_request_item_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
purchase_request_id	INTEGER	NOT NULL, REFERENCES purchase_request(purchase_request_id) ON DELETE CASCADE	Связь с заявкой
nomenclature_id	INTEGER	NOT NULL, REFERENCES nomenclature(nomenclature_id)	Связь с номенклатурой
quantity	INTEGER	NOT NULL	Количество
expected_price	DECIMAL(12,2)		Ожидаемая цена
priority	VARCHAR(20)	DEFAULT 'MEDIUM'	Приоритет

Таблица 5.31 – Структура таблицы «purchase_order»

Поле	Тип данных	Ограничения	Описание
purchase_order_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
purchase_order_number	VARCHAR(50)	UNIQUE, NOT NULL	Номер заказа
purchase_order_created_date	DATE	DEFAULT CURRENT_DATE	Дата создания
purchase_order_status	VARCHAR(30)	NOT NULL, DEFAULT 'DRAFT'	Статус
supplier_id	INTEGER	NOT NULL, REFERENCES supplier(supplier_id)	Поставщик
purchase_manager_id	INTEGER	NOT NULL, REFERENCES purchase_manager(purchase_manager_id)	Менеджер
total_amount	DECIMAL(14,2)		Итоговая сумма

expected_delivery_date	DATE		Ожидаемая дата поставки
purchase_order_created_at	TIMESTAMP	DEFAULT NOW()	Служебное поле

5.2.1.4. Таблицы группы «Складской учет»

Таблица 5.32 – Структура таблицы «inventory_item»

Поле	Тип данных	Ограничения	Описание
inventory_item_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
barcode	VARCHAR(50)	UNIQUE, NOT NULL	Штрих-код
nomenclature_id	INTEGER	NOT NULL, REFERENCES nomenclature(nomenclature_id)	Связь с номенклатурой
serial_number	VARCHAR(100)		Серийный номер
inventory_status	VARCHAR(30)	NOT NULL, DEFAULT 'AVAILABLE'	Статус единицы
warehouse_id	INTEGER	REFERENCES warehouse(warehouse_id)	Текущий склад
receipt_date	DATE		Дата поступления
write off date	DATE		Дата списания
purchase_cost	DECIMAL(12,2)		Стоимость закупки
size	VARCHAR(20)		Размер/ростовка
athlete_id	INTEGER	REFERENCES athlete(athlete_id)	Текущий держатель
inventory notes	TEXT		Примечания
inventory_created_at	TIMESTAMP	DEFAULT NOW()	Служебное поле

Таблица 5.33 – Структура таблицы «receipt_invoice»

Поле	Тип данных	Ограничения	Описание
receipt_invoice_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
receipt_invoice_number	VARCHAR(50)	UNIQUE, NOT NULL	Номер накладной
receipt invoice date	DATE	DEFAULT CURRENT DATE	Дата приемки
purchase_order_id	INTEGER	REFERENCES purchase_order(purchase_order_id)	Связь с заказом

storekeeper_id	INTEGER	NOT NULL, REFERENCES storekeeper(storekeeper_id)	Кладовщик
receipt_invoice_status	VARCHAR(30)	DEFAULT 'POSTED'	Статус
actual_amount	DECIMAL(14, 2)		Фактическая сумма
receipt_invoice_created at	TIMESTAMP	DEFAULT NOW()	Служебное поле

Таблица 5.34 – Структура таблицы «issuance_document»

Поле	Тип данных	Ограничения	Описание
issuance_document_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
inventory_item_id	INTEGER	NOT NULL, REFERENCES inventory_item(inventory_item_id)	Единица инвентаря
athlete_id	INTEGER	NOT NULL, REFERENCES athlete(athlete_id)	Спортсмен
storekeeper_id	INTEGER	NOT NULL, REFERENCES storekeeper(storekeeper_id)	Кладовщик
issuance_date	DATE	NOT NULL	Дата выдачи
planned_return_date	DATE		Плановая дата возврата
actual_return_date	DATE		Фактическая дата возврата
issuance_document_status	VARCHAR(30)	NOT NULL, DEFAULT 'ACTIVE'	Статус выдачи
condition_on_issue	VARCHAR(50)		Состояние при выдаче
condition_on_return	VARCHAR(50)		Состояние при возврате
issuance_document_notes	TEXT		Примечания
issuance_document_created at	TIMESTAMP	DEFAULT NOW()	Служебное поле

5.2.1.5. Таблицы группы «Вспомогательные»

Таблица 5.35 – Структура таблицы «operation_log»

Поле	Тип данных	Ограничения	Описание
operation_log_id	SERIAL	PRIMARY KEY	Уникальный идентификатор
user_id	INTEGER	REFERENCES user(user_id)	Пользователь
action	VARCHAR(100)	NOT NULL	Действие
entity type	VARCHAR(50)		Тип сущности
entity_id	INTEGER		ID сущности

details	JSONB		Детали в JSON
ip address	INET		IP-адрес
operation_log_created_at	TIMESTAMP	DEFAULT NOW()	Служебное поле

5.2.2. Связи между таблицами

Таблица 5.36 – Связи между таблицами

Родительская таблица	Дочерняя таблица	Тип связи	Внешний ключ	ON DELETE
user	athlete	1:1	user id	CASCADE
user	coach	1:1	user id	CASCADE
user	storekeeper	1:1	user id	CASCADE
user	purchase manager	1:1	user id	CASCADE
user	manager	1:1	user id	CASCADE
user	accountant	1:1	user id	CASCADE
user	administrator	1:1	user id	CASCADE
coach	athlete	1:M	coach id	SET NULL
coach	group	1:M	coach id	SET NULL
group	athlete	1:M	group id	SET NULL
warehouse	storekeeper	1:M	warehouse id	SET NULL
warehouse	inventory item	1:M	warehouse id	SET NULL
nomenclature	inventory item	1:M	nomenclature id	RESTRICT
nomenclature	purchase_request_item	1:M	nomenclature_id	RESTRICT
supplier	purchase_order	1:M	supplier id	RESTRICT
purchase_manager	purchase_order	1:M	purchase_manager_id	RESTRICT
purchase_order	receipt invoice	1:1	purchase_order id	SET NULL
storekeeper	receipt invoice	1:M	storekeeper id	RESTRICT
storekeeper	issuance document	1:M	storekeeper id	RESTRICT
athlete	inventory item	1:M	athlete id	SET NULL
athlete	issuance document	1:M	athlete id	RESTRICT
inventory_item	issuance document	1:M	inventory_item id	RESTRICT
user	purchase_request (автор)	1:M	author_id	RESTRICT
user	purchase_request (утвердил)	1:M	approved_by	SET NULL
purchase_request	purchase_request_item	1:M	purchase_request_id	CASCADE
user	operation log	1:M	user id	SET NULL

5.2.3. Соответствие свойств классов и полей ER-диаграммы

Таблица 5.37 – Соответствие свойств классов и полей ER-диаграммы

Класс	Свойство класса	Поле таблицы	Таблица БД
Пользователь	ФИО	full_name	user
	Email	email	
	Телефон	phone	
	Логин	login	
	ПарольHash	password_hash	
	Роль	role	
	ДатаРегистрации	registration_date	
	Активен	is_active	
Спортсмен	СпортивныйРазряд	sports_category	athlete
	ГодРождения	birth_year	
	ГруппаID	group_id	
	ТренерID	coach_id	
	—	has_active_debt	
Тренер	Специализация	specialization	coach
	Категория	coach_category	
Кладовщик	СкладID	warehouse_id	storekeeper
	НомерУдостоверения	certificate_number	
МенеджерПоЗакупкам	Отдел	department	purchase_manager
Руководитель	Должность	position	manager
	УровеньДоступа	manager_access_level	
Бухгалтер	Участок	section	accountant
Администратор	УровеньПрав	administrator_access_level	administrator
ЗаявкаНаИнвентарь	НомерЗаявки	purchase_request_number	purchase_request
	ДатаСозданияЗаявки	purchase_request_created_date	
	СтатусЗаявки	purchase_request_status	
	АвторЗаявкиID	author_id	
	ЗаявкуУтвердилID	approved_by	
	СуммаЗаявкиПлановая	planned_amount	
	КомментарийКЗаявке	purchase_request_comment	
ПозицияЗаявки	ЗаявкаID	purchase_request_id	purchase_request_item
	НоменклатураID	nomenclature_id	
	КоличествоПозиций	quantity	
	ЦенаПозицииОжидаемая	expected_price	
	ПриоритетПозиции	priority	
ЗаказПоставщику	НомерЗаказа	purchase_order_number	purchase_order
	ДатаСозданияЗаказа	purchase_order_created_date	
	СтатусЗаказа	purchase_order_status	
	ПоставщикID	supplier_id	

	МенеджерID	purchase manager id	
	СуммаИтоговая	total amount	
	ОжидаемаяДатаПоставки	expected_delivery_date	
Поставщик	Наименование	supplier name	supplier
	ИНН	inn	
	КПП	kpp	
	ЮридическийАдрес	legal_address	
	КонтактноеЛицо	contact_person	
	ТелефонПоставщика	supplier phone	
	EmailПоставщика	supplier_email	
	РейтингПоставщика	rating	
Номенклатура	АртикулТовара	article	nomenclature
	НаименованиеТовара	nomenclature_name	
	КатегорияТовара	nomenclature_category	
	ЕдиницаИзмерения	unit	
	НормативныйСрокСлужбы	standard_service_life	
	МинОстаток	min stock level	
	Производитель	manufacturer	
ЕдиницаИнвентаря	Штрихкод	barcode	inventory_item
	НоменклатураID	nomenclature_id	
	СерийныйНомер	serial number	
	СтатусИнвентаря	inventory_status	
	СкладID	warehouse_id	
	ДатаПоступления	receipt date	
	ДатаСписания	write off date	
	СтоимостьЗакупки	purchase cost	
	РазмерИнвентаря	size	
	ПримечаниеКИнвентарю	inventory_notes	
ВыдачаИнвентаря	ЕдиницаИнвентаряID	inventory item id	issuance_document
	СпортсменID	athlete id	
	КладовщикID	storekeeper id	
	ДатаВыдачи	issuance_date	
	ПлановаяДатаВозврата	planned_return_date	
	ФактическаяДатаВозврата	actual_return_date	
	СтатусВыдачи	issuance_document_status	
	СостояниеПриВыдаче	condition on issue	
	СостояниеПриВозврате	condition_on_return	
ПриходнаяНакладная	НомерНакладной	receipt invoice number	receipt_invoice
	ДатаПрихода	receipt invoice date	
	ЗаказID	order id	
	КладовщикID	storekeeper id	
	СтатусНакладной	receipt invoice status	
	СуммаФактическая	actual amount	

5.2.4. ER-диаграмма

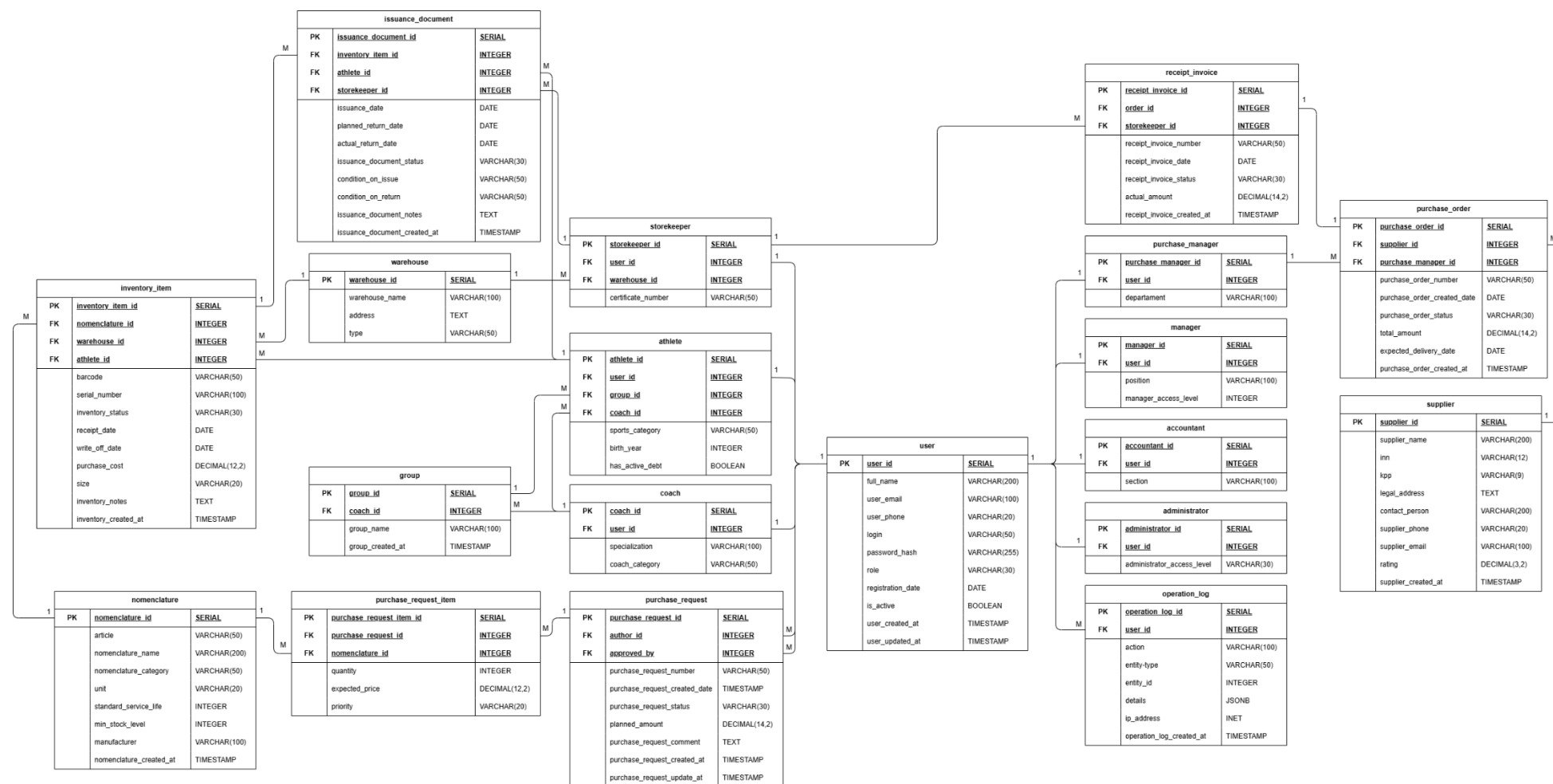


Рисунок 5.1 – Графическое представление ER-диаграммы

5.3. Выбор реляционной СУБД

5.3.1. Рассмотренные СУБД

Для создания базы данных АСУ МТО лыжной организации были рассмотрены три распространенные реляционные СУБД:

Таблица 5.38 – Рассмотренные СУБД

СУБД	Разработчик	Лицензия	Версия SQL
PostgreSQL	PostgreSQL Global Development Group	Открытая (PostgreSQL License)	SQL:2016 + расширения
MySQL	Oracle Corporation	GPL / Коммерческая	SQL:2003 + расширения
Microsoft SQL Server	Microsoft	Проприетарная / Express (бесплатно)	T-SQL (SQL Server 2022)

5.3.2. Критерии сравнения

Таблица 5.39 – Сравнительный анализ СУБД

Критерий	PostgreSQL	MySQL	Microsoft SQL Server
Соответствие стандартам SQL	Высокое (SQL:2016)	Среднее (SQL:2003)	T-SQL (диалект)
Производительность OLTP	Отличная	Хорошая	Отличная
Поддержка JSON/JSONB	Отличная (JSONB с индексами)	Хорошая (JSON)	Хорошая
Репликация и кластеризация	Встроенная, гибкая	Встроенная, ограниченная	Always On (Enterprise)
Работа с геоданными (PostGIS)	Отличная	Базовая	Средняя
Импортозамещение (реестр РФ)	Да (Postgres Pro)	Нет	Нет
Стоимость	Бесплатно / Платно (Postgres Pro)	Бесплатно (Community) / Платно (Enterprise)	Платно / Express (ограничена)
Сообщество и документация	Активное, много русскоязычных ресурсов	Огромное	Хорошее
Кроссплатформенность	Linux, Windows, macOS	Linux, Windows, macOS	Windows, Linux (ограничено)

5.3.3. Преимущества и недостатки

5.3.3.1. PostgreSQL

Преимущества:

- Полное соответствие стандартам SQL
- JSONB с поддержкой индексов GIN
- PostGIS для работы с гео данными
- Полнотекстовый поиск «из коробки»
- Поддержка оконных функций, CTE, рекурсивных запросов
- Есть российский сертифицированный форк (Postgres Pro)
- Бесплатная, открытая лицензия

Недостатки:

- Более высокие требования к администрированию
- Исторически немного медленнее MySQL на простых SELECT
- Меньшее количество GUI-инструментов (но DBeaver решает)

5.3.3.2. MySQL

Преимущества:

- Высокая скорость на простых запросах
- Огромное сообщество
- Простота настройки
- Репликация «из коробки»

Недостатки:

- Нет в реестре отечественного ПО
- Отклонения от стандартов SQL
- Ограниченная поддержка JSON (без индексации)
- Нет полноценной поддержки гео данных
- Менее строгая проверка целостности данных

5.3.3.3. Microsoft SQL Server

Преимущества:

- Отличная интеграция с Windows и .NET
- Мощный T-SQL с множеством расширений
- SQL Server Management Studio (удобный GUI)
- Always On Availability Groups
- Встроенные средства аналитики (SSAS, SSIS, SSRS)

Недостатки:

- Проприетарная лицензия (дорого)
- Основная платформа — Windows
- Ограниченная кроссплатформенность
- Нет в реестре отечественного ПО
- Высокие системные требования

5.3.4. Обоснование выбора СУБД

Выбрана СУБД: PostgreSQL 17.5

Обоснование:

1. Импортозамещение — PostgreSQL входит в реестр отечественного ПО (Postgres Pro), что соответствует требованиям к государственным и бюджетным организациям (спортивные школы, федерации)
2. Соответствие архитектуре проекта — согласно требованиям используется PostgreSQL как основная СУБД
3. Поддержка JSONB — поле details JSONB в таблице operation_logs позволяет гибко хранить данные аудита
4. Кроссплатформенность — возможность развертывания на Astra Linux (серверная ОС)
5. Расширяемость — возможность подключения PostGIS для учета географии складов и мест проведения соревнований

6. Стандарты SQL — строгое следование стандартам упрощает миграцию и поддержку
7. Бесплатность — отсутствие лицензионных отчислений

5.4. Создание базы данных в выбранной СУБД и заполнение данными

Листинги с SQL скриптами представлены в Приложение 2. SQL скрипты создания и заполнения БД. Ниже приведены результаты создания структуры БД, заполнения ее тестовыми данными и выполнение пробного запроса к БД.

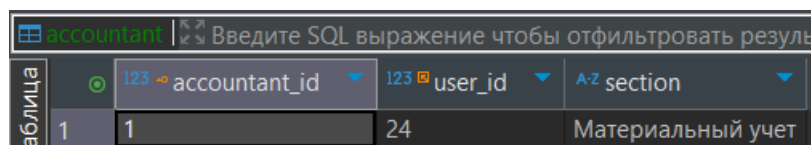
5.4.1. Создание структуры БД



Рисунок 5.2 – Структура созданной БД

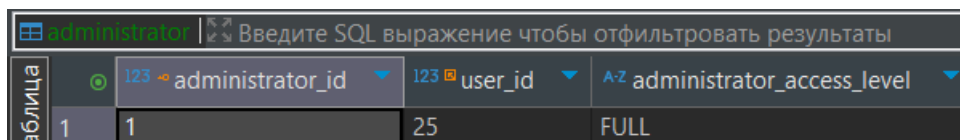
5.4.2. Заполнение базы тестовыми данными

Каждая таблица, за исключением справочных, была заполнена минимум десятью записями для наглядного отображения и возможности проведения тестовых запросов с видимыми результатами.



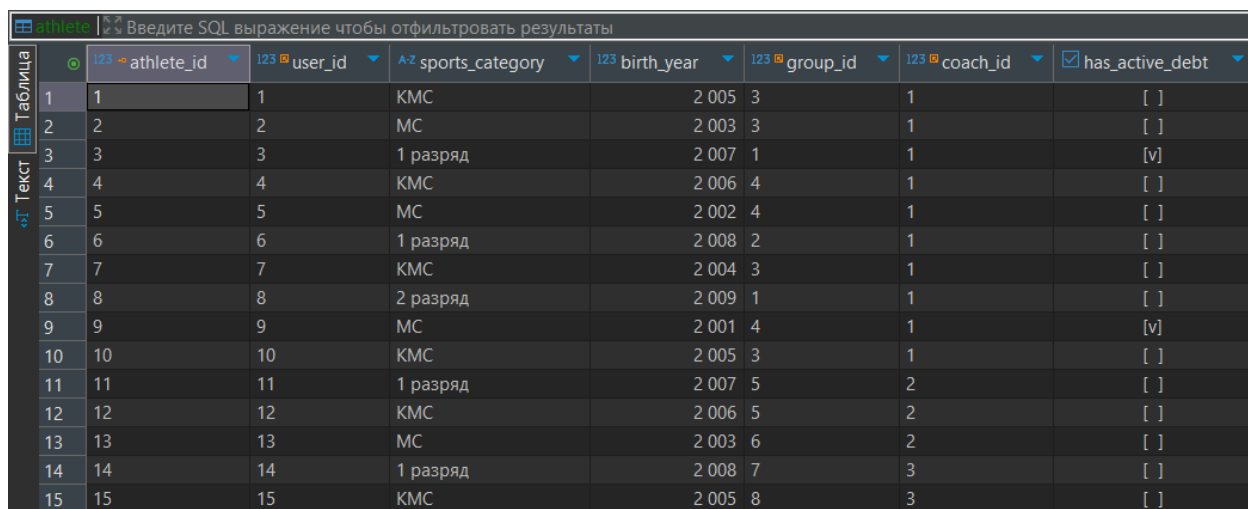
	123 accountant_id	123 user_id	A-Z section
1	1	24	Материальный учет

Рисунок 5.3 – Заполненная таблица "accountant"



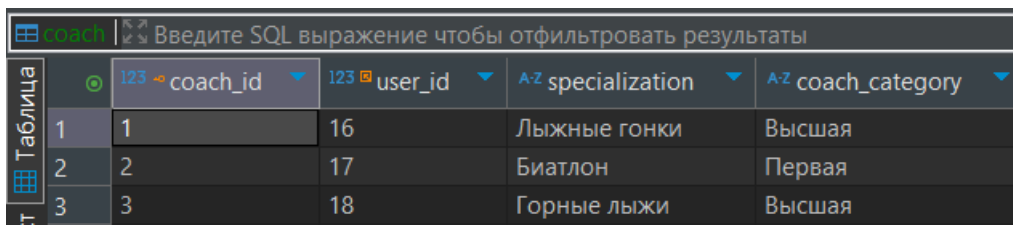
	123 administrator_id	123 user_id	A-Z administrator_access_level
1	1	25	FULL

Рисунок 5.4 – Заполненная таблица "administrator"



	123 athlete_id	123 user_id	A-Z sports_category	123 birth_year	123 group_id	123 coach_id	☒ has_active_debt
1	1	1	КМС	2 005	3	1	[]
2	2	2	МС	2 003	3	1	[]
3	3	3	1 разряд	2 007	1	1	[v]
4	4	4	КМС	2 006	4	1	[]
5	5	5	МС	2 002	4	1	[]
6	6	6	1 разряд	2 008	2	1	[]
7	7	7	КМС	2 004	3	1	[]
8	8	8	2 разряд	2 009	1	1	[]
9	9	9	МС	2 001	4	1	[v]
10	10	10	КМС	2 005	3	1	[]
11	11	11	1 разряд	2 007	5	2	[]
12	12	12	КМС	2 006	5	2	[]
13	13	13	МС	2 003	6	2	[]
14	14	14	1 разряд	2 008	7	3	[]
15	15	15	КМС	2 005	8	3	[]

Рисунок 5.5 – Заполненная таблица "athlete"



	123 coach_id	123 user_id	A-Z specialization	A-Z coach_category
1	1	16	Лыжные гонки	Высшая
2	2	17	Биатлон	Первая
3	3	18	Горные лыжи	Высшая

Рисунок 5.6 – Заполненная таблица "coach"

group Введите SQL выражение чтобы отфильтровать результаты				
Таблица	123 group_id	A-Z group_name	123 coach_id	group_created_at
1	1	Группа начальной подготовки №1	1	2024-09-01 00:00:00.000
2	2	Группа начальной подготовки №2	1	2024-09-01 00:00:00.000
3	3	Группа спортивного совершенствования	1	2024-09-01 00:00:00.000
4	4	Юниорская сборная (лыжные гонки)	1	2024-09-01 00:00:00.000
5	5	Биатлонная группа №1	2	2024-09-01 00:00:00.000
6	6	Биатлонная группа №2	2	2024-09-01 00:00:00.000
7	7	Горнолыжная группа "Слалом"	3	2024-09-01 00:00:00.000
8	8	Горнолыжная группа "Гигант"	3	2024-09-01 00:00:00.000
9	9	Спортивно-оздоровительная группа	1	2024-10-01 00:00:00.000
10	10	Группа высшего спортивного мастерства	1	2024-10-01 00:00:00.000

Рисунок 5.7 – Заполненная таблица "group"

Введите SQL выражение чтобы отфильтровать результаты										
Таблица	123 barcode	123 serial_number	A-Z inventory_status	123 receipt_date	write_off_date	123 purchase_cost	A-Z size	123 athlete_id	A-Z inventory_notes	inventory_created_at
1	SKI001-2024-001	1	FS-2024-001	AVAILABLE	1	2024-10-01	[NULL]	45 000 190	[NULL]	Новые
2	SKI001-2024-002	1	FS-2024-002	ISSUED	1	2024-10-01	[NULL]	45 000 195	1	Выданы Иванову
3	SKI001-2024-003	1	FS-2024-003	AVAILABLE	1	2024-10-01	[NULL]	45 000 200	[NULL]	Новые
4	SKI001-2024-004	1	FS-2024-004	RESERVED	1	2024-10-01	[NULL]	45 000 192	[NULL]	Зарезервированы для сборов
5	SKI001-2024-005	1	FS-2024-005	ISSUED	1	2024-10-01	[NULL]	45 000 198	2	Выданы Петрову
6	SKI002-2024-001	2	MR-2024-001	AVAILABLE	1	2024-10-15	[NULL]	50 000 192	[NULL]	
7	SKI002-2024-002	2	MR-2024-002	ISSUED	1	2024-10-15	[NULL]	50 000 198	4	Выданы Смирновой
8	SKI002-2024-003	2	MR-2024-003	UNDER REPAIR	2	2024-10-15	[NULL]	50 000 195	[NULL]	Ремонт скользящей поверхности
9	BOOT001-2024-001	6	SL-2024-001	AVAILABLE	1	2024-11-01	[NULL]	25 000 42	[NULL]	
10	BOOT001-2024-002	6	SL-2024-002	ISSUED	1	2024-11-01	[NULL]	25 000 43	1	Выданы Иванову
11	BOOT001-2024-003	6	SL-2024-003	AVAILABLE	1	2024-11-01	[NULL]	25 000 44	[NULL]	
12	BOOT001-2024-004	6	SL-2024-004	ISSUED	1	2024-11-01	[NULL]	25 000 41	5	Выданы Кузнецову
13	POLE001-2024-001	10	SW-2024-001	AVAILABLE	1	2024-11-15	[NULL]	8 000 155	[NULL]	
14	POLE001-2024-002	10	SW-2024-002	ISSUED	1	2024-11-15	[NULL]	8 000 160	2	Выданы Петрову
15	POLE001-2024-003	10	SW-2024-003	DEFECTIVE	1	2024-11-15	[NULL]	8 000 150	[NULL]	Трещина на древке
16	WAX001-2024-001	13	WAX-LF4-001	AVAILABLE	1	2024-12-01	[NULL]	500 [NULL]	[NULL]	
17	WAX001-2024-002	13	WAX-LF4-002	AVAILABLE	1	2024-12-01	[NULL]	500 [NULL]	[NULL]	
18	WAX002-2024-001	14	WAX-HF6-001	AVAILABLE	1	2024-12-01	[NULL]	600 [NULL]	[NULL]	
19	WAX002-2024-002	14	WAX-HF6-002	ISSUED	1	2024-12-01	[NULL]	600 [NULL]	3	Выданы сервисной бригаде
20	WAX003-2024-001	15	WAX-FC7-001	AVAILABLE	1	2024-12-01	[NULL]	1 200 [NULL]	[NULL]	

Рисунок 5.8 – Заполненная таблица "inventory_item"

Введите SQL выражение чтобы отфильтровать результаты												
fact	123	123	123	issuance_date	planned_return_date	actual_return_date	A-Z issuance_s	A-Z condi	A-Z condition_on_retr	A-Z issuance_document_notes	issuance_document_created_at	
1	1	2	1	2024-12-01	2025-03-30	[NULL]	ACTIVE	NEW	[NULL]	На сезон 2024-2025	2026-05-10 17:38:43.673	
2	2	5	2	1	2024-12-05	2025-03-30	[NULL]	ACTIVE	NEW	[NULL]	На сезон 2024-2025	2026-05-10 17:38:43.673
3	3	7	4	2	2024-12-10	2025-03-30	[NULL]	ACTIVE	NEW	[NULL]	На сезон 2024-2025	2026-05-10 17:38:43.673
4	4	12	5	1	2024-12-15	2025-03-30	[NULL]	ACTIVE	NEW	[NULL]	На сезон 2024-2025	2026-05-10 17:38:43.673
5	5	10	6	2	2025-01-10	2025-04-15	[NULL]	ACTIVE	NEW	[NULL]	Выданы Поповой	2026-05-10 17:38:43.673
6	6	3	7	1	2024-12-01	2024-11-28	RETURNED	NEW	GOOD	[NULL]	2026-05-10 17:38:43.673	
7	7	4	8	2	2024-11-05	2024-12-05	2024-12-01	RETURNED	NEW	GOOD	2026-05-10 17:38:43.673	
8	8	6	9	1	2024-11-10	2024-12-10	2024-12-05	RETURNED	NEW	MINOR_SCRATCHES	Небольшие царапины	2026-05-10 17:38:43.673
9	9	8	10	2	2024-11-15	2024-12-15	2024-12-10	RETURNED	NEW	GOOD	[NULL]	2026-05-10 17:38:43.673
10	10	11	11	1	2024-11-20	2024-12-20	2024-12-18	RETURNED	NEW	GOOD	[NULL]	2026-05-10 17:38:43.673
11	11	9	3	1	2024-10-01	2024-11-01	[NULL]	OVERDUE	USED	[NULL]	Просрочен возврат	2026-05-10 17:38:43.673
12	12	13	12	2	2024-10-15	2024-11-15	[NULL]	OVERDUE	USED	[NULL]	Просрочен возврат	2026-05-10 17:38:43.673
13	13	1	13	1	2024-11-01	2024-12-01	[NULL]	OVERDUE	USED	[NULL]	Просрочен возврат	2026-05-10 17:38:43.673
14	14	14	14	2	2024-11-01	2025-01-01	[NULL]	EXTENDED	NEW	[NULL]	Продлено до 01.02.2025	2026-05-10 17:38:43.673
15	15	15	15	1	2024-11-15	2025-01-15	[NULL]	EXTENDED	NEW	[NULL]	Продлено до 15.02.2025	2026-05-10 17:38:43.673

Рисунок 5.9 – Заполненная таблица "issuance_document"

manager Введите SQL выражение чтобы отфильтровать результаты				
Таблица	123 manager_id	123 user_id	A-Z position	123 manager_access_level
1	1	23	Директор спортивной школы	3

Рисунок 5.10 – Заполненная таблица "manager"

Введите SQL выражение чтобы отфильтровать результаты										
		nomenclature_id	article	nomenclature_name	nomenclature_category	unit	standard_service_life	min_stock_level	manufacturer	nomenclature_created_at
Текст Таблица	1	SKI-001	Лыжи гоночные Fischer Speedmax	skis	pair	24	5	Fischer	2026-05-10 17:38:43.628	
	2	SKI-002	Лыжи гоночные Madshus Redline	skis	pair	24	5	Madshus	2026-05-10 17:38:43.628	
	3	SKI-003	Лыжи тренировочные Fischer RCS	skis	pair	18	10	Fischer	2026-05-10 17:38:43.628	
	4	SKI-004	Лыжи горные Atomic Redster	skis	pair	24	3	Atomic	2026-05-10 17:38:43.628	
	5	SKI-005	Лыжи для классического хода Rossignol	skis	pair	24	5	Rossignol	2026-05-10 17:38:43.628	
	6	BOOT-001	Ботинки лыжные Salomon S/Lab	boots	pair	18	8	Salomon	2026-05-10 17:38:43.628	
	7	BOOT-002	Ботинки лыжные Fischer Carbonlite	boots	pair	18	8	Fischer	2026-05-10 17:38:43.628	
	8	BOOT-003	Ботинки тренировочные Madshus	boots	pair	12	12	Madshus	2026-05-10 17:38:43.628	
	9	BOOT-004	Ботинки горнолыжные Atomic Hawx	boots	pair	24	5	Atomic	2026-05-10 17:38:43.628	
	10	POLE-001	Палки лыжные Swix Triac	poles	pair	12	10	Swix	2026-05-10 17:38:43.628	
	11	POLE-002	Палки лыжные Leki HRC	poles	pair	12	10	Leki	2026-05-10 17:38:43.628	
	12	POLE-003	Палки горнолыжные Komperdell	poles	pair	12	5	Komperdell	2026-05-10 17:38:43.628	
	13	WAX-001	Парафин Swix LF4	wax	pcs	6	20	Swix	2026-05-10 17:38:43.628	
	14	WAX-002	Парафин Swix HF6	wax	pcs	6	15	Swix	2026-05-10 17:38:43.628	
	15	WAX-003	Порошок ускоритель Swix FC7	wax	pcs	12	10	Swix	2026-05-10 17:38:43.628	

Рисунок 5.11 – Заполненная таблица "nomenclature"

		123 * operation_log_id	123 user_id	A-Z action	A-Z entity_type	123 entity_id	() details	ip_address	operation_log_created_at
1	1	25	25	CREATE_USER	user	1	("role": "athlete", "created_by": "admin")	192.168.1.100	2024-09-01 09:00:00.000
2	2	25	25	CREATE_USER	user	16	("role": "coach", "created_by": "admin")	192.168.1.100	2024-09-01 09:05:00.000
3	3	16	16	CREATE_REQUEST	purchase_request	1	("amount": 250000)	192.168.1.101	2026-01-10 09:00:00.000
4	4	23	23	APPROVE_REQUEST	purchase_request	1	("approved": true)	192.168.1.104	2026-01-11 10:00:00.000
5	5	21	21	CREATE_ORDER	purchase_order	1	("total": 825000, "supplier_id": 3)	192.168.1.103	2026-01-12 11:00:00.000
6	6	19	19	RECEIVE_GOODS	receipt_invoice	1	("amount": 825000, "order_id": 1)	192.168.1.102	2024-10-01 14:00:00.000
7	7	19	19	ISSUE_INVENTORY	issuance_document	1	("barcode": "SKI001-2024-002", "athlete_id": 1)	192.168.1.102	2024-12-01 10:00:00.000
8	8	19	19	ISSUE_INVENTORY	issuance_document	2	("barcode": "SKI001-2024-005", "athlete_id": 2)	192.168.1.102	2024-12-05 10:30:00.000
9	9	20	20	ISSUE_INVENTORY	issuance_document	3	("barcode": "SKI002-2024-002", "athlete_id": 4)	192.168.1.105	2024-12-10 11:00:00.000
10	10	19	19	RETURN_INVENTORY	issuance_document	6	("condition": "GOOD")	192.168.1.102	2024-11-28 15:00:00.000
11	11	17	17	CREATE_REQUEST	purchase_request	2	("amount": 150000)	192.168.1.106	2026-01-15 10:30:00.000
12	12	22	22	CREATE_ORDER	purchase_order	2	("total": 160000, "supplier_id": 4)	192.168.1.107	2026-01-18 09:00:00.000
13	13	24	24	BLOCK_USER	user	3	("reason": "debt")	192.168.1.100	2024-11-02 08:00:00.000
14	14	25	25	EXPORT_REPORT	report	[NULL]	("type": "OSV", "period": "2024-Q4")	192.168.1.108	2025-01-10 16:00:00.000
15	15	23	23	VIEW_DASHBOARD	dashboard	[NULL]	("section": "analytics")	192.168.1.104	2025-01-15 09:00:00.000

Рисунок 5.12 – Заполненная таблица "operation_log"

		123 * purchase_manager_id	123 user_id	A-Z department
1	1	21		Отдел снабжения
2	2	22		Отдел закупок спортивного инвентаря

Рисунок 5.13 – Заполненная таблица "purchase_manager"

		123 * purchase_order_id	A-Z purchase_order_number	A-Z purchase_order_status	123 supplier_id	123 purchase_manager_id	123 total_amount	expected_delivery_date	purchase_order_created_at
1	1		ЗАКАЗ-2026-001	2026-01-12 SENT	3	1	825 000	2026-08-15	2026-05-10 17:38:43.651
2	2		ЗАКАЗ-2026-002	2026-01-18 DELIVERED	4	2	160 000	2026-02-01	2026-05-10 17:38:43.651
3	3		ЗАКАЗ-2026-003	2026-02-05 IN_TRANSIT	1	1	250 000	2026-03-10	2026-05-10 17:38:43.651
4	4		ЗАКАЗ-2026-004	2026-02-12 SENT	2	2	180 000	2026-09-01	2026-05-10 17:38:43.651
5	5		ЗАКАЗ-2026-005	2026-02-20 DRAFT	5	1	95 000	[NULL]	2026-05-10 17:38:43.651
6	6		ЗАКАЗ-2026-006	2026-03-01 DELIVERED	6	1	120 000	2026-03-20	2026-05-10 17:38:43.651
7	7		ЗАКАЗ-2026-007	2026-03-10 SENT	3	2	450 000	2026-10-01	2026-05-10 17:38:43.651
8	8		ЗАКАЗ-2026-008	2026-03-15 CLOSED	4	1	75 000	2026-03-25	2026-05-10 17:38:43.651
9	9		ЗАКАЗ-2026-009	2026-03-20 IN_TRANSIT	8	2	210 000	2026-04-15	2026-05-10 17:38:43.651
10	10		ЗАКАЗ-2026-010	2026-03-25 SENT	11	1	135 000	2026-08-20	2026-05-10 17:38:43.651

Рисунок 5.14 – Заполненная таблица "purchase_order"

		123 * purchase_request_item_id	123 purchase_request_id	123 nomenclature_id	123 quantity	123 expected_price	A-Z priority
1	1	1	1	1	10	45 000	HIGH
2	2	1	1	6	15	25 000	HIGH
3	3	1	1	10	20	8 000	MEDIUM
4	4	2	2	2	5	50 000	HIGH
5	5	2	2	7	10	28 000	MEDIUM
6	6	3	3	13	50	500	LOW
7	7	3	3	14	30	600	LOW
8	8	4	4	6	12	25 000	HIGH
9	9	4	4	8	10	12 000	MEDIUM
10	10	5	5	10	25	8 000	MEDIUM
11	11	6	6	5	8	52 000	HIGH
12	12	7	7	3	15	35 000	HIGH
13	13	8	8	15	20	1 200	LOW
14	14	9	9	1	20	45 000	HIGH
15	15	10	10	4	6	48 000	HIGH

Рисунок 5.15 – Заполненная таблица "purchase_request_item"

warehouse Введите SQL выражение чтобы отфильтровать результаты				
Таблица	warehouse_id	warehouse_name	address	type
1	1	Основной склад	г. Москва, ул. Лыжная, д. 1	main
2	2	Склад сервисного центра	г. Москва, ул. Техническая, д. 5	service
3	3	Резервный склад	г. Химки, ул. Запасная, д. 10	reserve
4	4	Склад лыжной базы "Снежинка"	г. Москва, Ленинградское шоссе, 45	main
5	5	Склад соревновательного инвентаря	г. Москва, ул. Спортивная, 12	main
6	6	Склад расходных материалов	г. Москва, ул. Складская, 3	service
7	7	Склад запасных частей	г. Химки, Вашутинское шоссе, 8	service
8	8	Склад сезонного хранения	г. Долгопрудный, ул. Лесная, 20	reserve
9	9	Склад экипировки сборной	г. Москва, Олимпийский пр-т, 5	main
10	10	Склад утилизации	г. Мытищи, ул. Промышленная, 7	reserve

Рисунок 5.21 – Заполненная таблица "warehouse"

5.4.3. Тестовый запрос на выборку

В тестовом запросе к БД запрашивались данные спортсменов-должников (с просроченным инвентарем) и сопутствующих параметров.

Введите SQL выражение чтобы отфильтровать результаты						
athlete_name	email	phone	sports_category	group_name	coach_name	overdue_items_count
Иванов Иван Иванович	ivanov@ski.ru	+79161234567	КМС	Группа спортивного совершенствования	Тренер Алексей Юрьевич	1 SK001-2024-002 (Лыжи гоночные Fischer Speedmax, просрочен на 406 дн.)
Кузнецов Дмитрий Алекс	kuznetsov@ski.ru	+79165678901	МС	Юниорская сборная (лыжные гонки)	Тренер Алексей Юрьевич	1 BOOT001-2024-004 (Ботинки лыжные Salomon S/Lab, просрочен на 406 дн.)
Лебедев Андрей Викторов	lebedev@ski.ru	+79163456780	МС	Биатлонная группа №2	Наставник Игорь Петрович	1 SK001-2024-001 (Лыжи гоночные Fischer Speedmax, просрочен на 525 дн.)
Орлова Татьяна Павловна	orlova@ski.ru	+79162345679	КМС	Биатлонная группа №1	Наставник Игорь Петрович	1 POLE001-2024-001 (Палки лыжные Swix Triac, просрочен на 541 дн.)
Петров Петр Петрович	petrov@ski.ru	+79162345678	МС	Группа спортивного совершенствования	Тренер Алексей Юрьевич	1 SK001-2024-005 (Лыжи гоночные Fischer Speedmax, просрочен на 406 дн.)
Попова Елена Викторовна	popova@ski.ru	+79166789012	1 разряд	Группа начальной подготовки №2	Тренер Алексей Юрьевич	1 BOOT001-2024-002 (Ботинки лыжные Salomon S/Lab, просрочен на 390 дн.)
Сидоров Сидор Сидорович	sidorov@ski.ru	+79163456789	1 разряд	Группа начальной подготовки №1	Тренер Алексей Юрьевич	1 BOOT001-2024-001 (Ботинки лыжные Salomon S/Lab, просрочен на 555 дн.)
Смирнова Анна Сергеевна	smirnova@ski.ru	+79164567890	КМС	Юниорская сборная (лыжные гонки)	Тренер Алексей Юрьевич	1 SK002-2024-002 (Лыжи гоночные Madshus Redline, просрочен на 406 дн.)

Рисунок 5.22 – Результат выполнения запроса

6. Схема сети для проектируемой АСУ

6.1. Расчет характеристик оборудования

6.1.1. Серверное оборудование

6.1.1.1. Сервер баз данных и приложений (основной)

В соответствии с архитектурой, сервер баз данных (PostgreSQL) и сервер приложений (Spring Boot / .NET Core), включая веб-сервер (Nginx + React), разворачиваются на одном физическом сервере с использованием контейнеризации (Docker Compose). Это оптимально для масштаба организации (50 пользователей) и позволяет эффективно использовать аппаратные ресурсы.

6.1.1.1.1. Исходные данные

СУБД: PostgreSQL 17 / Postgres Pro Standard 17.

Сервер приложений: Java 21 / Spring Boot 3.4 (или .NET Core 8).

Веб-сервер: Nginx (Reverse Proxy + раздача статики React SPA).

ОС: Astra Linux Special Edition 1.7.

Контейнеризация: Docker Compose (4 контейнера: БД, бэкенд, фронтенд, Nginx).

Количество таблиц БД: 19.

Объём данных БД: ~10 МБ (данные + индексы).

Одновременных пользователей: 50 (нефункциональное требование №1).

Активных соединений с БД: 20–30.

6.1.1.1.2. Расчёт оперативной памяти (RAM)

Таблица 6.1 – Расчёт оперативной памяти (RAM)

Компонент	Расчёт	Значение
ОС (Astra Linux)	Базовая потребность	2 ГБ
СУБД PostgreSQL		
shared_buffers	25% от 8 ГБ	2 ГБ

work mem	4 МБ × 20 соединений	0.08 ГБ
maintenance work mem	Для VACUUM	0.25 ГБ
wal_buffers + процессы	Буфер + postmaster	0.1 ГБ
Сервер приложений (JVM)		
Heap (Xmx)	Бизнес-логика 50 пользователей	2 ГБ
Metaspace + Threads + GC	Метаданные и потоки	0.5 ГБ
Веб-сервер Nginx		
Workers + буферы + кэш	2 workers, прокси	0.2 ГБ
Дополнительно		
Docker, мониторинг, утилиты	Агенты, демоны	0.5 ГБ
Запас (≈20%)	Пиковые нагрузки	1.4 ГБ

Итоговый расчёт:

$$2 + 2 + 0.08 + 0.25 + 0.1 + 2 + 0.5 + 0.2 + 0.5 + 1.4 = 9.03 \text{ ГБ.}$$

Округляем до стандартного объёма: 16 ГБ DDR4 ECC.

6.1.1.1.3. Расчёт процессора (CPU)

Таблица 6.2 – Расчёт процессора (CPU)

Компонент	Расчёт	Значение
PostgreSQL (20 активных соединений)	1 ядро на 10-15 соединений	2 ядра
Фоновые процессы БД + параллельные запросы	autovacuum, WAL, workers	1 ядро
Сервер приложений (HTTP-запросы)	50 пользователей	3 ядра
Бизнес-логика + JIT/GC	JVM	2 ядра
Nginx (соединения + SSL)	50 соединений, TLS	1 ядро
Системные процессы ОС	Ядро Linux	1 ядро
Резерв (≈20%)		2 ядра

Итоговый расчёт:

$$2 + 1 + 3 + 2 + 1 + 1 + 2 = 12 \text{ ядер.}$$

Оптимальный выбор: 12 ядер / 24 потока (Intel Xeon Silver 4310).

6.1.1.1.4. Расчёт дисковой подсистемы

Таблица 6.3 – Расчёт дисковой подсистемы

Компонент	Расчёт	Значение
ОС + Docker + утилиты		30 ГБ
БД (данные + индексы + рост 5 лет)	10 МБ × 60 мес. × 2	1.2 ГБ
WAL-файлы PostgreSQL	16 МБ × 64	1 ГБ
Логи (30 дней)	100 МБ/день	3 ГБ
Исполняемые файлы (JAR, образы)		5 ГБ
Свободное пространство (30%)		17 ГБ

Итого		$\approx 57 \text{ ГБ} \rightarrow 240 \text{ ГБ SSD}$
-------	--	--

RAID: 1 (зеркалирование $2 \times 240 \text{ ГБ SSD NVMe}$).

6.1.1.1.5. Выбранная конфигурация

Таблица 6.4 – Выбранная конфигурация

Параметр	Значение
Модель сервера	DEPO Storm 2400 / Aquarius Server T40
Процессор	$1 \times \text{Intel Xeon Silver 4310}$ (12 ядер, 2.1 ГГц, 24 потока)
Оперативная память	16 ГБ DDR4 ECC Registered ($2 \times 8 \text{ ГБ}$)
Дисковая подсистема	$2 \times 240 \text{ ГБ SSD NVMe}$, RAID 1
Сетевой интерфейс	$2 \times 1\text{GbE RJ45}$
Блок питания	$2 \times 550 \text{ Вт}$ (горячая замена)
Форм-фактор	1U Rackmount
Размещение ПО	Docker Compose (PostgreSQL, Spring Boot, Nginx+React)

6.1.1.2. Сервер аналитики и прогнозирования

6.1.1.2.1. Исходные данные

ПО: Python 3.12, statsmodels (ПП6), ClickHouse (OLAP), Redash/Superset (BI).

ОС: Astra Linux Special Edition 1.7.

Задача: Прогнозирование потребности (Холт-Уинтерс) для 15 позиций, параллельно до 3 моделей.

Частота обучения: еженедельно.

6.1.1.2.2. Расчёт оперативной памяти (RAM)

Таблица 6.5 – Расчёт оперативной памяти (RAM)

Компонент	Расчёт	Значение
ОС (Astra Linux)		1 ГБ
Python + pandas (данные)	$0.5 + 0.6 \text{ ГБ}$	1.1 ГБ
Обучение моделей (3 параллельно)	$3 \times 0.5 \text{ ГБ}$	1.5 ГБ
ClickHouse (OLAP)		2 ГБ
Redash/Superset (BI)		0.5 ГБ
Системные утилиты, мониторинг		0.5 ГБ
Запас ($\approx 20\%$)		1.4 ГБ

Итоговый расчёт:

$$1 + 1.1 + 1.5 + 2 + 0.5 + 0.5 + 1.4 = 8.0 \text{ ГБ.}$$

Округляем до: 16 ГБ DDR4 ECC (для уверенной работы с запасом).

6.1.1.2.3. Расчёт процессора (CPU)

Таблица 6.6 – Расчёт процессора (CPU)

Компонент	Расчёт	Значение
Обучение 3 моделей statsmodels	3×2 ядра	6 ядер
Предобработка данных (pandas)		1 ядро
ClickHouse (OLAP-запросы)		1 ядро
BI-платформа (Redash/Superset)		1 ядро
Системные процессы ОС		1 ядро
Резерв ($\approx 20\%$)		2 ядра

Итоговый расчёт:

$$6 + 1 + 1 + 1 + 1 + 2 = 12 \text{ ядер}$$

Оптимальный выбор: 12 ядер / 24 потока (Intel Xeon Silver 4310).

6.1.1.2.4. Расчёт дисковой подсистемы

Таблица 6.7 – Расчёт дисковой подсистемы

Компонент	Значение
ОС + Python-окружение	23 ГБ
Исторические данные (5 лет)	5 ГБ
ClickHouse (данные + индексы)	10 ГБ
Модели, графики, отчёты	6 ГБ
Логи (90 дней)	2 ГБ
Свободное пространство (30%)	20 ГБ
Итого	$\approx 66 \text{ ГБ} \rightarrow 240 \text{ ГБ SSD}$

6.1.1.2.5. Выбранная конфигурация

Таблица 6.8 – Выбранная конфигурация

Параметр	Значение
Модель сервера	DEPO Storm 2400 / Aquarius Server T40
Процессор	1 × Intel Xeon Silver 4310 (12 ядер, 2.1 ГГц, 24 потока)
Оперативная память	16 ГБ DDR4 ECC Registered (2 × 8 ГБ)
Дисковая подсистема	2 × 240 ГБ SSD NVMe, RAID 1
Сетевой интерфейс	2 × 1GbE RJ45
Блок питания	2 × 550 Вт (горячая замена)
Форм-фактор	1U Rackmount

6.1.1.3. Сервер резервного копирования

6.1.1.3.1. Исходные данные

ПО: Bacula / Veeam Backup & Replication. ОС: Astra Linux SE 1.7.

Источники: Основной сервер (БД, конфигу, логи), сервер аналитики (модели, данные).

Политика хранения: Ежедневно (30), Еженедельно (12), Ежемесячно (12), Ежегодно (5).

6.1.1.3.2. Расчёт дискового пространства

Ежедневные бэкапы (полный + инкременты):

- Основной сервер: $1 \times 3.1 \text{ ГБ} + 29 \times 0.3 \text{ ГБ} = 11.8 \text{ ГБ}$
- Сервер аналитики: $1 \times 5.8 \text{ ГБ} + 29 \times 0.2 \text{ ГБ} = 11.6 \text{ ГБ}$

Еженедельные, ежемесячные (полные копии, по 12 версий):

- Основной: $12 \times 3.1 \text{ ГБ} = 37.2 \text{ ГБ} \times 2 = 74.4 \text{ ГБ}$
- Аналитика: $12 \times 5.8 \text{ ГБ} = 69.6 \text{ ГБ} \times 2 = 139.2 \text{ ГБ}$

Ежегодные бэкапы (полные копии, 5 версий):

- Основной: $5 \times 3.1 \text{ ГБ} = 15.5 \text{ ГБ}$
- Аналитика: $5 \times 5.8 \text{ ГБ} = 29.0 \text{ ГБ}$

Суммарный объём:

$$11.8 + 11.6 + 74.4 + 139.2 + 15.5 + 29.0 = 281.5 \text{ ГБ.}$$

С запасом на рост данных 200%: $281.5 \times 3 \approx 845 \text{ ГБ.}$

Рекомендуемая полезная ёмкость: 2 ТБ (RAID 1, $2 \times 2 \text{ ТБ HDD}$).

6.1.1.3.3. Расчёт RAM

- ОС (Astra Linux): 1 ГБ
- Процессы Bacula/Veeam: Director, Storage Daemon, File Daemon — 0.5 ГБ
- Каталог базы данных Bacula (SQLite/PostgreSQL): 0.3 ГБ

- Кэширование при операциях бэкапа: 0.5 ГБ
- Запас: 0.7 ГБ

ИТОГО: $1 + 0.5 + 0.3 + 0.5 + 0.7 = 3$ ГБ. Приводим к стандартным 8 ГБ.

6.1.1.3.4. Расчёт процессора (CPU)

- Сжатие данных (gzip/lz4): 2 ядра
- Проверка целостности (SHA256/MD5): 1 ядро
- Фоновые процессы и планировщик: 1 ядро
- Резерв: 2 ядра

ИТОГО: $2 + 1 + 1 + 2 = 6$ ядер.

6.1.1.3.5. Выбранная конфигурация

Таблица 6.9 – Выбранная конфигурация

Параметр	Значение
Модель сервера	DEPO Storm 1300 / Aquarius Server T30
Процессор	1 × Intel Xeon E-2336 (6 ядер, 2.9 ГГц, 12 потоков)
Оперативная память	8 ГБ DDR4 ECC (2 × 4 ГБ)
Дисковая подсистема	2 × 2 ТБ HDD SATA, RAID 1
Сетевой интерфейс	2 × 1GbE RJ45
Блок питания	2 × 550 Вт (горячая замена)
Форм-фактор	2U Rackmount

6.1.2. Клиентское оборудование

6.1.2.1. Автоматизированные рабочие места (АРМ)

6.1.2.1.1. Исходные данные

ПО: Браузер (Chromium), офисный пакет (LibreOffice / Р7-Офис).

ОС: Astra Linux Common Edition.

Количество АРМ по ролям:

Менеджер по закупкам (2) + Кладовщик (2) + Тренер стационарный (3)
+ Руководитель (1) + Бухгалтер (1) + Администратор (1) = 10 АРМ.

6.1.2.1.2. Расчёт оперативной памяти (RAM)

- ОС (Astra Linux / Windows): 3 ГБ
- Браузер (с несколькими вкладками): 1.5 ГБ на вкладку × 5 вкладок = 7.5 ГБ
- Офисные приложения (LibreOffice / MS Office): 1 ГБ
- Антивирус и системные утилиты: 0.5 ГБ
- Запас: 2 ГБ

ИТОГО: $3 + 7.5 + 1 + 0.5 + 2 = 14$ ГБ. Приводим к стандарту 16 ГБ DDR4.

6.1.2.1.3. Расчёт процессора (CPU)

- Современный браузер (многопоточный рендеринг): рекомендуется 4 ядра
- Офисные приложения: 2 ядра
- Системные процессы: 1 ядро
- Резерв: 1 ядро

ИТОГО: $4 + 2 + 1 + 1 = 8$ ядер (Intel Core i5-12400).

6.1.2.1.4. Расчёт дисковой подсистемы

- ОС + обновления: 50 ГБ
- Офисные приложения: 10 ГБ
- Пользовательские данные (документы, отчёты): 30 ГБ
- Кэш браузера и временные файлы: 10 ГБ
- Свободное пространство (30%): 30 ГБ

ИТОГО минимальная полезная ёмкость: ~130 ГБ. Рекомендуется 256 ГБ SSD.

6.1.2.1.5. Выбранная конфигурация

Таблица 6.10 – Выбранная конфигурация

Параметр	Значение
Модель ПК	DEPO Neos 300 / Aquarius Elt E500

Процессор	Intel Core i5-12400 (6P+0E, 12 потоков, до 4.4 ГГц)
Оперативная память	16 ГБ DDR4 (2 × 8 ГБ)
Дисковая подсистема	256 ГБ SSD NVMe M.2
Монитор	23.8" Full HD, IPS
Сетевой интерфейс	1 × 1GbE RJ45
Количество	10 шт.

6.1.2.2. Мобильные устройства (планшеты)

6.1.2.2.1. Исходные данные

Пользователи: Тренеры на трассе (3 чел.), работа на открытом воздухе, низкие температуры (до -20°C), возможны осадки.

ПО: PWA-приложение АСУ (React) в браузере. ОС: Android 11+.

6.1.2.2.2. Расчёт оперативной памяти (RAM)

- ОС (Android 11+): 1.5 ГБ
- Браузер (Chromium) с PWA: 1 ГБ
- Фоновые службы (Wi-Fi, 4G, Bluetooth): 0.5 ГБ
- Запас: 1 ГБ

ИТОГО: $1.5 + 1 + 0.5 + 1 = 4$ ГБ.

6.1.2.2.3. Расчёт процессора (CPU)

- Веб-рендеринг и PWA: требуется 4-8 ядер среднего уровня
- Энергоэффективность важнее пиковой производительности.
- Рекомендуется: 8 ядер (big.LITTLE)

6.1.2.2.4. Расчёт встроенной памяти

- ОС Android: 15 ГБ
- PWA приложение (кеш, офлайн-данные): 5 ГБ
- Фотографии, документы: 15 ГБ
- Свободное пространство: 29 ГБ

ИТОГО: $15 + 5 + 15 + 29 = 64$ ГБ.

6.1.2.2.5. Дополнительные требования

Защита: IP68 (пыле- и водонепроницаемость), MIL-STD-810G (ударопрочность, работа при низких температурах).

Экран: Яркий (600+ нит), читаемый на солнце.

Связь: Wi-Fi 5/6, 4G LTE (для удалённых трасс), Bluetooth 5.0 (для подключения сканера).

6.1.2.2.6. Выбранная конфигурация

Таблица 6.11 – Выбранная конфигурация

Параметр	Значение
Модель	Getac T800
ОС	Android 11
Процессор	Qualcomm Snapdragon 660 (8 ядер, 2.2 ГГц)
Оперативная память	4 ГБ LPDDR4
Встроенная память	64 ГБ eMMC + microSD
Дисплей	8.1" WXGA (1280×800), 600 нит
Защита	IP68, MIL-STD-810G (–21°C)
Аккумулятор	8400 мАч (сменный)
Количество	3 шт.

6.1.2.3. Терминалы сбора данных (ТСД)

6.1.2.3.1. Исходные данные

Пользователи: Кладовщики (2 человека).

Задачи: Сканирование штрих-кодов при приёме, выдаче, инвентаризации.

ПО: Специализированное Android-приложение АСУ.

6.1.2.3.2. Расчёт оперативной памяти (RAM)

- ОС (Android 10+): 1.5 ГБ
- Приложение АСУ + фоновая синхронизация: 1 ГБ

- Драйверы сканера и системные службы: 0.5 ГБ
- Запас: 1 ГБ

ИТОГО: $1.5 + 1 + 0.5 + 1 = 4$ ГБ.

6.1.2.3.3. Расчёт процессора (CPU)

- Обработка данных сканирования: 2 ядра
- Синхронизация с сервером (Wi-Fi): 1 ядро
- Интерфейс и фоновые задачи: 1 ядро
- Резерв: 2 ядра

ИТОГО: 6 ядер. Приведем к стандарту 8 ядер.

6.1.2.3.4. Расчёт встроенной памяти

- ОС Android: 12 ГБ
- Приложение АСУ: 0.5 ГБ
- Локальная БД (SQLite) для офлайн-работы: 5 ГБ
- Свободное пространство: 14.5 ГБ

ИТОГО: $12 + 0.5 + 5 + 14.5 = 32$ ГБ. Рекомендуется 64 ГБ.

6.1.2.3.5. Требования к сканеру

Тип сканера: 1D/2D imager (читает повреждённые и бликующие коды).

Дальность считывания: от 5 см до 10 м.

Защита: IP67 (выдерживает падения и пыль).

6.1.2.3.6. Выбранная конфигурация

Таблица 6.12 – Выбранная конфигурация

Параметр	Значение
Модель	Zebra TC21
ОС	Android 10
Процессор	Qualcomm Snapdragon 660 (8 ядер, 2.2 ГГц)
Оперативная память	4 ГБ LPDDR4
Встроенная память	64 ГБ eMMC + microSD
Сканер	SE4710 1D/2D imager

Защита	IP67, падение с 1.5 м
Аккумулятор	3300 мАч (сменный)
Количество	2 шт.

6.1.2.4. Принтеры этикеток

6.1.2.4.1. Исходные данные

Назначение: Печать штрих-кодов для маркировки единиц инвентаря.

Объём печати: ~100-200 этикеток в день.

Тип этикеток: Самоклеящиеся, термотрансферные (стойкие к истиранию и влаге).

6.1.2.4.2. Расчёт характеристик

Разрешение печати: 203 dpi (стандарт для штрих-кодов).

Ширина печати: до 104 мм (стандартные этикетки 100×50 мм).

Скорость печати: 100-150 мм/сек (200 этикеток × 5 сек = ~15 минут работы в день).

Интерфейсы: USB (локально), Ethernet (сетевая печать), Bluetooth (для мобильной печати с ТСД).

6.1.2.4.3. Выбранная конфигурация

Таблица 6.13 – Выбранная конфигурация

Параметр	Значение
Модель	Zebra ZD421
Тип печати	Термотрансферная / прямая термопечать
Разрешение	203 dpi (8 точек/мм)
Ширина печати	104 мм
Скорость печати	152 мм/сек
Интерфейсы	USB 2.0, Ethernet, Bluetooth 4.1, Wi-Fi (опционально)
Количество	1 шт.

6.1.3. Сетевое оборудование

6.1.3.1. Коммутаторы

6.1.3.1.1. Исходные данные

Подключаемые устройства: 3 сервера $\times$ 2 порта (6) + 10 АРМ (10) + 1 МФУ/принтер (1) + 1 точка доступа Wi-Fi (1) = 19 портов.

Трафик: Веб-приложение (HTTP/HTTPS), база данных (SQL), бэкапы.

Топология: Звезда, резервирование L2.

6.1.3.1.2. Расчёт пропускной способности

- Пиковый трафик от 50 пользователей: 50×5 Мбит/с (веб + API) = 250 Мбит/с
- Трафик бэкапов (ночной): до 100 Мбит/с
- Внутренний трафик между серверами: до 100 Мбит/с
- Суммарный пиковый трафик: $250 + 100 + 100 = 450$ Мбит/с
- Требования к портам: 1 Гбит/с на порт (достаточно с 10-кратным запасом)
- Uplink между коммутаторами: 10 Гбит/с (для отказоустойчивости и скорости)

6.1.3.1.3. Выбранная конфигурация

Таблица 6.14 – Выбранная конфигурация

Параметр	Значение
Модель	Cisco Catalyst 9300-24T / Eltex MES2324
Порты доступа	$24 \times 1\text{GbE RJ45}$
Uplink	$4 \times 10\text{GbE SFP+}$
Управление	Web, CLI, SNMP
Пропускная способность	128 Гбит/с
Количество	2 шт. (стек)

6.1.3.2. Маршрутизатор / Межсетевой экран (NGFW)

6.1.3.2.1. Исходные данные

Внешний канал: 100 Мбит/с (Интернет).

Количество пользователей: 50 (внутренние) + 5 (удалённые через VPN).

Функции: Межсетевой экран, IPS/IDS, VPN-шлюз, контент-фильтрация.

6.1.3.2.2. Расчёт пропускной способности

- Интернет-трафик: $50 \text{ пользователей} \times 2 \text{ Мбит/с} = 100 \text{ Мбит/с}$
- VPN-трафик (удалённые тренеры): $5 \text{ пользователей} \times 5 \text{ Мбит/с} = 25 \text{ Мбит/с}$
- Суммарный пиковый трафик: $100 + 25 = 125 \text{ Мбит/с}$
- Требования к производительности FW: $125 \text{ Мбит/с} \times 1.5 \text{ (запас)} = 187.5 \text{ Мбит/с}$

С учётом включённого IPS/IDS: требуется 250 Мбит/с пропускной способности

6.1.3.2.3. Расчёт RAM и CPU

- Таблица состояний соединений: $60,000 \text{ одновременных сессий} \times 0.5 \text{ КБ} = 30 \text{ МБ}$
- Буферы пакетов: 128 МБ
- Сигнатуры IPS/IDS: 2 ГБ (загружаются в RAM)
- ОС и служебные процессы: 1 ГБ

ИТОГО RAM: $0.03 + 0.128 + 2 + 1 = 3.2 \text{ ГБ}$. Приведем к стандарту 4 ГБ.

CPU: Для обработки 500 Мбит/с с IPS требуется 2-4 ядра.

6.1.3.2.4. Выбранная конфигурация

Таблица 6.15 – Выбранная конфигурация

Параметр	Значение
Модель	UserGate C150 / Ideco UTM 150
Пропускная способность FW	1 Гбит/с

Пропускная способность с IPS	500 Мбит/с
Пиковая VPN-производительность	300 Мбит/с
Порты	6 × 1GbE RJ45
ОЗУ / Накопитель	8 ГБ / 120 ГБ SSD
Количество	1 шт.

6.1.3.3. VPN-шлюз

При использовании NGFW со встроенным VPN-сервером отдельный шлюз не требуется. Функционал VPN (до 10 подключений IPsec/SSL) полностью покрывается мощностями выбранного UserGate C150 / Ideco UTM 150.

6.2. Схема компьютерной сети АСУ МТО

На основе архитектуры, определённой ранее, разработана схема компьютерной сети, включающая серверное, клиентское и сетевое оборудование, необходимое для функционирования всех модулей АСУ МТО лыжной организации.

6.2.1. Состав оборудования:

- Основной сервер (1 шт.) — DEPO Storm 2400, Xeon Silver 4310, 16 ГБ RAM, 2×240 ГБ SSD RAID 1. Размещает: Docker Compose (PostgreSQL 17, Spring Boot, Nginx + React SPA).
- Сервер аналитики (1 шт.) — DEPO Storm 2400, Xeon Silver 4310, 16 ГБ RAM, 2×240 ГБ SSD RAID 1. Размещает: Python/statsmodels, ClickHouse, Redash/Superset.
- Сервер резервного копирования (1 шт.) — DEPO Storm 1300, Xeon E-2336, 8 ГБ RAM, 2×2 ТБ HDD RAID 1. Размещает: Bacula/Veeam Backup.

- АРМ пользователей (10 шт.) — DEPO Neos 300, Core i5-12400, 16 ГБ RAM, 256 ГБ SSD. Роли: менеджеры по закупкам, кладовщики, тренеры, руководитель, бухгалтер, администратор.
- Мобильные устройства тренеров (3 шт.) — Getac T800, Android 11, 4 ГБ RAM, 64 ГБ eMMC, защита IP68/MIL-STD-810G. Доступ через VPN по 4G LTE.
- Терминалы сбора данных (2 шт.) — Zebra TC21, Android 10, 4 ГБ RAM, 64 ГБ eMMC. Используются кладовщиками на складе.
- Принтер этикеток (1 шт.) – Zebra ZD421, термотрансферный, Ethernet
- Система видеонаблюдения (1 комплект) — NVR-регистратор + 4 IP-камеры (PoE)
- Сетевое оборудование: 2 коммутатора Cisco Catalyst 9300-24T (стек), межсетевой экран UserGate C150 / Ideco UTM 150 (NGFW с функцией VPN-шлюза).

6.2.2. Логическая структура сети:

Внутренняя сеть (LAN): 192.168.56.0/24. Все серверы и АРМ подключены через управляемые коммутаторы L3.

VPN-подсеть: 192.168.56.0/24. Выделяется для удалённых подключений тренеров и администратора.

Управление доступом: VLAN 10 (серверы), VLAN 20 (АРМ), VLAN 30 (гостевая/Wi-Fi), VLAN 40 (видеонаблюдение).

6.2.3. Графическое представление

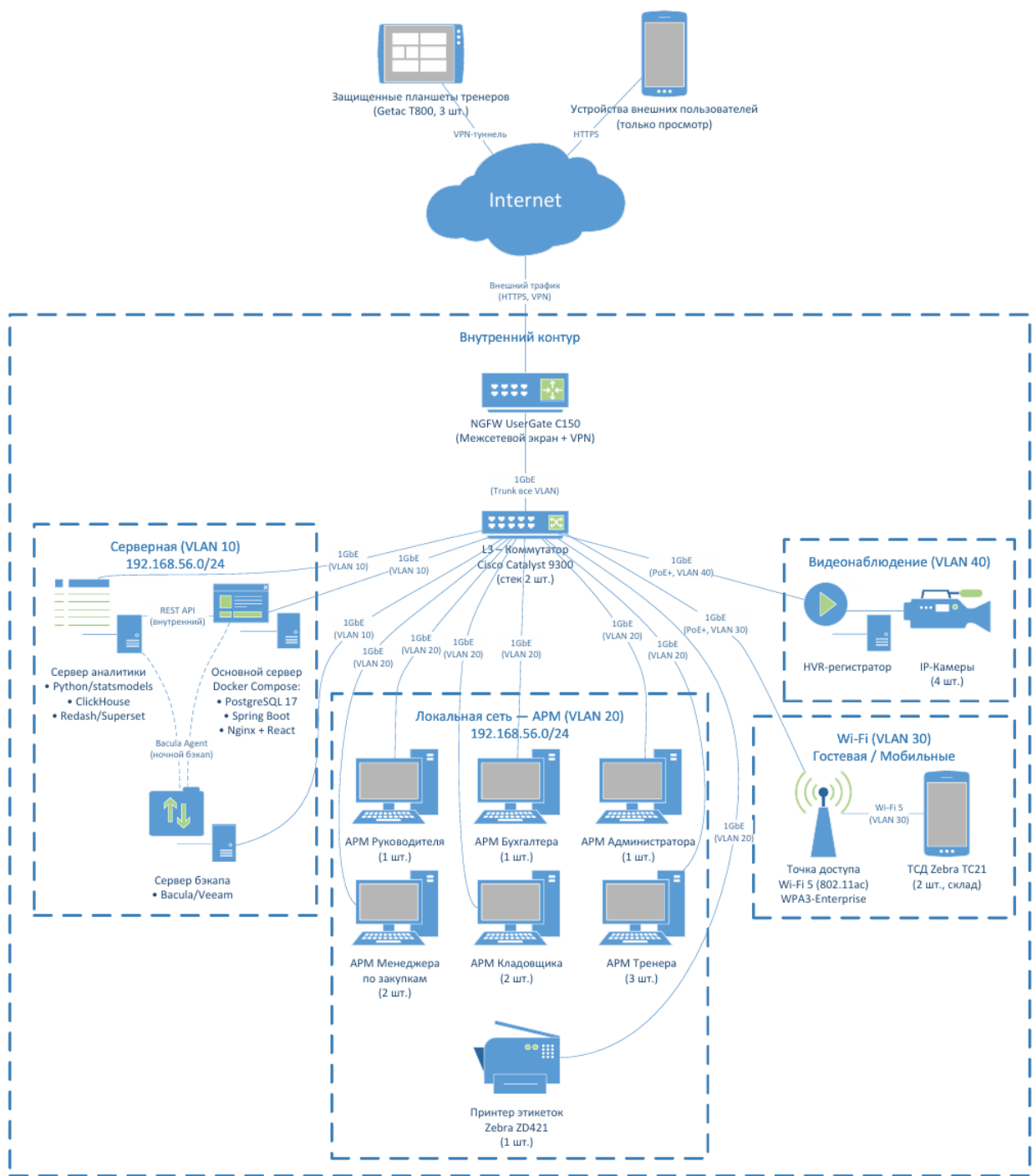


Рисунок 6.1 – Схема компьютерной сети

6.3. Описание и сравнение сред виртуализации

6.3.1. Рассмотренные среды виртуализации

Для создания виртуального стенда АСУ МТО лыжной организации были рассмотрены три распространенные среды виртуализации:

Таблица 6.16 – Рассмотренные среды виртуализации

Среда виртуализации	Разработчик	Тип	Лицензия
Oracle VirtualBox	Oracle Corporation	Гипервизор 2-го типа (настольный)	Бесплатная (GPL v2)
VMware Workstation Player	VMware (Broadcom)	Гипервизор 2-го типа (настольный)	Бесплатная (Player) / Платная (Pro)
Microsoft Hyper-V	Microsoft	Гипервизор 1-го типа (встроенный)	Бесплатная (в составе Windows)

6.3.2. Критерии сравнения

Таблица 6.17 – Сравнительный анализ сред виртуализации

Критерий	Oracle VirtualBox	VMware Workstation Player	Microsoft Hyper-V
Тип гипервизора	2-й тип (на ОС)	2-й тип (на ОС)	1-й тип (на железе)
Поддержка гостевых ОС	Windows, Linux, macOS, Solaris	Windows, Linux	Windows, Linux
Максимум vCPU на ВМ	32	16 (Player) / 32 (Pro)	240
Максимум RAM на ВМ	128 ГБ	64 ГБ	12 ТБ
Поддержка снапшотов	Да (встроено)	Нет (Player) / Да (Pro)	Да (через контрольные точки)
Поддержка сетей	NAT, Bridged, Internal, Host-Only, Generic	NAT, Bridged, Host-Only, Custom	NAT, External, Internal, Private
Клонирование ВМ	Полное / Связанное	Нет (Player)	Да (через экспорт/импорт)
Поддержка USB	Да (USB 1.1-3.0)	Да (USB 1.1-3.1)	Нет (только через RDP)
3D-ускорение	Да (экспериментальное)	Да	Нет
Поддержка Vagrant	Да	Да	Да
Кроссплатформенность	Windows, Linux, macOS, Solaris	Windows, Linux	Только Windows
Производительность	Средняя	Высокая	Очень высокая (близко к bare-metal)

Простота использования	Высокая (интуитивный GUI)	Высокая	Средняя (требуется настройка)
Русский язык интерфейса	Да	Нет	Да
Стоимость	Бесплатно	Бесплатно (Player)	Бесплатно

6.3.3. Преимущества и недостатки

Oracle VirtualBox

Преимущества:

- Полностью бесплатная, открытая лицензия (GPL v2)
- Кроссплатформенность — работает на Windows, Linux и macOS
- Встроенная поддержка снапшотов (мгновенных снимков состояния VM)
- Гибкая настройка виртуальных сетей: NAT, Bridged, Internal Network, Host-Only
- Поддержка русского языка интерфейса
- Возможность клонирования VM (полное и связанное)
- Большое сообщество и подробная документация
- Поддержка автоматизации через Vagrant

Недостатки:

- Гипервизор 2-го типа — производительность ниже, чем у Hyper-V
- 3D-ускорение работает нестабильно

VMware Workstation Player

Преимущества:

- Высокая производительность (оптимизированный гипервизор)
- Хорошая поддержка гостевых ОС
- Стабильная работа 3D-ускорения (DirectX 11, OpenGL 4.3)

Недостатки:

- Бесплатная версия (Player) не поддерживает снапшоты
- Бесплатная версия не поддерживает клонирование VM

- Отсутствует русский язык интерфейса
- Только Windows и Linux (нет поддержки macOS)

Microsoft Hyper-V

Преимущества:

- Гипервизор 1-го типа — производительность максимально близка к «голому железу»
- Глубокая интеграция с Windows (Active Directory, PowerShell)
- Динамическое выделение оперативной памяти

Недостатки:

- Работает только на Windows (не кроссплатформенный)
- Требуется редакций Windows Pro, Enterprise или Education
- Может конфликтовать с другими гипервизорами при одновременной работе
- Отсутствует поддержка USB-устройств в гостевых ВМ

6.3.4. Обоснование выбора среды виртуализации

Выбрана среда виртуализации: Oracle VirtualBox 7.0+

Обоснование выбора:

1. Кроссплатформенность — VirtualBox работает на Windows, Linux и macOS, что позволяет запустить стенд на любой ОС хоста без ограничений.
2. Бесплатность и открытая лицензия — отсутствие лицензионных отчислений критически важно для образовательного учреждения.
3. Поддержка снапшотов — возможность создания мгновенных снимков состояния ВМ позволяет безопасно экспериментировать с конфигурацией и быстро откатывать изменения при ошибках.
4. Гибкая настройка виртуальных сетей — поддержка Internal Network позволяет создать полностью изолированную сеть, моделирующую реальную инфраструктуру организации.

5. Русский язык интерфейса — упрощает документирование процесса создания стенда и его последующую демонстрацию.

6. Соответствие требованиям стенда:

- Стенду требуется 6 vCPU и 12 ГБ RAM суммарно
- VirtualBox поддерживает до 32 vCPU и 128 ГБ RAM на ВМ — с многократным запасом

6.4. Проектирование виртуального стенда

6.4.1. Общая архитектура стенда

Виртуальный стенд моделирует полную инфраструктуру АСУ МТО лыжной организации и включает следующие функциональные компоненты:

- Подсистема хранения данных — СУБД PostgreSQL со всеми 19 таблицами
- Подсистема бизнес-логики — Spring Boot приложение (модули управления закупками, складского учёта, администрирования и отчётности)
- Подсистема аналитики и прогнозирования — Python/statsmodels (модуль прогнозирования потребности)
- Подсистема веб-доступа — Nginx как единая точка входа для всех пользователей
- Клиентская подсистема — браузер для доступа к веб-интерфейсу АСУ

6.4.2. Состав виртуальных машин

Таблица 6.18 – Состав виртуального стенда

Метка	Назначение	Размещаемые компоненты АСУ	ОС	vCPU	RAM	Диск
BM1	Сервер баз данных	PostgreSQL 17 (19 таблиц)	Ubuntu Server 22.04 LTS	2	4 ГБ	25 ГБ

BM2	Сервер приложений, аналитики и веб-доступа	Spring Boot (модули 1-3), Python/statsmodels (модуль прогнозирования), Nginx (веб-сервер)	Ubuntu Server 22.04 LTS	2	4 ГБ	25 ГБ
BM3	Клиентская машина (АРМ пользователя)	Chromium (браузер), LibreOffice (офисный пакет)	Ubuntu Desktop 22.04 LTS	2	4 ГБ	20 ГБ

Таблица 6.19 – Соответствие ВМ модулям и функциям АСУ

Модуль АСУ	Функции	Размещение
Модуль 1. Управление закупками	Заявки, бюджетный контроль, заказы поставщикам, планирование	BM2 (Spring Boot)
Модуль 2. Складской учёт	Приёмка, выдача, возврат, инвентаризация, маркировка	BM2 (Spring Boot)
Модуль 3. Администрирование и отчётность	Управление пользователями, справочники, отчёты, аудит	BM2 (Spring Boot)
Модуль прогнозирования	Прогноз потребности, расчёт износа, обучение моделей	BM2 (Python/statsmodels)
База данных	19 таблиц, индексы, связи, хранимые данные	BM1 (PostgreSQL)
Веб-интерфейс	Пользовательский интерфейс для всех ролей	BM2 (Nginx)
Клиентское приложение	Браузер, офисный пакет	BM3 (Ubuntu Desktop)

6.4.3. Сетевая топология

Все виртуальные машины объединены в изолированную сеть Internal Network 192.168.56.0/24.

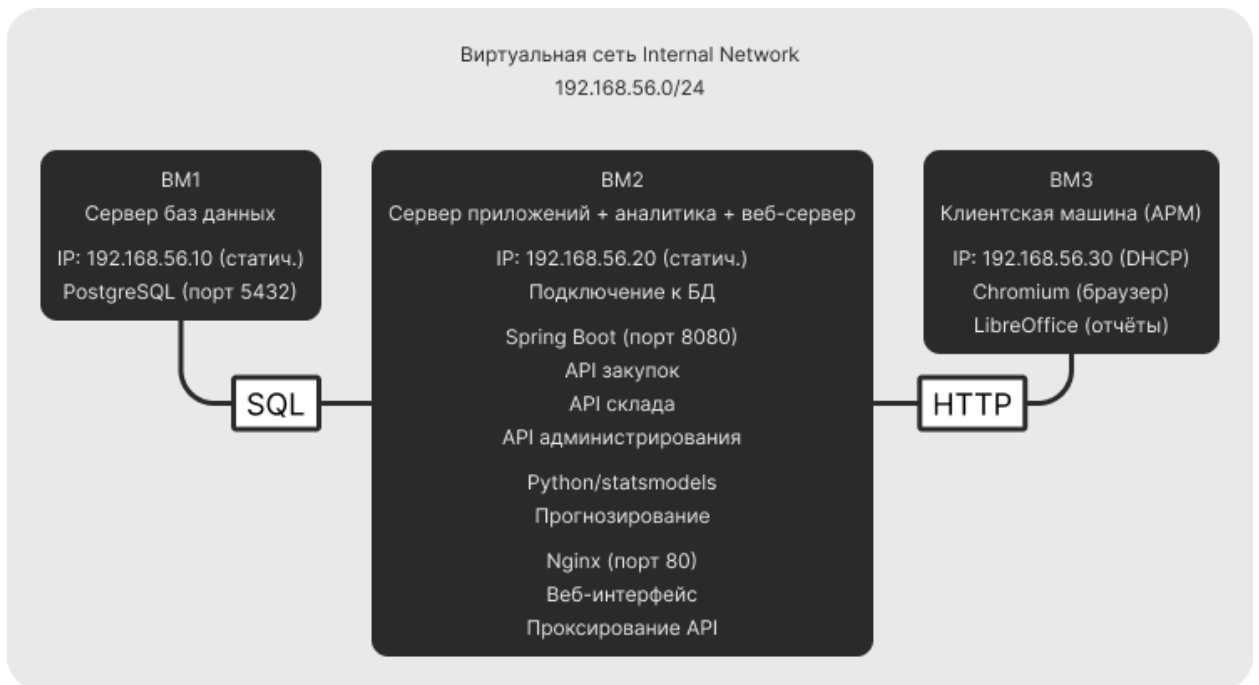


Рисунок 6.2 – Сетевая топология

Таблица 6.20 – Сетевые адреса и порты

Компонент	Размещение	IP-адрес	Порты	Назначение
PostgreSQL	BM1	192.168.56.10	5432	СУБД (все таблицы АСУ)
Spring Boot API	BM2	192.168.56.20	8080	Бизнес-логика (модули 1-3)
Python (прогноз)	BM2	192.168.56.20	—	Локальный запуск (main.py)
Ngix	BM2	192.168.56.20	80	Точка входа для пользователей
Клиент (браузер)	BM3	DHCP (192.168.56.100+)	—	Доступ ко всем функциям АСУ

6.4.4. Распределение программных компонентов АСУ по ВМ

Таблица 6.21 – Программные компоненты на ВМ

ВМ	ПО	Версия	Назначение
BM1	Ubuntu Server	22.04 LTS	ОС
	PostgreSQL	17	СУБД
BM2	Ubuntu Server	22.04 LTS	ОС
	OpenJDK	21	Среда выполнения Java
	Spring Boot	3.4	Бизнес-логика АСУ
	Python	3.12	Среда выполнения Python
	statsmodels	0.14.1	Экспоненциальное сглаживание
	pandas	2.2.0	Обработка данных
	Ngix	1.24+	Веб-сервер и reverse proxy

BM3	Ubuntu Desktop	22.04 LTS	ОС с GUI
	Chromium	120+	Браузер для доступа к АСУ
	LibreOffice	24.2+	Офисный пакет

6.5. Реализация виртуального стенда

На основе описанных при проектировании параметров были созданы 3 виртуальные машины: 2 серверные на Ubuntu 22.04 Server и одна на Ubuntu 22.04 Desktop.

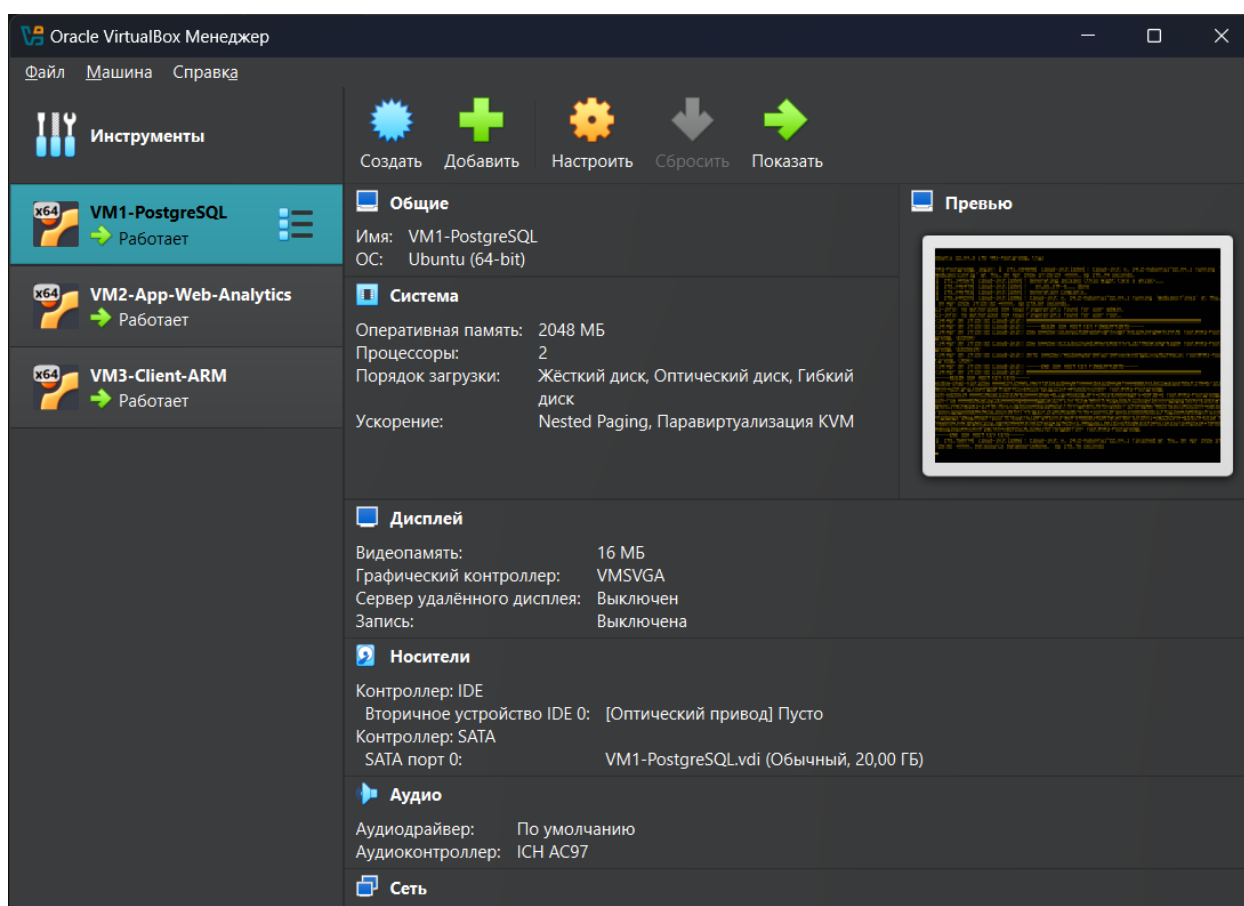
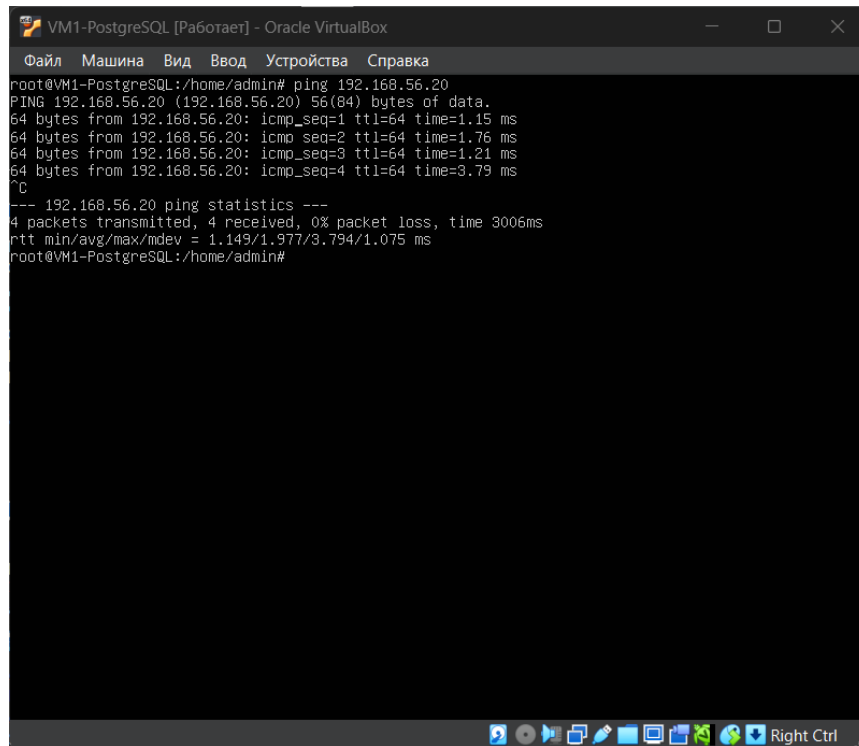


Рисунок 6.3 – Созданные ВМ

Далее была проведена настройка статического IP для ВМ1 (192.168.56.10) и ВМ2(192.168.56.20) (ВМ3 получает IP автоматически по DHCP). В результате была проведена проверка связи между узлами (рис 3.2-3.4).

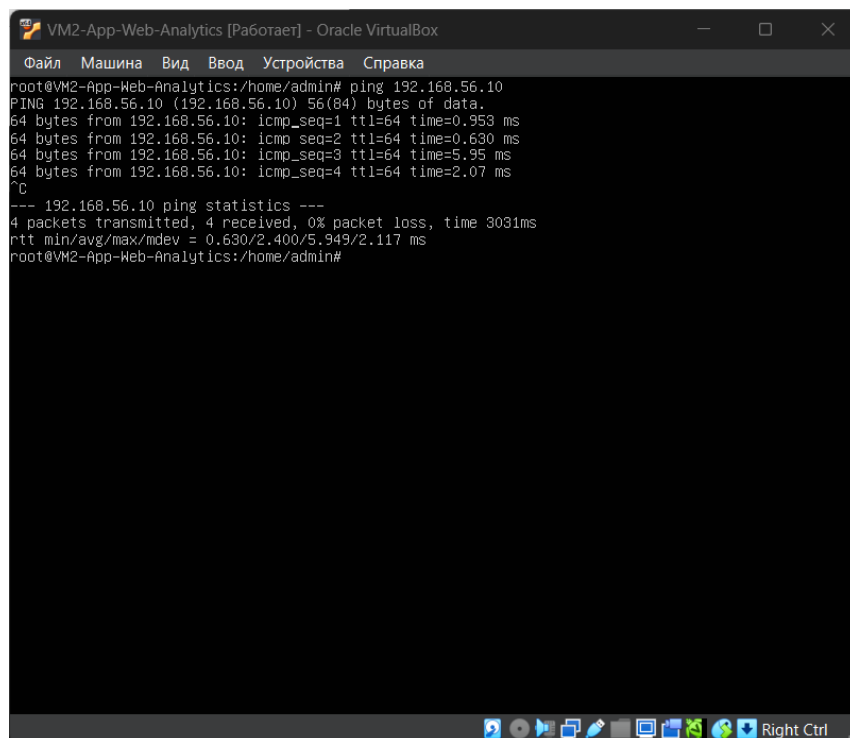
```
sudo nano /etc/netplan/00-installer-config.yaml
```

```
network:
  ethernet:
    enp0s3:
      dhcp4: no
      addresses: [192.168.56.10/24]
      nameservers:
        addresses: [8.8.8.8, 77.88.8.8]
  version: 2
```



```
VM1-PostgreSQL [Работает] - Oracle VirtualBox
Файл  Машина  Вид  Ввод  Устройства  Справка
root@VM1-PostgreSQL:/home/admin# ping 192.168.56.20
PING 192.168.56.20 (192.168.56.20) 56(84) bytes of data.
64 bytes from 192.168.56.20: icmp_seq=1 ttl=64 time=1.15 ms
64 bytes from 192.168.56.20: icmp_seq=2 ttl=64 time=1.76 ms
64 bytes from 192.168.56.20: icmp_seq=3 ttl=64 time=1.21 ms
64 bytes from 192.168.56.20: icmp_seq=4 ttl=64 time=3.79 ms
^C
--- 192.168.56.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 1.149/1.977/3.794/1.075 ms
root@VM1-PostgreSQL:/home/admin#
```

Рисунок 6.4 – Проверка пинга на VM1



```
VM2-App-Web-Analytics [Работает] - Oracle VirtualBox
Файл  Машина  Вид  Ввод  Устройства  Справка
root@VM2-App-Web-Analytics:/home/admin# ping 192.168.56.10
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.
64 bytes from 192.168.56.10: icmp_seq=1 ttl=64 time=0.953 ms
64 bytes from 192.168.56.10: icmp_seq=2 ttl=64 time=0.630 ms
64 bytes from 192.168.56.10: icmp_seq=3 ttl=64 time=5.95 ms
64 bytes from 192.168.56.10: icmp_seq=4 ttl=64 time=2.07 ms
^C
--- 192.168.56.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3031ms
rtt min/avg/max/mdev = 0.630/2.400/5.949/2.117 ms
root@VM2-App-Web-Analytics:/home/admin#
```

Рисунок 6.5 – Проверка пинга на VM2

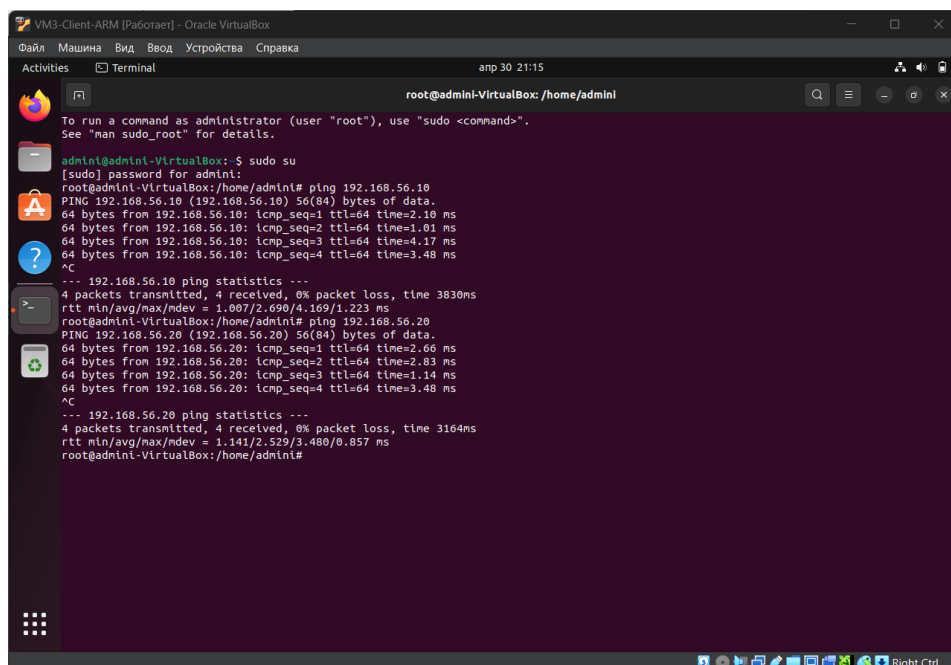


Рисунок 6.6 – Проверка пинга на VM3

Далее было установлено все минимально необходимое ПО на VM1 (сервер БД): непосредственно установка postgresql и настройка удаленного доступа к нему. В итоге была создана и заполнена БД asu_inventory через описанные ранее скрипты create_tables.sql и insert_data.sql.

```

# Обновление системы
sudo apt update && sudo apt upgrade -y

# Установка PostgreSQL
sudo apt install postgresql postgresql-contrib -y

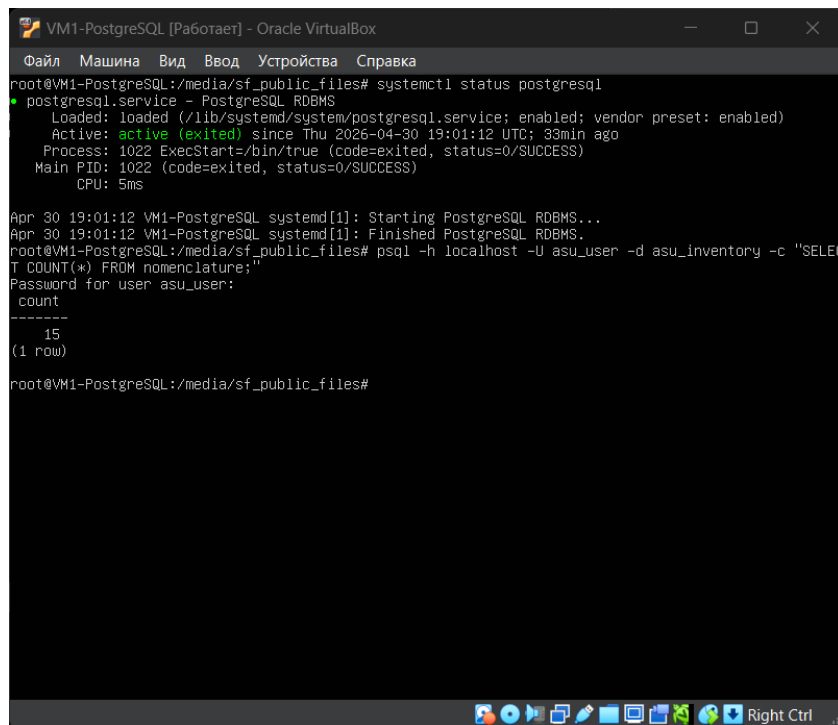
# Настройка удалённого доступа
sudo sed -i "s/#listen_addresses = 'localhost'/listen_addresses = '*'/"
/etc/postgresql/17/main/postgresql.conf
echo "host all all 192.168.56.0/24 md5" | sudo tee -a
/etc/postgresql/17/main/pg_hba.conf
sudo systemctl restart postgresql

# Создание БД и пользователя
sudo -u postgres psql <<EOF
CREATE USER asu_user WITH PASSWORD 'secure_password';
CREATE DATABASE asu_inventory OWNER asu_user;
GRANT ALL PRIVILEGES ON DATABASE asu_inventory TO asu_user;
EOF

# Создание структуры БД
psql -U asu_user -d asu_inventory -f create_tables.sql

# Заполнение тестовыми данными
psql -U asu_user -d asu_inventory -f insert_data.sql

```

```
VM1-PostgreSQL [Работает] - Oracle VirtualBox
Файл  Машина  Вид  Ввод  Устройства  Справка
root@VM1-PostgreSQL:/media/sf_public_files# systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/lib/systemd/system/postgresql.service; enabled; vendor preset: enabled)
   Active: active (exited) since Thu 2026-04-30 19:01:12 UTC; 33min ago
     Process: 1022 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
    Main PID: 1022 (code=exited, status=0/SUCCESS)
       CPU: 5ms

Apr 30 19:01:12 VM1-PostgreSQL systemd[1]: Starting PostgreSQL RDBMS...
Apr 30 19:01:12 VM1-PostgreSQL systemd[1]: Finished PostgreSQL RDBMS.
root@VM1-PostgreSQL:/media/sf_public_files# psql -h localhost -U asu_user -d asu_inventory -c "SELECT
COUNT(*) FROM nomenclature;"
Password for user asu_user:
 count
-----
      15
(1 row)

root@VM1-PostgreSQL:/media/sf_public_files#
```

Рисунок 6.7 – Проверка PostgreSQL на VM1

Затем ПО было установлено и протестировано на VM2. На сервер был установлен nginx и python со всеми необходимыми для работы модуля библиотеками. Статус nginx и результат работы main.py показали корректность установки компонентов.

```
# Обновление системы
sudo apt update && sudo apt upgrade -y

# Установка Java 21
sudo apt install openjdk-21-jdk -y

# Установка Python и библиотек
sudo apt install python3-pip python3-dev -y
pip3 install pandas numpy statsmodels scikit-learn psycopg2-binary
sqlalchemy loguru joblib

# Установка Nginx
sudo apt install nginx -y
sudo systemctl enable nginx
sudo systemctl start nginx

# Копирование модуля прогнозирования
mkdir -p /opt/asu-forecast

# Запуск модуля прогнозирования (демонстрация)
cd /opt/asu-forecast
python3 main.py
```

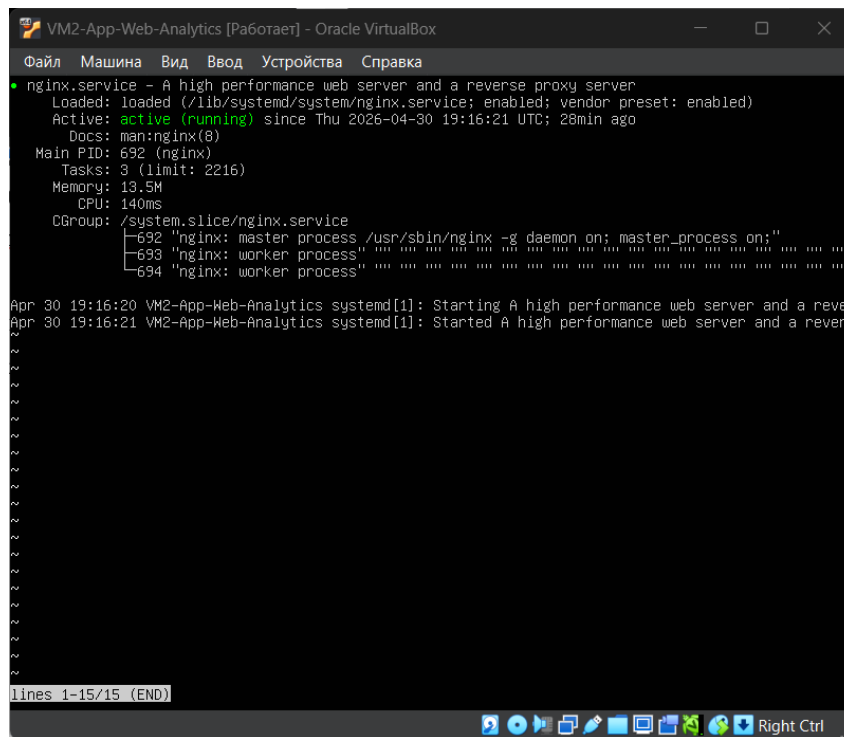


Рисунок 6.8 – Проверка статуса nginx на VM2

Ну и наконец установка ПО производилась на VM3 (веб-браузер и офисный пакет). Подключение к серверу через веб-браузер chromium подтвердило правильность настройки всех VM и локальной сети.

```
# Обновление системы
sudo apt update && sudo apt upgrade -y

# Установка браузера
sudo apt install chromium-browser -y

# Установка офисного пакета
sudo apt install libreoffice -y
```

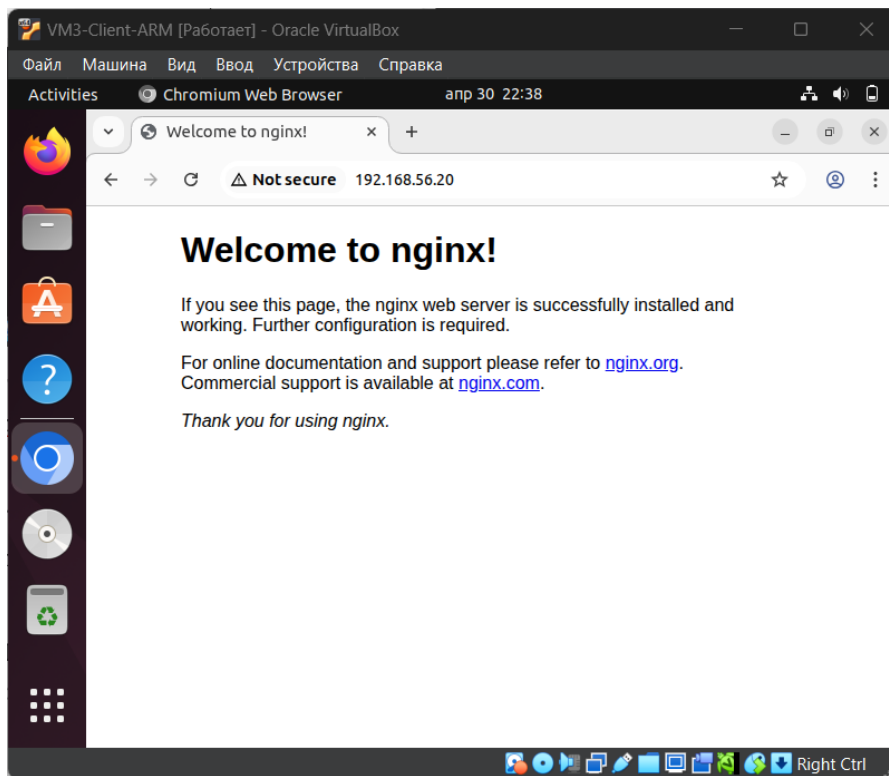


Рисунок 6.9 – Проверка доступности сервера на ВМ3

7. Web-интерфейс для работы с базой данных АСУ

7.1. Информационный веб-портал

7.1.1. Выбор инструмента разработки

Для создания информационного Web-портала были рассмотрены два подхода: использование CMS (WordPress) и разработка на чистом HTML/CSS/JavaScript.

Таблица 7.1 – Сравнение подходов к разработке

Критерий	CMS (WordPress)	Чистый HTML/CSS/JS
Время развёртывания	1-2 часа	30 минут
Требования к хостингу	PHP + MySQL	Любой веб-сервер или без него
Гибкость дизайна	Ограничена темой	Полный контроль
Зависимости	База данных, плагины	Отсутствуют
Размер проекта	50+ МБ	~10 КБ

Выбран подход: Чистый HTML + CSS + JavaScript

Обоснование:

1. Не требует установки сервера — портал открывается в браузере как статический файл.
2. Полный контроль над дизайном — все элементы интерфейса созданы с нуля и стилизованы в тематике лыжного спорта (сине-белая цветовая гамма, спортивная символика).
3. Отсутствие зависимостей — не требуется PHP, MySQL, WordPress или другие CMS. Достаточно папки с тремя файлами.
4. Использование CSS и веб-скриптов — полностью соответствует требованию задания. JavaScript применяется для интерактивных элементов: табы, слайдер отзывов, динамическая загрузка таблиц.

7.1.2. Структура web-портала

Портал реализован как одностраничное приложение (SPA) с плавной навигацией между разделами.

Таблица 7.2 – Разделы портала

Раздел	Якорь	Содержание
Главная	#home	Hero-баннер с названием АСУ, кратким описанием и кнопкой перехода
О системе	#about	Карточки пользователей (4 роли), функции системы (6 пунктов), преимущества автоматизации (3 блока)
Результаты работы	#results	Табы с таблицами складского учёта и выдач, график прогноза, слайдер отзывов
Контакты	#contact	Контактная информация и форма обратной связи

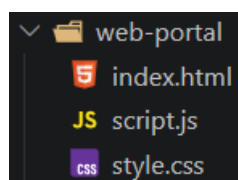


Рисунок 7.1 – Файловая структура

7.1.3. Описание разделов портала

7.1.3.1. Главная страница (Hero)

Назначение: Первое впечатление, приветствие, навигация.

Элементы:

- Фиксированная навигационная панель с логотипом « АСУ МТО» и ссылками на разделы
- Hero-баннер с градиентным фоном (тёмно-синий → голубой):
- Заголовок: «Автоматизированная система управления»
- Подзаголовок: «Материально-техническое обеспечение лыжного спорта»
- Описание: «Современное решение для учёта, закупок и прогнозирования потребности в спортивном инвентаре»
- Кнопка «Узнать больше ▼» (якорь к разделу «О системе»)

7.1.3.2. О системе

Назначение: Информация о проекте, его функциях и преимуществах.

Элементы:

7.1.3.2.1. Карточки пользователей

Менеджер по закупкам — формирует заказы, контролирует бюджет

Кладовщик — управляет складом, сканирует штрих-коды

Тренер — подаёт заявки на инвентарь для группы

Руководитель — утверждает бюджет, анализирует отчёты

7.1.3.2.2. Основные функции системы

Ведение номенклатуры (15 позиций)

Управление закупками и бюджетный контроль

Складской учёт с маркировкой штрих-кодами

Контроль выдачи и возврата инвентаря

Прогнозирование потребности

Автоматическое списание по истечении срока службы

7.1.3.2.3. Преимущества автоматизации

Экономия времени — сокращение оформления документов в 3 раза

Точность учёта — исключение ошибок ручного ввода

Прогнозирование — планирование закупок на основе реальных данных

7.1.3.3. Результаты работы АСУ

Назначение: Демонстрация конкретных данных, получаемых в результате работы системы.

Элементы:

7.1.3.3.1. Табы для переключения между блоками результатов

Вкладка «Складской учёт» — таблица из 8 записей доступного инвентаря с цветовой индикацией статусов

Вкладка «Активные выдачи» — таблица из 5 записей с информацией о выданном инвентаре

Вкладка «Прогноз потребности» — визуализация прогноза на 4 месяца (столбчатая диаграмма) и сводные показатели

7.1.3.3.2. Слайдер отзывов

Три реалистичных отзыва от имени конкретных пользователей (кладовщик, тренер, директор)

Автоматическая смена каждые 5 секунд

Ручное переключение по точкам-индикаторам

Аватары, кавычки, названия организаций

7.1.3.4. Контакты

Назначение: Информация об организации и обратная связь.

Элементы:

- Контактная информация: адрес, телефон, email, часы работы
- Форма обратной связи: имя, email, сообщение, кнопка «Отправить»

7.1.4. Исходный код

Исходный код веб-портала приведен в Приложение 3. Исходный код информационного веб-портала.

7.1.5. Результат разработанного портала

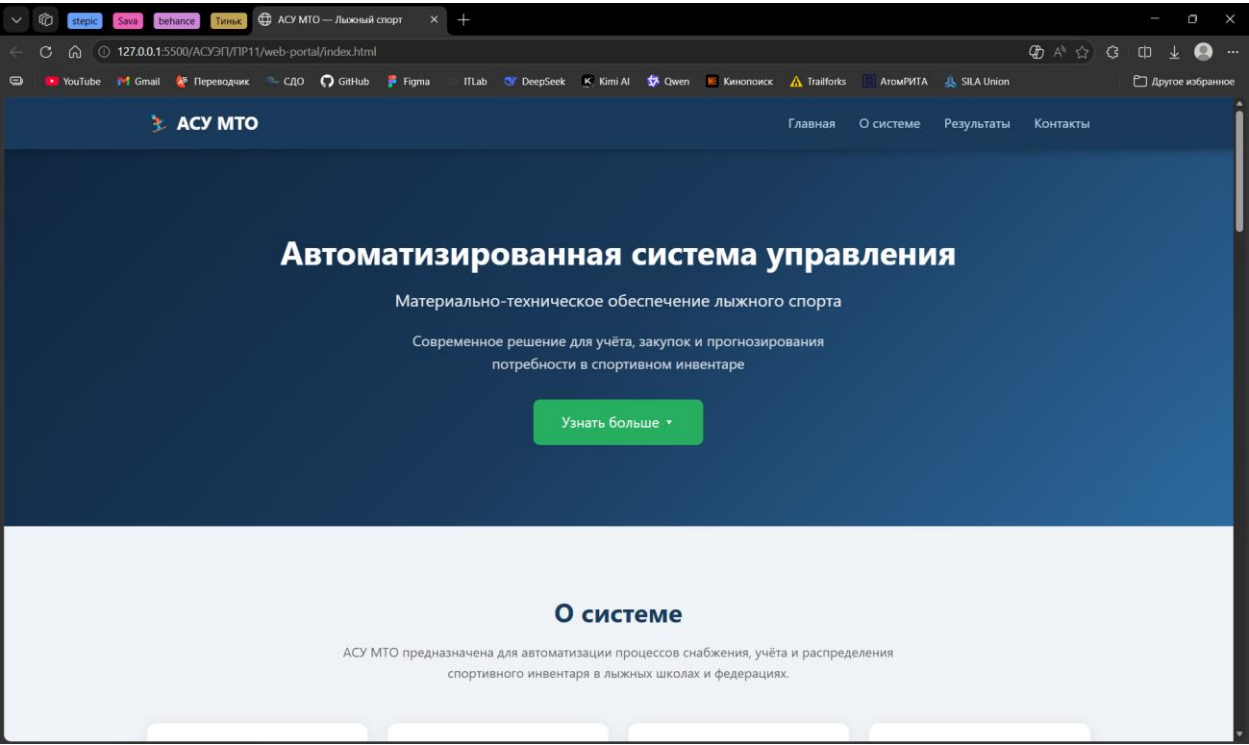


Рисунок 7.2 – Главная страница (Hero-баннер)

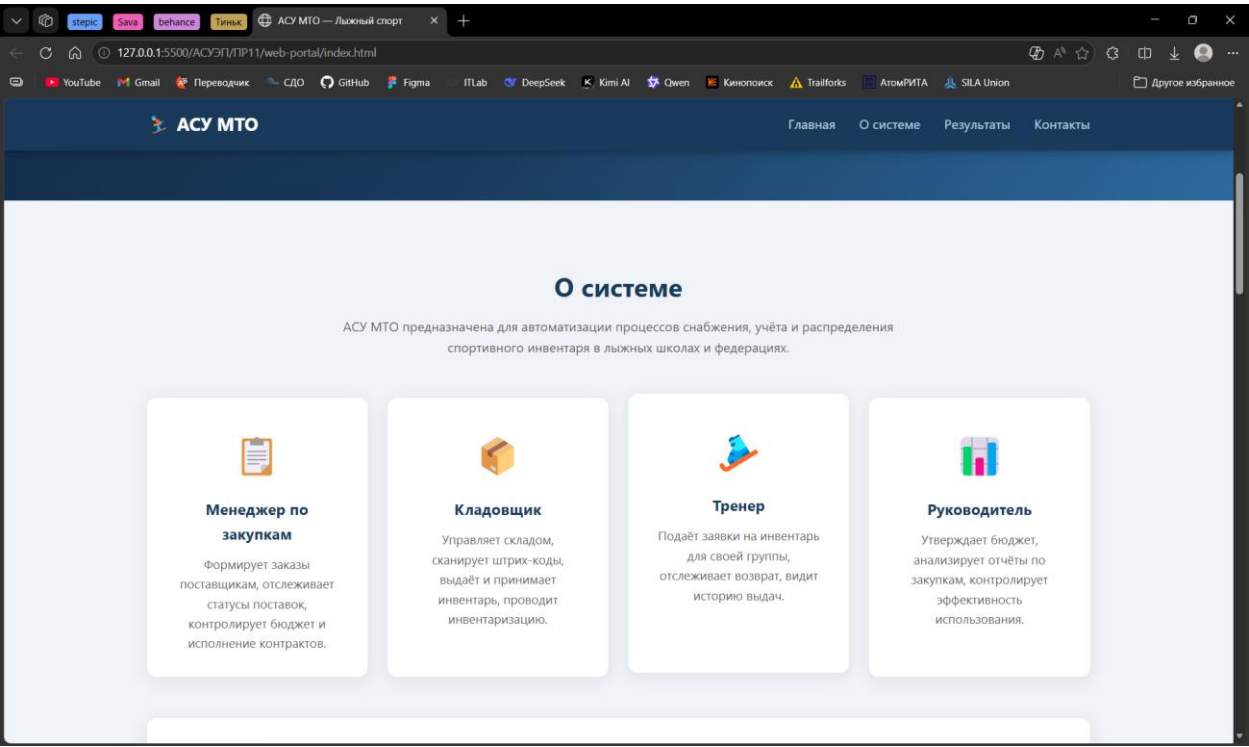


Рисунок 7.3 – Раздел «О системе» — карточки пользователей

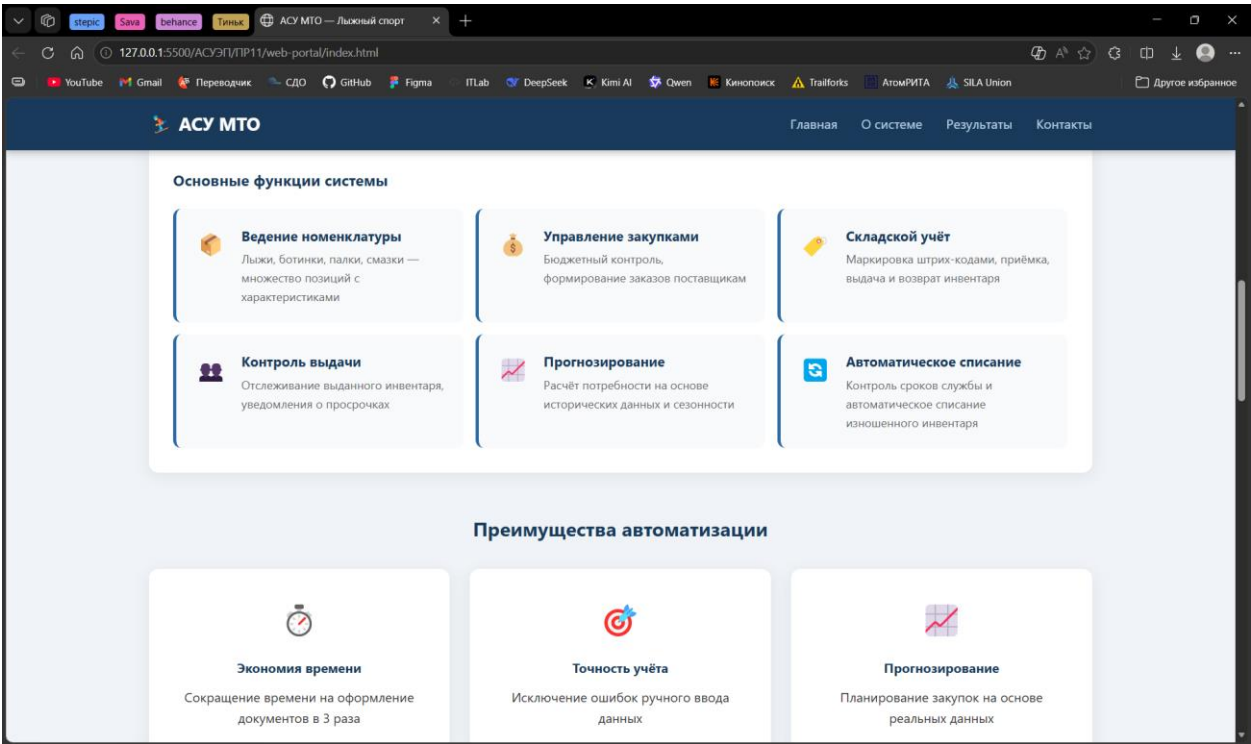


Рисунок 7.4 – Раздел «О системе» — функции и преимущества

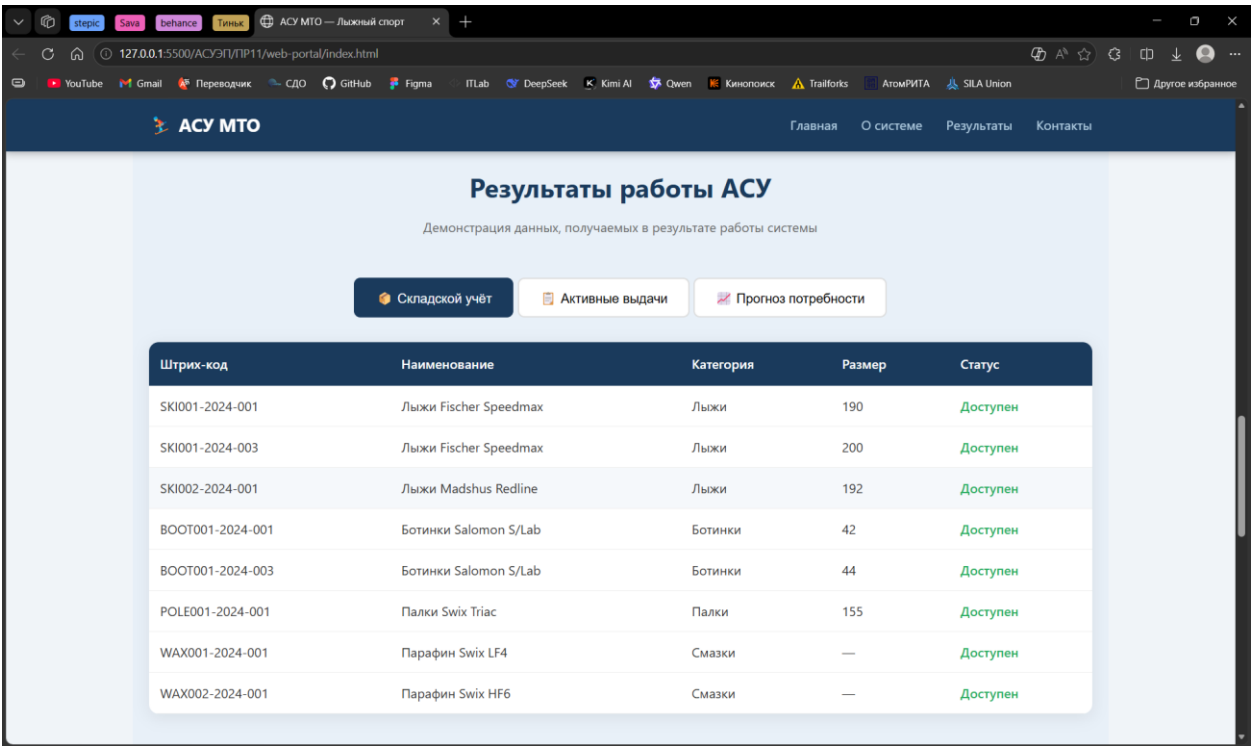


Рисунок 7.5 – Раздел «Результаты» — вкладка «Складской учёт»

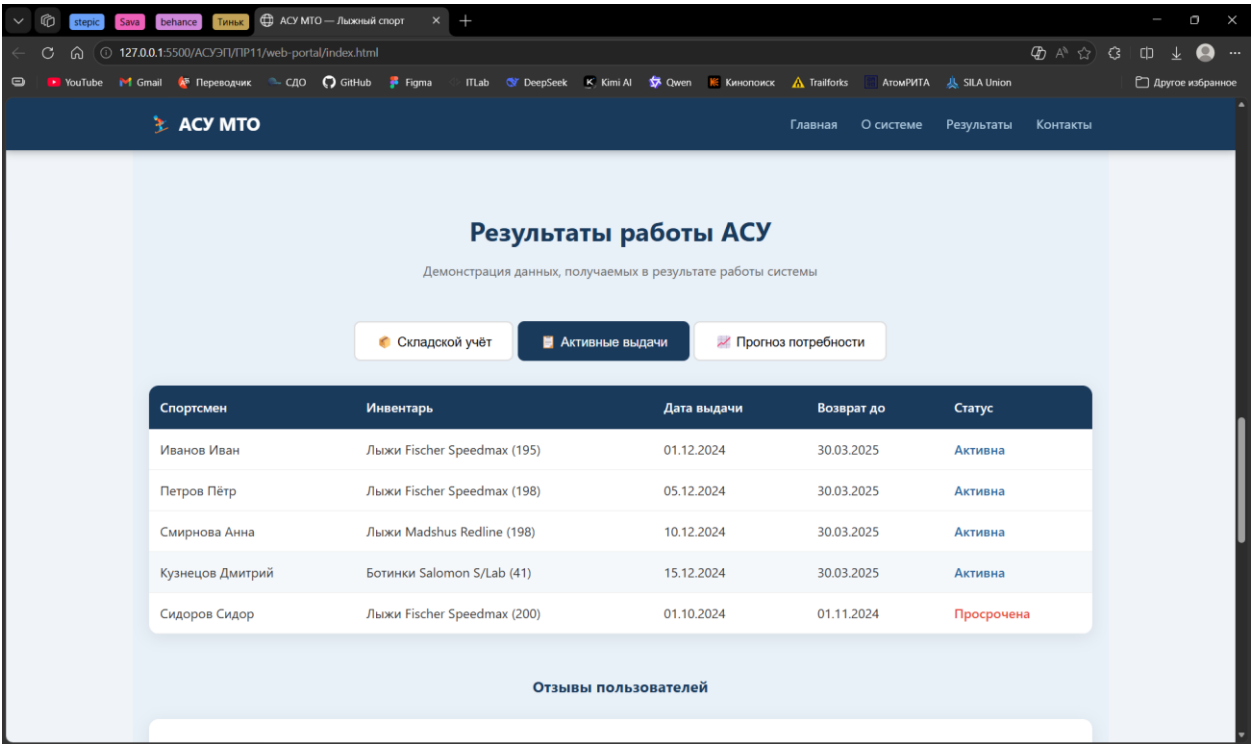


Рисунок 7.6 – Раздел «Результаты» — вкладка «Активные выдачи»

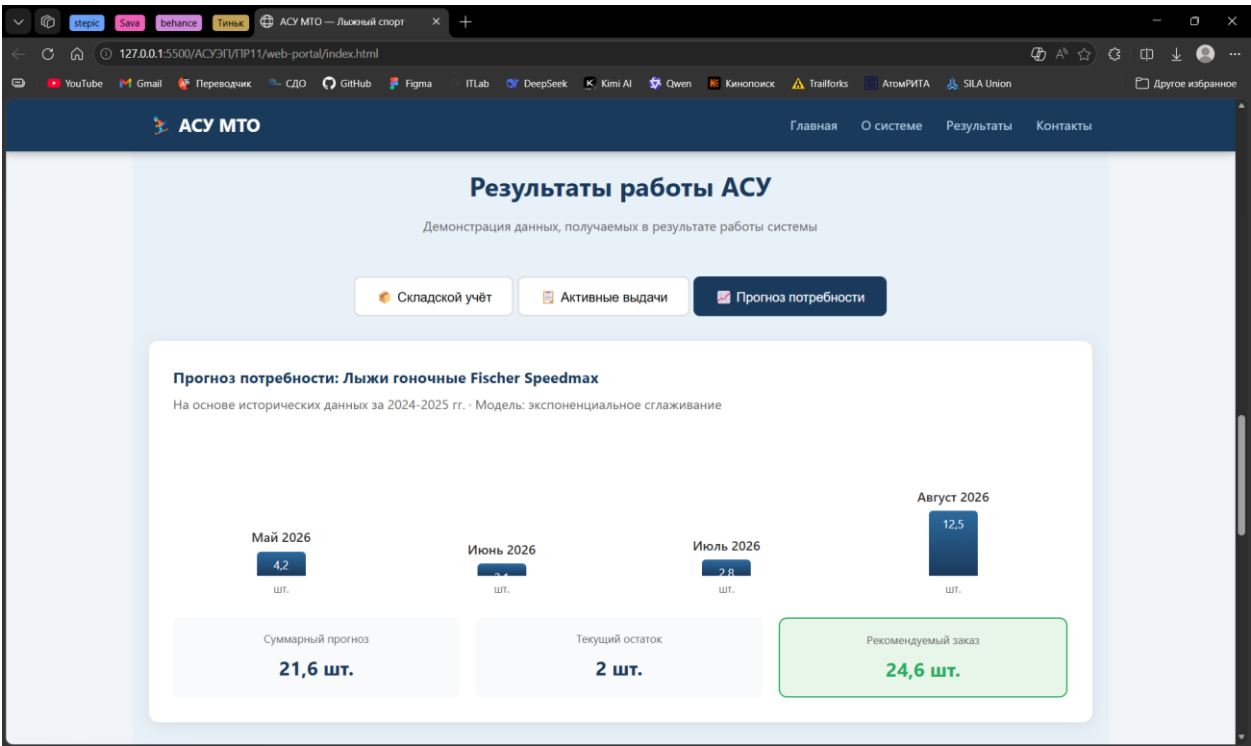


Рисунок 7.7 – Раздел «Результаты» — вкладка «Прогноз потребности»

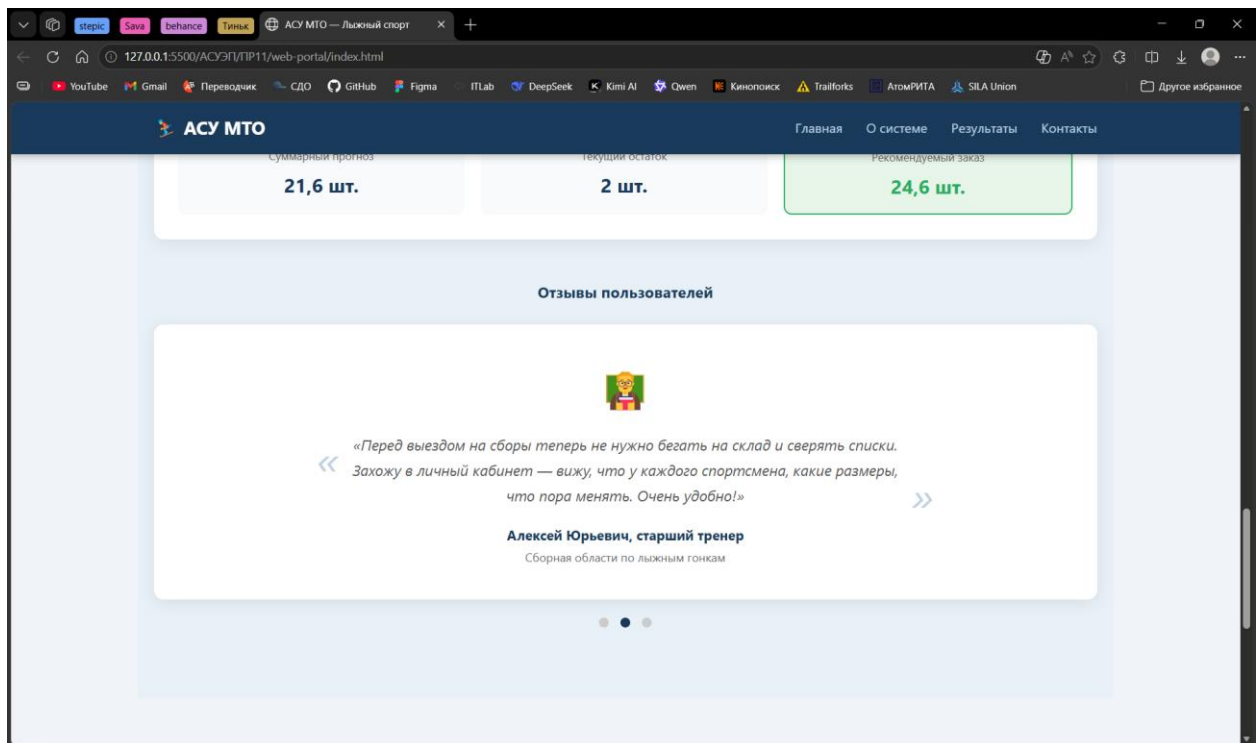


Рисунок 7.8 – Раздел «Результаты» — слайдер отзывов

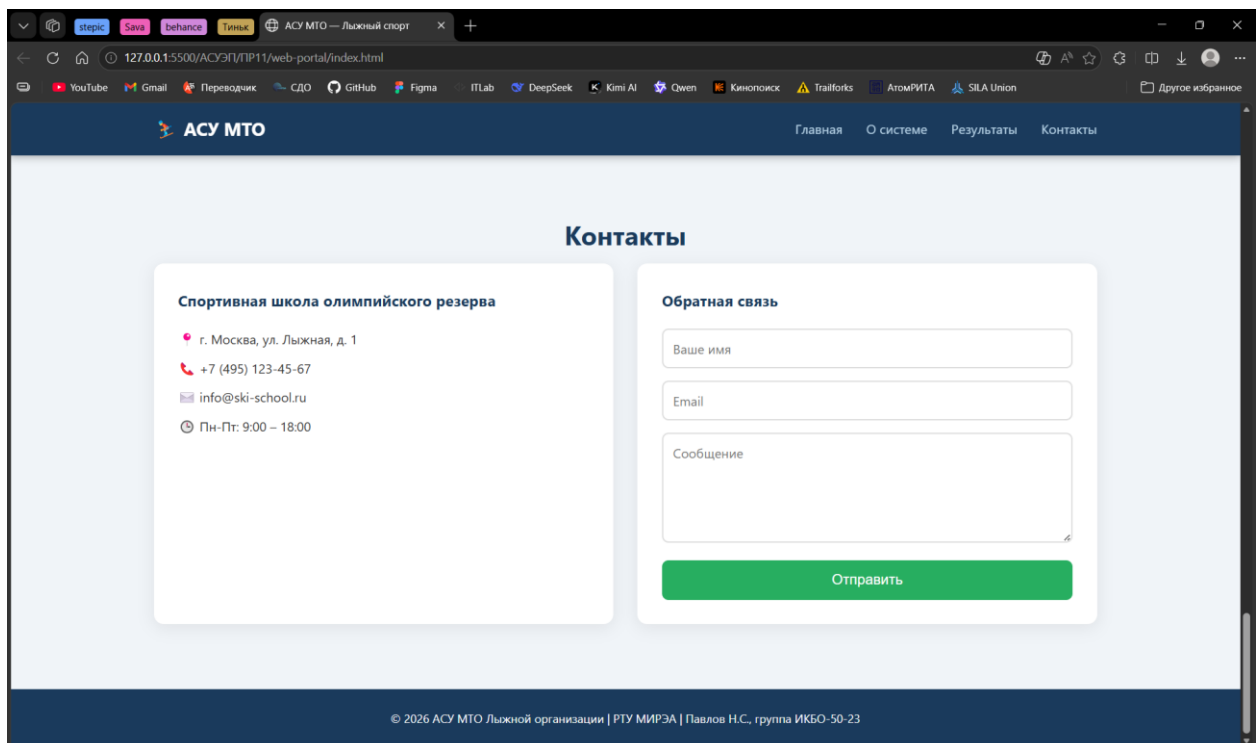


Рисунок 7.9 – Раздел «Контакты»

7.2. Веб-портал для работы с БД

7.2.1. Обоснование выбора технологий

Для разработки программного модуля с веб-интерфейсом, позволяющего работать с базой данных АСУ МТО, были выбраны следующие технологии:

Таблица 7.3 – Бэкенд: Python 3.12 + FastAPI

Критерий	Обоснование
Готовый код	Переиспользуется код database.py и config.py из модуля прогнозирования, что обеспечивает единство архитектуры
Созданная БД	Работает с той же БД PostgreSQL (схема asu_inventory, 19 таблиц), созданной ранее
Производительность	FastAPI — один из самых быстрых Python-фреймворков, поддерживает асинхронную обработку запросов
Документация	Автоматическая генерация Swagger UI (/docs) для тестирования API
Простота	Минимум кода для создания полноценного REST API

Таблица 7.4 – Фронтенд: HTML + CSS + JavaScript (Fetch API)

Критерий	Обоснование
Независимость	Не требует установки Node.js, npm или других инструментов сборки
Динамичность	Fetch API позволяет выполнять асинхронные запросы к бэкенду без перезагрузки страницы
Совместимость	Работает в любом современном браузере

7.2.2. Архитектура модуля и структура проекта

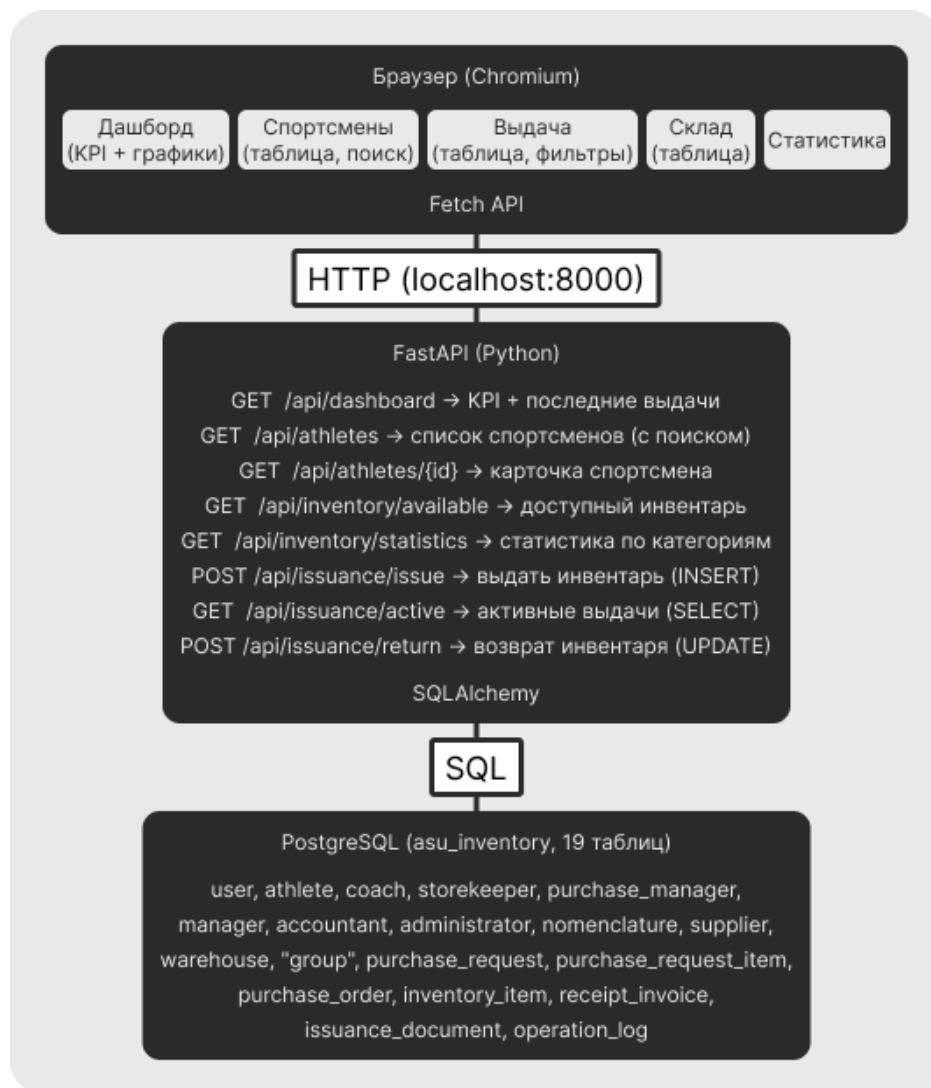


Рисунок 7.10 – Архитектура модуля

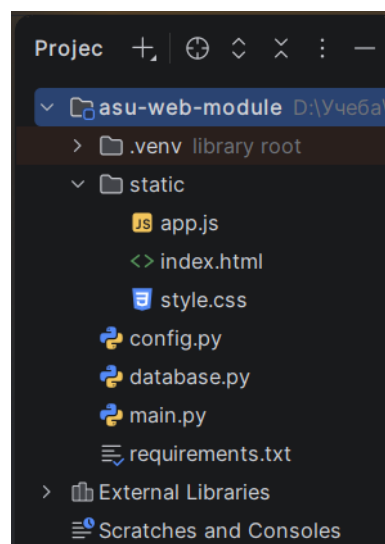


Рисунок 7.11 – Структура проекта

7.2.3. Описание функциональности

Модуль предоставляет веб-интерфейс с 5 вкладками, каждая из которых реализует определённый набор операций с базой данных.

7.2.3.1. Дашборд

Назначение: Отображение ключевых показателей системы в реальном времени.

API-эндпоинт: *GET /api/dashboard*

Выполняемые SQL-запросы:

- *SELECT COUNT(*) FROM athlete* — количество спортсменов
- *SELECT COUNT(*) FROM inventory_item WHERE status = 'AVAILABLE'* — доступный инвентарь
- *SELECT COUNT(*) FROM issuance_document WHERE status IN (...)* — активные выдачи
- *SELECT ... WHERE planned_return_date < CURRENT_DATE* — просроченные
- *SELECT ... GROUP BY nomenclature_category* — распределение по категориям
- *SELECT ... ORDER BY issuance_date DESC LIMIT 5* — последние выдачи
- *SELECT ... GROUP BY athlete ... ORDER BY overdue_items DESC LIMIT 5* — топ должников

Интерфейс:

- 5 KPI-карточек (спортсменов, доступно, выдач, просрочено, должников)
- График распределения инвентаря по категориям (CSS-бары)
- Таблица «Последние выдачи»
- Таблица «Топ должников»

7.2.3.2. Спортсмены

Назначение: Просмотр списка спортсменов с возможностью поиска и фильтрации.

API-эндпоинты:

- *GET /api/athletes?search=&has_debt=* — список с фильтрацией
- *GET /api/athletes/{id}* — детальная карточка

Выполняемые SQL-запросы:

- *SELECT ... FROM athlete JOIN "user" JOIN "group" JOIN coach WHERE full_name ILIKE ...* — поиск
- *SELECT ... WHERE athlete_id = :id* — карточка с email, телефоном, статистикой

Интерфейс:

- Поисковая строка с дебаунсом (300 мс)
- Фильтр по задолженности
- Таблица с подсветкой должников
- Кнопка просмотра детальной карточки (модальное окно)

7.2.3.3. Выдачи

Назначение: Управление выдачей и возвратом инвентаря.

API-эндпоинты:

- *GET /api/issuance/active?filter_status=* — активные выдачи
- *POST /api/issuance/issue* — выдача инвентаря (INSERT + UPDATE)
- *POST /api/issuance/return* — возврат инвентаря (UPDATE)

Выполняемые SQL-запросы:

- *SELECT ... FROM issuance_document JOIN athlete JOIN inventory_item JOIN nomenclature* — список выдач
- *INSERT INTO issuance_document (...) VALUES (...)* — создание документа

- *UPDATE inventory_item SET status = 'ISSUED'* — изменение статуса
- *UPDATE issuance_document SET status = 'RETURNED'* — возврат

Бизнес-логика:

- При выдаче проверяется, есть ли у спортсмена задолженность
- При выдаче проверяется, доступен ли инвентарь
- При возврате проверяется, не возвращён ли уже инвентарь

Интерфейс:

- Кнопка «Выдать инвентарь» → модальное окно с формой
- Выпадающие списки спортсменов и доступного инвентаря
- Таблица активных выдач с фильтром (все / просроченные / активные)
- Кнопка «Возврат» в каждой строке

7.2.3.4. Склад

Назначение: Просмотр доступного инвентаря.

API-эндпоинт: *GET /api/inventory/available*

Выполняемый SQL-запрос:

- *SELECT ... FROM inventory_item JOIN nomenclature WHERE status = 'AVAILABLE'*

Интерфейс:

- Таблица со всеми доступными единицами инвентаря

7.2.3.5. Статистика

Назначение: Анализ распределения инвентаря по категориям.

API-эндпоинт: *GET /api/inventory/statistics*

Выполняемый SQL-запрос:

- *SELECT ... GROUP BY nomenclature_category* — с разбивкой по статусам

Интерфейс:

- Карточки по категориям с числовыми показателями
- Визуализация пропорций через цветные полосы (доступен / выдан / проблемы)

7.2.4. Исходный код

Исходный код разработанного портала приведен в Приложение 4.
Исходный код портала для работы с БД

7.2.5. Результаты работы модуля

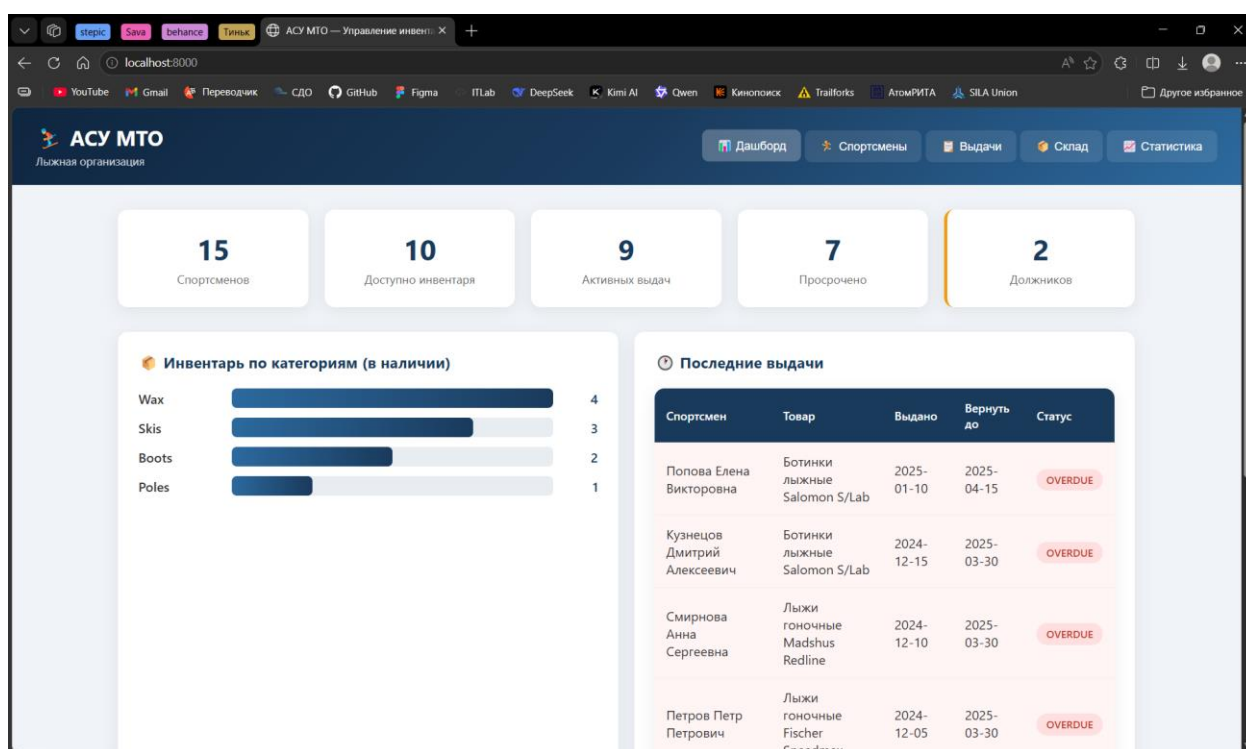


Рисунок 7.12 – Вкладка «Дашборд»

ФИО	Разряд	Год	Группа	Тренер	Выдач	Долг
Васильев Алексей Николаевич	KMC	2004	Группа спортивного совершенствования	Тренеров Алексей Юрьевич	0	✓
Волкова Ольга Дмитриевна	KMC	2005	Группа спортивного совершенствования	Тренеров Алексей Юрьевич	0	✓
Иванов Иван Иванович	KMC	2005	Группа спортивного совершенствования	Тренеров Алексей Юрьевич	1	✓
Козлова Наталья Игоревна	1 разряд	2008	Горнолыжная группа "Слалом"	Старцев Павел Николаевич	1	✓
Кузнецов Дмитрий Алексеевич	MC	2002	Юниорская сборная (лыжные гонки)	Тренеров Алексей Юрьевич	1	✓
Лебедев Андрей Викторович	MC	2003	Биатлонная группа №2	Наставников Игорь Петрович	1	✓
Морозов Сергей Владимирович	MC	2001	Юниорская сборная (лыжные гонки)	Тренеров Алексей Юрьевич	0	⚠
Новикова Мария Игоревна	2 разряд	2009	Группа начальной подготовки №1	Тренеров Алексей Юрьевич	0	✓
Орлова Татьяна Павловна	KMC	2006	Биатлонная группа №1	Наставников Игорь Петрович	1	✓
Петров Петр Петрович	MC	2003	Группа спортивного совершенствования	Тренеров Алексей Юрьевич	1	✓

Рисунок 7.13 – Вкладка «Спортсмены»

ФИО	Разряд	Год	Группа	Тренер	Выдач	Долг
Васильев Алексей Николаевич	KMC			Алексей Юрьевич	0	✓
Волкова Ольга Дмитриевна	KMC			Алексей Юрьевич	0	✓
Иванов Иван Иванович	KMC			Алексей Юрьевич	1	✓
Козлова Наталья Игоревна	1 разряд			авел Николаевич	1	✓
Кузнецов Дмитрий Алексеевич	MC			Алексей Юрьевич	1	✓
Лебедев Андрей Викторович	MC			ов Игорь Петрович	1	✓
Морозов Сергей Владимирович	MC			Алексей Юрьевич	0	⚠
Новикова Мария Игоревна	2 разряд	2009	Группа начальной подготовки №1	Тренеров Алексей Юрьевич	0	✓
Орлова Татьяна Павловна	KMC	2006	Биатлонная группа №1	Наставников Игорь Петрович	1	✓
Петров Петр Петрович	MC	2003	Группа спортивного совершенствования	Тренеров Алексей Юрьевич	1	✓

Кузнецов Дмитрий Алексеевич

Email: kuznetsov@ski.ru | Телефон: +79165678901

Разряд: MC | Год рождения: 2002

Группа: Юниорская сборная (лыжные гонки) | Тренер: Тренеров Алексей Юрьевич

Активных выдач: 1 | Возвращено: 0

Статус: Без долгов

Рисунок 7.14 – Модальное окно «Детали спортсмена»

stepicSavebehanсeТинькоАСУ МТО — Управление инвентарем

localhost:8000

YouTubeGmailПереводчикСДОGitHubFigmaITLabDeepSeekKimi AIQwenКинопоискTrailforksАтомРИТАSILA UnionДругое избранное

АСУ МТО
Лыжная организация

ДашбордСпортсменыВыдачиСкладСтатистика

Выдачи

Выдать инвентарьВсе

Спортсмен	Товар	Размер	Выдано	Вернуть до	Статус	
Орлова Татьяна Павловна	Палки лыжные Swix Triac	155	2024-10-15	2024-11-15	Просрочено (532д)	↗
Лебедев Андрей Викторович	Лыжи гоночные Fischer Speedmax	190	2024-11-01	2024-12-01	Просрочено (516д)	↗
Кузнецов Дмитрий Алексеевич	Ботинки лыжные Salomon S/Lab	41	2024-12-15	2025-03-30	Просрочено (397д)	↗
Смирнова Анна Сергеевна	Лыжи гоночные Madshus Redline	198	2024-12-10	2025-03-30	Просрочено (397д)	↗
Петров Петр Петрович	Лыжи гоночные Fischer Speedmax	198	2024-12-05	2025-03-30	Просрочено (397д)	↗
Иванов Иван Иванович	Лыжи гоночные Fischer Speedmax	195	2024-12-01	2025-03-30	Просрочено (397д)	↗
Попова Елена Викторовна	Ботинки лыжные Salomon S/Lab	43	2025-01-10	2025-04-15	Просрочено (381д)	↗
Волкова Ольга Дмитриевна	Лыжи гоночные Fischer Speedmax	190	2026-05-01	2026-06-06	Активна	↗
Петров Петр Петрович	Ботинки лыжные Salomon S/Lab	44	2026-05-01	2026-06-06	Активна	↗

Рисунок 7.15 – Вкладка «Выдачи»

stepicSavebehanсeТинькоАСУ МТО — Управление инвентарем

localhost:8000

YouTubeGmailПереводчикСДОGitHubFigmaITLabDeepSeekKimi AIQwenКинопоискTrailforksАтомРИТАSILA UnionДругое избранное

АСУ МТО
Лыжная организация

ДашбордСпортсменыВыдачиСкладСтатистика

Выдачи

Выдать инвентарьВсе

Выдача инвентаря

Спортсмен
— Выберите —

Инвентарь
— Сначала выберите спортсмена —

Дата возврата
06.06.2026

Выдать

Спортсмен	Товар	Размер	Выдано	Вернуть до	Статус	
Орлова Татьяна Павловна	Палки			2024-11-15	Просрочено (532д)	↗
Лебедев Андрей Викторович	Лыжи			2024-12-01	Просрочено (516д)	↗
Кузнецов Дмитрий Алексеевич	Ботинки			2025-03-30	Просрочено (397д)	↗
Смирнова Анна Сергеевна	Лыжи			2025-03-30	Просрочено (397д)	↗
Петров Петр Петрович	Лыжи			2025-03-30	Просрочено (397д)	↗
Иванов Иван Иванович	Лыжи			2025-03-30	Просрочено (397д)	↗
Попова Елена Викторовна	Ботинки			2025-04-15	Просрочено (381д)	↗
Волкова Ольга Дмитриевна	Лыжи гоночные Fischer Speedmax	190	2026-05-01	2026-06-06	Активна	↗
Петров Петр Петрович	Ботинки лыжные Salomon S/Lab	44	2026-05-01	2026-06-06	Активна	↗

Рисунок 7.16 – Модальное окно «Выдача инвентаря»

stepicSavebehanceТинькоАСУ МТО — Управление инвентарем

localhost:8000

YouTubeGmailПереводчикСДОGitHubFigmaITLabDeepSeekKimi AIQwenКинопоискTrailforksАтомРИТАSILA UnionДругое избранное

АСУ МТО
Лыжная организация

ДашбордСпортсменыВыдачиСкладСтатистика

Доступный инвентарь

Штрих-код	Наименование	Категория	Размер
BOOT001-2024-001	Ботинки лыжные Salomon S/Lab	boots	42
SKI001-2024-003	Лыжи гоночные Fischer Speedmax	skis	200
SKI002-2024-001	Лыжи гоночные Madshus Redline	skis	192
POLE001-2024-003	Палки лыжные Swix Triac	poles	150
POLE001-2024-001	Палки лыжные Swix Triac	poles	155
POLE001-2024-002	Палки лыжные Swix Triac	poles	160
WAX002-2024-001	Парафин Swix HF6	wax	—
WAX001-2024-002	Парафин Swix LF4	wax	—
WAX001-2024-001	Парафин Swix LF4	wax	—
WAX003-2024-001	Порошок ускоритель Swix FC7	wax	—

Рисунок 7.17 – Вкладка «Склад»

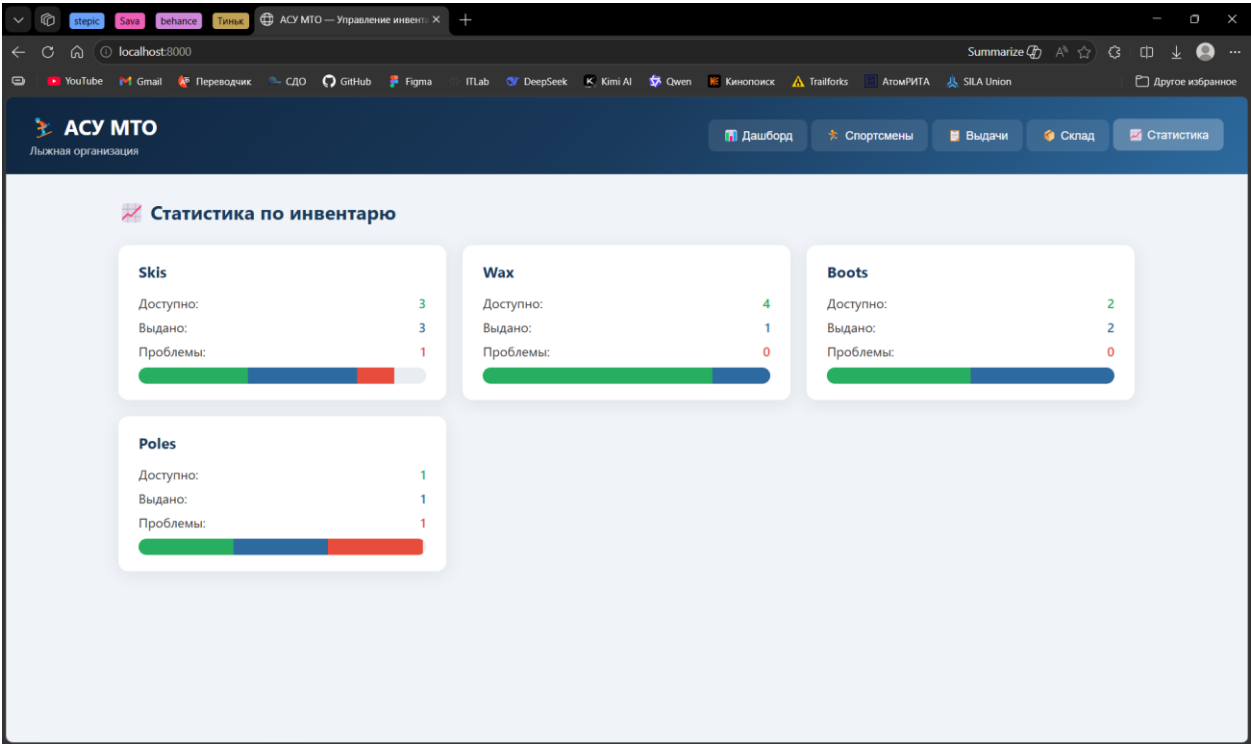


Рисунок 7.18 – Вкладка «Статистика»

8. Средства защиты информации для проектируемой АСУ

8.1. Угрозы безопасности

8.1.1. Угрозы безопасности хранения и обработки информации на рабочих станциях

На автоматизированных рабочих местах (АРМ) пользователей АСУ МТО хранится и обрабатывается следующая информация:

- Учётные данные пользователей (логины, пароли) — кэшируются браузером;
- Персональные данные спортсменов (ФИО, год рождения, спортивный разряд);
- Коммерческая информация о поставщиках (ИНН, КПП, договоры, цены);
- Отчёты по закупкам и движению ТМЦ (выгружаются в Excel/PDF);
- Данные бюджетного контроля (лимиты, фактические расходы).

Таблица 8.1 – Угрозы безопасности на рабочих станциях

№	Угроза	Описание	Источник угрозы	Последствия
1	Несанкционированный доступ к АРМ	Доступ постороннего лица к АРМ, оставленному без блокировки	Внутренний нарушитель	Утечка персональных данных, компрометация учётной записи
2	Заражение вредоносным ПО	Вирусы, трояны, кейлоггеры, шифровальщики, попавшие через фишинг или съёмные носители	Внешний нарушитель	Кража учётных данных, потеря отчётов, шифрование документов
3	Использование теневого ИТ-сервисов	Установка неучтённого ПО (мессенджеры, облачные	Внутренний нарушитель	Утечка конфиденциальных данных за периметр организации

		хранилища) сотрудниками		
4	Утрата или кража мобильного устройства	Потеря планшета Getac T800 или ТСД Zebra TC21 вне помещения	Внешний нарушитель	Доступ к данным АСУ, персональным данным спортсменов
5	Несанкционированное копирование данных	Выгрузка отчётов, базы спортсменов или коммерческой информации на USB-накопитель	Внутренний нарушитель	Утечка коммерческой тайны, нарушение 152-ФЗ
6	Недостаточная очистка кэша браузера	Хранение сессионных cookie и кэшированных страниц после завершения сеанса	Внутренний нарушитель	Повторный вход в систему под чужой учётной записью
7	Физическое повреждение АРМ	Выход из строя SSD-накопителя или блока питания без резервного копирования пользовательских данных	Технический сбой	Потеря пользовательских документов и отчётов

8.1.2. Угрозы безопасности передачи информации по компьютерной сети

Информация, передаваемая по компьютерной сети АСУ МТО, включает:

- Учётные данные пользователей при аутентификации (логин, пароль);
- Персональные данные спортсменов (ФИО, контакты, спортивные показатели);
- Коммерческую информацию о поставщиках (цены, банковские реквизиты);
- Данные складского учёта (штрих-коды, остатки, движение ТМЦ);
- Результаты прогнозирования и аналитики;
- Трафик резервного копирования (полные копии БД, конфигурации, модели).

Таблица 8.2 – Угрозы безопасности передачи информации по сети

№	Угроза	Описание	Участок сети	Последствия
1	Перехват трафика (Sniffing)	Пассивный перехват незашифрованного HTTP-трафика между клиентом и сервером	LAN, Wi-Fi	Компрометация паролей, перехват персональных данных
2	Атака «человек посередине» (MITM)	Активный перехват и модификация трафика между клиентом и сервером	LAN, публичный Wi-Fi, 4G	Подмена данных, перехват учётных данных, внедрение вредоносного кода
3	Несанкционированный доступ к Wi-Fi сети	Подключение постороннего устройства к корпоративной Wi-Fi точке доступа	Wi-Fi (гостевая/тренажерская сеть)	Доступ к внутренним ресурсам сети, перехват трафика
4	Перехват VPN-подключений	Атака на VPN-туннель (IPsec/SSL) при подключении тренажеров с удалённых трасс	Интернет, 4G-канал	Перехват учётных данных, доступ к БД через API
5	DDoS-атака на веб-сервер	Массированная отправка запросов к Nginx/Spring Boot с целью отказа в обслуживании	Интернет → DMZ	Недоступность АСУ для всех пользователей
6	Перехват трафика резервного копирования	Несанкционированный доступ к каналу передачи бэкапов между серверами	LAN (серверная)	Утечка полной копии БД и конфигураций системы
7	Атака на DNS (Spoofing)	Подмена DNS-записей для перенаправления пользователей на фишинговый сайт	Интернет, локальный DNS-сервер	Фишинг учётных данных, компрометация всей АСУ
8	Эксплуатация уязвимостей сетевого оборудования	Использование известных CVE в прошивках коммутаторов Cisco Catalyst 9300 или NGFW UserGate	LAN, периметр сети	Полный контроль над сетевой инфраструктурой, перехват всего трафика

8.2. Защита информации, хранимой и обрабатываемой на рабочих станциях

8.2.1. Анализ защищаемой информации и платформы рабочих станций

Рабочие станции АСУ МТО лыжной организации обрабатывают следующие категории защищаемой информации:

1. Персональные данные спортсменов (ФИО, год рождения, контактные данные, спортивный разряд) — подлежат защите в соответствии с Федеральным законом № 152-ФЗ «О персональных данных».
2. Коммерческая информация о поставщиках (ИНН, КПП, банковские реквизиты, договорные цены, история закупок) — составляет коммерческую тайну организации.
3. Данные бюджетного контроля (лимиты финансирования, фактические расходы, планы закупок) — финансовая информация ограниченного распространения.
4. Учётные данные пользователей (логины, пароли, сессионные ключи) — критичны для обеспечения целостности всей АСУ.
5. Отчёты и аналитические материалы (оборотные сальдовые ведомости, прогнозы потребности) — производные документы, содержащие консолидированную информацию.

Таблица 8.3 – Платформа рабочих станций

Параметр	АРМ (стационарные)	Мобильные устройства
ОС	Astra Linux Common Edition	Android 11 (Getac T800), Android 10 (Zebra TC21)
Пользователи	Менеджеры, кладовщики, тренеры, руководитель, бухгалтер, администратор	Тренеры (на трассе)
Ключевое ПО	Chromium, LibreOffice	Chromium (PWA- приложение АСУ)
Хранимые данные	Кэш браузера, выгруженные отчёты	Кэш PWA, офлайн-данные

8.2.2. Обзор и сравнение программных средств защиты

8.2.2.1. Антивирусная защита

В соответствии с политикой импортозамещения и требованиями к государственным учреждениям рассмотрены антивирусные решения, сертифицированные ФСТЭК России и совместимые с Astra Linux.

Таблица 8.4 – Сравнение антивирусных решений

Критерий	Kaspersky Endpoint Security	Dr.Web Enterprise Security Suite	ClamAV
Разработчик	Лаборатория Касперского (РФ)	Dr. Web (РФ)	Сообщество (Open Source)
Сертификация ФСТЭК	Да (НДВ-4, СКЗИ)	Да (НДВ-4)	Нет
Совместимость с Astra Linux	Да (специальная версия)	Да	Да (в репозитории)
Сигнатурный анализ	Да	Да	Да
Эвристический анализ	Да	Да	Нет
Поведенческий анализ	Да	Да	Нет
Централизованное управление	Kaspersky Security Center	Dr. Web Control Center	Нет
Защита от шифровальщиков	Да (Kaspersky Anti-Ransomware)	Да (Dr. Web Process Heuristic)	Нет
Стоимость	Платная	Платная	Бесплатная

8.2.2.2. Защита от несанкционированного доступа (СЗИ от НСД)

Для защиты АРМ от НСД на уровне операционной системы необходимы средства, обеспечивающие идентификацию и аутентификацию пользователей, контроль целостности, блокировку при бездействии.

Таблица 8.5 – Сравнение СЗИ от НСД

Критерий	Secret Net Studio	Dallas Lock Linux	Astra Linux Special Edition (СЗИ встроенная)
Разработчик	Код Безопасности (РФ)	Конфидент (РФ)	РусБИТех (РФ)
Сертификация ФСТЭК	Да (НДВ-4, межсетевые экраны)	Да (НДВ-4)	Да (встроенный СЗИ Astra Linux SE)
Уровень контроля	Высокий (мандатный доступ)	Средний (дискреционный доступ)	Высокий (мандатный + дискреционный)
Замкнутая программная среда	Да	Нет	Да

Контроль подключаемых устройств	Да (USB, съёмные носители)	Да	Да
Блокировка экрана при бездействии	Да	Да	Да
Централизованное управление	Да (Secret Net Studio)	Да (Dallas Lock Server)	Да (Astra Linux Directory)

8.2.2.3. Управление мобильными устройствами (MDM)

Для управления мобильными устройствами тренеров (Getac T800, 3 шт.) и терминалами сбора данных (Zebra TC21, 2 шт.) требуется MDM-решение, обеспечивающее удалённую блокировку, шифрование и контроль.

Таблица 8.6 – Сравнение MDM-решений

Критерий	SafeMobile	VMWare Workspace ONE (AirWatch)	Microsoft Intune
Разработчик	Код Безопасности (РФ)	Broadcom (США)	Microsoft (США)
Сертификация ФСТЭК	Да	Нет	Нет
Удалённая блокировка/стирание	Да	Да	Да
Полное шифрование устройства	Да	Да	Да
Контроль установки приложений	Да (белые списки)	Да	Да
Геолокация устройства	Да	Да	Да
Контроль целостности ОС	Да	Да	Да
Импортозамещение	Да (реестр отечественного ПО)	Нет	Нет

8.2.2.4. Предотвращение утечек данных (DLP)

Для предотвращения утечек через печать, копирование на съёмные носители и отправку данных по электронной почте с АРМ требуется DLP-система, интегрированная с Astra Linux.

Таблица 8.7 – Сравнение DLP-решений

Критерий	Solar Dozor	InfoWatch Traffic Monitor	DeviceLock DLP
Разработчик	ГК Солар (РФ)	InfoWatch (РФ)	Смарт Лайн Инк (РФ)
Сертификация ФСТЭК	Да	Да	Да
Контроль USB-портов	Да	Да	Да
Контроль печати	Да (теневой мониторинг)	Да	Да

Контроль сетевых каналов	Да (HTTP/HTTPS, SMTP, FTP)	Да	Да
Анализ контента по ключевым словам	Да	Да	Да
Совместимость с Astra Linux	Да	Да (специальная сборка)	Да
Централизованное управление	Да	Да	Да

8.2.3. Выбранный комплекс программных средств защиты

На основе анализа угроз, платформы рабочих станций и критериев сравнения выбран следующий комплекс программных средств защиты:

Таблица 8.8 – Выбранный комплекс средств защиты

№	Категория защиты	Выбранное средство	Обоснование выбора
1	Антивирусная защита	Kaspersky Endpoint Security для Linux (версия для Astra Linux)	Сертифицирован ФСТЭК России (НДВ-4), совместим с Astra Linux, имеет централизованное управление (Kaspersky Security Center), включает защиту от шифровальщиков и поведенческий анализ
2	СЗИ от НСД	Astra Linux Special Edition (встроенный СЗИ) + Secret Net Studio (для АРМ с критичными данными)	Astra Linux SE уже содержит сертифицированный СЗИ от НСД (мандатный и дискреционный доступ). Для АРМ руководителя и бухгалтера дополнительно разворачивается Secret Net Studio для усиленного контроля
3	Управление мобильными устройствами (MDM)	SafeMobile (Код Безопасности)	Сертифицирован ФСТЭК, входит в реестр отечественного ПО, обеспечивает удалённую блокировку/стирание, шифрование, геолокацию
4	Предотвращение утечек данных (DLP)	DeviceLock DLP	Сертифицирован ФСТЭК, обеспечивает контроль USB-портов, печати и сетевых каналов на Astra Linux, минимальное потребление ресурсов
5	Управление учётными записями и аутентификация	Keycloak (Open Source, уже в составе АСУ)	Централизованная аутентификация (SSO), поддержка MFA (двухфакторной аутентификации), ролевая модель доступа, аудит событий входа
6	Резервное копирование АРМ	Bacula (уже в составе АСУ)	Автоматическое резервное копирование пользовательских данных АРМ по расписанию на сервер бэкапа

Таблица 8.9 – Соответствие средств защиты выявленным угрозам

№	Угроза	Средство защиты	Механизм нейтрализации
1	Несанкционированный доступ к АРМ	Astra Linux SE (СЗИ НСД) + Keycloak (MFA)	Блокировка экрана при бездействии (5 мин), парольная политика, двухфакторная аутентификация
2	Заражение вредоносным ПО	Kaspersky Endpoint Security	Сигнатурный + эвристический + поведенческий анализ, защита от шифровальщиков
3	Использование теневого ИТ-сервисов	DeviceLock DLP + NGFW UserGate (из ПП13)	Блокировка облачных хранилищ и мессенджеров, контроль HTTP/HTTPS
4	Утрата или кража мобильного устройства	SafeMobile (MDM)	Полное шифрование (Android FBE), удалённая блокировка/стирание, геолокация
5	Несанкционированное копирование данных	DeviceLock DLP + Secret Net Studio	Блокировка USB-портов, аудит операций выгрузки, теневой мониторинг печати
6	Недостаточная очистка кэша браузера	Настройки Chromium (политики) + Keycloak	Автоочистка кэша при выходе, HttpOnly/Secure cookie, короткий срок жизни сессии
7	Физическое повреждение АРМ	Bacula (резервное копирование)	Автоматическое резервное копирование пользовательских данных на сервер бэкапа по расписанию

8.3. Обеспечение безопасности передачи информации по компьютерной сети

8.3.1. Анализ защищаемых каналов передачи данных

На основе схемы компьютерной сети, выделены следующие каналы передачи данных, требующие защиты. Каждый канал характеризуется типом передаваемого трафика, используемой технологией и участниками информационного обмена.

Таблица 8.10 – Защищаемые каналы передачи данных

№	Канал	Тип трафика	Технология	Участники обмена
1	WAN (внешний периметр)	Веб-интерфейс АСУ, VPN-подключения	HTTPS (TCP/443), IPsec/SSL	NGFW ↔ Интернет, тренеры (VPN)
2	LAN (внутренняя сеть)	HTTP-запросы к API (Spring Boot), SQL-запросы к БД	HTTP/HTTPS (TCP), SQL (TCP/5432)	АРМ ↔ Серверы, Сервер ↔ Сервер

3	Серверная сеть (VLAN 10)	Бэкапы БД и файлов, репликация данных	Bacula/Veeam (TCP), NFS/SSH	Серверы ↔ Сервер бэкапа
4	Wi-Fi (VLAN 30)	PWA-приложение АСУ, синхронизация ТСД	HTTPS (TCP/443), Wi-Fi 5 (802.11ac)	ТСД ↔ Точка доступа ↔ Коммутатор
5	VPN (удалённый доступ)	Веб-интерфейс АСУ, выгрузка отчётов	IPsec IKEv2 / SSL VPN	Тренеры (Getac T800) ↔ NGFW
6	Видеонаблюдение (VLAN 40)	Видеопоток (RTSP), запись на NVR	RTSP (TCP/UDP)	IP-камеры ↔ NVR-регистратор

8.3.2. Обзор и сравнение средств защиты сетевой передачи

8.3.2.1. Межсетевые экраны нового поколения (NGFW)

Межсетевой экран нового поколения (Next-Generation Firewall, NGFW) является ключевым элементом защиты периметра сети. В отличие от традиционных межсетевых экранов, NGFW обеспечивает фильтрацию трафика на уровнях L3–L7 модели OSI, выполняет функции системы предотвращения вторжений (IPS), VPN-шлюза и контентной фильтрации.

Для проектируемой АСУ МТО рассмотрены три NGFW-решения российского производства, сертифицированные ФСТЭК России.

Таблица 8.11 – Сравнение NGFW-решений

Критерий	UserGate C150	Ideco UTM 150	ViPNet Coordinator HW50
Разработчик	UserGate (РФ)	Айдеко (РФ)	ИнфоТеКС (РФ)
Сертификация ФСТЭК	Да (НДВ-4, МЭ)	Да (НДВ-4, МЭ)	Да (НДВ-4, СКЗИ)
Пропускная способность FW	1 Гбит/с	1 Гбит/с	500 Мбит/с
Пропускная способность с IPS	500 Мбит/с	500 Мбит/с	300 Мбит/с
Пиковая VPN-производительность	300 Мбит/с (IPsec)	200 Мбит/с (IPsec)	200 Мбит/с (IPsec)
Контент-фильтрация	Да (категории URL, приложения)	Да (категории URL)	Да (по реестру Минюста)
Защита от DDoS	Да (SYN-flood, UDP-flood)	Да (SYN-flood)	Да
Централизованное управление	UserGate Management Center	Ideco UTM Manager	ViPNet Administrator
Стоимость (ориентировочно)	От 60 000 Р/год	От 45 000 Р/год	От 90 000 Р/год

8.3.2.2. Системы обнаружения и предотвращения вторжений (IDS/IPS)

Системы обнаружения и предотвращения вторжений (Intrusion Detection/Prevention System) обеспечивают глубокий анализ сетевого трафика на предмет сигнатур известных атак и выявления аномального поведения. IDS/IPS может быть реализована как встроенный модуль NGFW, так и в виде отдельного программного или аппаратного решения.

Таблица 8.12 – Сравнение IDS/IPS решений

Критерий	Suricata	Snort	Континент-АП
Разработчик	Open Information Security Foundation (США)	Cisco (США) / Open Source	Код Безопасности (РФ)
Лицензия	GPLv2	GPLv2	Проприетарная
Сертификация ФСТЭК	Нет (используется как основа)	Нет (используется как основа)	Да
Сигнатурный анализ	Да	Да	Да
Анализ аномалий	Да (статистический и ML)	Да (частично)	Да
Производительность	до 10 Гбит/с	до 1 Гбит/с	до 1 Гбит/с
Интеграция с NGFW	Да (экспорт JSON-логов)	Да (Unified2)	Да (нативный агент)
Русскоязычная документация	Есть	Есть	Есть
Стоимость	Бесплатно	Бесплатно	Платная

8.3.2.3. VPN-шлюзы

VPN-шлюз (Virtual Private Network) обеспечивает защищённое подключение удалённых пользователей к внутренним ресурсам АСУ через зашифрованный туннель. Для проектируемой АСУ МТО критически важен VPN-доступ для тренеров, работающих на удалённых трассах и лыжных базах.

Таблица 8.13 – Сравнение VPN-решений

Критерий	StrongSwan	OpenVPN	ViPNet Client
Разработчик	StrongSwan Project (Швейцария)	OpenVPN Inc. / Community	ИнфоТеКС (РФ)
Лицензия	GPLv2	GPLv2	Проприетарная
Сертификация ФСТЭК	Нет (использует IPsec)	Нет (использует SSL)	Да (СКЗИ)
Протокол	IPsec IKEv2	SSL/TLS	IPsec IKEv2
Шифрование	AES-256-GCM	AES-256-GCM	ГОСТ 28147-89 (Кузнечик)
Производительность	Высокая (в ядре Linux)	Средняя (userspace)	Высокая

Поддержка одновременных клиентов	50+	50+	100+
Импортозамещение (реестр отечественного ПО)	Нет	Нет	Да
Стоимость	Бесплатно	Бесплатно	Платная

8.3.2.4. Средства защиты Wi-Fi сетей

Для защиты беспроводной сети (VLAN 30), к которой подключаются терминалы сбора данных (ТСД) и мобильные устройства тренеров, необходим комплекс мер, включающий современные стандарты шифрования и аутентификации. Выбор стандарта защиты Wi-Fi напрямую влияет на устойчивость сети к атакам перехвата и несанкционированного доступа.

Таблица 8.14 – Сравнение стандартов защиты Wi-Fi

Критерий	WPA2-Enterprise	WPA3-Enterprise	WPA2-PSK
Стандарт	IEEE 802.11i	IEEE 802.11ax (Wi-Fi 6)	IEEE 802.11i
Шифрование	AES-CCMP	AES-256-GCMP	AES-CCMP
Аутентификация	802.1X (RADIUS)	802.1X (RADIUS), Protected Management Frames (PMF)	PSK (предварительно заданный общий ключ)
Защита от перебора пароля	Частичная	Полная (Dragonfly)	Отсутствует
Защита от атак деаутентификации	Отсутствует	Да (PMF)	Отсутствует
Совершенная прямая секретность (PFS)	Да (после аутентификации)	Да (после аутентификации)	Нет
Рекомендация для корпоративной сети	Да (базовый уровень)	Да (настоятельно рекомендован)	Нет (устарел)

8.3.2.5. Средства шифрования каналов и мониторинга трафика

Для контроля и анализа трафика на предмет аномалий и инцидентов безопасности требуется система мониторинга сетевой активности (Network Security Monitoring, NSM). Кроме того, все каналы передачи конфиденциальных данных должны быть зашифрованы с использованием актуальных криптографических протоколов.

Таблица 8.15 – Сравнение систем мониторинга сетевой безопасности

Критерий	Zeek (Bro)	MaxPatrol SIEM	Wireshark
Разработчик	Zeek Project (США, Open Source)	Positive Technologies (РФ)	Wireshark Foundation

Тип системы	NSM (Network Security Monitor)	SIEM (Security Information and Event Management)	Анализатор сетевых протоколов
Сертификация ФСТЭК	Нет	Да	Нет
Анализ в реальном времени	Да	Да	Нет (только захват пакетов)
Выявление аномалий	Да (настраиваемые скрипты)	Да (машинное обучение, корреляция событий)	Нет
Интеграция с существующей экосистемой	Да (экспорт JSON-логов)	Да (агенты, API)	Нет
Русскоязычная документация	Есть	Есть	Есть
Стоимость	Бесплатно	Платная	Бесплатно

8.3.3. Выбранный комплекс средств защиты сети

На основе анализа угроз безопасности передачи информации, архитектуры компьютерной сети и сравнительного анализа средств защиты, представленного в разделе 2, сформирован следующий комплекс программных и аппаратных средств обеспечения безопасности передачи данных.

Таблица 8.16 – Выбранный комплекс средств защиты сети

№	Категория защиты	Выбранное средство	Обоснование выбора
1	Межсетевой экран (NGFW)	UserGate C150	Сертифицирован ФСТЭК, обеспечивает фильтрацию L3–L7, встроенные модули IPS/IDS и VPN-шлюза, контент-фильтрацию, защиту от DDoS-атак. Пропускная способность (1 Гбит/с — FW, 500 Мбит/с — IPS) полностью покрывает потребности сети из 50 пользователей при внешнем канале 100 Мбит/с
2	Система обнаружения и предотвращения вторжений (IDS/IPS)	Встроенный IPS в NGFW UserGate + Suricata (дополнительный сенсор)	Модуль IPS в составе UserGate C150 обеспечивает базовую сигнатурную защиту и блокировку атак на периметре. Suricata (Open Source, GPLv2) разворачивается как дополнительный сетевой сенсор для анализа аномалий трафика с использованием машинного обучения, что усиливает защиту внутренней сети
3	VPN-шлюз	Встроенный VPN-модуль UserGate C150 + StrongSwan (для	Встроенный VPN в NGFW UserGate реализует IPsec IKEv2 + AES-256-GCM для подключения удалённых тренеров. Для мобильных устройств (Getac T800)

		серверов) + ViPNet Client (для мобильных устройств)	используется ViPNet Client (сертифицирован ФСТЭК как СКЗИ), реализующий ГОСТ-шифрование («Кузнечик»). Серверные межсерверные туннели защищаются StrongSwan (Open Source, IPsec в ядре Linux)
4	Защита Wi-Fi	WPA3-Enterprise + RADIUS (FreeRADIUS)	Наивысший уровень защиты беспроводной сети: шифрование AES-256-GCM, полная защита от перебора пароля (алгоритм Dragonfly), обязательные Protected Management Frames (PMF) для предотвращения атак деаутентификации. Аутентификация 802.1X через RADIUS (FreeRADIUS) интегрирована с Keycloak (SSO)
5	Шифрование каналов передачи данных	TLS 1.3 (HTTPS) + DNSSEC	Обязательное использование HTTPS (TLS 1.3) для всего веб-трафика между клиентами и сервером (Nginx + Spring Boot). Отключение устаревших протоколов (TLS 1.1, SSL 3.0). Внедрение DNSSEC на уровне DNS-провайдера для криптографической проверки подлинности DNS-ответов
6	Мониторинг сетевой безопасности	Zeek (Bro) + Zabbix (уже используется, PP5)	Zeek (NSM) обеспечивает глубокий анализ сетевого трафика в реальном времени, выявление аномалий и экспорт событий в формате JSON. Zabbix (выбран в PP5 для мониторинга серверов) расширяется шаблонами для мониторинга сетевых устройств (SNMP), оповещения о CVE-уязвимостях и контроля доступности каналов
7	Фильтрация и защита DNS	DNSSEC + локальный DNS-сервер BIND9	DNSSEC на стороне провайдера защищает от атак подмены DNS-ответов (DNS Spoofing). Локальный кэширующий DNS-сервер BIND9 размещается во внутренней сети (VLAN 10), ускоряет разрешение имён и фильтрует запросы к вредоносным доменам по актуальным чёрным спискам (RPZ — Response Policy Zones)

Таблица 8.17 – Соответствие выбранных средств защиты угрозам передачи информации

№	Угроза	Выбранное средство защиты	Механизм нейтрализации
1	Перехват трафика (Sniffing)	TLS 1.3 (HTTPS) на всех веб-сервисах	Шифрование всего трафика веб-приложения между клиентом (APM, ТСД) и сервером (Nginx). Принудительное отключение устаревших протоколов (TLS 1.1, SSL) и небезопасных cipher suites
2	Атака «человек посередине» (MITM)	TLS 1.3 + HSTS (HTTP Strict	Принудительное использование HTTPS с валидными сертификатами.

		Transport Security) + DNSSEC	HSTS исключает возможность перехвата на этапе первого запроса. DNSSEC предотвращает подмену IP-адреса сервера на уровне DNS
3	Несанкционированный доступ к Wi-Fi сети	WPA3-Enterprise + RADIUS + изоляция VLAN 30	Аутентификация каждого устройства через 802.1X (индивидуальные учётные данные). Гостевая/мобильная сеть изолирована в отдельный VLAN 30 с ACL-правилами, запрещающими доступ к серверной сети (VLAN 10)
4	Перехват VPN-подключений	IPsec IKEv2 + AES-256-GCM (StrongSwan) + ГОСТ-шифрование (ViPNet Client)	Шифрование туннеля на уровне IP-пакетов с использованием стойких алгоритмов (AES-256). Для мобильных устройств — ГОСТ-шифрование (сертифицированное СКЗИ). Отклонение подключений с невалидными или просроченными сертификатами
5	DDoS-атака на веб-сервер	UserGate C150 (защита от DDoS) + Nginx rate limiting + Kong/APISEX	Многоуровневая защита: фильтрация SYN-flood и UDP-flood на уровне NGFW; ограничение числа запросов с одного IP-адреса (rate limiting) на уровне Nginx и API-шлюза Kong; блокировка IP-адресов, превысивших порог
6	Перехват трафика резервного копирования	Bacula (шифрование на стороне агента) + изоляция в VLAN 10	Все резервные копии шифруются агентом Bacula на стороне сервера-источника до передачи по сети. Сервер бэкапа и серверы-источники находятся в изолированном серверном VLAN 10, недоступном из пользовательской сети (VLAN 20)
7	DNS Spoofing	DNSSEC + локальный DNS-сервер BIND9 с RPZ	Криптографическая верификация каждого DNS-ответа на стороне клиента через DNSSEC. Локальный DNS-сервер BIND9 фильтрует запросы к вредоносным доменам по спискам Response Policy Zones, блокируя фишинговые сайты
8	Эксплуатация уязвимостей сетевого оборудования	Zabbix (мониторинг) + регулярные обновления прошивок	Непрерывный мониторинг версий прошивок коммутаторов Cisco Catalyst 9300 и NGFW UserGate C150. Автоматические оповещения администратора о выходе обновлений безопасности и новых CVE. Плановая процедура обновления прошивок в окна обслуживания

Заключение

В результате выполнения курсовой работы была спроектирована автоматизированная система управления материально-техническим обеспечением лыжной спортивной организации.

В ходе работы были получены следующие результаты:

1. Анализ предметной области. С использованием технологии OSINT проведен анализ деятельности спортивных организаций в области обеспечения лыжным инвентарем. Выявлены ключевые бизнес-процессы: сбор заявок от тренеров, формирование заказов поставщикам, складской учет, выдача инвентаря спортсменам и контроль возврата.

2. Функциональные и нефункциональные требования. Разработано 20 функциональных требований (включая ведение справочников, регистрацию заявок, бюджетный контроль, формирование заказов, приемку и маркировку товаров, выдачу и возврат инвентаря) и 20 нефункциональных требований (многопользовательский доступ, разграничение прав, круглосуточная доступность, производительность, безопасность и др.).

3. UML-моделирование. Система разбита на три модуля: «Управление закупками, планирование и бюджетный контроль», «Складской учет, логистика и управление имуществом», «Администрирование и отчетность». Для каждого модуля построены диаграммы вариантов использования. Разработаны три диаграммы последовательности (формирование и согласование заявки, приемка и маркировка инвентаря, выдача инвентаря и контроль возврата) и диаграмма классов, содержащая 16 классов.

4. Программно-аппаратная платформа. Определен состав оборудования (сервер БД, сервер приложений, сервер аналитики, сервер резервного копирования, АРМ, ТСД, принтеры этикеток). Выбрано программное обеспечение (Astra Linux, PostgreSQL, Spring Boot, React.js, Python/statsmodels). Разработан модуль прогнозирования потребности в

лыжном инвентаре на основе экспоненциального сглаживания (библиотека statsmodels).

5. База данных. На основе диаграммы классов построена ER-диаграмма, содержащая 19 таблиц. Выбрана СУБД PostgreSQL (обоснование: импортозамещение, поддержка JSONB, кроссплатформенность, бесплатность). Созданы SQL-скрипты создания таблиц и заполнения тестовыми данными (минимум по 10 записей в каждую таблицу). Выполнен тестовый запрос на выборку спортсменов-должников.

6. Схема сети и виртуализация. Выполнен расчет характеристик серверного и клиентского оборудования. Построена схема компьютерной сети. Выбрана среда виртуализации Oracle VirtualBox (обоснование: кроссплатформенность, бесплатность, поддержка снапшотов). Создан виртуальный стенд из трех виртуальных машин (сервер БД, сервер приложений и аналитики, клиентская машина). Проведена настройка сетевого взаимодействия.

7. Web-интерфейс. Разработан информационный веб-портал (HTML/CSS/JavaScript) с разделами: главная, о системе, результаты работы (складской учёт, активные выдачи, прогноз потребности, слайдер отзывов), контакты. Создан программный модуль с Web-интерфейсом для работы с БД на FastAPI + HTML/JS, реализующий пять вкладок: дашборд с KPI, управление спортсменами, выдачу и возврат инвентаря, просмотр склада, статистику.

8. Безопасность. Проведен анализ угроз безопасности (7 угроз на рабочих станциях, 8 угроз в сети). Выбран комплекс средств защиты: Kaspersky Endpoint Security (антивирус), Astra Linux SE (СЗИ от НСД), SafeMobile (MDM), DeviceLock DLP (предотвращение утечек), UserGate C150 (NGFW), WPA3-Enterprise + RADIUS (защита Wi-Fi), DNSSEC + BIND9 (защита DNS).

Все задачи курсовой работы выполнены в полном объеме. Разработанная АСУ МТО позволяет автоматизировать процессы

планирования закупок, складского учета и контроля выдачи инвентаря, что обеспечивает прозрачность движения материальных ценностей, снижение дефицита и избытка товаров, а также повышение эффективности использования бюджетных средств спортивной организации.

Список источников

1. ГОСТ 34.601-90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания. – Введ. 1992-01-01. – М.: Изд-во стандартов, 1990. – 12 с.
2. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. – Введ. 2011-12-01. – М.: Стандартиформ, 2011. – 98 с.
3. ГОСТ Р ИСО 9241-210-2016. Эргономика взаимодействия человек-система. Человеко-ориентированное проектирование интерактивных систем. – Введ. 2017-06-01. – М.: Стандартиформ, 2016. – 38 с.
4. Федеральный закон от 04.12.2007 № 329-ФЗ (ред. от 08.08.2024) «О физической культуре и спорте в Российской Федерации». – Доступ из справ.-правовой системы «КонсультантПлюс».
5. Федеральный закон от 27.07.2006 № 152-ФЗ (ред. от 21.12.2021) «О персональных данных». – Доступ из справ.-правовой системы «КонсультантПлюс».
6. Буч, Г. Язык UML. Руководство пользователя / Г. Буч, Д. Рамбо, И. Якобсон. – 2-е изд. – М.: ДМК Пресс, 2021. – 496 с.
7. Ларман, К. Применение UML и шаблонов проектирования / К. Ларман. – 3-е изд. – М.: Вильямс, 2020. – 736 с.
8. Маккинни, У. Python и анализ данных / У. Маккинни. – 2-е изд. – М.: ДМК Пресс, 2020. – 540 с.
9. Холл, М. PostgreSQL 17. Администрирование и оптимизация / М. Холл. – М.: ДМК Пресс, 2024. – 380 с.
10. Документация PostgreSQL [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/> (дата обращения: 17.05.2026).
11. Документация FastAPI [Электронный ресурс]. – Режим доступа: <https://fastapi.tiangolo.com/> (дата обращения: 17.05.2026).

12. Документация statsmodels [Электронный ресурс]. – Режим доступа: <https://www.statsmodels.org/stable/> (дата обращения: 17.05.2026).
13. Документация Oracle VirtualBox [Электронный ресурс]. – Режим доступа: <https://www.virtualbox.org/wiki/Documentation> (дата обращения: 17.05.2026).
14. UserGate. Документация NGFW [Электронный ресурс]. – Режим доступа: <https://usergate.com/products/next-generation-firewall> (дата обращения: 17.05.2026).
15. Zebra Products: Printers, Barcode Scanners and More [Электронный ресурс]. – Режим доступа: <https://www.zebra.com/us/en/products.html> (дата обращения: 17.05.2026).
16. Kaspersky Lab. Kaspersky Endpoint Security [Электронный ресурс]. – Режим доступа: <https://www.kaspersky.ru/enterprise-security> (дата обращения: 17.05.2026).

Приложения

Приложение 1. Программный код модуля для доработки

Листинг 1 - Файл requirements.txt

```
pandas==2.2.0
numpy==1.26.0
statsmodels==0.14.1
scikit-learn==1.4.0
psycopg2-binary==2.9.9
sqlalchemy==2.0.25
matplotlib==3.8.2
loguru==0.7.2
joblib==1.3.2
python-dotenv==1.0.0
```

Листинг 2 – Файл config.py

```
"""Конфигурация модуля прогнозирования."""
import os
from dataclasses import dataclass, field

@dataclass
class DatabaseConfig:
    """Настройки подключения к PostgreSQL."""
    host: str = field(default_factory=lambda: os.getenv("DB_HOST", "localhost"))
    port: int = field(default_factory=lambda: int(os.getenv("DB_PORT", "5432")))
    database: str = field(default_factory=lambda: os.getenv("DB_NAME", "postgres"))
    schema: str = field(default_factory=lambda: os.getenv("DB_SCHEMA", "asu_inventory"))
    username: str = field(default_factory=lambda: os.getenv("DB_USERNAME", "postgres"))
    password: str = field(default_factory=lambda: os.getenv("DB_PASSWORD", ""))

    @property
    def connection_string(self) -> str:
        return f"postgresql://{self.username}:{self.password}@{self.host}:{self.port}/{self.database}"

@dataclass
class ForecastConfig:
    """Настройки модели прогнозирования."""
    forecast_days: int = 90
    seasonality_mode: str = "multiplicative"
    yearly_seasonality: bool = True
    weekly_seasonality: bool = True
    min_train_days: int = 30

@dataclass
class AppConfig:
    """Общая конфигурация приложения."""
    db: DatabaseConfig = field(default_factory=DatabaseConfig)
    forecast: ForecastConfig = field(default_factory=ForecastConfig)
    models_dir: str = field(default_factory=lambda: os.getenv("MODELS_DIR", "./models"))
```

```
config = AppConfig()
```

Листинг 3 – Файл database.py

```
"""Модуль для работы с базой данных PostgreSQL."""
import pandas as pd
from sqlalchemy import create_engine, text
from loguru import logger
from typing import Optional, Dict, Any

from config import config

class DatabaseManager:
    """Менеджер для работы с БД АСУ."""

    def __init__(self):
        self.engine = create_engine(
            config.db.connection_string,
            connect_args={'options': f'-c search_path={con-
fig.db.schema},public'}
        )
        logger.info(f"Подключение к БД: {config.db.host}:{con-
fig.db.port}/{config.db.database}")
        logger.info(f"Схема: {config.db.schema}")

    def get_historical_consumption(self, nomenclature_id: int, start_date:
str, end_date: Optional[str] = None) -> pd.DataFrame:
        """Загрузка исторических данных по расходу инвентаря."""
        query = f"""
        SELECT
            DATE(d.issuance_date) as ds,
            COUNT(*) as y
        FROM issuance_document d
        JOIN inventory_item i ON d.inventory_item_id = i.inventory_item_id
        WHERE i.nomenclature_id = :nomenclature_id
            AND d.issuance_date >= :start_date
        """
        params = {"nomenclature_id": nomenclature_id, "start_date":
start_date}

        if end_date:
            query += " AND d.issuance_date <= :end_date"
            params["end_date"] = end_date

        query += " GROUP BY DATE(d.issuance_date) ORDER BY ds"

        logger.info(f"Загрузка данных для номенклатуры ID={nomencla-
ture_id}")

        with self.engine.connect() as conn:
            df = pd.read_sql(text(query), conn, params=params)

            if not df.empty:
                df['ds'] = pd.to_datetime(df['ds'])
                date_range = pd.date_range(start=df['ds'].min(),
end=df['ds'].max())
                df = df.set_index('ds').reindex(date_range, fill_value=0).re-
set_index()
                df.columns = ['ds', 'y']

            logger.info(f"Загружено {len(df)} записей")
```

```

        return df

    def get_current_stock(self, nomenclature_id: int) -> int:
        """Получение текущего остатка на складе."""
        query = f"""
        SELECT COUNT(*) as stock
        FROM inventory_item
        WHERE nomenclature_id = :nomenclature_id
            AND inventory_status = 'AVAILABLE'
        """
        with self.engine.connect() as conn:
            result = conn.execute(text(query), {"nomenclature_id": nomenclature_id})
            return result.scalar() or 0

    def get_nomenclature_info(self, nomenclature_id: int) -> Dict[str, Any]:
        """Получение информации о номенклатуре."""
        query = f"""
        SELECT
            nomenclature_id,
            nomenclature_name,
            nomenclature_category,
            min_stock_level,
            standard_service_life,
            manufacturer
        FROM nomenclature
        WHERE nomenclature_id = :nomenclature_id
        """
        with self.engine.connect() as conn:
            result = conn.execute(text(query), {"nomenclature_id": nomenclature_id})
            row = result.fetchone()
            if row:
                return dict(row._mapping)
            return {}

    def get_all_nomenclature(self) -> pd.DataFrame:
        """Получение списка всей номенклатуры."""
        query = f"""
        SELECT
            nomenclature_id,
            nomenclature_name,
            nomenclature_category,
            min_stock_level,
            manufacturer
        FROM nomenclature
        ORDER BY nomenclature_id
        """
        with self.engine.connect() as conn:
            return pd.read_sql(text(query), conn)

```

Листинг 4 – Файл *forecast_model.py*

```

"""Модель прогнозирования потребности в лыжном инвентаре (statsmodels)."""
import pandas as pd
import numpy as np
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from loguru import logger
import joblib
import os
from typing import Optional, Dict, Any
from datetime import datetime, timedelta

```

```

from config import config

class SkiInventoryForecastModel:
    """Модель прогнозирования на основе экспоненциального сглаживания."""

    def __init__(self, model_name: str = "ski_forecast"):
        self.model_name = model_name
        self.model = None
        self.training_metrics: Dict[str, float] = {}
        self._simple_mean = None
        self._simple_std = None
        self._seasonal_periods = 7

    def prepare_data(self, df: pd.DataFrame):
        """Подготовка данных."""
        df = df.copy()
        df['ds'] = pd.to_datetime(df['ds'])
        df = df.set_index('ds')
        df = df.asfreq('D', fill_value=0)
        series = df['y'].astype(float)
        logger.info(f"Данные подготовлены: {len(series)} точек")
        return series

    def fit(self, df: pd.DataFrame) -> 'SkiInventoryForecastModel':
        """Обучение модели."""
        logger.info(f"Обучение модели {self.model_name}...")
        series = self.prepare_data(df)

        if len(series) < config.forecast.min_train_days:
            logger.warning(f"Мало данных: {len(series)} точек. Использую среднее.")
            self._simple_mean = series.mean()
            self._simple_std = series.std()
            return self

        # Определяем seasonal_periods на основе длины ряда
        data_len = len(series)
        if data_len < 14:
            self._seasonal_periods = 3
            use_seasonal = False
            use_trend = False
        elif data_len < 30:
            self._seasonal_periods = 5
            use_seasonal = True
            use_trend = False
        elif data_len < 60:
            self._seasonal_periods = 7
            use_seasonal = True
            use_trend = False
        else:
            self._seasonal_periods = 7
            use_seasonal = True
            use_trend = True

        logger.info(f"Параметры: seasonal_periods={self._seasonal_periods}, "
                    f"seasonal={use_seasonal}, trend={use_trend}")

        try:
            if use_seasonal:
                self.model = ExponentialSmoothing(
                    series,
                    seasonal_periods=self._seasonal_periods,

```

```

        trend='add' if use_trend else None,
        seasonal='add',
        damped_trend=True,
        initialization_method='estimated'
    ).fit()
else:
    self.model = ExponentialSmoothing(
        series,
        trend=None,
        seasonal=None,
        initialization_method='estimated'
    ).fit()

    fitted = self.model.fittedvalues
    valid_idx = ~np.isnan(fitted)
    if valid_idx.sum() > 0:
        mae = np.mean(np.abs(series[valid_idx] - fitted[valid_idx]))
        mape = np.mean(np.abs((series[valid_idx] - fitted[valid_idx]) /
                                np.where(series[valid_idx] == 0, 1,
series[valid_idx]))) * 100
        self.training_metrics = {'mae': round(mae, 2), 'mape':
round(mape, 1)}
        logger.info(f"Модель обучена. MAE={mae:.2f},
MAPE={mape:.1f}%")
    except Exception as e:
        logger.error(f"Ошибка обучения: {e}. Использую среднее.")
        self.model = None
        self._simple_mean = series.mean()
        self._simple_std = series.std()

    return self

def predict(self, periods: int = None) -> pd.DataFrame:
    """Прогноз."""
    if periods is None:
        periods = config.forecast.forecast_days

    start_date = datetime.now().date()
    dates = [start_date + timedelta(days=i) for i in range(periods)]

    if self.model is not None:
        try:
            forecast_values = self.model.forecast(steps=periods)
            forecast_values = np.maximum(forecast_values, 0)

            # Ограничиваем рост: не больше максимума из исторических
            max_historical = self._simple_mean * 3 if self._simple_mean
else 5
            forecast_values = np.minimum(forecast_values, max_historical)

            return pd.DataFrame({
                'ds': dates,
                'yhat': forecast_values,
                'yhat_lower': np.maximum(forecast_values * 0.6, 0),
                'yhat_upper': np.minimum(forecast_values * 1.5, max_historical * 1.5)
            })
        except Exception as e:
            logger.error(f"Ошибка прогноза: {e}")

    # Запасной вариант: простое среднее

```

```

mean_val = self._simple_mean or 0
std_val = self._simple_std or 0
values = np.full(periods, mean_val)
return pd.DataFrame({
    'ds': dates,
    'yhat': values,
    'yhat_lower': np.maximum(values - 2 * std_val, 0),
    'yhat_upper': values + 2 * std_val
})

def save(self, path: Optional[str] = None):
    if path is None:
        path = os.path.join(config.models_dir, f"{self.model_name}.pkl")
    os.makedirs(os.path.dirname(path), exist_ok=True)
    joblib.dump({
        'model': self.model,
        'model_name': self.model_name,
        'training_metrics': self.training_metrics,
        '_simple_mean': self._simple_mean,
        '_simple_std': self._simple_std,
        '_seasonal_periods': self._seasonal_periods
    }, path)
    logger.info(f"Модель сохранена: {path}")

    @classmethod
    def load(cls, path: str) -> 'SkiInventoryForecastModel':
        data = joblib.load(path)
        instance = cls(model_name=data['model_name'])
        instance.model = data['model']
        instance.training_metrics = data['training_metrics']
        instance._simple_mean = data.get('_simple_mean')
        instance._simple_std = data.get('_simple_std')
        instance._seasonal_periods = data.get('_seasonal_periods', 7)
        logger.info(f"Модель загружена: {path}")
        return instance

```

Листинг 5 – Файл main.py

```

"""Главный скрипт запуска модуля прогнозирования."""
import os
from datetime import datetime
from loguru import logger

from config import config
from database import DatabaseManager
from forecast_model import SkiInventoryForecastModel

def print_header(title: str):
    """Вывод заголовка раздела."""
    print("\n" + "=" * 70)
    print(f" {title}")
    print("=" * 70)

def main():
    """Основная функция."""
    print_header("МОДУЛЬ ПРОГНОЗИРОВАНИЯ ПОТРЕБНОСТИ В ЛЫЖНОМ ИНВЕНТАРЕ")

    # 1. Подключение к БД
    print_header("1. ПОДКЛЮЧЕНИЕ К БАЗЕ ДАННЫХ")
    db = DatabaseManager()

```

```

# 2. Загрузка списка номенклатуры
print_header("2. ДОСТУПНАЯ НОМЕНКЛАТУРА")
nomenclature_df = db.get_all_nomenclature()
print(nomenclature_df.to_string(index=False))

# 3. Выбор номенклатуры для прогноза
print_header("3. ВЫБОР НОМЕНКЛАТУРЫ")
nomenclature_id = 1
info = db.get_nomenclature_info(nomenclature_id)
print(f"Выбрана номенклатура: ID={nomenclature_id}")
print(f"    Наименование:    {info.get('nomenclature_name', '-')}")
print(f"    Категория:        {info.get('nomenclature_category', '-')}")
print(f"    Производитель:    {info.get('manufacturer', '-')}")
print(f"    Мин. остаток:      {info.get('min_stock_level', '-')} шт.")
print(f"    Срок службы:       {info.get('standard_service_life', '-')}
мес.")

# 4. Загрузка исторических данных
print_header("4. ЗАГРУЗКА ИСТОРИЧЕСКИХ ДАННЫХ")
start_date = '2024-01-01'
df = db.get_historical_consumption(nomenclature_id, start_date)
print(f"Загружено {len(df)} записей (с {df['ds'].min().date() if not
df.empty else '-'} "
      f"no {df['ds'].max().date() if not df.empty else '-'}")

if len(df) > 0:
    print("\nПервые 5 записей:")
    print(df.head().to_string(index=False))
    print(f"\nСтатистика: среднее={df['y'].mean():.1f} шт./день, "
          f"максимум={df['y'].max():.0f} шт./день")

# 5. Обучение модели
print_header("5. ОБУЧЕНИЕ МОДЕЛИ")
model = SkiInventoryForecastModel(model_name=f"forecast_nomenclature_{nomenclature_id}")
model.fit(df)
print(f"Метрики качества:")
if model.training_metrics:
    print(f"    MAE (средняя абсолютная ошибка): {model.training_metrics['mae']} шт.")
    print(f"    MAPE (средняя процентная ошибка): {model.training_metrics['mape']}%")

# 6. Прогноз
print_header("6. ПРОГНОЗ НА 90 ДНЕЙ")
forecast = model.predict(periods=90)

forecast['month'] = forecast['ds'].apply(lambda x: x.month)
month_names = {5: 'Май', 6: 'Июнь', 7: 'Июль', 8: 'Август', 9: 'Сентябрь', 10: 'Октябрь'}
forecast['month_name'] = forecast['month'].map(month_names)
monthly = forecast.groupby('month_name')['yhat'].sum().round(1)

print("\nПрогноз потребности по месяцам:")
for month, value in monthly.items():
    bar = "█" * max(1, int(value))
    print(f"    {month:10}: {value:6.1f} шт.    {bar}")

total_demand = forecast['yhat'].sum()
total_lower = forecast['yhat_lower'].sum()
total_upper = forecast['yhat_upper'].sum()

print(f"\nСуммарный прогноз на 90 дней:")
print(f"    Ожидаемый спрос:    {total_demand:.1f} шт.")

```

```

    print(f"    Доверительный интервал: [{total_lower:.1f} – {total_upper:.1f}] шт.")

    # 7. Сравнение с текущим остатком
    print_header("7. РЕКОМЕНДАЦИИ ПО ЗАКУПКЕ")
    current_stock = db.get_current_stock(nomenclature_id)
    min_stock = info.get('min_stock_level', 5)

    print(f"    Текущий остаток:      {current_stock} шт.")
    print(f"    Минимальный запас:      {min_stock} шт.")
    print(f"    Прогноз спроса:          {total_demand:.1f} шт.")

    if current_stock >= total_demand + min_stock:
        print(f"\n    ✓ Запас достаточен. Закупка не требуется.")
    else:
        need = total_demand + min_stock - current_stock
        print(f"\n    ! Рекомендуется закупить: {need:.1f} шт.")

    # 8. Сохранение модели
    print_header("8. СОХРАНЕНИЕ МОДЕЛИ")
    model_path = os.path.join(config.models_dir, f"model_nomenclature_{nomenclature_id}.pkl")
    model.save(model_path)
    print(f"Модель сохранена в: {model_path}")

    print_header("РАБОТА МОДУЛЯ ЗАВЕРШЕНА")

if __name__ == "__main__":
    main()

```


Приложение 2. SQL скрипты создания и заполнения БД

Листинг 6 – Скрипт создания таблиц (create tables.sql)

```
-- Базовая таблица пользователей
CREATE TABLE "user" (
    user_id SERIAL PRIMARY KEY,
    full_name VARCHAR(200) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(20),
    login VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(30) NOT NULL CHECK (role IN ('athlete', 'coach', 'storekeeper',
'purchase_manager', 'manager', 'accountant', 'administrator')),
    registration_date DATE DEFAULT CURRENT_DATE,
    is_active BOOLEAN DEFAULT TRUE,
    user_created_at TIMESTAMP DEFAULT NOW(),
    user_updated_at TIMESTAMP DEFAULT NOW()
);

-- Справочник складов
CREATE TABLE warehouse (
    warehouse_id SERIAL PRIMARY KEY,
    warehouse_name VARCHAR(100) NOT NULL,
    address TEXT,
    type VARCHAR(50) CHECK (type IN ('main', 'reserve', 'service'))
);

-- Тренеры
CREATE TABLE coach (
    coach_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES "user"(user_id) ON DELETE CASCADE,
    specialization VARCHAR(100),
    coach_category VARCHAR(50)
);

-- Группы спортсменов
CREATE TABLE "group" (
    group_id SERIAL PRIMARY KEY,
    group_name VARCHAR(100) NOT NULL,
    coach_id INTEGER REFERENCES coach(coach_id) ON DELETE SET NULL,
    group_created_at TIMESTAMP DEFAULT NOW()
);

-- Спортсмены
CREATE TABLE athlete (
    athlete_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES "user"(user_id) ON DELETE CASCADE,
    sports_category VARCHAR(50),
    birth_year INTEGER,
    group_id INTEGER REFERENCES "group"(group_id) ON DELETE SET NULL,
    coach_id INTEGER REFERENCES coach(coach_id) ON DELETE SET NULL,
    has_active_debt BOOLEAN DEFAULT FALSE
);

-- Кладовщики
CREATE TABLE storekeeper (
    storekeeper_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES "user"(user_id) ON DELETE CASCADE,
    warehouse_id INTEGER REFERENCES warehouse(warehouse_id) ON DELETE SET NULL,
    certificate_number VARCHAR(50)
```

```

);

-- Менеджеры по закупкам
CREATE TABLE purchase_manager (
    purchase_manager_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES "user"(user_id) ON DELETE CASCADE,
    department VARCHAR(100)
);

-- Руководители
CREATE TABLE manager (
    manager_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES "user"(user_id) ON DELETE CASCADE,
    position VARCHAR(100),
    manager_access_level INTEGER DEFAULT 1 CHECK (manager_access_level BETWEEN 1
AND 3)
);

-- Бухгалтеры
CREATE TABLE accountant (
    accountant_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES "user"(user_id) ON DELETE CASCADE,
    section VARCHAR(100)
);

-- Администраторы
CREATE TABLE administrator (
    administrator_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES "user"(user_id) ON DELETE CASCADE,
    administrator_access_level VARCHAR(30) DEFAULT 'FULL' CHECK (administrator_ac-
cess_level IN ('FULL', 'PARTIAL', 'READONLY'))
);

-- Номенклатура
CREATE TABLE nomenclature (
    nomenclature_id SERIAL PRIMARY KEY,
    article VARCHAR(50) UNIQUE NOT NULL,
    nomenclature_name VARCHAR(200) NOT NULL,
    nomenclature_category VARCHAR(50) NOT NULL CHECK (nomenclature_category IN
('skis', 'boots', 'poles', 'bindings', 'wax', 'accessories')),
    unit VARCHAR(20) NOT NULL DEFAULT 'pcs',
    standard_service_life INTEGER,
    min_stock_level INTEGER DEFAULT 0,
    manufacturer VARCHAR(100),
    nomenclature_created_at TIMESTAMP DEFAULT NOW()
);

-- Поставщики
CREATE TABLE supplier (
    supplier_id SERIAL PRIMARY KEY,
    supplier_name VARCHAR(200) NOT NULL,
    inn VARCHAR(12) UNIQUE NOT NULL,
    kpp VARCHAR(9),
    legal_address TEXT,
    contact_person VARCHAR(200),
    supplier_phone VARCHAR(20),
    supplier_email VARCHAR(100),
    rating DECIMAL(3,2) DEFAULT 0 CHECK (rating BETWEEN 0 AND 5),
    supplier_created_at TIMESTAMP DEFAULT NOW()
);

-- Заявки на инвентарь

```

```

CREATE TABLE purchase_request (
    purchase_request_id SERIAL PRIMARY KEY,
    purchase_request_number VARCHAR(50) UNIQUE NOT NULL,
    purchase_request_created_date TIMESTAMP DEFAULT NOW(),
    purchase_request_status VARCHAR(30) NOT NULL DEFAULT 'DRAFT' CHECK (purchase_request_status IN ('DRAFT', 'ON_APPROVAL', 'APPROVED', 'REJECTED')),
    author_id INTEGER NOT NULL REFERENCES "user"(user_id),
    approved_by INTEGER REFERENCES "user"(user_id),
    planned_amount DECIMAL(14,2),
    purchase_request_comment TEXT,
    purchase_request_created_at TIMESTAMP DEFAULT NOW(),
    purchase_request_updated_at TIMESTAMP DEFAULT NOW()
);

-- Позиции заявок
CREATE TABLE purchase_request_item (
    purchase_request_item_id SERIAL PRIMARY KEY,
    purchase_request_id INTEGER NOT NULL REFERENCES purchase_request(purchase_request_id) ON DELETE CASCADE,
    nomenclature_id INTEGER NOT NULL REFERENCES nomenclature(nomenclature_id) ON DELETE RESTRICT,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    expected_price DECIMAL(12,2),
    priority VARCHAR(20) DEFAULT 'MEDIUM' CHECK (priority IN ('HIGH', 'MEDIUM', 'LOW'))
);

-- Заказы поставщикам
CREATE TABLE purchase_order (
    purchase_order_id SERIAL PRIMARY KEY,
    purchase_order_number VARCHAR(50) UNIQUE NOT NULL,
    purchase_order_created_date DATE DEFAULT CURRENT_DATE,
    purchase_order_status VARCHAR(30) NOT NULL DEFAULT 'DRAFT' CHECK (purchase_order_status IN ('DRAFT', 'SENT', 'IN_TRANSIT', 'DELIVERED', 'CLOSED')),
    supplier_id INTEGER NOT NULL REFERENCES supplier(supplier_id),
    purchase_manager_id INTEGER NOT NULL REFERENCES purchase_manager(purchase_manager_id),
    total_amount DECIMAL(14,2),
    expected_delivery_date DATE,
    purchase_order_created_at TIMESTAMP DEFAULT NOW()
);

-- Единицы инвентаря
CREATE TABLE inventory_item (
    inventory_item_id SERIAL PRIMARY KEY,
    barcode VARCHAR(50) UNIQUE NOT NULL,
    nomenclature_id INTEGER NOT NULL REFERENCES nomenclature(nomenclature_id) ON DELETE RESTRICT,
    serial_number VARCHAR(100),
    inventory_status VARCHAR(30) NOT NULL DEFAULT 'AVAILABLE' CHECK (inventory_status IN ('AVAILABLE', 'RESERVED', 'ISSUED', 'UNDER_REPAIR', 'DEFECTIVE', 'WRITTEN_OFF')),
    warehouse_id INTEGER REFERENCES warehouse(warehouse_id) ON DELETE SET NULL,
    receipt_date DATE,
    write_off_date DATE,
    purchase_cost DECIMAL(12,2),
    size VARCHAR(20),
    athlete_id INTEGER REFERENCES athlete(athlete_id) ON DELETE SET NULL,
    inventory_notes TEXT,
    inventory_created_at TIMESTAMP DEFAULT NOW()
);

```

```

-- Приходные накладные
CREATE TABLE receipt_invoice (
    receipt_invoice_id SERIAL PRIMARY KEY,
    receipt_invoice_number VARCHAR(50) UNIQUE NOT NULL,
    receipt_invoice_date DATE DEFAULT CURRENT_DATE,
    purchase_order_id INTEGER REFERENCES purchase_order(purchase_order_id) ON DE-
LETE SET NULL,
    storekeeper_id INTEGER NOT NULL REFERENCES storekeeper(storekeeper_id),
    receipt_invoice_status VARCHAR(30) DEFAULT 'POSTED' CHECK (receipt_invoice_sta-
tus IN ('DRAFT', 'POSTED', 'CANCELLED')),
    actual_amount DECIMAL(14,2),
    receipt_invoice_created_at TIMESTAMP DEFAULT NOW()
);

-- Документы выдачи
CREATE TABLE issuance_document (
    issuance_document_id SERIAL PRIMARY KEY,
    inventory_item_id INTEGER NOT NULL REFERENCES inventory_item(inventory_item_id)
ON DELETE RESTRICT,
    athlete_id INTEGER NOT NULL REFERENCES athlete(athlete_id) ON DELETE RESTRICT,
    storekeeper_id INTEGER NOT NULL REFERENCES storekeeper(storekeeper_id),
    issuance_date DATE NOT NULL,
    planned_return_date DATE,
    actual_return_date DATE,
    issuance_document_status VARCHAR(30) NOT NULL DEFAULT 'ACTIVE' CHECK (issu-
ance_document_status IN ('ACTIVE', 'RETURNED', 'OVERDUE', 'EXTENDED')),
    condition_on_issue VARCHAR(50) DEFAULT 'NEW',
    condition_on_return VARCHAR(50),
    issuance_document_notes TEXT,
    issuance_document_created_at TIMESTAMP DEFAULT NOW()
);

-- Журнал операций (аудит)
CREATE TABLE operation_log (
    operation_log_id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES "user"(user_id) ON DELETE SET NULL,
    action VARCHAR(100) NOT NULL,
    entity_type VARCHAR(50),
    entity_id INTEGER,
    details JSONB,
    ip_address INET,
    operation_log_created_at TIMESTAMP DEFAULT NOW()
);

```

Листинг 7 – Скрипт заполнения данными (inser_data.sql)

```

INSERT INTO "user" (full_name, email, phone, login, password_hash, role, registra-
tion_date, is_active) VALUES
('Иванов Иван Иванович', 'ivanov@ski.ru', '+79161234567', 'ivanov',
'hash_ivanov_123', 'athlete', '2024-09-01', TRUE),
('Петров Петр Петрович', 'petrov@ski.ru', '+79162345678', 'petrov', 'hash_pe-
trov_123', 'athlete', '2024-09-01', TRUE),
('Сидоров Сидор Сидорович', 'sidorov@ski.ru', '+79163456789', 'sidorov', 'hash_si-
dorov_123', 'athlete', '2024-09-02', TRUE),
('Смирнова Анна Сергеевна', 'smirnova@ski.ru', '+79164567890', 'asmirnova',
'hash_smirnova_123', 'athlete', '2024-09-02', TRUE),
('Кузнецов Дмитрий Алексеевич', 'kuznetsov@ski.ru', '+79165678901', 'dkuznetsov',
'hash_kuznetsov_123', 'athlete', '2024-09-03', TRUE),
('Попова Елена Викторовна', 'popova@ski.ru', '+79166789012', 'epopova',
'hash_popova_123', 'athlete', '2024-09-03', TRUE),

```

```
(
'Васильев Алексей Николаевич', 'vasiliev@ski.ru', '+79167890123', 'avasiliev',
'hash_vasiliev_123', 'athlete', '2024-09-04', TRUE),
('Новикова Мария Игоревна', 'novikova@ski.ru', '+79168901234', 'mnovikova',
'hash_novikova_123', 'athlete', '2024-09-04', TRUE),
('Морозов Сергей Владимирович', 'morozov@ski.ru', '+79169012345', 'smorozov',
'hash_morozov_123', 'athlete', '2024-09-05', TRUE),
('Волкова Ольга Дмитриевна', 'volkova@ski.ru', '+79160123456', 'ovolkova',
'hash_volkova_123', 'athlete', '2024-09-05', TRUE),
('Соколов Роман Андреевич', 'sokolov@ski.ru', '+79161234568', 'rsokolov',
'hash_sokolov_123', 'athlete', '2024-09-06', TRUE),
('Орлова Татьяна Павловна', 'orlova@ski.ru', '+79162345679', 'torlova', 'hash_or-
lova_123', 'athlete', '2024-09-06', TRUE),
('Лебедев Андрей Викторович', 'lebedev@ski.ru', '+79163456780', 'alebedev',
'hash_lebedev_123', 'athlete', '2024-09-07', TRUE),
('Козлова Наталья Игоревна', 'kozlova@ski.ru', '+79164567891', 'nkozlova',
'hash_kozlova_123', 'athlete', '2024-09-07', TRUE),
('Федоров Михаил Сергеевич', 'fedorov@ski.ru', '+79165678902', 'mfedorov',
'hash_fedorov_123', 'athlete', '2024-09-08', TRUE),
('Тренеров Алексей Юрьевич', 'trenеров@ski.ru', '+79161239876', 'trenеров',
'hash_trenеров_123', 'coach', '2024-08-01', TRUE),
('Наставников Игорь Петрович', 'nastavnikov@ski.ru', '+79162348765', 'nastavnikov',
'hash_nastavnikov_123', 'coach', '2024-08-15', TRUE),
('Старцев Павел Николаевич', 'startsev@ski.ru', '+79163459876', 'pstartsev',
'hash_startsev_123', 'coach', '2024-09-01', TRUE),
('Складской Михаил Семенович', 'skladskoy@ski.ru', '+79163457654', 'skladskoy',
'hash_skladskoy_123', 'storekeeper', '2024-08-01', TRUE),
('Инвентарев Игорь Васильевич', 'inventarev@ski.ru', '+79164568765', 'inventarev',
'hash_inventarev_123', 'storekeeper', '2024-09-01', TRUE),
('Закупочный Владимир Ильич', 'zakupochny@ski.ru', '+79164566543', 'zakupochny',
'hash_zakupochny_123', 'purchase_manager', '2024-08-01', TRUE),
('Снабженцев Андрей Петрович', 'snabzhentsev@ski.ru', '+79165677654', 'asnab-
zhentsev', 'hash_snabzhentsev_123', 'purchase_manager', '2024-09-01', TRUE),
('Руководителей Павел Борисович', 'rukovoditelev@ski.ru', '+79165675432',
'rukovoditelev', 'hash_rukovoditelev_123', 'manager', '2024-07-01', TRUE),
('Бухгалтерова Наталья Алексеевна', 'buhgalterova@ski.ru', '+79166784321',
'buhgalterova', 'hash_buhgalterova_123', 'accountant', '2024-08-01', TRUE),
('Админов Денис Валерьевич', 'adminov@ski.ru', '+79167893210', 'adminov', 'hash_ad-
minov_123', 'administrator', '2024-07-01', TRUE);
```

```
INSERT INTO warehouse (warehouse_name, address, type) VALUES
```

```
('Основной склад', 'г. Москва, ул. Лыжная, д. 1', 'main'),
('Склад сервисного центра', 'г. Москва, ул. Техническая, д. 5', 'service'),
('Резервный склад', 'г. Химки, ул. Запасная, д. 10', 'reserve'),
('Склад лыжной базы "Снежинка"', 'г. Москва, Ленинградское шоссе, 45', 'main'),
('Склад соревновательного инвентаря', 'г. Москва, ул. Спортивная, 12', 'main'),
('Склад расходных материалов', 'г. Москва, ул. Складская, 3', 'service'),
('Склад запасных частей', 'г. Химки, Вашутинское шоссе, 8', 'service'),
('Склад сезонного хранения', 'г. Долгопрудный, ул. Лесная, 20', 'reserve'),
('Склад экипировки сборной', 'г. Москва, Олимпийский пр-т, 5', 'main'),
('Склад утилизации', 'г. Мытищи, ул. Промышленная, 7', 'reserve');
```

```
INSERT INTO coach (user_id, specialization, coach_category) VALUES
```

```
(16, 'Лыжные гонки', 'Высшая'),
(17, 'Биатлон', 'Первая'),
(18, 'Горные лыжи', 'Высшая');
```

```
INSERT INTO "group" (group_name, coach_id, group_created_at) VALUES
```

```
('Группа начальной подготовки №1', 1, '2024-09-01'),
('Группа начальной подготовки №2', 1, '2024-09-01'),
('Группа спортивного совершенствования', 1, '2024-09-01'),
('Юниорская сборная (лыжные гонки)', 1, '2024-09-01'),
('Биатлонная группа №1', 2, '2024-09-01'),
```

```

('Биатлонная группа №2', 2, '2024-09-01'),
('Горнолыжная группа "Слалом"', 3, '2024-09-01'),
('Горнолыжная группа "Гигант"', 3, '2024-09-01'),
('Спортивно-оздоровительная группа', 1, '2024-10-01'),
('Группа высшего спортивного мастерства', 1, '2024-10-01');

INSERT INTO athlete (user_id, sports_category, birth_year, group_id, coach_id,
has_active_debt) VALUES
(1, 'КМС', 2005, 3, 1, FALSE),
(2, 'МС', 2003, 3, 1, FALSE),
(3, '1 разряд', 2007, 1, 1, TRUE),
(4, 'КМС', 2006, 4, 1, FALSE),
(5, 'МС', 2002, 4, 1, FALSE),
(6, '1 разряд', 2008, 2, 1, FALSE),
(7, 'КМС', 2004, 3, 1, FALSE),
(8, '2 разряд', 2009, 1, 1, FALSE),
(9, 'МС', 2001, 4, 1, TRUE),
(10, 'КМС', 2005, 3, 1, FALSE),
(11, '1 разряд', 2007, 5, 2, FALSE),
(12, 'КМС', 2006, 5, 2, FALSE),
(13, 'МС', 2003, 6, 2, FALSE),
(14, '1 разряд', 2008, 7, 3, FALSE),
(15, 'КМС', 2005, 8, 3, FALSE);

INSERT INTO storekeeper (user_id, warehouse_id, certificate_number) VALUES
(19, 1, 'СКЛ-2024-001'),
(20, 2, 'СКЛ-2024-002');

INSERT INTO purchase_manager (user_id, department) VALUES
(21, 'Отдел снабжения'),
(22, 'Отдел закупок спортивного инвентаря');

INSERT INTO manager (user_id, position, manager_access_level) VALUES
(23, 'Директор спортивной школы', 3);

INSERT INTO accountant (user_id, section) VALUES
(24, 'Материальный учет');

INSERT INTO administrator (user_id, administrator_access_level) VALUES
(25, 'FULL');

INSERT INTO nomenclature (article, nomenclature_name, nomenclature_category, unit,
standard_service_life, min_stock_level, manufacturer) VALUES
('SKI-001', 'Лыжи гоночные Fischer Speedmax', 'skis', 'pair', 24, 5, 'Fischer'),
('SKI-002', 'Лыжи гоночные Madshus Redline', 'skis', 'pair', 24, 5, 'Madshus'),
('SKI-003', 'Лыжи тренировочные Fischer RCS', 'skis', 'pair', 18, 10, 'Fischer'),
('SKI-004', 'Лыжи горные Atomic Redster', 'skis', 'pair', 24, 3, 'Atomic'),
('SKI-005', 'Лыжи для классического хода Rossignol', 'skis', 'pair', 24, 5, 'Ros-
signol'),
('BOOT-001', 'Ботинки лыжные Salomon S/Lab', 'boots', 'pair', 18, 8, 'Salomon'),
('BOOT-002', 'Ботинки лыжные Fischer Carbonlite', 'boots', 'pair', 18, 8, 'Fisch-
er'),
('BOOT-003', 'Ботинки тренировочные Madshus', 'boots', 'pair', 12, 12, 'Madshus'),
('BOOT-004', 'Ботинки горнолыжные Atomic Hawx', 'boots', 'pair', 24, 5, 'Atomic'),
('POLE-001', 'Палки лыжные Swix Triac', 'poles', 'pair', 12, 10, 'Swix'),
('POLE-002', 'Палки лыжные Leki HRC', 'poles', 'pair', 12, 10, 'Leki'),
('POLE-003', 'Палки горнолыжные Komperdell', 'poles', 'pair', 12, 5, 'Komperdell'),
('WAX-001', 'Парафин Swix LF4', 'wax', 'pcs', 6, 20, 'Swix'),
('WAX-002', 'Парафин Swix HF6', 'wax', 'pcs', 6, 15, 'Swix'),
('WAX-003', 'Порошок ускоритель Swix FC7', 'wax', 'pcs', 12, 10, 'Swix');

```



```

INSERT INTO supplier (supplier_name, inn, kpp, legal_address, contact_person, supplier_phone, supplier_email, rating) VALUES
('000 "СпортТовары"', '7712345678', '771201001', 'г. Москва, ул. Спортивная, 10', 'Иванов А.А.', '+74951234567', 'info@sporttovary.ru', 4.8),
('000 "Лыжный Мир"', '7723456789', '772301001', 'г. Москва, ул. Снежная, 5', 'Петров Б.Б.', '+74952345678', 'sales@skimirs.ru', 4.5),
('000 "Фишер Рус"', '7734567890', '773401001', 'г. Москва, ул. Горнолыжная, 15', 'Сидоров В.В.', '+74953456789', 'info@fischer-rus.ru', 4.9),
('000 "Свикс Россия"', '7745678901', '774501001', 'г. Санкт-Петербург, пр. Лыжный, 20', 'Козлов Г.Г.', '+78123456789', 'sales@swix-russia.ru', 4.7),
('000 "Саломон СНГ"', '7756789012', '775601001', 'г. Москва, ул. Зимняя, 8', 'Новиков Д.Д.', '+74954567890', 'info@salomon-cis.ru', 4.6),
('000 "СпортМастер"', '7767890123', '776701001', 'г. Москва, ул. Торговая, 1', 'Морозов Е.Е.', '+74955678901', 'corp@sportmaster.ru', 4.4),
('000 "Декатлон"', '7778901234', '777801001', 'г. Москва, ш. Дмитровское, 100', 'Волков Ж.Ж.', '+74956789012', 'b2b@decathlon.ru', 4.3),
('000 "Кант"', '7789012345', '778901001', 'г. Москва, пр. Мира, 50', 'Зайцев З.З.', '+74957890123', 'opt@kant.ru', 4.5),
('000 "Триал-Спорт"', '7790123456', '779001001', 'г. Москва, ул. Профсоюзная, 30', 'Медведев И.И.', '+74958901234', 'wholesale@trial-sport.ru', 4.2),
('000 "Альпиндустрия"', '7701234567', '770101001', 'г. Москва, ул. Первомайская, 40', 'Волков К.К.', '+74959012345', 'opt@alpindustria.ru', 4.4),
('000 "Спорт-Экспресс"', '7711122233', '771101001', 'г. Москва, ул. Ленина, 15', 'Быстров С.С.', '+74951239876', 'info@sport-express.ru', 4.6),
('000 "Лыжный стандарт"', '7722233344', '772201001', 'г. Екатеринбург, ул. Уральская, 10', 'Уралов М.М.', '+73431234567', 'sales@skistandard.ru', 4.3);

INSERT INTO purchase_request (purchase_request_number, purchase_request_created_date, purchase_request_status, author_id, approved_by, planned_amount, purchase_request_comment) VALUES
('ЗАЯВ-2026-001', '2026-01-10 09:00:00', 'APPROVED', 16, 23, 250000.00, 'Закупка инвентаря на сезон 2026'),
('ЗАЯВ-2026-002', '2026-01-15 10:30:00', 'ON_APPROVAL', 17, NULL, 150000.00, 'Дополнительная закупка лыж для юниоров'),
('ЗАЯВ-2026-003', '2026-01-20 11:45:00', 'DRAFT', 18, NULL, 50000.00, 'Закупка смазочных материалов'),
('ЗАЯВ-2026-004', '2026-02-01 08:15:00', 'APPROVED', 16, 23, 180000.00, 'Закупка ботинок для группы НР-1'),
('ЗАЯВ-2026-005', '2026-02-10 14:20:00', 'REJECTED', 17, 23, 95000.00, 'Закупка папок (отклонено - пересмотреть бюджет)'),
('ЗАЯВ-2026-006', '2026-02-15 16:00:00', 'APPROVED', 18, 23, 75000.00, 'Закупка креплений'),
('ЗАЯВ-2026-007', '2026-03-01 10:00:00', 'ON_APPROVAL', 16, NULL, 220000.00, 'Плановая закупка на весну'),
('ЗАЯВ-2026-008', '2026-03-05 11:30:00', 'DRAFT', 17, NULL, 35000.00, 'Закупка расходных материалов'),
('ЗАЯВ-2026-009', '2026-03-10 09:45:00', 'APPROVED', 16, 23, 310000.00, 'Крупная закупка лыж Fischer'),
('ЗАЯВ-2026-010', '2026-03-15 13:15:00', 'ON_APPROVAL', 18, NULL, 120000.00, 'Закупка горнолыжного инвентаря');

INSERT INTO purchase_request_item (purchase_request_id, nomenclature_id, quantity, expected_price, priority) VALUES
(1, 1, 10, 45000.00, 'HIGH'),
(1, 6, 15, 25000.00, 'HIGH'),
(1, 10, 20, 8000.00, 'MEDIUM'),
(2, 2, 5, 50000.00, 'HIGH'),
(2, 7, 10, 28000.00, 'MEDIUM'),
(3, 13, 50, 500.00, 'LOW'),
(3, 14, 30, 600.00, 'LOW'),
(4, 6, 12, 25000.00, 'HIGH'),
(4, 8, 10, 12000.00, 'MEDIUM'),

```

```

(5, 10, 25, 8000.00, 'MEDIUM'),
(6, 5, 8, 52000.00, 'HIGH'),
(7, 3, 15, 35000.00, 'HIGH'),
(8, 15, 20, 1200.00, 'LOW'),
(9, 1, 20, 45000.00, 'HIGH'),
(10, 4, 6, 48000.00, 'HIGH');

INSERT INTO purchase_order (purchase_order_number, purchase_order_created_date,
purchase_order_status, supplier_id, purchase_manager_id, total_amount, expected_de-
livery_date) VALUES
('3AKA3-2026-001', '2026-01-12', 'SENT', 3, 1, 825000.00, '2026-08-15'),
('3AKA3-2026-002', '2026-01-18', 'DELIVERED', 4, 2, 160000.00, '2026-02-01'),
('3AKA3-2026-003', '2026-02-05', 'IN_TRANSIT', 1, 1, 250000.00, '2026-03-10'),
('3AKA3-2026-004', '2026-02-12', 'SENT', 2, 2, 180000.00, '2026-09-01'),
('3AKA3-2026-005', '2026-02-20', 'DRAFT', 5, 1, 95000.00, NULL),
('3AKA3-2026-006', '2026-03-01', 'DELIVERED', 6, 1, 120000.00, '2026-03-20'),
('3AKA3-2026-007', '2026-03-10', 'SENT', 3, 2, 450000.00, '2026-10-01'),
('3AKA3-2026-008', '2026-03-15', 'CLOSED', 4, 1, 75000.00, '2026-03-25'),
('3AKA3-2026-009', '2026-03-20', 'IN_TRANSIT', 8, 2, 210000.00, '2026-04-15'),
('3AKA3-2026-010', '2026-03-25', 'SENT', 11, 1, 135000.00, '2026-08-20');

INSERT INTO inventory_item (barcode, nomenclature_id, serial_number, inventory_sta-
tus, warehouse_id, receipt_date, purchase_cost, size, athlete_id, inventory_notes)
VALUES
('SKI001-2024-001', 1, 'FS-2024-001', 'AVAILABLE', 1, '2024-10-01', 45000.00,
'190', NULL, 'Новые'),
('SKI001-2024-002', 1, 'FS-2024-002', 'ISSUED', 1, '2024-10-01', 45000.00, '195',
1, 'Выданы Иванову'),
('SKI001-2024-003', 1, 'FS-2024-003', 'AVAILABLE', 1, '2024-10-01', 45000.00,
'200', NULL, 'Новые'),
('SKI001-2024-004', 1, 'FS-2024-004', 'RESERVED', 1, '2024-10-01', 45000.00, '192',
NULL, 'Зарезервированы для сборов'),
('SKI001-2024-005', 1, 'FS-2024-005', 'ISSUED', 1, '2024-10-01', 45000.00, '198',
2, 'Выданы Петрову'),
('SKI002-2024-001', 2, 'MR-2024-001', 'AVAILABLE', 1, '2024-10-15', 50000.00,
'192', NULL, NULL),
('SKI002-2024-002', 2, 'MR-2024-002', 'ISSUED', 1, '2024-10-15', 50000.00, '198',
4, 'Выданы Смирновой'),
('SKI002-2024-003', 2, 'MR-2024-003', 'UNDER_REPAIR', 2, '2024-10-15', 50000.00,
'195', NULL, 'Ремонт скользящей поверхности'),
('BOOT001-2024-001', 6, 'SL-2024-001', 'AVAILABLE', 1, '2024-11-01', 25000.00,
'42', NULL, NULL),
('BOOT001-2024-002', 6, 'SL-2024-002', 'ISSUED', 1, '2024-11-01', 25000.00, '43',
1, 'Выданы Иванову'),
('BOOT001-2024-003', 6, 'SL-2024-003', 'AVAILABLE', 1, '2024-11-01', 25000.00,
'44', NULL, NULL),
('BOOT001-2024-004', 6, 'SL-2024-004', 'ISSUED', 1, '2024-11-01', 25000.00, '41',
5, 'Выданы Кузнецову'),
('POLE001-2024-001', 10, 'SW-2024-001', 'AVAILABLE', 1, '2024-11-15', 8000.00,
'155', NULL, NULL),
('POLE001-2024-002', 10, 'SW-2024-002', 'ISSUED', 1, '2024-11-15', 8000.00, '160',
2, 'Выданы Петрову'),
('POLE001-2024-003', 10, 'SW-2024-003', 'DEFECTIVE', 1, '2024-11-15', 8000.00,
'150', NULL, 'Трещина на древке'),
('WAX001-2024-001', 13, 'WAX-LF4-001', 'AVAILABLE', 1, '2024-12-01', 500.00, NULL,
NULL, NULL),
('WAX001-2024-002', 13, 'WAX-LF4-002', 'AVAILABLE', 1, '2024-12-01', 500.00, NULL,
NULL, NULL),
('WAX002-2024-001', 14, 'WAX-HF6-001', 'AVAILABLE', 1, '2024-12-01', 600.00, NULL,
NULL, NULL),
('WAX002-2024-002', 14, 'WAX-HF6-002', 'ISSUED', 1, '2024-12-01', 600.00, NULL, 3,
'Выданы сервисной бригаде'),

```



```

('WAX003-2024-001', 15, 'WAX-FC7-001', 'AVAILABLE', 1, '2024-12-01', 1200.00, NULL,
NULL, NULL);

INSERT INTO receipt_invoice (receipt_invoice_number, receipt_invoice_date, pur-
chase_order_id, storekeeper_id, receipt_invoice_status, actual_amount) VALUES
('ПН-2024-001', '2024-10-01', 1, 1, 'POSTED', 825000.00),
('ПН-2024-002', '2024-11-15', 2, 1, 'POSTED', 160000.00),
('ПН-2024-003', '2024-12-01', 3, 2, 'POSTED', 250000.00),
('ПН-2025-001', '2025-01-15', 4, 1, 'POSTED', 180000.00),
('ПН-2025-002', '2025-02-01', 6, 2, 'POSTED', 120000.00),
('ПН-2025-003', '2025-02-15', 8, 1, 'POSTED', 75000.00),
('ПН-2025-004', '2025-03-01', 7, 1, 'DRAFT', 450000.00),
('ПН-2025-005', '2025-03-10', 9, 2, 'POSTED', 210000.00),
('ПН-2025-006', '2025-03-20', 5, 1, 'CANCELLED', 0.00),
('ПН-2025-007', '2025-03-25', 10, 2, 'DRAFT', 135000.00);

INSERT INTO issuance_document (inventory_item_id, athlete_id, storekeeper_id, issu-
ance_date, planned_return_date, actual_return_date, issuance_document_status, con-
dition_on_issue, condition_on_return, issuance_document_notes) VALUES
(2, 1, 1, '2024-12-01', '2025-03-30', NULL, 'ACTIVE', 'NEW', NULL, 'На сезон 2024-
2025'),
(5, 2, 1, '2024-12-05', '2025-03-30', NULL, 'ACTIVE', 'NEW', NULL, 'На сезон 2024-
2025'),
(7, 4, 2, '2024-12-10', '2025-03-30', NULL, 'ACTIVE', 'NEW', NULL, 'На сезон 2024-
2025'),
(12, 5, 1, '2024-12-15', '2025-03-30', NULL, 'ACTIVE', 'NEW', NULL, 'На сезон 2024-
2025'),
(10, 6, 2, '2025-01-10', '2025-04-15', NULL, 'ACTIVE', 'NEW', NULL, 'Выданы Попо-
вой'),
(3, 7, 1, '2024-11-01', '2024-12-01', '2024-11-28', 'RETURNED', 'NEW', 'GOOD',
NULL),
(4, 8, 2, '2024-11-05', '2024-12-05', '2024-12-01', 'RETURNED', 'NEW', 'GOOD',
NULL),
(6, 9, 1, '2024-11-10', '2024-12-10', '2024-12-05', 'RETURNED', 'NEW', 'MI-
NOR_SCRATCHES', 'Небольшие царапины'),
(8, 10, 2, '2024-11-15', '2024-12-15', '2024-12-10', 'RETURNED', 'NEW', 'GOOD',
NULL),
(11, 11, 1, '2024-11-20', '2024-12-20', '2024-12-18', 'RETURNED', 'NEW', 'GOOD',
NULL),
(9, 3, 1, '2024-10-01', '2024-11-01', NULL, 'OVERDUE', 'USED', NULL, 'Просрочен
возврат!'),
(13, 12, 2, '2024-10-15', '2024-11-15', NULL, 'OVERDUE', 'USED', NULL, 'Просрочен
возврат!'),
(1, 13, 1, '2024-11-01', '2024-12-01', NULL, 'OVERDUE', 'USED', NULL, 'Просрочен
возврат!'),
(14, 14, 2, '2024-11-01', '2025-01-01', NULL, 'EXTENDED', 'NEW', NULL, 'Продлено до
01.02.2025'),
(15, 15, 1, '2024-11-15', '2025-01-15', NULL, 'EXTENDED', 'NEW', NULL, 'Продлено до
15.02.2025');

INSERT INTO operation_log (user_id, action, entity_type, entity_id, details, ip_ad-
dress, operation_log_created_at) VALUES
(25, 'CREATE_USER', 'user', 1, '{"created_by": "admin", "role": "athlete"}',
'192.168.1.100', '2024-09-01 09:00:00'),
(25, 'CREATE_USER', 'user', 16, '{"created_by": "admin", "role": "coach"}',
'192.168.1.100', '2024-09-01 09:05:00'),
(16, 'CREATE_REQUEST', 'purchase_request', 1, '{"amount": 250000}',
'192.168.1.101', '2026-01-10 09:00:00'),
(23, 'APPROVE_REQUEST', 'purchase_request', 1, '{"approved": true}',
'192.168.1.104', '2026-01-11 10:00:00'),
(21, 'CREATE_ORDER', 'purchase_order', 1, '{"supplier_id": 3, "total": 825000}',
'192.168.1.103', '2026-01-12 11:00:00'),

```

```
(19, 'RECEIVE_GOODS', 'receipt_invoice', 1, '{"order_id": 1, "amount": 825000}',
'192.168.1.102', '2024-10-01 14:00:00'),
(19, 'ISSUE_INVENTORY', 'issuance_document', 1, '{"athlete_id": 1, "barcode":
"SKI001-2024-002"}', '192.168.1.102', '2024-12-01 10:00:00'),
(19, 'ISSUE_INVENTORY', 'issuance_document', 2, '{"athlete_id": 2, "barcode":
"SKI001-2024-005"}', '192.168.1.102', '2024-12-05 10:30:00'),
(20, 'ISSUE_INVENTORY', 'issuance_document', 3, '{"athlete_id": 4, "barcode":
"SKI002-2024-002"}', '192.168.1.105', '2024-12-10 11:00:00'),
(19, 'RETURN_INVENTORY', 'issuance_document', 6, '{"condition": "GOOD"}',
'192.168.1.102', '2024-11-28 15:00:00'),
(17, 'CREATE_REQUEST', 'purchase_request', 2, '{"amount": 150000}',
'192.168.1.106', '2026-01-15 10:30:00'),
(22, 'CREATE_ORDER', 'purchase_order', 2, '{"supplier_id": 4, "total": 160000}',
'192.168.1.107', '2026-01-18 09:00:00'),
(25, 'BLOCK_USER', 'user', 3, '{"reason": "debt"}', '192.168.1.100', '2024-11-02
08:00:00'),
(24, 'EXPORT_REPORT', 'report', NULL, '{"type": "OSV", "period": "2024-Q4"}',
'192.168.1.108', '2025-01-10 16:00:00'),
(23, 'VIEW_DASHBOARD', 'dashboard', NULL, '{"section": "analytics"}',
'192.168.1.104', '2025-01-15 09:00:00');
```

Листинг 8 – Пример поискового запроса (search_request.sql)

```
-- Поиск спортсменов-должников (с просроченным инвентарем)
SELECT
    u.full_name AS athlete_name,
    u.email,
    u.phone,
    a.sports_category,
    g.group_name,
    cu.full_name AS coach_name,
    COUNT(d.issuance_document_id) AS overdue_items_count,
    STRING_AGG(i.barcode || ' (' || n.nomenclature_name || ', просрочен на ' ||
        (CURRENT_DATE - d.planned_return_date) || ' дн.),' ;') AS over-
due_items
FROM athlete a
JOIN "user" u ON a.user_id = u.user_id
LEFT JOIN "group" g ON a.group_id = g.group_id
LEFT JOIN coach c ON a.coach_id = c.coach_id
LEFT JOIN "user" cu ON c.user_id = cu.user_id
JOIN issuance_document d ON a.athlete_id = d.athlete_id
JOIN inventory_item i ON d.inventory_item_id = i.inventory_item_id
JOIN nomenclature n ON i.nomenclature_id = n.nomenclature_id
WHERE d.issuance_document_status IN ('ACTIVE', 'OVERDUE')
    AND d.planned_return_date < CURRENT_DATE
GROUP BY u.full_name, u.email, u.phone, a.sports_category, g.group_name,
cu.full_name
ORDER BY overdue_items_count DESC;
```

Приложение 3. Исходный код информационного веб-портала

Листинг 9 – index.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>АСУ МТО – Лыжный спорт</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- Навигация -->
  <nav class="navbar">
    <div class="nav-container">
      <a href="#home" class="nav-logo">  АСУ МТО</a>
      <div class="nav-links">
        <a href="#home">Главная</a>
        <a href="#about">О системе</a>
        <a href="#results">Результаты</a>
        <a href="#contact">Контакты</a>
      </div>
    </div>
  </nav>

  <!-- Главный экран -->
  <header id="home" class="hero">
    <div class="hero-content">
      <h1>Автоматизированная система управления</h1>
      <h2>Материально-техническое обеспечение лыжного спорта</h2>
      <p>Современное решение для учёта, закупок и прогнозирования потребности в спортивном инвентаре</p>
      <a href="#about" class="btn-hero">Узнать больше ▾</a>
    </div>
  </header>

  <!-- Раздел: О системе -->
  <section id="about" class="section">
    <h2 class="section-title">О системе</h2>
    <p class="section-desc">АСУ МТО предназначена для автоматизации процессов снабжения, учёта и распределения спортивного инвентаря в лыжных школах и федерациях.</p>

    <div class="cards-grid">
      <div class="card">
        <div class="card-icon">  </div>
        <h3>Менеджер по закупкам</h3>
        <p>Формирует заказы поставщикам, отслеживает статусы поставок, контролирует бюджет и исполнение контрактов.</p>
      </div>
    </div>
  </section>
</body>
```

```

        <div class="card">
            <div class="card-icon">📦</div>
            <h3>Кладовщик</h3>
            <p>Управляет складом, сканирует штрих-коды, выдаёт и принимает
инвентарь, проводит инвентаризацию.</p>
        </div>
        <div class="card">
            <div class="card-icon">👤</div>
            <h3>Тренер</h3>
            <p>Подает заявки на инвентарь для своей группы, отслеживает
возврат, видит историю выдач.</p>
        </div>
        <div class="card">
            <div class="card-icon">📊</div>
            <h3>Руководитель</h3>
            <p>Утверждает бюджет, анализирует отчёты по закупкам, контроли-
рует эффективность использования.</p>
        </div>
    </div>

    <div class="features-section">
        <h3>Основные функции системы</h3>
        <div class="features-grid">
            <div class="feature-card">
                <div class="feature-icon">📦</div>
                <div class="feature-text">
                    <h4>Ведение номенклатуры</h4>
                    <p>Лыжи, ботинки, палки, смазки — множество позиций с
характеристиками</p>
                </div>
            </div>
            <div class="feature-card">
                <div class="feature-icon">💰</div>
                <div class="feature-text">
                    <h4>Управление закупками</h4>
                    <p>Бюджетный контроль, формирование заказов поставщи-
кам</p>
                </div>
            </div>
            <div class="feature-card">
                <div class="feature-icon">🏷️</div>
                <div class="feature-text">
                    <h4>Складской учёт</h4>
                    <p>Маркировка штрих-кодами, приёмка, выдача и возврат
инвентаря</p>
                </div>
            </div>
            <div class="feature-card">
                <div class="feature-icon">👥</div>
                <div class="feature-text">
                    <h4>Контроль выдачи</h4>

```

```

        <p>Отслеживание выданного инвентаря, уведомления о про-
сроках</p>
    </div>
</div>
<div class="feature-card">
    <div class="feature-icon">☑</div>
    <div class="feature-text">
        <h4>Прогнозирование</h4>
        <p>Расчёт потребности на основе исторических данных и
сезонности</p>
    </div>
</div>
<div class="feature-card">
    <div class="feature-icon">📅</div>
    <div class="feature-text">
        <h4>Автоматическое списание</h4>
        <p>Контроль сроков службы и автоматическое списание из-
ношенного инвентаря</p>
    </div>
</div>
</div>
</div>

<div class="benefits">
    <h3>Преимущества автоматизации</h3>
    <div class="benefits-grid">
        <div class="benefit-item">
            <div class="benefit-icon">⌚</div>
            <h4>Экономия времени</h4>
            <p>Сокращение времени на оформление документов в 3 раза</p>
        </div>
        <div class="benefit-item">
            <div class="benefit-icon">🎯</div>
            <h4>Точность учёта</h4>
            <p>Исключение ошибок ручного ввода данных</p>
        </div>
        <div class="benefit-item">
            <div class="benefit-icon">☑</div>
            <h4>Прогнозирование</h4>
            <p>Планирование закупок на основе реальных данных</p>
        </div>
    </div>
</div>
</div>
</section>

<!-- Раздел: Результаты работы АСУ -->
<section id="results" class="section section-alt">
    <h2 class="section-title">Результаты работы АСУ</h2>
    <p class="section-desc">Демонстрация данных, получаемых в результате
работы системы</p>

```

```

    <!-- Табы -->
    <div class="tabs">
        <button class="tab-btn active" onclick="showResultTab('inventory',
this)">📦 Складской учёт</button>
        <button class="tab-btn" onclick="showResultTab('issuance',
this)">📋 Активные выдачи</button>
        <button class="tab-btn" onclick="showResultTab('forecast',
this)">📈 Прогноз потребности</button>
    </div>

    <div class="tab-content active" id="tab-inventory">
        <div class="table-wrapper">
            <table>
                <thead><tr><th>Штрих-
код</th><th>Наименование</th><th>Категория</th><th>Размер</th><th>Ста-
тус</th></tr></thead>
                <tbody id="result-inventory-tbody"></tbody>
            </table>
        </div>
    </div>

    <div class="tab-content" id="tab-issuance">
        <div class="table-wrapper">
            <table>
                <thead><tr><th>Спортсмен</th><th>Инвентарь</th><th>Дата вы-
дачи</th><th>Возврат до</th><th>Статус</th></tr></thead>
                <tbody id="result-issuance-tbody"></tbody>
            </table>
        </div>
    </div>

    <div class="tab-content" id="tab-forecast">
        <div class="forecast-demo">
            <h3>Прогноз потребности: Лыжи гоночные Fischer Speedmax</h3>
            <p style="color: var(--text-light); margin-bottom: 25px;">На
основе исторических данных за 2024-2025 гг. • Модель: экспоненциальное сглажи-
вание</p>

            <div class="forecast-months">
                <div class="forecast-month">
                    <span>Май 2026</span>
                    <div class="forecast-bar" style="height:
30px;">4,2</div>

                    <small>шт.</small>
                </div>
                <div class="forecast-month">
                    <span>Июнь 2026</span>
                    <div class="forecast-bar" style="height:
15px;">2,1</div>

                    <small>шт.</small>
                </div>
                <div class="forecast-month">

```

```

        <span>Июль 2026</span>
        <div class="forecast-bar" style="height:
20px;">2,8</div>
        <small>шт.</small>
    </div>
    <div class="forecast-month">
        <span>Август 2026</span>
        <div class="forecast-bar" style="height:
80px;">12,5</div>
        <small>шт.</small>
    </div>
</div>
<div class="forecast-summary">
    <div class="forecast-summary-item">
        <span class="fs-label">Суммарный прогноз</span>
        <span class="fs-value">21,6 шт.</span>
    </div>
    <div class="forecast-summary-item">
        <span class="fs-label">Текущий остаток</span>
        <span class="fs-value">2 шт.</span>
    </div>
    <div class="forecast-summary-item highlight">
        <span class="fs-label">Рекомендуемый заказ</span>
        <span class="fs-value">24,6 шт.</span>
    </div>
</div>
</div>
</div>

<!-- Слайдер отзывов -->
<div class="testimonials">
    <h3>Отзывы пользователей</h3>
    <div class="slider" id="testimonial-slider">
        <div class="slide active">
            <div class="testimonial-avatar">👤</div>
            <p class="testimonial-text">«Раньше инвентаризация занимала
два полных рабочих дня. Сейчас – не больше двух часов. Штрих-кодирование ре-
ально изменило нашу работу. Всё прозрачно, ничего не теряется.»</p>
            <span class="testimonial-author">Михаил Семёнович, кладов-
щик</span>
            <span class="testimonial-org">СШОР №5, г. Москва</span>
        </div>
        <div class="slide">
            <div class="testimonial-avatar">👤</div>
            <p class="testimonial-text">«Перед выездом на сборы теперь
не нужно бегать на склад и сверять списки. Захожу в личный кабинет – вижу, что
у каждого спортсмена, какие размеры, что пора менять. Очень удобно!»</p>
            <span class="testimonial-author">Алексей Юрьевич, старший
тренер</span>
            <span class="testimonial-org">Сборная области по лыжным
гонкам</span>

```

```

        </div>
        <div class="slide">
            <div class="testimonial-avatar"><img alt="Avatar of Pavel Borisovich" data-bbox="678 104 698 120"/></div>
            <p class="testimonial-text">«Модуль прогнозирования помог избежать дефицита лыж Fischer перед сезоном 2025/2026. Система рассчитала потребность с учётом износа – и мы закупили ровно столько, сколько нужно. Ни больше, ни меньше.»</p>
            <span class="testimonial-author">Павел Борисович, директор</span>
            <span class="testimonial-org">Лыжная федерация, г. Екатеринбург</span>
        </div>
    </div>
    <div class="slider-dots">
        <span class="dot active" onclick="currentSlide(0)"></span>
        <span class="dot" onclick="currentSlide(1)"></span>
        <span class="dot" onclick="currentSlide(2)"></span>
    </div>
</div>
</section>

<!-- Раздел: Контакты -->
<section id="contact" class="section">
    <h2 class="section-title">Контакты</h2>
    <div class="contact-grid">
        <div class="contact-info">
            <h3>Спортивная школа олимпийского резерва</h3>
            <p>📍 г. Москва, ул. Лыжная, д. 1</p>
            <p>☎ +7 (495) 123-45-67</p>
            <p>✉ info@ski-school.ru</p>
            <p>🕒 Пн-Пт: 9:00 – 18:00</p>
        </div>
        <form class="contact-form" onsubmit="event.preventDefault(); alert('Спасибо за обращение!');">
            <h3>Обратная связь</h3>
            <input type="text" placeholder="Ваше имя" required>
            <input type="email" placeholder="Email" required>
            <textarea placeholder="Сообщение" rows="5" required></textarea>
            <button type="submit" class="btn-submit">Отправить</button>
        </form>
    </div>
</section>

<!-- Подвал -->
<footer>
    <p>© 2026 АСУ МТО Лыжной организации | РТУ МИРЭА | Павлов Н.С., группа ИКБО-50-23</p>
</footer>

<script src="script.js"></script>
</body>

```


</html>

Листинг 10 – style.css

```
/* ===== ПЕРЕМЕННЫЕ ===== */
:root {
  --primary: #1a3a5c;
  --primary-light: #2d6a9f;
  --success: #27ae60;
  --warning: #f39c12;
  --danger: #e74c3c;
  --bg: #f0f4f8;
  --card: #ffffff;
  --text: #333333;
  --text-light: #666666;
  --radius: 12px;
  --shadow: 0 4px 20px rgba(0, 0, 0, 0.08);
}

* { margin: 0; padding: 0; box-sizing: border-box; }

body {
  font-family: 'Segoe UI', 'Arial', sans-serif;
  background: var(--bg);
  color: var(--text);
  line-height: 1.6;
}

/* ===== НАВИГАЦИЯ ===== */
.navbar {
  position: fixed;
  top: 0;
  width: 100%;
  background: var(--primary);
  color: white;
  z-index: 1000;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.3);
}

.nav-container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 15px 20px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.nav-logo {
  font-size: 1.4em;
  font-weight: bold;
```

```

        color: white;
        text-decoration: none;
    }

    .nav-links a {
        color: #b8d4e8;
        text-decoration: none;
        margin-left: 25px;
        transition: color 0.3s;
    }

    .nav-links a:hover { color: white; }

    /* ===== HERO ===== */
    .hero {
        margin-top: 60px;
        background: linear-gradient(135deg, #0f2640, var(--primary), var(--primary-
light));
        color: white;
        text-align: center;
        padding: 100px 20px;
    }

    .hero h1 { font-size: 2.5em; margin-bottom: 10px; }
    .hero h2 { font-size: 1.3em; font-weight: normal; opacity: 0.9; margin-bottom:
20px; }
    .hero p { font-size: 1.1em; opacity: 0.8; margin-bottom: 30px; max-width:
600px; margin-left: auto; margin-right: auto; }

    .btn-hero {
        display: inline-block;
        background: var(--success);
        color: white;
        padding: 14px 35px;
        border-radius: 8px;
        text-decoration: none;
        font-size: 1.1em;
        transition: all 0.3s;
    }

    .btn-hero:hover { background: #219a52; transform: translateY(-2px); }

    /* ===== СЕКЦИИ ===== */
    .section {
        max-width: 1200px;
        margin: 0 auto;
        padding: 80px 20px;
    }

    .section-alt { background: #e8f0f8; }

```

```

.section-title {
  text-align: center;
  font-size: 2em;
  color: var(--primary);
  margin-bottom: 10px;
}

.section-desc {
  text-align: center;
  color: var(--text-light);
  margin-bottom: 50px;
  max-width: 700px;
  margin-left: auto;
  margin-right: auto;
}

/* ===== КАРТОЧКИ ===== */
.cards-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 25px;
  margin-bottom: 50px;
}

.card {
  background: var(--card);
  padding: 30px;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  text-align: center;
  transition: transform 0.3s;
}

.card:hover { transform: translateY(-5px); }

.card-icon { font-size: 3em; margin-bottom: 15px; }
.card h3 { color: var(--primary); margin-bottom: 10px; }
.card p { color: var(--text-light); font-size: 0.95em; }

/* ===== ФУНКЦИИ ===== */
.features-section {
  background: var(--card);
  padding: 30px;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  margin-bottom: 50px;
}

.features-section h3 { color: var(--primary); margin-bottom: 20px; }

.features-grid {

```

```

    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(320px, 1fr));
    gap: 15px;
}

.feature-card {
    display: flex;
    align-items: flex-start;
    gap: 15px;
    padding: 20px;
    background: #f8fafc;
    border-radius: 10px;
    border-left: 4px solid var(--primary-light);
    transition: all 0.3s;
}

.feature-card:hover {
    background: #eef3f8;
    transform: translateX(3px);
}

.feature-icon {
    font-size: 1.8em;
    min-width: 45px;
    text-align: center;
}

.feature-text h4 {
    color: var(--primary);
    margin-bottom: 4px;
    font-size: 1.05em;
}

.feature-text p {
    color: var(--text-light);
    font-size: 0.9em;
    margin: 0;
}

/* ===== ПРЕИМУЩЕСТВА ===== */
.benefits h3 { text-align: center; color: var(--primary); margin-bottom: 30px; font-size: 1.5em; }

.benefits-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(300px, 1fr)); gap: 25px; }

.benefit-item {
    background: var(--card);
    padding: 30px;
    border-radius: var(--radius);
    box-shadow: var(--shadow);
}

```

```

    text-align: center;
}

.benefit-icon { font-size: 2.5em; margin-bottom: 15px; }
.benefit-item h4 { color: var(--primary); margin-bottom: 10px; }

/* ===== ТАБЫ ===== */
.tabs { display: flex; gap: 5px; margin-bottom: 30px; justify-content: center;
flex-wrap: wrap; }

.tab-btn {
    background: white;
    border: 2px solid #ddd;
    padding: 12px 25px;
    border-radius: 8px;
    cursor: pointer;
    font-size: 1em;
    transition: all 0.3s;
}

.tab-btn.active, .tab-btn:hover {
    background: var(--primary);
    color: white;
    border-color: var(--primary);
}

.tab-content { display: none; }
.tab-content.active { display: block; }

/* ===== ТАБЛИЦЫ ===== */
.table-wrapper {
    background: var(--card);
    border-radius: var(--radius);
    box-shadow: var(--shadow);
    overflow-x: auto;
}

table {
    width: 100%;
    border-collapse: collapse;
}

th { background: var(--primary); color: white; padding: 14px; text-align: left;
font-weight: 500; }
td { padding: 12px 14px; border-bottom: 1px solid #eee; }
tr:hover { background: #f5f8fb; }

.status-available { color: var(--success); font-weight: 600; }
.status-issued { color: var(--primary-light); font-weight: 600; }
.status-overdue { color: var(--danger); font-weight: 600; }

```

```

/* ===== ПРОГНОЗ ===== */
.forecast-demo {
  background: var(--card);
  padding: 30px;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}

.forecast-demo h3 { color: var(--primary); margin-bottom: 5px; }

.forecast-months {
  display: flex;
  justify-content: space-around;
  align-items: flex-end;
  height: 200px;
  margin-bottom: 25px;
}

.forecast-month { text-align: center; }
.forecast-month span { display: block; margin-bottom: 5px; font-weight: 500; }
.forecast-month small { display: block; margin-top: 5px; color: var(--text-light); }

.forecast-bar {
  width: 60px;
  background: linear-gradient(to top, var(--primary), var(--primary-light));
  border-radius: 5px 5px 0 0;
  color: white;
  display: flex;
  align-items: flex-start;
  justify-content: center;
  padding-top: 5px;
  font-size: 0.9em;
  margin: 0 auto;
}

.forecast-summary {
  display: flex;
  gap: 20px;
  margin-top: 25px;
  flex-wrap: wrap;
}

.forecast-summary-item {
  flex: 1;
  min-width: 150px;
  background: #f8fafc;
  padding: 15px 20px;
  border-radius: 10px;
  text-align: center;
}

```

```

.forecast-summary-item.highlight {
  background: #e8f5e9;
  border: 2px solid var(--success);
}

.fs-label {
  display: block;
  color: var(--text-light);
  font-size: 0.85em;
  margin-bottom: 5px;
}

.fs-value {
  font-size: 1.5em;
  font-weight: 700;
  color: var(--primary);
}

.forecast-summary-item.highlight .fs-value { color: var(--success); }

/* ===== СЛАЙДЕР ===== */
.testimonials { margin-top: 50px; }

.testimonials h3 { text-align: center; color: var(--primary); margin-bottom: 25px; }

.slider {
  background: var(--card);
  padding: 40px;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  text-align: center;
  min-height: 200px;
  position: relative;
}

.slide { display: none; }
.slide.active { display: block; }

.testimonial-avatar { font-size: 3em; margin-bottom: 15px; }

.testimonial-text {
  font-size: 1.15em;
  line-height: 1.7;
  font-style: italic;
  color: #555;
  max-width: 700px;
  margin: 0 auto 20px;
  position: relative;
}

```

```

.testimonial-text::before {
  content: '«';
  font-size: 3em;
  color: var(--primary-light);
  opacity: 0.3;
  position: absolute;
  left: -30px;
  top: -15px;
}

.testimonial-text::after {
  content: '»';
  font-size: 3em;
  color: var(--primary-light);
  opacity: 0.3;
  position: absolute;
  right: -30px;
  bottom: -30px;
}

.testimonial-author {
  display: block;
  color: var(--primary);
  font-weight: 700;
  font-size: 1.05em;
}

.testimonial-org {
  display: block;
  color: var(--text-light);
  font-size: 0.9em;
  margin-top: 3px;
}

.slider-dots { text-align: center; margin-top: 15px; }

.dot {
  display: inline-block;
  width: 12px;
  height: 12px;
  background: #ccc;
  border-radius: 50%;
  margin: 0 5px;
  cursor: pointer;
  transition: background 0.3s;
}

.dot.active, .dot:hover { background: var(--primary); }

/* ===== КОМПАНИИ ===== */

```



```

.contact-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 30px;
}

.contact-info {
  background: var(--card);
  padding: 30px;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}

.contact-info h3 { color: var(--primary); margin-bottom: 20px; }
.contact-info p { margin-bottom: 10px; }

.contact-form {
  background: var(--card);
  padding: 30px;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}

.contact-form h3 { color: var(--primary); margin-bottom: 20px; }

.contact-form input, .contact-form textarea {
  width: 100%;
  padding: 12px;
  border: 2px solid #ddd;
  border-radius: 8px;
  margin-bottom: 15px;
  font-family: inherit;
  font-size: 1em;
}

.contact-form input:focus, .contact-form textarea:focus {
  border-color: var(--primary-light);
  outline: none;
}

.btn-submit {
  width: 100%;
  padding: 14px;
  background: var(--success);
  color: white;
  border: none;
  border-radius: 8px;
  font-size: 1.1em;
  cursor: pointer;
  transition: 0.3s;
}

```

```

.btn-submit:hover { background: #219a52; }

/* ===== ПОДВАЛ ===== */
footer {
  background: var(--primary);
  color: white;
  text-align: center;
  padding: 25px;
  font-size: 0.9em;
}

/* ===== АДАПТИВ ===== */
@media (max-width: 768px) {
  .hero h1 { font-size: 1.8em; }
  .hero h2 { font-size: 1.1em; }
  .nav-links a { margin-left: 10px; font-size: 0.9em; }
  .contact-grid { grid-template-columns: 1fr; }
  .forecast-months { height: 150px; }
  .forecast-bar { width: 40px; }
  .testimonial-text::before, .testimonial-text::after { display: none; }
  .testimonial-text { font-size: 1em; }
}

```

Листинг 11 – script.js

```

// ===== ТАБЫ В РАЗДЕЛЕ "РЕЗУЛЬТАТЫ" =====

function showResultTab(tabName, btn) {
  document
    .querySelectorAll(".tab-btn")
    .forEach((b) => b.classList.remove("active"));
  document
    .querySelectorAll(".tab-content")
    .forEach((t) => t.classList.remove("active"));
  btn.classList.add("active");
  document.getElementById("tab-" + tabName).classList.add("active");
}

// ===== СЛАЙДЕР ОТЗЫВОВ =====

let slideIndex = 0;
let slideInterval;

function currentSlide(n) {
  slideIndex = n;
  updateSlider();
  resetAutoSlide();
}

function updateSlider() {

```

```

document.querySelectorAll(".slide").forEach((slide, i) => {
    slide.classList.toggle("active", i === slideIndex);
});
document.querySelectorAll(".dot").forEach((dot, i) => {
    dot.classList.toggle("active", i === slideIndex);
});
}

function autoSlide() {
    const totalSlides = document.querySelectorAll(".slide").length;
    slideIndex = (slideIndex + 1) % totalSlides;
    updateSlider();
}

function resetAutoSlide() {
    clearInterval(slideInterval);
    slideInterval = setInterval(autoSlide, 5000);
}

slideInterval = setInterval(autoSlide, 5000);

// ===== ДАННЫЕ ДЛЯ ТАБЛИЦ =====

const inventoryData = [
    {
        barcode: "SKI001-2024-001",
        name: "Льжи Fischer Speedmax",
        category: "Льжи",
        size: "190",
        status: "available",
    },
    {
        barcode: "SKI001-2024-003",
        name: "Льжи Fischer Speedmax",
        category: "Льжи",
        size: "200",
        status: "available",
    },
    {
        barcode: "SKI002-2024-001",
        name: "Льжи Madshus Redline",
        category: "Льжи",
        size: "192",
        status: "available",
    },
    {
        barcode: "BOOT001-2024-001",
        name: "Ботинки Salomon S/Lab",
        category: "Ботинки",
        size: "42",
        status: "available",
    }
]

```

```

    },
    {
      barcode: "BOOT001-2024-003",
      name: "Ботинки Salomon S/Lab",
      category: "Ботинки",
      size: "44",
      status: "available",
    },
    {
      barcode: "POLE001-2024-001",
      name: "Палки Swix Triac",
      category: "Палки",
      size: "155",
      status: "available",
    },
    {
      barcode: "WAX001-2024-001",
      name: "Парафин Swix LF4",
      category: "Смазки",
      size: "-",
      status: "available",
    },
    {
      barcode: "WAX002-2024-001",
      name: "Парафин Swix HF6",
      category: "Смазки",
      size: "-",
      status: "available",
    },
  ],
];

const issuanceData = [
  {
    athlete: "Иванов Иван",
    item: "Лыжи Fischer Speedmax (195)",
    date: "01.12.2024",
    return: "30.03.2025",
    status: "active",
  },
  {
    athlete: "Петров Пётр",
    item: "Лыжи Fischer Speedmax (198)",
    date: "05.12.2024",
    return: "30.03.2025",
    status: "active",
  },
  {
    athlete: "Смирнова Анна",
    item: "Лыжи Madshus Redline (198)",
    date: "10.12.2024",
    return: "30.03.2025",
  },
];

```

```

        status: "active",
    },
    {
        athlete: "Кузнецов Дмитрий",
        item: "Ботинки Salomon S/Lab (41)",
        date: "15.12.2024",
        return: "30.03.2025",
        status: "active",
    },
    {
        athlete: "Сидоров Сидор",
        item: "Лыжи Fischer Speedmax (200)",
        date: "01.10.2024",
        return: "01.11.2024",
        status: "overdue",
    },
];

const statusLabels = {
    available: "Доступен",
    active: "Активна",
    overdue: "Просрочена",
};

const statusClasses = {
    available: "status-available",
    active: "status-issued",
    overdue: "status-overdue",
};

function loadResultTables() {
    document.getElementById("result-inventory-tbody").innerHTML = inventoryData
        .map(
            (item) => `
                <tr>
                    <td>${item.barcode}</td>
                    <td>${item.name}</td>
                    <td>${item.category}</td>
                    <td>${item.size}</td>
                    <td class="${statusClasses[item.status]}">${statusLabels[item.sta-
tus]}</td>
                </tr>
            `,
        )
        .join("");

    document.getElementById("result-issuance-tbody").innerHTML = issuanceData
        .map(
            (item) => `
                <tr>
                    <td>${item.athlete}</td>

```

```
        <td>${item.item}</td>
        <td>${item.date}</td>
        <td>${item.return}</td>
        <td class="${statusClasses[item.status]}">${statusLabels[item.sta-
tus]}</td>
    </tr>
    `
    )
    .join("");
}

document.addEventListener("DOMContentLoaded", loadResultTables);
```

Приложение 4. Исходный код портала для работы с БД

Листинг 12 – requirements.txt

```
fastapi==0.108.0
uvicorn[standard]==0.25.0
sqlalchemy==2.0.25
psycopg2-binary==2.9.9
loguru==0.7.2
pydantic==2.5.0
python-dotenv==1.0.0
```

Листинг 13 – config.py

```
import os
from dataclasses import dataclass, field

@dataclass
class DatabaseConfig:
    host: str = field(default_factory=lambda: os.getenv("DB_HOST", "localhost"))
    port: int = field(default_factory=lambda: int(os.getenv("DB_PORT", "5432")))
    database: str = field(default_factory=lambda: os.getenv("DB_NAME", "postgres"))
    schema: str = field(default_factory=lambda: os.getenv("DB_SCHEMA", "asu_inventory"))
    username: str = field(default_factory=lambda: os.getenv("DB_USERNAME", "postgres"))
    password: str = field(default_factory=lambda: os.getenv("DB_PASSWORD", ""))

    @property
    def connection_string(self) -> str:
        return f"postgresql://{self.username}:{self.password}@{self.host}:{self.port}/{self.database}"

@dataclass
class AppConfig:
    db: DatabaseConfig = field(default_factory=DatabaseConfig)

config = AppConfig()
```

Листинг 14 – database.py

```
from sqlalchemy import create_engine
from loguru import logger
from config import config

class DatabaseManager:
    def __init__(self):
        self.engine = create_engine(
            config.db.connection_string,
            pool_size=5,
            max_overflow=10,
            echo=False,
            connect_args={'options': f'-c search_path={config.db.schema},public'}
        )
```

```

    )
    logger.info(f"Подключение к БД: {config.db.host}:{config.db.port}/{config.db.database}")

```

Листинг 15 – main.py

```

from fastapi import FastAPI, HTTPException, Query
from fastapi.staticfiles import StaticFiles
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import Optional, List
from datetime import date, datetime

from database import DatabaseManager
from config import config

app = FastAPI(title="ACY MTO Web API", version="2.0.0")

app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
allow_headers=["*"])

db = DatabaseManager()

# ===== Pydantic-схемы =====

class AthleteResponse(BaseModel):
    athlete_id: int
    full_name: str
    sports_category: Optional[str]
    birth_year: Optional[int]
    group_name: Optional[str]
    coach_name: Optional[str]
    has_active_debt: bool

class IssueRequest(BaseModel):
    athlete_id: int
    inventory_item_id: int
    planned_return_date: date

class ReturnRequest(BaseModel):
    issuance_id: int

class MessageResponse(BaseModel):
    status: str
    message: str

# ===== ДАШБОРД =====

@app.get("/api/dashboard")
async def get_dashboard():
    """Ключевые метрики, последние выдачи, топ-должники."""
    with db.engine.connect() as conn:
        from sqlalchemy import text

        athletes_count = conn.execute(text("SELECT COUNT(*) FROM athlete")).scalar()
        available_items = conn.execute(text(
            "SELECT COUNT(*) FROM inventory_item WHERE inventory_status = 'AVAILABLE'"
        )).scalar()
        active_issuances = conn.execute(text(

```



```

        "SELECT COUNT(*) FROM issuance_document WHERE issuance_document_status IN ('ACTIVE', 'EXTENDED', 'OVERDUE')")
    ))).scalar()
    overdue_count = conn.execute(text("""
        SELECT COUNT(*) FROM issuance_document
        WHERE planned_return_date < CURRENT_DATE
        AND issuance_document_status IN ('ACTIVE', 'OVERDUE')
    """)).scalar()
    debtors = conn.execute(text(
        "SELECT COUNT(*) FROM athlete WHERE has_active_debt = TRUE"
    )).scalar()

    categories = conn.execute(text("""
        SELECT n.nomenclature_category, COUNT(*) as cnt
        FROM inventory_item i
        JOIN nomenclature n ON i.nomenclature_id = n.nomenclature_id
        WHERE i.inventory_status = 'AVAILABLE'
        GROUP BY n.nomenclature_category ORDER BY cnt DESC
    """)).fetchall()

    recent_issuances = conn.execute(text("""
        SELECT u.full_name AS athlete_name, n.nomenclature_name AS product_name,
            d.issuance_date, d.planned_return_date,
            CASE WHEN d.planned_return_date < CURRENT_DATE THEN
'OVERDUE' ELSE 'ACTIVE' END AS effective_status
        FROM issuance_document d
        JOIN athlete a ON d.athlete_id = a.athlete_id
        JOIN "user" u ON a.user_id = u.user_id
        JOIN inventory_item i ON d.inventory_item_id = i.inventory_item_id
        JOIN nomenclature n ON i.nomenclature_id = n.nomenclature_id
        WHERE d.issuance_document_status IN ('ACTIVE', 'EXTENDED',
'OVERDUE')
        ORDER BY d.issuance_date DESC LIMIT 5
    """)).fetchall()

    top_debtors = conn.execute(text("""
        SELECT u.full_name AS athlete_name, a.sports_category,
            COUNT(d.issuance_document_id) AS overdue_items,
            MAX(CURRENT_DATE - d.planned_return_date) AS max_days
        FROM athlete a
        JOIN "user" u ON a.user_id = u.user_id
        JOIN issuance_document d ON a.athlete_id = d.athlete_id
        WHERE d.planned_return_date < CURRENT_DATE
            AND d.issuance_document_status IN ('ACTIVE', 'OVERDUE')
        GROUP BY u.full_name, a.sports_category
        ORDER BY overdue_items DESC LIMIT 5
    """)).fetchall()

    return {
        "athletes_count": athletes_count,
        "available_items": available_items,
        "active_issuances": active_issuances,
        "overdue_count": overdue_count,
        "debtors": debtors,
        "categories": [{"category": r[0], "count": r[1]} for r in categories],
        "recent_issuances": [dict(r._mapping) for r in recent_issuances],
        "top_debtors": [dict(r._mapping) for r in top_debtors]
    }

```

```

# ===== СПОРТСМЕНЫ =====

@app.get("/api/athletes")
async def get_athletes(
    search: str = Query("", description="Поиск по фамилии"),
    has_debt: Optional[str] = Query(None, description="Фильтр по задолженно-
сти")
):
    query = """
    SELECT
        a.athlete_id, u.full_name, a.sports_category, a.birth_year,
        g.group_name, cu.full_name AS coach_name, a.has_active_debt,
        (SELECT COUNT(*) FROM issuance_document d
         WHERE d.athlete_id = a.athlete_id
          AND d.issuance_document_status IN ('ACTIVE','EXTENDED','OVERDUE'))
    AS active_items
    FROM athlete a
    JOIN "user" u ON a.user_id = u.user_id
    LEFT JOIN "group" g ON a.group_id = g.group_id
    LEFT JOIN coach c ON a.coach_id = c.coach_id
    LEFT JOIN "user" cu ON c.user_id = cu.user_id
    WHERE 1=1
    """
    params = {}
    if search:
        query += " AND u.full_name ILIKE :search"
        params["search"] = f"%{search}%"
    if has_debt == "true":
        query += " AND a.has_active_debt = TRUE"
    elif has_debt == "false":
        query += " AND a.has_active_debt = FALSE"
    query += " ORDER BY u.full_name"

    with db.engine.connect() as conn:
        from sqlalchemy import text
        result = conn.execute(text(query), params)
        return [dict(row._mapping) for row in result]

@app.get("/api/athletes/{athlete_id}")
async def get_athlete_detail(athlete_id: int):
    """Детальная информация о спортсмене."""
    query = """
    SELECT
        a.athlete_id, u.full_name, u.email, u.phone,
        a.sports_category, a.birth_year, a.has_active_debt,
        g.group_name, cu.full_name AS coach_name,
        (SELECT COUNT(*) FROM issuance_document d
         WHERE d.athlete_id = a.athlete_id
          AND d.issuance_document_status IN ('ACTIVE','EXTENDED','OVERDUE'))
    AS active_items,
        (SELECT COUNT(*) FROM issuance_document d
         WHERE d.athlete_id = a.athlete_id
          AND d.issuance_document_status = 'RETURNED') AS returned_items
    FROM athlete a
    JOIN "user" u ON a.user_id = u.user_id
    LEFT JOIN "group" g ON a.group_id = g.group_id
    LEFT JOIN coach c ON a.coach_id = c.coach_id
    LEFT JOIN "user" cu ON c.user_id = cu.user_id
    WHERE a.athlete_id = :athlete_id
    """
    with db.engine.connect() as conn:
        from sqlalchemy import text
        result = conn.execute(text(query), {"athlete id": athlete_id})

```

```

        row = result.fetchone()
        if row:
            return dict(row._mapping)
        raise HTTPException(status_code=404, detail="Спортсмен не найден")

# ===== ИМБЕХТАРЬ =====

@app.get("/api/inventory/available")
async def get_available_inventory():
    query = """
        SELECT
            i.inventory_item_id, i.barcode, i.size,
            n.nomenclature_name, n.nomenclature_category
        FROM inventory_item i
        JOIN nomenclature n ON i.nomenclature_id = n.nomenclature_id
        WHERE i.inventory_status = 'AVAILABLE'
        ORDER BY n.nomenclature_name, i.size
        """
    with db.engine.connect() as conn:
        from sqlalchemy import text
        result = conn.execute(text(query))
        return [dict(row._mapping) for row in result]

@app.get("/api/inventory/statistics")
async def get_inventory_statistics():
    query = """
        SELECT
            n.nomenclature_category,
            COUNT(*) FILTER (WHERE i.inventory_status = 'AVAILABLE') AS available,
            COUNT(*) FILTER (WHERE i.inventory_status = 'ISSUED') AS issued,
            COUNT(*) FILTER (WHERE i.inventory_status IN ('DEFECTIVE', 'UNDER_REPAIR')) AS problems,
            COUNT(*) AS total
        FROM inventory_item i
        JOIN nomenclature n ON i.nomenclature_id = n.nomenclature_id
        GROUP BY n.nomenclature_category ORDER BY total DESC
        """
    with db.engine.connect() as conn:
        from sqlalchemy import text
        result = conn.execute(text(query))
        return [dict(row._mapping) for row in result]

# ===== ВЫДАЧА =====

@app.post("/api/issuance/issue", response_model=MessageResponse)
async def issue_inventory(request: IssueRequest):
    try:
        with db.engine.connect() as conn:
            from sqlalchemy import text

            result = conn.execute(
                text("SELECT has_active_debt FROM athlete WHERE athlete_id = :id"),
                {"id": request.athlete_id}
            )
            row = result.fetchone()
            if not row:
                raise HTTPException(status_code=404, detail="Спортсмен не найден")
            if row[0]:

```

```

        return MessageResponse(status="error", message="⚠ Спортсмен имеет задолженность! Выдача невозможна.")

    result = conn.execute(
        text("SELECT inventory_status FROM inventory_item WHERE inventory_item_id = :id"),
        {"id": request.inventory_item_id}
    )
    row = result.fetchone()
    if not row or row[0] != 'AVAILABLE':
        return MessageResponse(status="error", message="⚠ Инвентарь недоступен для выдачи.")

    result = conn.execute(
        text("""
            INSERT INTO issuance_document
            (inventory_item_id, athlete_id,
             storekeeper_id, issuance_date, planned_return_date,
             issuance_document_status, condition_on_issue)
            VALUES (:inv_id, :ath_id, 1, CURRENT_DATE, :ret_date,
            'ACTIVE', 'NEW')
            RETURNING issuance_document_id
        """),
        {"inv_id": request.inventory_item_id, "ath_id": request.athlete_id, "ret_date": request.planned_return_date}
    )
    issuance_id = result.scalar()

    conn.execute(
        text("""
            UPDATE inventory_item
            SET inventory_status = 'ISSUED', athlete_id = :ath_id
            WHERE inventory_item_id = :inv_id
        """),
        {"ath_id": request.athlete_id, "inv_id": request.inventory_item_id}
    )
    conn.commit()

    return MessageResponse(status="success", message=f"✅ Инвентарь успешно выдан! Документ №{issuance_id}")

except HTTPException:
    raise
except Exception as e:
    return MessageResponse(status="error", message=str(e))

# ===== АКТИВНЫЕ ВЫДАЧИ =====

@app.get("/api/issuance/active")
async def get_active_issuances(filter_status: str = Query("all")):
    query = """
    SELECT
        d.issuance_document_id, u.full_name AS athlete_name,
        i.barcode, n.nomenclature_name AS product_name,
        i.size, d.issuance_date, d.planned_return_date,
        CASE
            WHEN d.planned_return_date < CURRENT_DATE AND d.issuance_document_status = 'ACTIVE'
            THEN 'OVERDUE'
            ELSE d.issuance_document_status
        END AS effective_status,
    """

```

```

        CASE
            WHEN d.planned_return_date < CURRENT_DATE
            THEN (CURRENT_DATE - d.planned_return_date)
            ELSE 0
        END AS days_overdue
FROM issuance_document d
JOIN athlete a ON d.athlete_id = a.athlete_id
JOIN "user" u ON a.user_id = u.user_id
JOIN inventory_item i ON d.inventory_item_id = i.inventory_item_id
JOIN nomenclature n ON i.nomenclature_id = n.nomenclature_id
WHERE d.issuance_document_status IN ('ACTIVE', 'EXTENDED', 'OVERDUE')
"""

params = {}
if filter_status == "overdue":
    query += " AND d.planned_return_date < CURRENT_DATE"
elif filter_status == "active":
    query += " AND d.planned_return_date >= CURRENT_DATE"
query += " ORDER BY days_overdue DESC, d.issuance_date DESC"

with db.engine.connect() as conn:
    from sqlalchemy import text
    result = conn.execute(text(query), params)
    return [dict(row._mapping) for row in result]

# ===== БОЗБПАТ =====

@app.post("/api/issuance/return", response_model=MessageResponse)
async def return_inventory(request: ReturnRequest):
    try:
        with db.engine.connect() as conn:
            from sqlalchemy import text

            result = conn.execute(
                text("""
                    SELECT inventory_item_id, issuance_document_status
                    FROM issuance_document WHERE issuance_document_id = :id
                """),
                {"id": request.issuance_id}
            )
            row = result.fetchone()
            if not row:
                return MessageResponse(status="error", message="Документ вы-
дачи не найден.")
            if row[1] == 'RETURNED':
                return MessageResponse(status="error", message="Инвентарь
уже возвращён.")

            conn.execute(
                text("""
                    UPDATE issuance_document
                    SET actual_return_date = CURRENT_DATE,
                      issuance_document_status = 'RETURNED',
                      condition_on_return = 'GOOD'
                    WHERE issuance_document_id = :id
                """),
                {"id": request.issuance_id}
            )

            conn.execute(
                text("""
                    UPDATE inventory_item
                    SET inventory_status = 'AVAILABLE', athlete_id = NULL
                    WHERE inventory item id = :inv id
                """)
            )

```

```

        """),
        {"inv_id": row[0]}
    )
    conn.commit()

    return MessageResponse(status="success", message="✅ Инвентарь
успешно возвращён!")

except Exception as e:
    return MessageResponse(status="error", message=str(e))

# ===== СТАТИКА =====

app.mount("/", StaticFiles(directory="static", html=True), name="static")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)

```

Листинг 16 – static/index.html

```

<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>АСУ МТО – Управление инвентарём</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <!-- Шапка -->
    <header>
        <div class="header-content">
            <h1>🏃 АСУ МТО</h1>
            <span class="subtitle">Лыжная организация</span>
        </div>
        <nav>
            <button onclick="showTab('dashboard')" class="active">🏠
Дашборд</button>
            <button onclick="showTab('athletes')">🏃 Спортсмены</button>
            <button onclick="showTab('issuances')">📄 Выдачи</button>
            <button onclick="showTab('inventory')">📦 Склад</button>
            <button onclick="showTab('statistics')">📊 Статистика</button>
        </nav>
    </header>

    <!-- Тост -->
    <div id="toast" class="toast hidden"></div>

    <main>
        <!-- Дашборд -->
        <section id="tab-dashboard" class="tab active">
            <div class="kpi-grid" id="kpi-grid">
                <div class="kpi-card"><div class="kpi-value skeleton">--
</div><div class="kpi-label">Спортсменов</div></div>
                <div class="kpi-card"><div class="kpi-value skeleton">--
</div><div class="kpi-label">Доступно инвентаря</div></div>
                <div class="kpi-card"><div class="kpi-value skeleton">--
</div><div class="kpi-label">Активных выдач</div></div>
                <div class="kpi-card"><div class="kpi-value skeleton">--

```

```

</div><div class="kpi-label">Просрочено</div></div>
    <div class="kpi-card warning"><div class="kpi-value skeleton"></div><div class="kpi-label">Должников</div></div>
    </div>
    <div class="dashboard-grid">
        <div class="chart-card">
            <h3>📦 Инвентарь по категориям (в наличии)</h3>
            <div id="category-chart"></div>
        </div>
        <div class="chart-card">
            <h3>🔔 Последние выдачи</h3>
            <div class="table-wrapper">
                <table>

<thead><tr><th>Спортсмен</th><th>Товар</th><th>Выдано</th><th>Вернуть
до</th><th>Статус</th></tr></thead>
                <tbody id="recent-issuances-tbody"></tbody>
            </table>
        </div>
        <div class="chart-card">
            <h3>⚠️ Топ должников</h3>
            <div class="table-wrapper">
                <table>

<thead><tr><th>Спортсмен</th><th>Разряд</th><th>Просрочено</th><th>Макс.
дней</th></tr></thead>
                <tbody id="top-debtors-tbody"></tbody>
            </table>
        </div>
    </div>
</div>
</section>

<!-- Спортсмены -->
<section id="tab-athletes" class="tab">
    <div class="section-header">
        <h2>🏃 Спортсмены</h2>
        <div class="controls">
            <input type="text" id="athlete-search" placeholder="🔍
Поиск по фамилии..." oninput="debounceSearch()">
            <select id="debt-filter" onchange="loadAthletes()">
                <option value="">Все</option>
                <option value="true">Должники</option>
                <option value="false">Без долгов</option>
            </select>
        </div>
    </div>
    <div class="table-wrapper">
        <table>

<thead><tr><th>ФИО</th><th>Разряд</th><th>Год</th><th>Группа</th><th>Тренер<
/th><th>Выдач</th><th>Долг</th><th></th></tr></thead>
        <tbody id="athletes-tbody"></tbody>
    </table>
    </div>
</section>

<!-- Выдачи -->
<section id="tab-issuances" class="tab">
    <div class="section-header">
        <h2>📄 Выдачи</h2>

```

```

        <div class="controls">
            <button class="btn-primary" onclick="openIssue-
Modal()"><img alt="document icon" data-bbox="248 98 265 112"/> Выдать инвентарь</button>
            <select id="issuance-filter" onchange="loadIssuances()">
                <option value="all">Все</option>
                <option value="overdue">Просроченные</option>
                <option value="active">Активные</option>
            </select>
        </div>
    </div>
    <div class="table-wrapper">
        <table>

<thead><tr><th>Спортсмен</th><th>Товар</th><th>Размер</th><th>Выдано</th><th>
>Вернуть до</th><th>Статус</th><th></th></tr></thead>
        <tbody id="issuances-tbody"></tbody>
    </table>
    </div>
</section>

<!-- Склад -->
<section id="tab-inventory" class="tab">
    <div class="section-header"><h2><img alt="box icon" data-bbox="305 370 322 384"/> Доступный инвен-
тарь</h2></div>
    <div class="table-wrapper">
        <table>
            <thead><tr><th>Штрих-
код</th><th>Наименование</th><th>Категория</th><th>Размер</th></tr></thead>
            <tbody id="inventory-tbody"></tbody>
        </table>
    </div>
</section>

<!-- Статистика -->
<section id="tab-statistics" class="tab">
    <div class="section-header"><h2><img alt="chart icon" data-bbox="305 548 322 562"/> Статистика по инвен-
тарю</h2></div>
    <div id="stats-container"></div>
</section>
</main>

<!-- Модальное окно: Выдача -->
<div id="issue-modal" class="modal hidden">
    <div class="modal-content">
        <span class="modal-close" onclick="closeIssueModal()"><img alt="close icon" data-bbox="885 670 902 684"/></span>
        <h2><img alt="document icon" data-bbox="315 685 332 699"/> Выдача инвентаря</h2>
        <form id="issue-form" onsubmit="event.preventDefault(); issue-
Inventory();">
            <label>Спортсмен</label>
            <select id="issue-athlete" required><option value="">— Выбе-
рите —</option></select>
            <label>Инвентарь</label>
            <select id="issue-inventory" required><option value="">—
Сначала выберите спортсмена —</option></select>
            <label>Дата возврата</label>
            <input type="date" id="issue-return-date" required>
            <button type="submit" class="btn-primary-full"><img alt="rocket icon" data-bbox="790 835 807 849"/> Вы-
дать</button>
        </form>
    </div>
</div>

<!-- Модальное окно: Детали спортсмена -->

```



```

    <div id="athlete-modal" class="modal hidden">
      <div class="modal-content">
        <span class="modal-close" onclick="closeAthleteModal()">×</span>
        <div id="athlete-detail"></div>
      </div>
    </div>

    <script src="app.js"></script>
  </body>
</html>

```

Листинг 17 – static/style.css

```

/* ===== ПЕРЕМЕННЫЕ ===== */
:root {
  --primary: #1a3a5c;
  --primary-light: #2d6a9f;
  --success: #27ae60;
  --warning: #f39c12;
  --danger: #e74c3c;
  --bg: #f0f4f8;
  --card: #ffffff;
  --text: #333333;
  --text-light: #666666;
  --radius: 12px;
  --shadow: 0 4px 20px rgba(0, 0, 0, 0.08);
}

/* ===== БАЗОВЫЕ СТИЛИ ===== */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Segoe UI', 'Arial', sans-serif;
  background: var(--bg);
  color: var(--text);
  min-height: 100vh;
}

/* ===== ШАПКА ===== */
header {
  background: linear-gradient(135deg, #0f2640, var(--primary), var(--primary-light));
  color: white;
  padding: 20px 30px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  gap: 15px;
}

.header-content h1 {
  font-size: 1.6em;
}

.header-content .subtitle {
  font-size: 0.85em;
  opacity: 0.8;
}

```

```

nav {
  display: flex;
  gap: 5px;
  flex-wrap: wrap;
}

nav button {
  background: rgba(255, 255, 255, 0.1);
  color: white;
  border: none;
  padding: 10px 18px;
  border-radius: 8px;
  cursor: pointer;
  font-size: 0.9em;
  transition: all 0.3s;
  white-space: nowrap;
}

nav button:hover,
nav button.active {
  background: rgba(255, 255, 255, 0.25);
  transform: translateY(-1px);
}

/* ===== TOCT ===== */
.toast {
  position: fixed;
  top: 20px;
  right: 20px;
  padding: 15px 25px;
  border-radius: var(--radius);
  color: white;
  font-weight: 500;
  z-index: 9999;
  transition: all 0.4s;
  box-shadow: var(--shadow);
}

.toast.success {
  background: var(--success);
}

.toast.error {
  background: var(--danger);
}

.toast.hidden {
  opacity: 0;
  transform: translateX(100%);
}

/* ===== KOHTEHT ===== */
main {
  max-width: 1300px;
  margin: 30px auto;
  padding: 0 25px;
}

.tab {
  display: none;
  animation: fadeIn 0.3s;
}

```

```

.tab.active {
  display: block;
}

@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(10px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* ===== KPI ===== */
.kpi-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 20px;
  margin-bottom: 30px;
}

.kpi-card {
  background: var(--card);
  padding: 25px;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  text-align: center;
  transition: transform 0.3s;
}

.kpi-card:hover {
  transform: translateY(-3px);
}

.kpi-card.warning {
  border-left: 4px solid var(--warning);
}

.kpi-value {
  font-size: 2.2em;
  font-weight: 700;
  color: var(--primary);
}

.kpi-value.skeleton {
  color: #ccc;
}

.kpi-label {
  color: var(--text-light);
  margin-top: 5px;
  font-size: 0.9em;
}

/* ===== ДАШБОРД-ГРИД ===== */
.dashboard-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 20px;
}

@media (max-width: 900px) {

```

```

.dashboard-grid {
  grid-template-columns: 1fr;
}

/* ===== ГРАФИК КАТЕГОРИЙ ===== */
.chart-card {
  background: var(--card);
  padding: 25px;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}

.chart-card h3 {
  margin-bottom: 20px;
  color: var(--primary);
}

.chart-row {
  display: flex;
  align-items: center;
  gap: 15px;
  margin-bottom: 12px;
}

.chart-label {
  width: 100px;
  text-transform: capitalize;
  font-weight: 500;
}

.chart-bar-wrapper {
  flex: 1;
  background: #e8edf2;
  border-radius: 6px;
  height: 24px;
  overflow: hidden;
}

.chart-bar {
  height: 100%;
  background: linear-gradient(90deg, var(--primary-light), var(--primary));
  border-radius: 6px;
  transition: width 0.5s;
}

.chart-value {
  width: 40px;
  text-align: right;
  font-weight: 600;
  color: var(--primary);
}

/* ===== ЗАГОЛОВКИ СЕКЦИЙ ===== */
.section-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  gap: 15px;
  margin-bottom: 25px;
}

```

```

.section-header h2 {
  color: var(--primary);
}

.controls {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
}

.controls input,
.controls select {
  padding: 8px 15px;
  border: 2px solid #ddd;
  border-radius: 8px;
  font-size: 0.9em;
  transition: border-color 0.3s;
}

.controls input:focus,
.controls select:focus {
  border-color: var(--primary-light);
  outline: none;
}

/* ===== ТАБЛИЦЫ ===== */
.table-wrapper {
  background: var(--card);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  overflow-x: auto;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th {
  background: var(--primary);
  color: white;
  padding: 14px;
  text-align: left;
  font-weight: 500;
  font-size: 0.9em;
}

td {
  padding: 12px 14px;
  border-bottom: 1px solid #eee;
}

tr:hover {
  background: #f5f8fb;
}

tr.row-warning {
  background: #fffd5;
}

tr.row-danger {
  background: #fff5f5;
}

```

```

/* ===== БЕЙДЖИ ===== */
.badge {
  padding: 5px 12px;
  border-radius: 20px;
  font-size: 0.8em;
  font-weight: 600;
}

.badge-active,
.badge-extended {
  background: #d4edff;
  color: #1a6dbb;
}

.badge-overdue {
  background: #ffe0e0;
  color: #c0392b;
}

/* ===== КНОПКИ ===== */
.btn-primary {
  background: var(--success);
  color: white;
  border: none;
  padding: 10px 20px;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 500;
  transition: all 0.3s;
}

.btn-primary:hover {
  background: #219a52;
  transform: translateY(-1px);
}

.btn-sm {
  background: var(--primary-light);
  color: white;
  border: none;
  padding: 6px 12px;
  border-radius: 6px;
  cursor: pointer;
}

.btn-return {
  background: #3498db;
}

.btn-return:hover {
  background: #2980b9;
}

/* ===== МОДАЛЬНОЕ ОКНО ===== */
.modal {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background: rgba(0, 0, 0, 0.5);
  display: flex;
  align-items: center;
  justify-content: center;
}

```

```

    z-index: 1000;
}

.modal.hidden {
    display: none;
}

.modal-content {
    background: white;
    padding: 35px;
    border-radius: var(--radius);
    max-width: 500px;
    width: 90%;
    position: relative;
    animation: fadeIn 0.3s;
}

.modal-close {
    position: absolute;
    top: 15px;
    right: 20px;
    font-size: 1.5em;
    cursor: pointer;
    color: #999;
}

.modal-content h2,
.modal-content h3 {
    color: var(--primary);
    margin-bottom: 20px;
}

.modal-content label {
    display: block;
    margin: 15px 0 5px;
    font-weight: 500;
    color: var(--primary);
}

.modal-content select,
.modal-content input {
    width: 100%;
    padding: 10px;
    border: 2px solid #ddd;
    border-radius: 8px;
    font-size: 1em;
}

.modal-content select:focus,
.modal-content input:focus {
    border-color: var(--primary-light);
    outline: none;
}

.btn-primary-full {
    width: 100%;
    margin-top: 20px;
    padding: 14px;
    background: var(--success);
    color: white;
    border: none;
    border-radius: 8px;
    font-size: 1.1em;
    cursor: pointer;
}

```

```

    transition: 0.3s;
}

.btn-primary-full:hover {
    background: #219a52;
}

/* ===== ДЕТАЛИ СПОРТСМЕНА ===== */
.detail-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 12px;
}

.detail-item {
    background: var(--bg);
    padding: 10px 14px;
    border-radius: 6px;
}

.detail-label {
    display: block;
    font-size: 0.8em;
    color: var(--text-light);
    margin-bottom: 3px;
}

.text-success {
    color: var(--success);
}

.text-danger {
    color: var(--danger);
}

/* ===== СТАТИСТИКА ===== */
#stats-container {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    gap: 20px;
}

.stat-card {
    background: var(--card);
    padding: 20px 25px;
    border-radius: var(--radius);
    box-shadow: var(--shadow);
}

.stat-card h3 {
    color: var(--primary);
    margin-bottom: 15px;
    text-transform: capitalize;
}

.stat-row {
    display: flex;
    justify-content: space-between;
    margin-bottom: 8px;
}

.stat-available {
    color: var(--success);
    font-weight: 600;
}

```



```

}

.stat-issued {
  color: var(--primary-light);
  font-weight: 600;
}

.stat-problems {
  color: var(--danger);
  font-weight: 600;
}

.stat-bar {
  display: flex;
  height: 20px;
  border-radius: 10px;
  overflow: hidden;
  margin-top: 10px;
  background: #e8edf2;
}

.stat-bar-available {
  background: var(--success);
}

.stat-bar-issued {
  background: var(--primary-light);
}

.stat-bar-problems {
  background: var(--danger);
}

/* ===== АДАПТИВ ===== */
@media (max-width: 768px) {
  header {
    flex-direction: column;
    text-align: center;
  }

  nav {
    justify-content: center;
  }

  nav button {
    padding: 8px 12px;
    font-size: 0.8em;
  }

  .kpi-grid {
    grid-template-columns: repeat(2, 1fr);
  }

  .section-header {
    flex-direction: column;
    align-items: flex-start;
  }

  .detail-grid {
    grid-template-columns: 1fr;
  }

  #stats-container {
    grid-template-columns: 1fr;
  }
}

```

```

    }
  }
}

```

Листинг 18 – static/app.js

```

const API = 'http://localhost:8000';
let searchTimeout;

// ===== НАВИГАЦИЯ =====

function showTab(name) {
  document.querySelectorAll('.tab').forEach(t => t.classList.remove('active'));
  document.querySelectorAll('nav button').forEach(b => b.classList.remove('active'));
  document.getElementById('tab-' + name).classList.add('active');
  event.target.classList.add('active');

  const loaders = {
    dashboard: loadDashboard,
    athletes: loadAthletes,
    issuances: loadIssuances,
    inventory: loadInventory,
    statistics: loadStatistics,
  };
  if (loaders[name]) loaders[name]();
}

// ===== ТОСТЫ =====

function showToast(message, type) {
  const toast = document.getElementById('toast');
  toast.textContent = message;
  toast.className = `toast ${type}`;
  setTimeout(() => toast.className = 'toast hidden', 3000);
}

// ===== ДАШБОРД =====

async function loadDashboard() {
  const res = await fetch(`${API}/api/dashboard`);
  const d = await res.json();

  // KPI
  const kpiValues = [d.athletes_count, d.available_items, d.active_issuances, d.overdue_count, d.debtors];
  document.querySelectorAll('.kpi-value').forEach((el, i) => {
    el.textContent = kpiValues[i] ?? '-';
    el.classList.remove('skeleton');
  });

  // Категории
  const maxCat = Math.max(...d.categories.map(c => c.count), 1);
  document.getElementById('category-chart').innerHTML = d.categories.map(c => `
    <div class="chart-row">
      <span class="chart-label">${c.category}</span>
      <div class="chart-bar-wrapper">
        <div class="chart-bar" style="width:${(c.count / maxCat * 100).toFixed(0)}%></div>
      </div>
      <span class="chart-value">${c.count}</span>
    </div>
  `);
}

```

```

    `).join('');

    // Последние выдачи
    document.getElementById('recent-issuances-tbody').innerHTML = d.recent_issuances.map(r => `
        <tr class="${r.effective_status === 'OVERDUE' ? 'row-danger' : ''}">
            <td>${r.athlete_name}</td>
            <td>${r.product_name}</td>
            <td>${r.issuance_date}</td>
            <td>${r.planned_return_date || '-'}</td>
            <td><span class="badge badge-${r.effective_status.toLowerCase()}>${r.effective_status}</span></td>
        </tr>
    `).join('');

    // Топ-должники
    document.getElementById('top-debtors-tbody').innerHTML = d.top_debtors.map(t => `
        <tr class="row-danger">
            <td>${t.athlete_name}</td>
            <td>${t.sports_category || '-'}</td>
            <td>${t.overdue_items}</td>
            <td>${t.max_days} дн.</td>
        </tr>
    `).join('');
}

// ===== СПОРТСМЕНЫ =====

function debounceSearch() {
    clearTimeout(searchTimeout);
    searchTimeout = setTimeout(loadAthletes, 300);
}

async function loadAthletes() {
    const search = document.getElementById('athlete-search').value;
    const debt = document.getElementById('debt-filter').value;
    const res = await fetch(`${API}/api/athletes?search=${search}&has_debt=${debt}`);
    const data = await res.json();
    document.getElementById('athletes-tbody').innerHTML = data.map(a => `
        <tr class="${a.has_active_debt ? 'row-warning' : ''}">
            <td><strong>${a.full_name}</strong></td>
            <td>${a.sports_category || '-'}</td>
            <td>${a.birth_year || '-'}</td>
            <td>${a.group_name || '-'}</td>
            <td>${a.coach_name || '-'}</td>
            <td>${a.active_items}</td>
            <td>${a.has_active_debt ? '⚠' : '✅'}</td>
            <td><button class="btn-sm" onclick="viewAthlete(${a.athlete_id})">🔍</button></td>
        </tr>
    `).join('');
}

async function viewAthlete(id) {
    const res = await fetch(`${API}/api/athletes/${id}`);
    if (!res.ok) return showToast('Спортсмен не найден', 'error');
    const a = await res.json();
    document.getElementById('athlete-modal').classList.remove('hidden');
    document.getElementById('athlete-detail').innerHTML = `
        <h3>${a.full_name}</h3>
        <div class="detail-grid">
    `
}

```

```

        <div class="detail-item"><span class="detail-label">Email</span><span>${a.email || '-'}</span></div>
        <div class="detail-item"><span class="detail-label">Телефон</span><span>${a.phone || '-'}</span></div>
        <div class="detail-item"><span class="detail-label">Разряд</span><span>${a.sports_category || '-'}</span></div>
        <div class="detail-item"><span class="detail-label">Год рождения</span><span>${a.birth_year || '-'}</span></div>
        <div class="detail-item"><span class="detail-label">Группа</span><span>${a.group_name || '-'}</span></div>
        <div class="detail-item"><span class="detail-label">Тренер</span><span>${a.coach_name || '-'}</span></div>
        <div class="detail-item"><span class="detail-label">Активных выдач</span><span>${a.active_items}</span></div>
        <div class="detail-item"><span class="detail-label">Возвращено</span><span>${a.returned_items}</span></div>
        <div class="detail-item"><span class="detail-label">Статус</span><span class="${a.has_active_debt ? 'text-danger' : 'text-success'}">${a.has_active_debt ? '⚠ Должник' : '✅ Без долгов'}</span></div>
    </div>
    `;
}

function closeAthleteModal() {
    document.getElementById('athlete-modal').classList.add('hidden');
}

// ===== ВЫДАЧИ =====

async function loadIssuances() {
    const filter = document.getElementById('issuance-filter').value;
    const res = await fetch(`${API}/api/issuance/active?filter_status=${filter}`);
    const data = await res.json();
    document.getElementById('issuances-tbody').innerHTML = data.map(d => `
        <tr class="${d.effective_status === 'OVERDUE' ? 'row-danger' : ''}">
            <td><strong>${d.athlete_name}</strong></td>
            <td>${d.product_name}</td>
            <td>${d.size || '-'}</td>
            <td>${d.issuance_date}</td>
            <td>${d.planned_return_date || '-'}</td>
            <td><span class="badge badge-${d.effective_status.toLowerCase()}>${d.effective_status === 'OVERDUE' ? `Просрочено (${d.days_overdue}д)` : 'Активна'}</span></td>
            <td><button class="btn-sm btn-return" onclick="returnInventory(${d.issuance_document_id})">↩</button></td>
        </tr>
    `).join('');
}

async function openIssueModal() {
    document.getElementById('issue-modal').classList.remove('hidden');
    const res = await fetch(`${API}/api/athletes`);
    const data = await res.json();
    document.getElementById('issue-athlete').innerHTML = '<option value="">— Выберите —</option>' +
        data.map(a => `<option value="${a.athlete_id}">${a.full_name}
    ${a.has_active_debt ? '⚠' : ''}</option>`).join('');
    document.getElementById('issue-inventory').innerHTML = '<option value="">— Сначала выберите спортсмена —</option>';
}

```

```

function closeIssueModal() {
    document.getElementById('issue-modal').classList.add('hidden');
}

document.getElementById('issue-athlete').addEventListener('change', async
function () {
    const res = await fetch(`${API}/api/inventory/available`);
    const data = await res.json();
    document.getElementById('issue-inventory').innerHTML = '<option
value="">- Выберите -</option>' +
        data.map(i => `<option value="${i.inventory_item_id}">${i.barcode} -
${i.nomenclature_name} (${i.size || '6/p'})</option>`).join('');
});

async function issueInventory() {
    const res = await fetch(`${API}/api/issuance/issue`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            athlete_id: parseInt(document.getElementById('issue-ath-
lete').value),
            inventory_item_id: parseInt(document.getElementById('issue-in-
ventory').value),
            planned_return_date: document.getElementById('issue-return-
date').value,
        }),
    });
    const data = await res.json();
    showToast(data.message, data.status);
    if (data.status === 'success') {
        closeIssueModal();
        loadDashboard();
        loadAthletes();
        loadIssuances();
        loadInventory();
    }
}

async function returnInventory(id) {
    const res = await fetch(`${API}/api/issuance/return`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ issuance_id: id }),
    });
    const data = await res.json();
    showToast(data.message, data.status);
    loadDashboard();
    loadIssuances();
    loadInventory();
    loadAthletes();
}

// ===== CKJIAД =====

async function loadInventory() {
    const res = await fetch(`${API}/api/inventory/available`);
    const data = await res.json();
    document.getElementById('inventory-tbody').innerHTML = data.map(i => `
        <tr>
            <td>${i.barcode}</td>
            <td>${i.nomenclature_name}</td>
            <td>${i.nomenclature_category}</td>
            <td>${i.size || '-'}</td>
        </tr>
    `);
}

```

```

    `).join('');
}

// ===== СТАТИСТИКА =====

async function loadStatistics() {
    const res = await fetch(`${API}/api/inventory/statistics`);
    const data = await res.json();
    document.getElementById('stats-container').innerHTML = data.map(s => `
        <div class="stat-card">
            <h3>${s.nomenclature_category}</h3>
            <div class="stat-row">
                <span>Доступно:</span><span class="stat-availa-
ble">${s.available}</span>
            </div>
            <div class="stat-row">
                <span>Выдано:</span><span class="stat-issued">${s.is-
sued}</span>
            </div>
            <div class="stat-row">
                <span>Проблемы:</span><span class="stat-problems">${s.prob-
lems}</span>
            </div>
            <div class="stat-bar">
                <div class="stat-bar-available" style="width:${(s.available /
s.total * 100).toFixed(0)}%"></div>
                <div class="stat-bar-issued" style="width:${(s.issued /
s.total * 100).toFixed(0)}%"></div>
                <div class="stat-bar-problems" style="width:${(s.problems /
s.total * 100).toFixed(0)}%"></div>
            </div>
        `).join('');
}

// ===== ИНИЦИАЛИЗАЦИЯ =====

loadDashboard();

```